
MATH FOUNDATIONS

Toán học Nền tảng & Bản chất Hệ thống trong AI/ML

Ngày cập nhật gần nhất: Ngày 6 tháng 10 năm 2026

Chú thích: Tài liệu được tổng hợp trong quá trình mình học tập, mọi đóng góp xin gửi về
work.thanhnguyenviet@gmail.com

Mục lục

Bảng Tra Cấu Thuật Ngữ, Thuật Toán & Kiến Thức Chuyên Môn (Lesson 1 – 22)	1
Lesson 1: Linear Algebra Intuition	18
I. Độc lập tuyến tính & đa cộng tuyến	18
II. Tại sao $w = (X^T \cdot X)^{-1}X^Ty$ mà không phải $w = X^{-1}y$ với $Xw = y$	18
III. Phép chiếu trực giao (Orthogonal projection)	19
IV. Gram Schmidt	20
V. Độ tương đồng Cosine (Cosine Similarity), Embedding & Kỹ thuật LoRA	21
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	23
Lesson 2: Vectors, Matrices, Operations	24
I. Tầng Dense: vì sao là $Z = X @ W^T + b$	24
II. Broadcasting & Bẫy Tràn bộ nhớ	24
III. Lưu ý và Backpropagation	27
IV. Phép nhân Hadamard vs Nhân ma trận & Công thức đếm tham số Mạng Nơ-ron	31
V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	33
Lesson 3: Matrix Transformations and Eigenvalues	34
I. Ma trận là cỗ máy biến đổi không gian	34
II. Composition — Thứ tự phép biến đổi: $A @ B \neq B @ A$	35
III. Định thức $\det(A)$: Thước đo tỷ lệ co giãn thể tích	35
IV. Eigenvalues & Eigenvectors (Trị riêng & Vectơ riêng)	35
V. Vì sao Eigenvalues là trọng tâm trong ML	36
VI. Biểu diễn Vectơ qua Phép biến đổi tuyến tính & Định lý Phổ (Spectral Theorem)	38
VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	40
Lesson 4: Calculus for Machine Learning	41
I. Bản chất và cơ chế	41
II. Thuật toán Gradient Descent and Learning Rate	42
III. Đạo hàm số và Đạo hàm giải tích	43
IV. Ma trận Hessian — Đo độ cong không gian	44
V. Xấp xỉ chuỗi Taylor (Taylor Series Approximation)	46
VI. Tích phân trong Học máy (Integrals in ML)	46
VII. Quy tắc chuỗi đa biến trong Đồ thị tính toán	47
VIII. Ma trận Jacobian & Bảng đạo hàm các Hàm kích hoạt hiện đại	47

IX. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	49
Lesson 5: Chain Rule and Autodiff	50
I. Vấn đề	50
II. The Concept	50
III. Tại sao phải gọi <code>optimizer.zero_grad()</code> & Vấn đề Đạo hàm trong RNN / LSTM	54
IV. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	56
Lesson 6: Probability and Distributions	57
I. Concept	57
II. Common Distributions (5 phân phối xác suất cốt lõi)	57
III. Expected Value & Variance (Kỳ vọng và Phương sai)	58
IV. Joint and Marginal Distributions (Phân phối đồng thời & biên)	59
V. Why the Normal Distribution Shows Up Everywhere	59
VI. Log Probabilities (Xác suất Logarit)	60
VII. Softmax Function (trong vở đánh số VI)	61
VIII. Sampling (Lấy mẫu ngẫu nhiên)	62
IX. Các chiến lược Sampling trong LLM, Tổ hợp xác suất & Kiểm định giả thuyết . .	63
X. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	65
Lesson 7: Bayes Theorem	66
I. The Concept	66
II. Naive Bayes, MLE and Laplace Smoothing	67
III. Maximum A Posteriori (MAP) vs MLE và Tư duy Bayesian trong ML	67
IV. Bản chất Bayes của Regularization (L1/L2) & Ma trận Nhầm lẫn (Confusion Matrix)	69
V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	71
Lesson 8: Optimization: Gradient Descent Family	72
I. The Problem & The Concept	72
II. SGD vs Batch vs Mini-batch Gradient Descent	72
III. Momentum (Quán tính)	73
IV. ADAM: Adaptive Learning Rates	74
V. Learning Rate Schedules (Lịch trình tốc độ học)	75
VI. Convex vs Non-Convex & Loss Landscape	75
VII. Các Kỹ thuật Chống Overfitting & Chiến lược Tìm kiếm Siêu tham số	76
VIII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	77

Lesson 9: Information Theory	78
I. The Concept	78
II. Mô hình Ngôn ngữ N-grams & Các Chỉ số Đánh giá trong NLP	83
III. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	85
Lesson 10: Dimensionality Reduction	86
I. The Problem	86
II. The Concept: Curse of Dimensionality (Lời nguyên chiều cao)	86
III. PCA: Find the Directions That Matter	87
IV. t-SNE ⁽¹⁾ : Bảo toàn Láng giềng Cục bộ	87
V. Thuật toán UMAP	88
VI. When to Use Which (Khi nào dùng thuật toán nào?)	89
VII. Kernel PCA	89
VIII. Reconstruction Error (Sai số Tái tạo)	90
IX. Các Kỹ thuật Chuẩn hóa Dữ liệu (Feature Scaling), Chống Rò rỉ Dữ liệu & K-Means / Kernel SVM	92
X. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	93
Lesson 11: Singular Value Decomposition (SVD)	94
I. The Problem	94
II. The Concept	94
III. Matrix Factorization trong Hệ Gợi ý (Recommender Systems) & Toán học trong CNN / Transformer	97
IV. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	99
Lesson 12: Tensor Operations	100
I. Vấn đề (The Problem)	100
II. Bản chất Tensor & Quy ước Shape trong AI	100
III. Cấu trúc Bộ nhớ Vật lý 1D & Bước nhảy (Strides)	101
IV. Thủ thuật <code>stride = 0</code> trong Broadcasting & Nhân Ma trận theo Lô	104
V. Ký hiệu Tổng Einstein (Einstein Summation — <code>einsum</code>)	106
VI. Ứng dụng : Truy vết Shape trong Multi-Head Attention & KV-Cache	108
VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	110
Lesson 13: Numerical Stability	111
I. Vấn đề (The Problem)	111
II. Chuẩn IEEE 754: Cấu trúc Số Dấu phẩy động (FP32, FP16, BF16, FP8)	111
III. Machine Epsilon, Swamping & Triệt tiêu Thảm khốc	112

IV. Trần số, Shift-Invariant Softmax & Thủ thuật Log-Sum-Exp	115
V. Kiểm soát Lỗi Số học & Ổn định Huấn luyện	116
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	120
Lesson 14: Norms and Distances	121
I. Vấn đề (The Problem)	121
II. Chuẩn Vector (Vector Norms) & Họ Khoảng cách L_p	121
III. Khoảng cách Euclid (L_2) vs Độ tương đồng Cosine (Cosine Similarity)	123
IV. Khoảng cách Mahalanobis (Mahalanobis Distance)	124
V. Khoảng cách trên Tập hợp, Chuỗi ký tự & Phân phối Xác suất	125
VI. Bảng Chọn Khoảng cách theo Bài toán & Liên hệ với các Hàm Loss	128
VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	129
Lesson 15: Statistics for ML	130
I. Vấn đề (The Problem)	130
II. Thống kê Mô tả & Hiệu chỉnh Bessel (Descriptive Statistics)	130
III. Luật Số Lớn (LLN) vs Định lý Giới hạn Trung tâm (CLT)	133
IV. Khoảng Tin Cậy (CI) & Lấy mẫu lại Bootstrap	134
V. Kiểm định Giả thuyết, A/B Testing & Đa kiểm định	135
VI. Nghịch lý Simpson (Simpson's Paradox)	137
VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	138
Lesson 16: Sampling Methods	139
I. Vấn đề (The Problem)	139
II. Biến đổi Ngược CDF & Reparameterization	139
III. Lấy mẫu Bác bỏ (Rejection) & Lấy mẫu Quan trọng (Importance)	141
IV. Markov Chain Monte Carlo (MCMC) & Metropolis-Hastings	142
V. Chiến lược Giải mã & Lấy mẫu Token trong LLM (LLM Decoding)	144
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	146
Lesson 17: Linear Systems	147
I. Vấn đề (The Problem)	147
II. Khử Gauss, Đảo hàng (Partial Pivoting) & Phân rã $PA = LU$	147
III. Phân rã Cholesky ($A = LL^T$) cho Ma trận SPD	150
IV. Số Điều Kiện $\kappa(A)$ (Condition Number)	151
V. Hệ Quá Xác Định ($m > n$), Phân rã QR & Ổn định hóa Ridge	152
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	154

Lesson 18: Convex Optimization	155
I. Vấn đề (The Problem)	155
II. Tập Lồi (Convex Sets) & Hàm Lồi (Convex Functions)	155
III. Điều kiện Khả vi Bậc 1, Bậc 2 & Hội tụ Tuyến tính	157
IV. Bất đẳng thức Jensen (Jensen's Inequality) — Trụ cột của Xác suất & AI	158
V. Tối ưu có Ràng buộc: Nhân tử Lagrange, Đối ngẫu, KKT & SVM	159
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	162
Lesson 19: Complex Numbers for AI	163
I. Vấn đề (The Problem)	163
II. Dạng Đại số (Cartesian Form), Liên hợp & Mặt phẳng Phức	163
III. Dạng Cực (Polar Form) & Công thức Euler Huyền thoại	165
IV. Ứng dụng Chuyên sâu: Rotary Position Embedding (RoPE) trong LLMs	166
V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	168
Lesson 20: The Fourier Transform	169
I. Vấn đề (The Problem) & Trực giác Cốt lõi	169
II. Biến đổi Fourier Rời rạc (Discrete Fourier Transform — DFT)	169
III. Thuật toán Fast Fourier Transform (FFT — Cooley-Tukey)	170
IV. Định lý Lấy mẫu Nyquist-Shannon & Định lý Tích chập (Convolution Theorem)	171
V. Ứng dụng Chuyên sâu trong AI: STFT, Lọc Ảnh & Spectral Bias	172
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	174
Lesson 21: Graph Theory for ML	175
I. Vấn đề (The Problem)	175
II. Biểu diễn Đại số của Đồ thị (Algebraic Representation)	176
III. Ma trận Graph Laplacian & Lý thuyết Đồ thị Phổ (Spectral Graph Theory)	179
IV. Ứng dụng Chuyên sâu trong AI hiện đại (Deep AI Applications)	182
V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	184
Lesson 22: Stochastic Processes	185
I. Vấn đề (The Problem)	185
II. Chuỗi Markov Thời gian Rời rạc (DTMC)	186
III. Bước đi Ngẫu nhiên, Quá trình Poisson & Chuyển động Brown	189
IV. Ứng dụng Chuyên sâu 1: Học Tăng cường & Phương trình Bellman (MDP)	190
V. Ứng dụng Chuyên sâu 2: Mô hình Khuếch tán (DDPM & Langevin)	191
VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)	193

BẢNG TRA CỨU NHANH THUẬT NGỮ & THUẬT TOÁN (LESSON 1 – 22)

Mẹo sử dụng: Bấm trực tiếp vào liên kết màu xanh ở cột ngoài cùng bên phải để nhảy thẳng đến đúng đề mục lý thuyết tương ứng.

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
LESSON 1: LINEAR ALGEBRA INTUITION (TRỰC GIÁC ĐẠI SỐ TUYẾN TÍNH)			
Multicol- linearity	Đa cộng tuyến & Phụ thuộc tuyến tính (<i>Linear Dependence</i>)	Khi các cột đặc trưng trùng lặp thông tin làm ma trận $X^T X$ suy biến (không khả nghịch), khiến trọng số hồi quy dao động hỗn loạn.	L.1 – Mục I
Normal Equation (OLS)	Phương trình Pháp tuyến (<i>Ordinary Least Squares</i>)	Nghiệm đóng $w = (X^T X)^{-1} X^T y$ tìm siêu phẳng khớp dữ liệu nhất khi hệ có nhiều phương trình hơn ẩn ($m > n$).	L.1 – Mục II
Orthogonal Projection	Phép chiếu trực giao	Hạ đường vuông góc từ vector mục tiêu y xuống không gian cột của X để cực tiểu hóa sai số bình phương $\ Xw - y\ _2^2$.	L.1 – Mục III
Gram- Schmidt & QR	Trực chuẩn hóa Gram-Schmidt & Phân rã QR	Biến hệ vector bất kỳ thành hệ cơ sở vuông góc từng đôi một (khử phần trùng lặp thông tin), dùng để giải hệ phương trình ổn định số học.	L.1 – Mục IV
Dot Product & Cosine	Tích vô hướng & Độ tương đồng Cosine (<i>Cosine Similarity</i>)	Đo mức độ cùng hướng ($\cos \theta$) giữa 2 vector ngữ nghĩa bất chấp độ dài văn bản dài hay ngắn.	L.1 – Mục V.1
LoRA	<i>Low-Rank Adaptation</i> — Tinh chỉnh Hạng thấp	Phân tách ma trận cập nhật trọng số lớn $\Delta W_{d \times k} = B_{d \times r} A_{r \times k}$ ($r \ll d$) giúp tinh chỉnh LLM giảm tới 99% số tham số cần huấn luyện.	L.1 – Mục V.2
Embedding, Span & Basis	Không gian Nhúng, Không gian Sinh & Hệ Cơ sở	Biểu diễn từ vựng/ảnh thành tọa độ thực liên tục; hệ cơ sở là tập trực tối thiểu vừa đủ để tổ hợp ra mọi điểm trong không gian.	L.1 – Mục V.3
LESSON 2: VECTORS, MATRICES & OPERATIONS (VECTOR, MA TRẬN & PHÉP TOÁN)			
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Dense / Linear Layer	Tầng Kết nối Đầy đủ ($Z = XW^T + b$)	Phép biến đổi tuyến tính đồng thời cả một lô dữ liệu (<i>batch</i>) qua ma trận trọng số chuyển vị W^T .	L.2 – Mục I
Broadcasting	Cơ chế Tự động Mở rộng Chiều Tensor	Tự kéo giãn chiều có kích thước 1 để cộng/nhân hai Tensor khác shape; giúp tránh bẫy nhân bản chiều ảo làm tràn bộ nhớ RAM/VRAM.	L.2 – Mục II
Matrix Backprop	Lan truyền ngược dạng Ma trận	Công thức truyền đạo hàm qua phép nhân ma trận: $\nabla_X = \nabla_Z W$ và $\nabla_W = \nabla_Z^T X$ dựa trên quy tắc khớp chiều.	L.2 – Mục III.1
Hadamard Product ($\odot$)	Phép nhân từng phần tử (<i>Element-wise Product</i>)	Nhân trực tiếp các phần tử cùng vị trí $A_{ij} \cdot B_{ij}$ (dùng trong các cổng đóng/mở của LSTM, Attention Mask, Dropout).	L.2 – Mục IV.1
MLP Parameter Count	Công thức Đếm Tham số Mạng Nơ-ron	Tính tổng số trọng số và bias $\sum(d_{in} \cdot d_{out} + d_{out})$ để ước lượng dung lượng VRAM cần thiết cho mô hình.	L.2 – Mục IV.2
Outer Product & Frobenius	Tích ngoài ($\vec{u}\vec{v}^T$) & Chuẩn Frobenius ($\ A\ _F$)	Tích ngoài tạo ma trận hạng 1 từ 2 vector; chuẩn Frobenius đo tổng độ lớn năng lượng của toàn bộ các phần tử trong ma trận.	L.2 – Mục IV.3
LESSON 3: MATRIX TRANSFORMATIONS & EIGENVALUES (BIẾN ĐỔI KHÔNG GIAN & TRỊ RIÊNG)			
Linear Transformation	Phép biến đổi Không gian Tuyến tính	Nhìn ma trận như cỗ máy bóp méo lưới tọa độ (xoay, co giãn, trượt <i>shear</i> , phản xạ) thông qua quỹ đạo của các vector cơ sở.	L.3 – Mục I
Determinant ($\det(A)$)	Định thức của Ma trận	Đo hệ số phóng đại/thu nhỏ diện tích (2D) hoặc thể tích (3D); $\det(A) = 0$ báo hiệu không gian bị ép bẹp làm mất thông tin vĩnh viễn.	L.3 – Mục III
Eigenvalues & Eigenvectors	Trị riêng (λ) & Vectơ riêng ($\vec{v}$)	Những trục phương đặc biệt không bị xoay đi khi nhân ma trận ($A\vec{v} = \lambda\vec{v}$), chỉ bị co giãn theo tỷ lệ λ .	L.3 – Mục IV
Spectral Theorem	Định lý Phổ ($A = Q\Lambda Q^T$)	Mọi ma trận thực đối xứng (như ma trận hiệp phương sai, Hessian, Graph Laplacian) đều có các trục vectơ riêng vuông góc với nhau từng đôi một.	L.3 – Mục VI.2
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
PageRank & ResNet Skip	Ứng dụng Trị riêng ($\lambda = 1$) trong PageRank & ResNet	Xếp hạng trang web của Google là tìm vectơ riêng ứng với $\lambda = 1$; kết nối tắt $x + f(x)$ trong ResNet giữ trị riêng quanh 1 chống tắt đạo hàm.	L.3 – Mục VI.3
LESSON 4: CALCULUS FOR MACHINE LEARNING (GIẢI TÍCH CHO HỌC MÁY)			
Gradient (∇f)	Vectơ Đạo hàm riêng bậc một	Chỉ hướng làm hàm số tăng dốc nhất tại một điểm; đi ngược hướng Gradient ($-\nabla L$) là nguyên lý cốt lõi giúp mô hình giảm hàm Loss.	L.4 – Mục I
Gradient Descent (GD)	Thuật toán Hạ Gradient & Tốc độ học (<i>Learning Rate</i> η)	Cập nhật lặp $\theta \leftarrow \theta - \eta \nabla L(\theta)$ để dò đáy thung lũng hàm mất mát; chọn η quá lớn sẽ gây văng nghiệm phân kỳ.	L.4 – Mục II
Centered Difference	Công thức Sai phân Trung tâm	Xấp xỉ đạo hàm số học $\frac{f(x+h)-f(x-h)}{2h}$ đạt độ chính xác bậc hai $\mathcal{O}(h^2)$, dùng để kiểm tra code đạo hàm giải tích.	L.4 – Mục III
Hessian Matrix (H)	Ma trận Đạo hàm riêng bậc hai ($\nabla^2 f$)	Đo độ cong cục bộ của bề mặt hàm Loss theo mọi hướng; trị riêng của H giúp phân biệt đáy cực tiểu, đỉnh cực đại và điểm yên ngựa (<i>Saddle Point</i>).	L.4 – Mục IV
Taylor Series	Khai triển Xấp xỉ Chuỗi Taylor	Xấp xỉ hàm phức tạp quanh một điểm bằng đa thức bậc 1 (tiếp tuyến — GD) hoặc bậc 2 (parabol — phương pháp Newton).	L.4 – Mục V
Jacobian Matrix (J)	Ma trận Đạo hàm của Hàm Vectơ	Chứa mọi đạo hàm riêng $\frac{\partial f_i}{\partial x_j}$ khi cả đầu vào lẫn đầu ra đều là nhiều chiều (như tầng Softmax hay mạng biến đổi tọa độ).	L.4 – Mục VIII.1
Activation Functions	Các Hàm Kích hoạt Phi tuyến (ReLU, GELU, SiLU/Swish)	Bề cong không gian giúp mạng nơ-ron học được hàm phi tuyến phức tạp; khắc phục lỗi bão hòa đạo hàm của Sigmoid/Tanh và lỗi chết nơ-ron (<i>Dying ReLU</i>).	L.4 – Mục VIII.2
LESSON 5: CHAIN RULE & AUTODIFF (QUY TẮC CHUỖI & VI PHÂN TỰ ĐỘNG)			
Chain Rule	Quy tắc Đạo hàm Hàm hợp	Nhân các mắt xích đạo hàm cục bộ để tính độ nhạy của đầu ra cuối cùng theo từng tham số ở các tầng ẩn sâu bên trong.	L.5 – Mục II.1
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Reverse- Mode Autodiff	Vì phân Tự động Chế độ Ngược (<i>Autograd</i>)	Tính đạo hàm của hàm Loss vô hướng theo hàng tỷ tham số chỉ trong đúng 1 lượt lan truyền ngược trên Đồ thị tính toán (<i>Computational Graph</i>).	L.5 – Mục II.2
<code>zero_grad()</code>	Xóa Bộ nhớ Dệm Đạo hàm Cộng dồn	PyTorch mặc định cộng dồn (+= vào <code>.grad</code> để hỗ trợ tích lũy mini-batch; phải gọi <code>zero_grad()</code> đầu mỗi bước nếu không đạo hàm sẽ bị cộng chồng sai lệch.	L.5 – Mục III.1
BPTT & LSTM Cell State	<i>Backpropagation Through Time</i> — Lan truyền ngược qua Thời gian	Giải thích vì sao nhân ma trận lặp lại trong RNN gây biến mất/bùng nổ đạo hàm, và cách đường cao tốc cộng tuyến tính của LSTM/GRU cứu đạo hàm.	L.5 – Mục III.2
Local Gradient Gates	Các Cổng Đạo hàm Cục bộ (Add, Mul, Copy, Max)	Cổng Cộng phân phối nguyên vẹn đạo hàm; cổng Nhân trao đổi giá trị hai nhánh; cổng Max chỉ cho đạo hàm đi qua nhánh lớn nhất.	L.5 – Mục III.3
LESSON 6: PROBABILITY & DISTRIBUTIONS (XÁC SUẤT & CÁC PHÂN PHỐI CỐT LỖI)			
PMF vs PDF	<i>Probability Mass / Density Function</i> — Hàm Khối vs Hàm Mật độ	PMF gán xác suất trực tiếp cho biến rời rạc (đếm được); PDF đo mật độ của biến liên tục (xác suất tại 1 điểm lẻ bằng 0, phải lấy tích phân trên khoảng).	L.6 – Mục I
5 Core Distributions	Bernoulli, Binomial, Categorical, Uniform, Gaussian	5 khối Lego xác suất xây dựng nên mọi mô hình AI: phân loại nhị phân (Bernoulli), đa lớp (Categorical), khởi tạo trọng số (Uniform/Gaussian).	L.6 – Mục II
Expectation & Variance	Kỳ vọng ($\mathbb{E}[X]$) & Phương sai ($\text{Var}(X)$)	Kỳ vọng là tâm trọng lực trung bình dài hạn; phương sai đo mức độ dao động bình phương quanh tâm.	L.6 – Mục III
Joint & Marginal Prob.	Phân phối Đồng thời $P(X, Y)$ & Phân phối Biên $P(X)$	Quy tắc cộng triệt tiêu biến không quan tâm (<i>Marginalization / Sum Rule</i>) và quy tắc nhân xác suất có điều kiện (<i>Product Rule</i>).	L.6 – Mục IV
Log- Probabilities	Xác suất Logarit ($\sum \log P(x_i)$)	Biến phép nhân hàng ngàn xác suất nhỏ (dễ tràn số dưới về 0.0) thành phép cộng các số âm an toàn tuyệt đối trên máy tính.	L.6 – Mục VI
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Softmax Function	Hàm Chuẩn hóa Xác suất Đa lớp	Biến vector điểm thô (<i>Logits</i>) $\vec{z} \in \mathbb{R}^K$ thành phân phối xác suất hợp lệ dương và có tổng bằng 1.	L.6 – Mục VII
MSE & BCE Origin	Nguồn gốc Xác suất của Hàm Mất mất MSE & BCE	Giả định sai số nhiễu tuân theo phân phối Gaussian sẽ dẫn xuất ra hàm MSE; giả định nhân tuân theo phân phối Bernoulli dẫn xuất ra hàm Binary Cross-Entropy.	L.6 – Mục IX.4
LESSON 7: BAYES THEOREM (ĐỊNH LÝ BAYES & SUY LUẬN THỐNG KÊ)			
Bayes’ Theorem	Định lý Bayes ($P(H D) = \frac{P(D H)P(H)}{P(D)}$)	Quy tắc cập nhật niềm tin từ Tiên nghiệm (<i>Prior</i>) sang Hậu nghiệm (<i>Posterior</i>) khi có thêm bằng chứng mới; tránh bẫy Ngụy biện Tỷ lệ Nền (<i>Base Rate Fallacy</i>).	L.7 – Mục I
Naive Bayes & Laplace	Bộ phân loại Naive Bayes & Làm mượt Laplace (<i>Add-1 Smoothing</i>)	Giả định các từ độc lập có điều kiện để lọc thư rác siêu tốc; cộng thêm $\alpha = 1$ vào tần suất đếm để tránh 1 từ mới có xác suất 0 xóa số cả tích.	L.7 – Mục II
MLE vs MAP	<i>Maximum Likelihood</i> vs <i>Maximum A Posteriori</i>	MLE chỉ tối ưu khớp dữ liệu quan sát; MAP kết hợp thêm phân phối tiên nghiệm $P(\theta)$ đóng vai trò như bộ điều chuẩn chống học vẹt khi ít dữ liệu.	L.7 – Mục III
Bayesian L1 / L2	Bản chất Bayes của Ridge (L_2) và Lasso (L_1)	Phạt chuẩn L_2 tương đương với MAP khi đặt tiên nghiệm Gaussian lên trọng số; phạt chuẩn L_1 tương đương với tiên nghiệm Laplace nhọn đỉnh tại 0.	L.7 – Mục IV.1
Precision, Recall & F1	Ma trận Nhầm lẫn (<i>Confusion Matrix</i>) & Điểm F1	Đánh giá mô hình trên dữ liệu mất cân bằng: Precision đo độ chuẩn khi báo dương tính; Recall đo khả năng không bỏ sót; F1 là trung bình điều hòa của cả hai.	L.7 – Mục IV.2
Conjugate Priors	Tiên nghiệm Liên hợp (Beta – Binomial)	Khi Tiên nghiệm và Hậu nghiệm cùng thuộc một họ phân phối, giúp cập nhật xác suất Bayes trực tiếp bằng phép cộng tham số mà không cần tính tích phân.	L.7 – Mục IV.3
LESSON 8: OPTIMIZATION: GRADIENT DESCENT FAMILY (HỌ THUẬT TOÁN TỐI ƯU HÓA)			
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
SGD & Flat Minima	<i>Stochastic / Mini-batch Gradient Descent</i>	Tính đạo hàm trên từng lô nhỏ vừa tăng tốc trên GPU, vừa tạo nhiễu ngẫu nhiên có ích giúp mô hình nhảy khỏi đáy nhọn (<i>Sharp Minima</i>) sang đáy phẳng tổng quát tốt.	L.8 – Mục II
Momentum (EMA)	Động lượng / Quán tính Trung bình Trượt Hàm mũ	Tích lũy vận tốc từ các bước trước ($v_t = \beta v_{t-1} + g_t$) giúp triệt tiêu dao động dích dắc ngang vách hẻm núi và tăng tốc dọc theo đáy thung lũng.	L.8 – Mục III
Adam Optimizer	<i>Adaptive Moment Estimation</i> & Hiệu chỉnh Chệch	Kết hợp quán tính bậc 1 (m_t) và bình phương đạo hàm bậc 2 (v_t) kèm hiệu chỉnh ($1 - \beta^t$) để tự động co giãn tốc độ học riêng cho từng tham số.	L.8 – Mục IV
Warmup & Cosine Decay	Lịch trình Tốc độ học (<i>Learning Rate Schedules</i>)	Tăng dần LR ở đầu quá trình huấn luyện (<i>Linear Warmup</i>) để ổn định thống kê Adam, sau đó hạ mượt theo đường cong Cosine để hội tụ êm vào đáy.	L.8 – Mục V
Early Stopping & Dropout	Chẩn đoán Overfitting & Kỹ thuật Chống Học vẹt	Theo dõi khoảng cách giữa Train Loss và Val Loss để dừng huấn luyện đúng lúc; tắt ngẫu nhiên nơ-ron giúp mạng không phụ thuộc cục bộ.	L.8 – Mục VII.1
Grid vs Random Search	Chiến lược Dò tìm Siêu tham số (<i>Hyperparameter Tuning</i>)	Giải thích vì sao lấy mẫu ngẫu nhiên (Random Search) luôn khám phá được nhiều giá trị quan trọng hơn lưới cố định (Grid Search) với cùng ngân sách thử nghiệm.	L.8 – Mục VII.2
AdamW	<i>Adam with Decoupled Weight Decay</i>	Tách riêng bước kéo trọng số về 0 ra khỏi mẫu số chia thích nghi $\sqrt{v_t}$ của Adam; bộ tối ưu tiêu chuẩn bắt buộc khi huấn luyện Transformer/LLM.	L.8 – Mục VII.3
LESSON 9: INFORMATION THEORY (LÝ THUYẾT THÔNG TIN)			
Shannon Entropy (H)	Lượng tin Tự thân & Độ bất ngờ Trung bình	Đo mức độ hỗn loạn/không chắc chắn của một phân phối xác suất ($H(P) = -\sum p_i \log p_i$); phân phối đều có Entropy đạt cực đại.	L.9 – Mục I.2
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Cross-Entropy ($H(P, Q)$)	Entropy Chéo — Hàm Mất mát Phân loại	Do số bit trung bình tổn kém khi dùng phân phối dự đoán Q của mô hình để mã hóa nhãn sự thật P ; đạo hàm đẹp gọn thành $(\hat{y} - y)$.	L.9 – Mục I.3
KL Divergence (D_{KL})	<i>Kullback-Leibler Divergence</i> — Độ phân kỳ KL	Do lượng thông tin bị thất thoát khi xấp xỉ P bằng Q ($D_{KL} = H(P, Q) - H(P) \geq 0$); bất đối xứng dẫn tới hành vi <i>Mode-covering</i> vs <i>Mode-seeking</i> .	L.9 – Mục I.4
Mutual Information (I)	Thông tin Tương hỗ ($I(X; Y)$)	Do lượng độ bất ngờ về X giảm đi khi biết Y ; phát hiện được mọi sự phụ thuộc phi tuyến tính mà hệ số tương quan Pearson bỏ sót.	L.9 – Mục I.5
Label Smoothing	Kỹ thuật Làm mịn Nhãn	Thay nhãn cứng $[1, 0, 0]$ bằng $[1 - \epsilon, \frac{\epsilon}{K-1}, \dots]$ để ngăn logits phình ra vô cùng và giúp mô hình hiệu chuẩn độ tự tin tốt hơn.	L.9 – Mục I.9
Perplexity (PPL)	Độ bối rối của Mô hình Ngôn ngữ ($PPL = e^{\text{Cross-Entropy}}$)	Biểu thị số lượng lựa chọn từ vựng trung bình mà LLM đang phân vân ở mỗi bước sinh từ; PPL càng thấp mô hình dự đoán càng sắc bén.	L.9 – Mục I.12
N-grams, BLEU & ROUGE	Mô hình Cụm n từ & Chỉ số Đánh giá NLP	BLEU thiên về độ chuẩn (<i>Precision</i>) của các cụm n -grams dùng trong Dịch máy; ROUGE thiên về độ phủ (<i>Recall</i>) dùng trong Tóm tắt văn bản.	L.9 – Mục II
LESSON 10: DIMENSIONALITY REDUCTION (GIẢM CHIỀU DỮ LIỆU)			
Curse of Dimensionality	Lời nguyền Chiều cao	Khi số chiều d quá lớn, dữ liệu trở nên thưa thớt cực độ, thể tích dồn hết ra vỏ ngoài và khoảng cách giữa mọi điểm trở nên xấp xỉ bằng nhau.	L.10 – Mục II
PCA & EVR	<i>Principal Component Analysis</i> & Tỷ lệ Phương sai Giữ lại	Tìm các trục trực giao (vectơ riêng của ma trận hiệp phương sai) có phương sai lớn nhất để nén giảm chiều tuyến tính mà giữ lại tối đa thông tin.	L.10 – Mục III
t-SNE	<i>t-Distributed Stochastic Neighbor Embedding</i>	Giảm chiều phi tuyến xuống 2D/3D để trực quan hóa cụm bằng cách khớp xác suất láng giềng với phân phối Student- t đuôi nặng chống dồn cục (<i>Crowding Problem</i>).	L.10 – Mục IV
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
UMAP	<i>Uniform Manifold Approximation and Projection</i>	Thuật toán giảm chiều trên đa tập dựa trên lý thuyết đồ thị mờ, chạy nhanh hơn t-SNE nhiều lần và bảo toàn tốt hơn cấu trúc khoảng cách toàn cục.	L.10 – Mục V
Kernel PCA	Phân tích Thành phần Chính Hạt nhân (<i>Kernel Trick</i>)	Dùng hàm nhân (RBF, đa thức) để tính tích vô hướng ngầm trong không gian nhiều chiều, giúp tách rời các đám dữ liệu có hình khuyên đồng tâm phi tuyến.	L.10 – Mục VII
Reconstruc- tion Error	Sai số Tái tạo & Phát hiện Bất thường (<i>Anomaly Detection</i>)	Đo khoảng cách $\ x - \hat{x}\ _2^2$ khi nén xuống không gian PCA rồi chiếu ngược lại; điểm gian lận/bất thường sẽ có sai số tái tạo vọt lên rất cao.	L.10 – Mục VIII.2
Feature Scaling	Chuẩn hóa Đặc trưng (Standard, MinMax, Robust, MaxAbs)	Đưa các cột dữ liệu khác đơn vị đo về cùng thang tỷ lệ trước khi chạy PCA, KNN, SVM hay Gradient Descent; dùng <code>RobustScaler</code> (Median/IQR) khi có Outliers.	L.10 – Mục IX.1
Data Leakage	Rò rỉ Dữ liệu khi Tiền xử lý	Tuyệt đối chỉ gọi <code>.fit()</code> bộ chuẩn hóa/PCA trên tập Train rồi dùng thông số đó <code>.transform()</code> tập Test; gộp chung trước khi chia sẽ làm ảo tưởng kết quả.	L.10 – Mục IX.2
K-Means Clustering	Phân cụm Không giám sát K-Means	Lập hai bước gán cụm gần nhất và cập nhật trọng tâm cụm để cực tiểu hóa tổng phương sai nội cụm (<i>Inertia / WCSS</i>).	L.10 – Mục IX.3
LESSON 11: SINGULAR VALUE DECOMPOSITION (PHÂN RÃ GIÁ TRỊ KỶ DỊ – SVD)			
Full SVD ($U\Sigma V^T$)	<i>Singular Value Decomposition</i> — Phân rã Giá trị Kỳ dị	Phân tích mọi ma trận $m \times n$ bất kỳ thành 3 bước hình học: Xoay trực giao (V^T) → Co giãn dọc trục (Σ) → Xoay trực giao (U).	L.11 – Mục II.2
Truncated SVD (A_k)	Xấp xỉ Hạng thấp Tối ưu (Định lý Eckart-Young- Mirsky)	Chỉ giữ lại k giá trị kỳ dị lớn nhất $\sigma_1, \dots, \sigma_k$ để nén ảnh, khử nhiễu và thu được ma trận hạng k gần với ma trận gốc nhất.	L.11 – Mục II.5
Moore- Penrose (A^+)	Ma trận Giả Nghịch đảo (<i>Pseudoinverse</i>)	Tổng quát hóa nghịch đảo cho ma trận không vuông hoặc suy biến ($A^+ = V\Sigma^+U^T$), tìm nghiệm bình phương tối thiểu có chuẩn nhỏ nhất.	L.11 – Mục II.6
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Matrix Factorization & LSA	Phân rã Ma trận Hệ gợi ý & Phân tích Ngữ nghĩa Ẩn	Dự đoán điểm đánh giá phim/sản phẩm còn khuyết (<i>Collaborative Filtering</i>) và trích xuất chủ đề ẩn từ ma trận Từ – Văn bản trong NLP.	L.11 – Mục III.1
CNN Output & 1×1 Conv	Công thức Kích thước Đầu ra CNN & Tích chập Điểm 1×1	Tính chiều dài ảnh sau lọc $\lfloor \frac{W-K+2P}{S} \rfloor + 1$; dùng bộ lọc 1×1 như phép chiếu tuyến tính để nén giảm số kênh đặc trưng.	L.11 – Mục III.2
Paged-Attention	Độ phức tạp $\mathcal{O}(N^2)$ của Self-Attention & Phân trang KV-Cache	Quản lý bộ nhớ KV-Cache theo các khối trang không liên tục (giống RAM hệ điều hành) trong vLLM để chống phân mảnh VRAM khi phục vụ nhiều người dùng.	L.11 – Mục III.3
LESSON 12: TENSOR OPERATIONS (CẤU TRÚC TENSOR, BỘ NHỚ & EINSUM)			
Tensor Strides	Bước nhảy Bộ nhớ Vật lý 1D (<i>Memory Strides</i>)	Mọi Tensor nhiều chiều đều nằm phẳng trên 1 dải RAM 1D; mảng strides quy định số ô nhớ cần nhảy qua khi tăng chỉ số ở từng chiều.	L.12 – Mục III.1
Zero-Copy View vs Copy	Biến đổi Khung nhìn (transpose , view) vs Sao chép (contiguous)	transpose chỉ trao đổi metadata shape/strides trong $\mathcal{O}(1)$ mà không chép dữ liệu; khi mảng mất tính liên tục thì phải gọi .contiguous() trước khi .view() .	L.12 – Mục III.2
stride = 0 & bmm	Thủ thuật Bước nhảy 0 & Nhân Ma trận theo Lô (bmm)	Đặt stride = 0 giúp đọc lặp lại cùng một vùng nhớ mà không tốn thêm 1 byte RAM nào khi Broadcasting; bmm nhân song song hàng loạt cặp ma trận trên GPU.	L.12 – Mục IV
Einstein Summation (einsum)	Ký hiệu Tổng Einstein	Ngôn ngữ ký hiệu chỉ số (VD bhqd, bhkd->bhqk) thay thế mọi tổ hợp transpose , sum , bmm rắc rối bằng một dòng lệnh tự động tối ưu.	L.12 – Mục V
MHA & KV-Cache Sizing	Truy vết Shape Multi-Head Attention & Tính Dung lượng KV-Cache	Tách chiều ẩn $D = H \times d_k$ để tính Attention song song; công thức tính số GB VRAM mà KV-Cache tiêu thụ theo độ dài ngữ cảnh và kích thước batch.	L.12 – Mục VI
LESSON 13: NUMERICAL STABILITY (ĐỘ ỔN ĐỊNH SỐ HỌC TRONG AI)			
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
IEEE 754 (FP32 / FP16 / BF16)	Chuẩn Biểu diễn Số Thực Dấu phẩy động	Số bit mũ (<i>Exponent</i>) quyết định dải động chống tràn số; số bit định trị (<i>Mantissa</i>) quyết định độ chi tiết. BF16 giữ nguyên 8 bit mũ của FP32 nên huấn luyện LLM cực kỳ ổn định.	L.13 – Mục II
Machine Epsilon & Swamping	Độ phân giải Máy (ϵ_{mach}) & Hiện tượng Nuốt số	Khi cộng một số quá nhỏ (như cập nhật đạo hàm ηg) vào trọng số lớn trong FP16, phần nhỏ bị làm tròn biến mất hoàn toàn khiến mạng ngừng học.	L.13 – Mục III.1
Catastrophic Cancellation	Hiện tượng Triệt tiêu Thảm khốc	Khi trừ hai số thực xấp xỉ bằng nhau, các chữ số chính xác ở đầu bị triệt tiêu sạch, chỉ còn lại phần sai số làm tròn ở đuôi gây sai lệch nghiêm trọng.	L.13 – Mục III.2
Shift- Invariant Softmax & LSE	Softmax Bất biến Tịnh tiến & Thủ thuật <i>Log-Sum-Exp</i>	Trừ đi giá trị lớn nhất $m = \max(z)$ trước khi tính $e^{z_i - m}$ để số mũ lớn nhất luôn là $e^0 = 1$, triệt tiêu hoàn toàn lỗi tràn số Inf/NaN.	L.13 – Mục IV
Mixed Precision (AMP)	Huấn luyện Độ chính xác Hỗn hợp & <i>Loss Scaling</i>	Nhân ma trận ở FP16/BF16 để tăng tốc Tensor Cores nhưng bắt buộc giữ bản sao <i>Master Weights</i> ở FP32 để tích lũy cập nhật đạo hàm không bị nuốt số.	L.13 – Mục V.3
Gradient Clipping	Cắt ngưỡng Chuẩn Đạo hàm Toàn cục	Thu nhỏ toàn bộ vector đạo hàm nếu tổng chuẩn $\ \vec{g}\ _2$ vượt ngưỡng c (giữ nguyên hướng đi), ngăn một batch nhiều phá hỏng trọng số mô hình.	L.13 – Mục V.4
BatchNorm, LayerNorm, RMSNorm	Các Tầng Chuẩn hóa Kích hoạt	Định tâm và co giãn đầu ra mỗi tầng về vùng trung bình 0, phương sai 1 để giữ đạo hàm và dải số học ổn định xuyên suốt hàng trăm tầng mạng.	L.13 – Mục V.5
LESSON 14: NORMS & DISTANCES (CHUẨN VECTOR & CÁC ĐỘ ĐO KHOẢNG CÁCH)			
Họ chuẩn L_p (L_1, L_2, L_∞)	Chuẩn Manhattan (L_1), Euclid (L_2) & Chebyshev (L_∞)	Đo kích thước vector; mặt cầu đơn vị hình thoi nhọn của L_1 ép nghiệm chạm trục tọa độ tạo tính thưa (<i>Sparsity</i> — Lasso), còn mặt cầu tròn của L_2 ép trọng số nhỏ đều (Ridge).	L.14 – Mục II
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
RAG Length Bias	Thiên lệch Độ dài Văn bản khi Tìm kiếm Vector	Khi vector chưa chuẩn hóa độ dài ($\ \vec{x}\ _2 \neq 1$), khoảng cách Euclid L_2 và Cosine cho thứ tự xếp hạng trái ngược nhau; phải chuẩn hóa L_2 trước khi nạp vào Vector DB.	L.14 – Mục III.2
Mahalanobis Distance	Khoảng cách Mahalanobis ($\sqrt{\Delta^T S^{-1} \Delta}$)	Nhân với nghịch đảo ma trận hiệp phương sai S^{-1} để tự động chia chuẩn hóa đơn vị đo và khử tương quan chéo giữa các đặc trưng khi bất điểm ngoại lai/gian lận.	L.14 – Mục IV
Jaccard Similarity	Độ tương đồng Tập hợp Jaccard ($\frac{ A \cap B }{ A \cup B }$)	Đo tỷ lệ phần tử chung trên tổng phần tử duy nhất giữa 2 tập hợp; kết hợp với MinHash để lọc văn bản trùng lặp (<i>Deduplication</i>) cho tập dữ liệu LLM.	L.14 – Mục V.1
Edit / Levenshtein Distance	Khoảng cách Chỉnh sửa Chuỗi ký tự	Đếm số bước Chèn, Xóa, Thay thế tối thiểu để biến chuỗi A thành chuỗi B bằng Quy hoạch động; nền tảng tính chỉ số lỗi nhận dạng giọng nói WER/CER.	L.14 – Mục V.2
Wasserstein / EMD	<i>Earth Mover's Distance</i> — Khoảng cách Vận chuyển Tối ưu	Đo công sức tối thiểu để san lấp đồng đất phân phối P sang Q ; vẫn cung cấp đạo hàm mượt khi 2 phân phối tách rời hoàn toàn (cứu cánh cho WGAN và theo dõi Data Drift).	L.14 – Mục V.3
Huber Loss	Hàm Mất mát Lai Huber	Kết hợp ưu điểm của MSE (mượt bậc hai khi sai số nhỏ quanh đáy) và MAE (tuyến tính bậc một khi sai số lớn để không bị Outliers kéo lệch mô hình).	L.14 – Mục VI.2
LESSON 15: STATISTICS FOR MACHINE LEARNING (THỐNG KÊ SUY DIỄN CHO HỌC MÁY)			
Latency P_{50}, P_{95}, P_{99} & IQR	Phân vị Độ trễ Hệ thống & Khoảng Tứ phân vị (<i>Interquartile Range</i>)	Trung bình (Mean) bị bóp méo nặng bởi 1 vài request nghẽn mạng; kỹ sư hệ thống luôn dùng Trung vị (P_{50}) và phân vị đuôi (P_{95}, P_{99}) để giám sát SLA thực tế.	L.15 – Mục II.1
Bessel's Correction	Hiệu chỉnh Bessel ($\frac{1}{n-1}$ khi tính Phương sai Mẫu)	Do trung bình mẫu $\bar{x}$ luôn nằm gần các điểm mẫu hơn tâm thật μ , tổng bình phương độ lệch bị hụt; chia cho $n - 1$ (số bậc tự do độc lập) giúp ước lượng không bị chệch.	L.15 – Mục II.2
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Pearson (r) vs Spearman (ρ_s)	Tương quan Tuyến tính Pearson vs Tương quan Hạng Spearman	Pearson chỉ đo quan hệ đường thẳng và dễ hỏng vì ngoại lai; Spearman đổi giá trị sang thứ hạng (<i>Rank</i>) nên bắt được mọi quan hệ đồng biến/nghịch biến phi tuyến.	L.15 – Mục II.3
LLN, CLT & Standard Error	Luật Số Lớn, Định lý Giới hạn Trung tâm & Sai số Chuẩn ($\frac{\sigma}{\sqrt{n}}$)	LLN đảm bảo trung bình mẫu hội tụ về μ ; CLT đảm bảo phân phối của $\bar{X}$ tiến về hình chuông Gaussian với độ rộng thu hẹp theo $\frac{1}{\sqrt{n}}$ (muốn giảm nửa sai số phải gấp 4 mẫu).	L.15 – Mục III
Confidence Interval & Bootstrap	Khoảng Tin Cậy (CI) & Lấy mẫu lại <i>Bootstrap</i>	Rút mẫu lại có hoàn lại B lần từ tập test để tạo khoảng tin cậy 95% cho các chỉ số phức tạp (F1, AUC, BLEU) mà không cần giả định công thức phân phối.	L.15 – Mục IV
t-test, p-value & Bonferroni	Kiểm định Giả thuyết, Độ lớn Hiệu ứng (<i>Cohen's d</i>) & Hiệu chỉnh Đa kiểm định	Đánh giá chênh lệch giữa 2 mô hình có vượt qua nhiễu ngẫu nhiên hay không; chia mức ý nghĩa α/m (Bonferroni) để chống bẫy dương tính giả khi thử nhiều mô hình.	L.15 – Mục V
Simpson's Paradox	Nghịch lý Thống kê Simpson	Hiện tượng một mô hình thắng ở từng nhóm con nhưng lại thua khi gộp chung toàn bộ dữ liệu do tỷ lệ phân bổ mẫu không đều (biến gây nhiều ẩn — <i>Confounder</i>).	L.15 – Mục VI
LESSON 16: SAMPLING METHODS (CÁC PHƯƠNG PHÁP LẤY MẪU & GIẢI MÃ LLM)			
Inverse Transform Sampling	Lấy mẫu Biến đổi Ngược CDF ($X = F^{-1}(U)$)	Biến số ngẫu nhiên đều $U \sim \text{Uniform}(0, 1)$ thành biến ngẫu nhiên tuân theo phân phối bất kỳ thông qua hàm ngược của phân phối tích lũy CDF.	L.16 – Mục II.1
Box-Muller & Reparameter- ization	Biến đổi Box-Muller & Mẹo Tái tham số hóa ($z = \mu + \sigma \odot \epsilon$)	Tách nguồn nhiễu ngẫu nhiên $\epsilon \sim \mathcal{N}(0, I)$ sang nhánh phụ để đạo hàm lan truyền ngược chảy xuyên qua tầng lấy mẫu trong mạng sinh Variational Autoencoder (VAE).	L.16 – Mục II.2
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Rejection Sampling	Phương pháp Lấy mẫu Bác bỏ	Sinh mẫu từ phân phối đề xuất dễ lấy $q(x)$ bao phủ phía trên $p(x)$ rồi giữ/loại theo tỷ lệ chiều cao; bị suy giảm hiệu suất nặng khi số chiều cao.	L.16 – Mục III.1
Importance Sampling	Phương pháp Lấy mẫu Quan trọng	Ước lượng kỳ vọng dưới phân phối $p(x)$ bằng mẫu rút từ $q(x)$ nhân với trọng số hiệu chỉnh $\frac{p(x)}{q(x)}$; nền tảng của thuật toán PPO (<i>Off-policy RL</i>) tinh chỉnh RLHF.	L.16 – Mục III.2
MCMC & Metropolis- Hastings	<i>Markov Chain Monte Carlo</i> — Lấy mẫu Chuỗi Markov	Thiết kế bước nhảy ngẫu nhiên chỉ phụ thuộc tỷ số $\frac{p(x')}{p(x)}$ để lấy mẫu chính xác từ các phân phối phức tạp nhiều chiều mà không cần tính hằng số chuẩn hóa Z .	L.16 – Mục IV
Temperature, Top-k & Top-p	Chiến lược Lấy mẫu Giải mã Token trong LLM (<i>Nucleus Sampling</i>)	Nhiệt độ T điều chỉnh độ sắc nhọn của Softmax; Top- k giữ k từ cao nhất; Top- p linh hoạt giữ nhóm từ có tổng xác suất tích lũy đạt ngưỡng p để cắt bỏ đuôi từ rác.	L.16 – Mục V
LESSON 17: LINEAR SYSTEMS (HỆ PHƯƠNG TRÌNH TUYẾN TÍNH & PHÂN RÃ MA TRẬN)			
Partial Pivoting & $PA = LU$	Khử Gauss có Chọn Phần tử Chốt & Phân rã LU	Đổi hàng để đưa số có trị tuyệt đối lớn nhất làm chốt chia (tránh chia cho số gần 0 gây nổ sai số); tách thành tích 2 ma trận tam giác giúp giải nhiều vế phải siêu tốc.	L.17 – Mục II
Cholesky ($A = LL^T$)	Phân rã Cholesky cho Ma trận Xác định Dương (SPD)	Tận dụng tính đối xứng xác định dương (như ma trận hiệp phương sai, Kernel Gaussian Process) để phân rã nhanh gấp đôi và tốn nửa bộ nhớ so với LU .	L.17 – Mục III
Condition Number ($\kappa(A)$)	Số Điều Kiện của Ma trận ($\frac{\sigma_{\max}}{\sigma_{\min}}$)	Đo mức độ nhạy cảm khuếch đại nhiều khi giải hệ phương trình; mỗi $\log_{10} \kappa(A)$ đơn vị làm mất đi 1 chữ số thập phân chính xác của nghiệm.	L.17 – Mục IV
QR Solver & Tikhonov Ridge	Giải Bình phương Tối thiểu bằng QR & Ổn định hóa Ridge	Tránh lập ma trận $X^T X$ vì nó làm bình phương số điều kiện $\kappa(X^T X) = \kappa(X)^2$; hoặc cộng thêm λI vào đường chéo để nâng trị riêng nhỏ nhất lên khỏi 0.	L.17 – Mục V
LESSON 18: CONVEX OPTIMIZATION (TỐI ƯU HÓA LỖI, ĐỐI NGẪU & SVM)			
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Convex Sets & Functions	Tập Lồi & Hàm Lồi (Hình bát hướng lên)	Đảm bảo dây cung nối 2 điểm luôn nằm phía trên đồ thị và đường tiếp tuyến luôn nằm dưới đồ thị; giúp mọi điểm cực tiểu cục bộ đồng thời là cực tiểu toàn cục duy nhất.	L.18 – Mục II
Newton's Method	Phương pháp Tối ưu Bậc hai của Newton ($x - H^{-1}\nabla f$)	Nhân nghịch đảo Hessian để tự động nắn tròn các thung lũng hẹp dẹt và hội tụ bậc hai cực nhanh, nhưng tốn kém $\mathcal{O}(d^3)$ khi số tham số lớn.	L.18 – Mục III.2
Jensen's Inequality	Bất đẳng thức Lồi Jensen ($f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$)	Công cụ nền tảng chứng minh Độ phân kỳ KL luôn không âm ($D_{\text{KL}} \geq 0$) và thiết lập cận dưới bằng chứng ELBO trong thuật toán EM / mô hình sinh VAE.	L.18 – Mục IV
Lagrange, Duality & KKT	Nhân tử Lagrange, Bài toán Đối ngẫu & Hệ Điều kiện KKT	Đưa các ràng buộc bất đẳng thức vào hàm mục tiêu kèm điều kiện Độ lệch bù (<i>Complementary Slackness</i> $\lambda_i g_i(x^*) = 0$) để giải bài toán tối ưu có điều kiện.	L.18 – Mục V.1
SVM & Support Vectors	<i>Support Vector Machine</i> — Máy Vectơ Hỗ trợ Lề Cực đại	Nhờ điều kiện KKT $\alpha_i g_i = 0$, chỉ những điểm dữ liệu nằm ngay trên ranh giới lề ($\alpha_i > 0$ — <i>Support Vectors</i>) mới quyết định siêu phẳng phân loại.	L.18 – Mục V.2
LESSON 19: COMPLEX NUMBERS FOR AI (SỐ PHỨC & MÃ HÓA VỊ TRÍ ROPE TRONG LLM)			
Complex Plane & Conjugate	Mặt phẳng Phức Argand, Số phức Liên hợp ($\bar{z}$) & Mô-đun ($ z $)	Biểu diễn số phức $a + bi$ như tọa độ 2D; nhân với đơn vị ảo i ($i^2 = -1$) chính là phép xoay vector ngược chiều kim đồng hồ một góc vuông 90° .	L.19 – Mục II
Euler's Formula	Công thức Euler ($e^{i\theta} = \cos \theta + i \sin \theta$) & Dạng Cực	Kết nối hàm mũ phức với lượng giác, biến phép nhân hai số phức thành phép nhân độ dài và cộng trực tiếp hai góc xoay ($\theta_1 + \theta_2$).	L.19 – Mục III
Roots of Unity (ω_N^k)	Căn bậc N của Đơn vị ($z^N = 1$)	N điểm chia đều đường tròn đơn vị có tính chất trực giao triệt tiêu tổng bằng 0, làm cơ sở toán học để tách các tần số sóng trong biến đổi Fourier.	L.19 – Mục III.3
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
RoPE	<i>Rotary Position Embedding</i> — Mã hóa Vị trí Xoay	Ghép từng cặp chiều của vector Query/Key thành số phức và xoay góc $m\theta$; tích vô hướng Attention tự động chỉ phụ thuộc vào khoảng cách tương đối ($m - n$) giữa 2 từ.	L.19 – Mục IV
LESSON 20: THE FOURIER TRANSFORM (BIẾN ĐỔI FOURIER, FFT & XỬ LÝ TÍN HIỆU)			
DFT & IDFT	<i>Discrete Fourier Transform</i> — Biến đổi Fourier Rời rạc	Chiều tín hiệu miền thời gian lên các sóng cơ sở phức trực giao để tách thành phổ biên độ và pha của từng tần số thành phần.	L.20 – Mục II
FFT (Cooley-Tukey)	<i>Fast Fourier Transform</i> — Biến đổi Fourier Nhanh	Khai thác tính đối xứng tuần hoàn của căn đơn vị để chia đôi đệ quy chẵn/lẻ, giảm độ phức tạp từ $\mathcal{O}(N^2)$ xuống $\mathcal{O}(N \log_2 N)$ (tăng tốc hàng vạn lần).	L.20 – Mục III
Nyquist-Shannon & Aliasing	Định lý Lấy mẫu Nyquist-Shannon & Hiện tượng Chồng phổ	Tần số lấy mẫu phải lớn hơn ít nhất gấp đôi tần số cao nhất của tín hiệu ($f_s \geq 2f_{\max}$), nếu không sóng tần số cao sẽ bị giả dạng thành sóng tần số thấp (<i>Aliasing</i>).	L.20 – Mục IV.1
Convolution Theorem	Định lý Tích chập ($\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \odot \mathcal{F}\{g\}$)	Phép trượt tích chập tốn kém $\mathcal{O}(N^2)$ trong miền thời gian tương đương với phép nhân từng phần tử siêu tốc trong miền tần số (ứng dụng trong CNN cửa sổ lớn, S4/Hyena).	L.20 – Mục IV.2
STFT & Mel-Spectrogram	Biến đổi Fourier Thời gian Ngắn & Phổ Thang Mel	Cắt âm thanh thành các khung cửa sổ trượt để tạo ảnh phổ Thời gian – Tần số theo thang cảm nhận tai người; đầu vào tiêu chuẩn của Whisper và các mô hình Speech AI.	L.20 – Mục V.1
Spectral Bias	Thiên lệch Phổ Tần số của Mạng Nơ-ron	Mạng MLP thông thường luôn học các thành phần tần số thấp (mượt) rất nhanh nhưng học chi tiết tần số cao cực chậm; khắc phục bằng mã hóa <i>Fourier Features</i> (như trong NeRF).	L.20 – Mục V.2
LESSON 21: GRAPH THEORY FOR ML (LÝ THUYẾT ĐỒ THỊ PHỔ & MẠNG GNN)			
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Adjacency (A), Degree (D) & CSR	Ma trận Kề, Ma trận Bậc & Lưu trữ Thừa (<i>Sparse COO/CSR</i>)	Biểu diễn cấu trúc mạng lưới thành ma trận đại số; dùng định dạng nén chỉ số CSR/COO để lưu đồ thị hàng triệu đỉnh mà không bị tràn bộ nhớ vì các số 0.	L.21 – Mục II.1
Walks & Powers of A (A^k)	Đếm Đường đi trên Đồ thị qua Lũy thừa Ma trận Kề	Phần tử $(A^k)_{ij}$ cho biết chính xác số lượng đường đi có độ dài k bước nối từ đỉnh i đến đỉnh j (nền tảng của vùng cảm thụ k -hop trong GNN).	L.21 – Mục II.2
BFS vs DFS	<i>Breadth-First Search vs Depth-First Search</i>	BFS dùng Hàng đợi (FIFO) tìm đường đi ngắn nhất và lấy mẫu láng giềng theo lớp (GraphSAGE); DFS dùng Ngăn xếp (LIFO) phát hiện chu trình và sắp xếp Tô-pô.	L.21 – Mục II.3
Graph Laplacian ($L = D - A$)	Ma trận Laplacian của Đồ thị & Dạng Toàn phương	Đo độ chênh lệch tín hiệu giữa các đỉnh kề nhau ($\vec{x}^T L \vec{x} = \sum_{(i,j) \in E} (x_i - x_j)^2 \geq 0$); số lượng trị riêng bằng 0 đếm chính xác số thành phần liên thông.	L.21 – Mục III.1
Fiedler Value (λ_2) & Spectral Clustering	Giá trị Fiedler & Thuật toán Phân cụm Phổ	Trị riêng nhỏ thứ hai λ_2 đo độ gắn kết của đồ thị; dấu âm/dương của vectơ riêng Fiedler $\vec{v}_2$ tự động cắt đồ thị thành các cụm tối ưu nhất.	L.21 – Mục III.2
GNN / GCN & Message Passing	<i>Graph Neural Networks</i> & Cơ chế Truyền Thông điệp	Cập nhật vector biểu diễn của mỗi đỉnh bằng cách trung bình có chuẩn hóa ($\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} XW$) thông tin từ các đỉnh láng giềng xung quanh.	L.21 – Mục IV.1
Attention as Complete Graph	Self-Attention dưới Góc nhìn Đồ thị Đầy đủ	Transformer thực chất là một mạng GNN hoạt động trên đồ thị đầy đủ mà trọng số cạnh giữa các tử được tính linh hoạt ở mỗi tầng bằng $\text{Softmax}(QK^T/\sqrt{d_k})$.	L.21 – Mục IV.2
LESSON 22: STOCHASTIC PROCESSES (QUÁ TRÌNH NGẪU NHIÊN, RL & DIFFUSION)			
Markov Chain (DTMC)	Chuỗi Markov Thời gian Rời rạc & Ma trận Chuyển P	Quá trình ngẫu nhiên có tính chất Không nhớ (<i>Memoryless</i>): tương lai chỉ phụ thuộc vào trạng thái hiện tại; phân phối sau k bước tính bằng nhân ma trận $\vec{\pi}^{(0)} P^k$.	L.22 – Mục II.1
Xem tiếp ở trang sau...			

Khái niệm / Viết tắt	Tên đầy đủ & Dịch nghĩa chuẩn	Bản chất & Tác dụng thực tế trong AI / ML	Tra cứu nhanh
Stationary Dist. & Detailed Balance	Phân phối Dừng ($\vec{\pi} = \vec{\pi}P$), Tính Ergodic & Cân bằng Chi tiết	Trạng thái cân bằng dài hạn bất chấp điểm xuất phát ban đầu; điều kiện $\pi_i P_{ij} = \pi_j P_{ji}$ giúp thiết kế chuỗi lấy mẫu MCMC hội tụ đúng về phân phối mục tiêu.	<u>L.22 – Mục II.2</u>
Random Walk & Brownian Motion	Bước đi Ngẫu nhiên, Quá trình Poisson & Chuyển động Brown (W_t)	Phương sai vị trí tăng tuyến tính theo thời gian ($\text{Var} \propto t$) nên độ lệch chuẩn khuếch tán theo quy luật căn bậc hai $\sqrt{t}$; giới hạn liên tục sinh ra quá trình Wiener W_t .	<u>L.22 – Mục III</u>
MDP & Bellman Equation	<i>Markov Decision Process</i> & Phương trình Quy hoạch động Bellman	Khung toán học của Học tăng cường (<i>Reinforcement Learning</i>): phân rã giá trị dài hạn của trạng thái thành Phần thưởng tức thời cộng với Giá trị tương lai chiết khấu γ .	<u>L.22 – Mục IV</u>
DDPM & Langevin Dynamics	<i>Denoising Diffusion Probabilistic Models</i> — Mô hình Khuếch tán Sinh ảnh	Quá trình thuận cộng nhiễu Gaussian đóng công thức nhảy thẳng tới bước t bất kỳ; quá trình ngược dùng mạng nơ-ron dự đoán nhiều kết hợp động lực học Langevin để tái tạo ảnh từ nhiễu trắng.	<u>L.22 – Mục V</u>

PHASE 01: MATH FOUNDATIONS

Lesson 1: Linear Algebra Intuition

I. Độc lập tuyến tính & đa cộng tuyến

Nếu huấn luyện mô hình có Matrix dữ liệu:

$$X = \begin{bmatrix} 30 & 15 & 45 \\ 40 & 20 & 60 \\ 25 & 10 & 35 \end{bmatrix} \begin{array}{l} \longrightarrow \vec{v}_1 \parallel \vec{v}_2 \\ \longrightarrow \text{cột } 3 = \text{cột } 2 + \text{cột } 1 \end{array}$$

$\Rightarrow$ Không mang lại thông tin gì, $\text{rank}(X) = 2 < 3$

$\Rightarrow$ Khuyết hạng $\Rightarrow$ **Rank - deficient**

$\Rightarrow$ Khi tính $\det(X^T \cdot X) = 0 \Rightarrow$ Không $\exists$ ma trận nghịch đảo $[X^T \cdot X]^{-1}$.

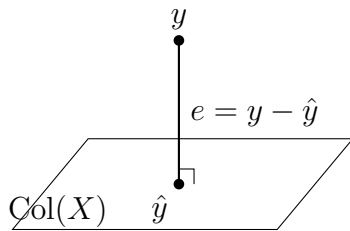
II. Tại sao $w = (X^T \cdot X)^{-1}X^T y$ mà không phải $w = X^{-1}y$ với $Xw = y$

- Vấn đề X^{-1} chỉ $\exists$ cho ma trận vuông. Trong thực tế số lượng mẫu $n \gg$ số lượng đặc trưng d :

$$X \begin{bmatrix} 0 & 1 & \dots & d \\ 1 \\ \vdots \\ n \end{bmatrix} \times \begin{bmatrix} 0 \\ 1 \\ \vdots \\ d \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ n \end{bmatrix} \quad \text{nên thường } \nexists X^{-1}$$

- Giống bài toán n phương trình, d ẩn ($d < n$) thì vô nghiệm hoặc vô số nghiệm.
- Vì không tìm được $Xw = y$, ta tìm 1 vector $\hat{y} = Xw$ sao cho $\hat{y}$ và y có khoảng cách min:

$$\Rightarrow \min |\hat{y} - y| = \min \|y - Xw\|^2$$

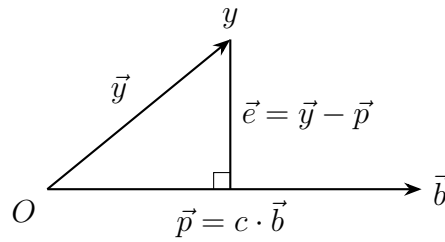


$$\hat{y} \in \text{Col}(X) \begin{cases} \text{vì } y = Xw \\ \text{vì không tìm được nên cần} \\ \hat{y} = Xw \end{cases}$$

Đoạn thẳng nối $y, \hat{y}$ là đường vuông góc với mặt $\text{Col}(X)$

III. Phép chiếu trực giao (Orthogonal projection)

①



$$\vec{e} = \vec{y} - \vec{p} = \vec{y} - c \cdot \vec{b}$$

WRONG

$$\forall \vec{e} \cdot \vec{b} = 0 \ (\perp) \Rightarrow \vec{y} \cdot \vec{b} - c \cdot \vec{b} \cdot \vec{b} = 0$$

$$\Rightarrow c = \frac{\vec{y} \cdot \vec{b}}{\vec{b} \cdot \vec{b}}$$

$$\text{Khi đó } \vec{p} = c \cdot \vec{b} = \left(\frac{\vec{y} \cdot \vec{b}}{\|\vec{b}\|^2} \right) \cdot \vec{b}$$

$$\text{Khi đó hình chiếu } \vec{y} \text{ lên } \vec{b} \text{ là } \vec{p} = \left(\frac{\vec{y} \cdot \vec{b}}{\|\vec{b}\|^2} \right) \vec{b}$$

$$\vec{e} \cdot \vec{b} = 0 \Rightarrow \vec{b}^T \cdot \vec{e} = 0$$

$$\text{Thay } \vec{e} = \vec{y} - \vec{p} \text{ có: } \vec{b}^T \cdot (\vec{y} - \vec{p}) = 0 \quad (\text{mà } \vec{p} = c\vec{b})$$

$$\Rightarrow \vec{b}^T \cdot \vec{y} - \vec{b}^T \vec{b} \cdot c = 0$$

$$\Rightarrow c = \frac{\vec{b}^T \cdot \vec{y}}{\vec{b}^T \cdot \vec{b}} = \frac{\vec{b}^T \cdot \vec{y}}{\|\vec{b}\|^2}$$

$$\Rightarrow \vec{p} = c \cdot \vec{b} = \frac{\vec{b}^T \cdot \vec{y}}{\|\vec{b}\|^2} \cdot \vec{b}$$

[?] Vì sao không để luôn $\vec{e} \cdot \vec{b}$ mà cần $\vec{b}^T \cdot \vec{e}$?

Vì quy ước mọi vectơ đều ở dạng cột:

$$b = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}, \quad e = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix} \quad \text{nên } \vec{b} \cdot \vec{e} = e_1 b_1 + e_2 b_2 + e_3 b_3 \text{ như này nó lại về tích vô hướng.}$$

Để máy tính hiểu là nhân ma trận ta phải xoay $\vec{b}$:

$$b : (3 \times 1) @ e : (3 \times 1) \neq \quad (\otimes)$$

$$b^T : (1 \times 3) @ e : (3 \times 1) = (1 \times 1) \quad (\checkmark)$$

$\Rightarrow$ ② Mở rộng lên không gian nhiều chiều

Thay vì chiếu y lên b , ta chiếu lên không gian cột của ma trận X $[\text{Col}(X)]$:

$$\begin{cases} \text{hình chiếu là } \hat{y} = Xw \\ \text{vectơ sai số } e = y - \hat{y} = y - Xw \quad (e \perp \text{Col}(X)) \end{cases}$$

Vì $e \perp (\text{Col}(X)) \Rightarrow X^T \cdot e = 0$

Thay $e = y - Xw$:

$$\Rightarrow X^T y - X^T \cdot Xw = 0 \quad (\text{trong vở ghi } X^T y - X \cdot X^T w = 0, \text{ chuẩn đại số là } X^T X w = X^T y)$$

$$\Rightarrow \boxed{w = (X^T \cdot X)^{-1} X^T \cdot y} \quad (\text{trong vở ghi } (X \cdot X^T)^{-1} X^T \cdot y, \text{ chuẩn đại số là } (X^T X)^{-1} X^T y)$$

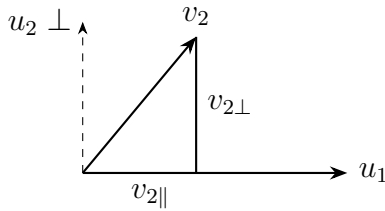
IV. Gram Schmidt

① Giả sử có 3 vectơ $\{v_1, v_2, v_3\}$ độc lập tuyến tính nhưng góc giữa chúng $\neq 90^\circ$

$\Rightarrow$ Để tạo 3 trục tọa độ $\{u_1, u_2, u_3\} \perp$ đôi một:

Bước 1: Chọn vectơ v_1 làm chuẩn, $u_1 = v_1$.

Bước 2: Với v_2 , tách thành 1 phần nằm theo hướng u_1 là $(v_{2\parallel})$ và 1 thành phần vuông góc u_1 :



Vì $v_{2\parallel}$ chính là hình chiếu của v_2 lên u_1

$$\Rightarrow v_{2\perp} = v_2 - v_{2\parallel} = v_2 - \text{proj}_{u_1}(v_2)$$

Bước 3: Với v_3 , ta tìm $u_3 \perp u_1$ và u_2 :

- Để tìm thành phần $\perp u_1$ lặp luận tương tự ta tìm được $v_3 - \text{proj}_{u_1}(v_3) = a_1$
- Và ta lấy thành phần a_1 này chỉ trừ tiếp đi thành phần hình chiếu của nó lên u_2 ta sẽ được 1 vectơ kết quả là:

$$u_3 = [v_3 - \text{proj}_{u_1}(v_3)] - \text{proj}_{u_2}(v_3)$$

$$\Rightarrow \boxed{u_3 = v_3 - \text{proj}_{u_1}(v_3) - \text{proj}_{u_2}(v_3)}$$

Bước 4: Chuẩn hóa độ dài:

$$\left\{ q_1 = \frac{u_1}{\|u_1\|}, \quad q_2 = \frac{u_2}{\|u_2\|}, \quad q_3 = \frac{u_3}{\|u_3\|} \right\}$$

② Phân rã $A = Q \cdot R$

- Từ quá trình trên ta tách A thành tích $Q \cdot R$:

$$\begin{cases} Q : \text{Orthogonal matrix thỏa } Q^T \cdot Q = I \{Q^T = Q^{-1}\} \\ \quad \langle \text{chứa các cột } q_i \text{ ở bước 4} \rangle \\ R : \text{Ma trận tam giác trên lưu hệ số hình chiếu trong quá trình Gram-Schmidt.} \end{cases}$$

⇒ **Cách tìm:** $Xw = y$

Giả sử $X = [x_1 \ x_2 \ x_3]$ (x_i là vectơ cột)

Sau Gram-Schmidt thu được $Q = [q_1 \ q_2 \ q_3]$

Ta biểu diễn:

$$\begin{cases} x_1 = r_{11}q_1 + 0q_2 + 0q_3 \\ x_2 = r_{12}q_1 + r_{22}q_2 + 0q_3 \\ x_3 = r_{13}q_1 + r_{23}q_2 + r_{33}q_3 \end{cases}$$

(Do ta chọn x_1 là vectơ đầu tiên để dựng ra trục q_1 nên x_1 chỉ $\in q_1$, tương tự x_2, x_3)

Khi đó:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ 0 & r_{22} & r_{23} \\ 0 & 0 & r_{33} \end{bmatrix}$$

$$Q = [q_1, q_2, q_3]$$

❓ Vì sao làm như này:

$$\begin{aligned} Xw = y &\iff Q \cdot Rw = y \\ &\iff Q^T \cdot Q \cdot Rw = Q^T y \\ &\iff I \cdot Rw = Q^T y \\ &\iff Rw = Q^T y \end{aligned}$$

Vì R là ma trận Δ trên nên việc giải hệ tìm w (trong vở ghi x) dễ hơn nhiều = cách thế từ dưới lên mà không cần nghịch đảo ma trận.

V. Độ tương đồng Cosine (Cosine Similarity), Embedding & Kỹ thuật LoRA

1. Tích vô hướng (Dot Product) và Cosine Similarity

① Công thức hình học: Cho 2 vectơ $\vec{a}, \vec{b} \in \mathbb{R}^d$:

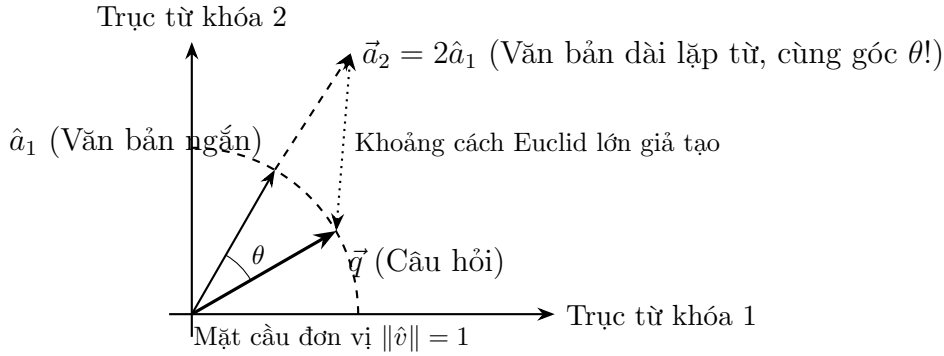
$$\vec{a} \cdot \vec{b} = \sum_{i=1}^d a_i b_i = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \cos \theta \implies \boxed{\cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|}}$$

$$\begin{cases} \cos \theta = 1 \ (\theta = 0^\circ) : 2 \text{ vectơ cùng hướng tuyệt đối (tương đồng cực đại)} \\ \cos \theta = 0 \ (\theta = 90^\circ) : 2 \text{ vectơ trực giao (độc lập / không liên quan)} \\ \cos \theta = -1 \ (\theta = 180^\circ) : 2 \text{ vectơ ngược hướng (đối lập hoàn toàn)} \end{cases}$$

❓ Vì sao trong hệ thống RAG / Vector Search ta chia cho $\|\vec{a}\|\|\vec{b}\|$ thay vì dùng luôn $\vec{a} \cdot \vec{b}$?

- Nếu dùng tích vô hướng thuần $\vec{a} \cdot \vec{b}$, một văn bản dài gấp 10 lần văn bản gốc (lặp lại từ khóa nhiều lần làm độ dài $\|\vec{a}\|$ lớn) sẽ có tích vô hướng lớn bất thường dù nội dung không sát bằng.

- Phép chia cho $\|\vec{a}\|\|\vec{b}\|$ thực chất là đưa cả 2 vectơ về mặt cầu đơn vị ($\|\hat{a}\| = \|\hat{b}\| = 1$), giúp đo thuần túy **góc lệch về mặt ngữ nghĩa** mà không bị nhiễu bởi độ dài:

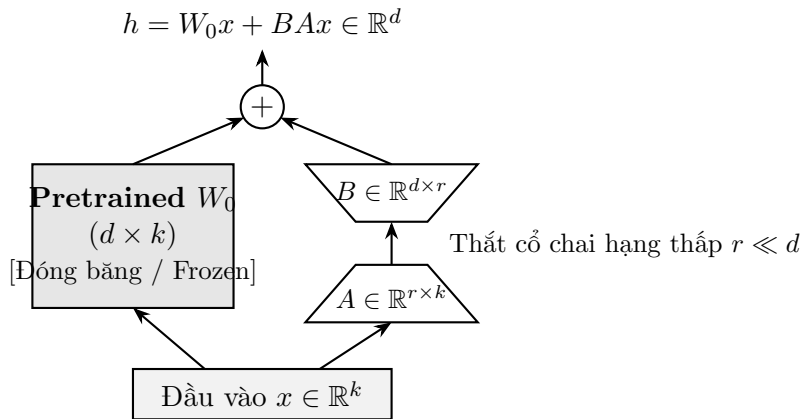


2. Ứng dụng Hạng ma trận vào LoRA (Low-Rank Adaptation)

① **Bản chất toán học:** Trong các mô hình ngôn ngữ lớn (LLM), ma trận trọng số $W_0 \in \mathbb{R}^{d \times k}$ cực lớn (VD: $4096 \times 4096 \approx 16,77$ triệu tham số). Khi Fine-tune, thay vì cập nhật toàn bộ $\Delta W \in \mathbb{R}^{d \times k}$, ta phân rã ma trận cập nhật thành tích của 2 ma trận hạng thấp $r \ll \min(d, k)$:

$$W = W_0 + \Delta W = W_0 + \underset{(d \times r)}{B} @ \underset{(r \times k)}{A}$$

- $\text{rank}(\Delta W) \leq r$: Mọi cập nhật trọng số được ép nằm gọn trong một không gian con r chiều.
- Số tham số cần huấn luyện giảm từ $d \times k$ xuống còn $r \times (d + k)$.



VD: Với $d = k = 4096$ và chọn hạng $r = 16$:

Full Fine-tuning : $4096 \times 4096 = 16\,777\,216$ tham số

LoRA ($r = 16$) : $4096 \times 16 + 16 \times 4096 = 131\,072$ tham số (giảm $> 99,2\%$ VRAM!)

3. Bản chất Embedding, Không gian Sinh (Span) & Hệ Cơ sở (Basis)

- **Embedding trong AI:** Là phép biểu diễn các đối tượng rời rạc (từ vựng, câu văn, bức ảnh, người dùng) thành các vectơ thực dày đặc $\vec{v} \in \mathbb{R}^d$ (VD $d = 768$ hoặc 1536) sao cho khoảng cách / góc hình học phản ánh độ tương đồng về ngữ nghĩa.

- **Độc lập tuyến tính (Linear Independence):** Hệ vectơ $\{\vec{v}_1, \dots, \vec{v}_k\}$ độc lập tuyến tính khi không có vectơ nào biểu diễn được dưới dạng tổ hợp tuyến tính của các vectơ còn lại:

$$c_1\vec{v}_1 + c_2\vec{v}_2 + \dots + c_k\vec{v}_k = \vec{0} \iff c_1 = c_2 = \dots = c_k = 0$$

- **Span & Basis:** **Span** là toàn bộ không gian tạo bởi mọi tổ hợp tuyến tính của hệ vectơ; **Basis (Hệ cơ sở)** là hệ vectơ **vừa độc lập tuyến tính vừa sinh ra (span)** toàn bộ không gian đó. Số lượng vectơ độc lập tuyến tính tối đa trong các cột của ma trận A chính là **Hạng của ma trận $\text{rank}(A)$** (số chiều thực sự của dữ liệu).

VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 1

- **Tích vô hướng & Cosine Similarity:** $\vec{a} \cdot \vec{b} = \sum_{i=1}^d a_i b_i = \|\vec{a}\| \|\vec{b}\| \cos \theta \implies \cos \theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$ (chuẩn hóa độ dài về mặt cầu đơn vị để đo góc ngữ nghĩa trong Vector Search / RAG).
- **Hình chiếu trực giao (Orthogonal Projection):** Hình chiếu của $\vec{y}$ lên $\vec{b}$ là $\vec{p} = \text{proj}_{\vec{b}}(\vec{y}) = \frac{\vec{b}^T \vec{y}}{\|\vec{b}\|^2} \vec{b}$ (với phần dư trực giao $\vec{e} = \vec{y} - \vec{p} \perp \vec{b}$).
- **Phương trình Pháp tuyến (Normal Equation):** Khi hệ $Xw = y$ vô nghiệm ($n \gg d$), chiếu y lên không gian cột $\text{Col}(X)$ cho $X^T(y - Xw) = 0 \implies \boxed{w = (X^T X)^{-1} X^T y}$.
- **Trực chuẩn hóa Gram-Schmidt & Phân rã QR:** Tách $X = Q \cdot R$ (với $Q^T Q = I$ và R tam giác trên) giúp giải hệ bình phương tối thiểu qua $\boxed{Rw = Q^T y}$ bằng phép thế ngược ổn định số học hơn nghịch đảo $(X^T X)^{-1}$.
- **Hạng ma trận (Rank) & LoRA:** Nếu $\text{rank}(X) < d$ (đa cộng tuyến) thì $\det(X^T X) = 0$ (suy biến). Ngược lại, chủ động ép ma trận cập nhật về hạng thấp $\Delta W = B_{d \times r} @ A_{r \times k}$ ($r \ll d$) chính là cốt lõi của **LoRA Fine-tuning**.

Lesson 2: Vectors, Matrices, Operations

I. Tầng Dense: vì sao là $Z = X @ W^T + b$

- Cấu tạo ma trận trọng số W :

$$W : (d_{\text{out}} \times d_{\text{in}}) : \text{số hàng} = d_{\text{out}}, \quad \text{cột} = d_{\text{in}}$$

- Mỗi hàng là 1 vectơ trọng số của 1 nơ-ron.
- Số cột là số đặc trưng đầu vào.

❓ **Sao không lưu ngược lại:** Trích xuất nơ-ron thứ i chỉ cần gọi $W[i]$ ($O(1)$), chứ lưu ngược lại thì truy cập full 1 nơ-ron cần $W[j][i]$ với j chạy từ thuộc tính 0 đến cuối.

- Cấu tạo ma trận dữ liệu X trong thực tế:

$$X : (B \times d_{\text{in}})$$

- B : số mẫu batch. Batch là tập hợp 1 nhóm các mẫu dữ liệu được gom lại để đưa vào mô hình cùng lúc.
- d_{in} : số lượng đặc trưng của mỗi mẫu.

❓ Vì sao là $X @ W^T$:

- Nếu để $X @ W : (B \times d_{\text{in}}) @ (d_{\text{out}} \times d_{\text{in}}) \rightarrow$ không nhân ma trận được.

$\Rightarrow$

$$\begin{array}{ccc} X @ W^T : (B \times d_{\text{in}}) @ (d_{\text{in}} \times d_{\text{out}}) \\ \parallel & & \parallel \\ Z & : & B \times d_{\text{out}} \end{array}$$

- **Ý nghĩa:** Dữ liệu gốc có B mẫu, mỗi mẫu có d_{in} đặc trưng (VD: Tên, Tuổi, ...). Ta muốn chuyển thành các dạng khác (năm sinh, gia đình, ...).

II. Broadcasting & Bẫy Tràn bộ nhớ

① Definition

- Broadcasting là cơ chế tự động trong các thư viện PyTorch, Numpy, TensorFlow cho phép thực hiện các phép toán $+$, $-$, $\times$, $/$ giữa 2 Tensor có shape khác nhau = cách kéo giãn ảo mảng có kích thước nhỏ hơn để khớp với mảng có kích thước lớn hơn.

Bản chất: Nó không sao chép hay nhân bản data thật trên RAM, VRAM. Nó đọc lại cùng 1 vùng nhớ nhiều lần.

② Rules:

- **B1:** So khớp kích thước từ phải sang trái, tensor nào có số chiều ít hơn, máy tính đệm số 1 vào trái đến lúc =.
- **B2:**
 - Nếu kích thước 2 bên = nhau $\rightarrow$ giữ nguyên.
 - Nếu 1 bên có kích thước = 1, bên kia là $N \Rightarrow$ bên số 1 kéo dẫn ảo thành N .
- **B3:** Nếu tại 1 chiều nào đó, 2 số $\neq$ nhau và không số nào = 1 $\rightarrow$ `ValueError` / `RuntimeError`.

③ VD1

$$Z = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \quad \text{shape (2,3) 2D}$$

$$b = [10 \quad 20 \quad 30] \quad \text{shape (3,) 1D}$$

Lưu ý mảng 1 chiều thì shape cũng 1 chiều:

$$\begin{array}{ccc} b = [10 & 20 & 30] \neq b = [[10 & 20 & 30]] \\ \downarrow & & \downarrow \\ \text{shape (3,)} & & \text{shape (1,3) (trong vở ghi (3,1))} \end{array}$$

Bản chất $Z = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$.

- **Tensor:** Tên gọi tổng quát cho mảng dữ liệu ở mọi chiều:
 - Tensor 0 chiều: số : VD: 5
 - 1 chiều: mảng 1D: $[1, 2, 3]$
 - 2 chiều: Ma trận 2D
 - N chiều: N -D Tensor

	Chiều 1	Chiều 2
Z	2	3
b	?	3

Bước 1: Có chiều 2 khớp, nó sang chiều 1 nó đệm thêm số 1:

$$\begin{array}{c|c|c} & \text{Chiều 1} & \text{Chiều 2} \\ \hline Z & 2 & 3 \\ \hline b & 1 & 3 \end{array} \implies b = [[10, 20, 30]]$$

Bước 2: Vì chiều 2 khớp nên so chiều 1. Có 1 thẳng = 1, 1 thẳng $\neq$ 1 = 2 nên nó kéo thẳng 1 thành 2:

$$\begin{aligned} \Rightarrow b &= \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix} \\ \Rightarrow Z + b &= \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix} \\ &= \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix} \end{aligned}$$

VD2 **Báo lỗi:** VD: $Z : \text{shape} = (2, 5, 6)$, $b : \text{shape} (4, 5)$

	$D1$	$D2$	$D3$
Z	2	5	6
b	?	4	5

B1: Đếm số 1:

	$D1$	$D2$	$D3$
Z	2	5	6
b	1	4	5

B2: So $D3$ trước: $6 \neq 5$ mà trong 2 thàng không thàng nào $= 1 \Rightarrow \text{ValueError}$.

VD3

- Tensor A : shape $(3, 1)$: $A = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}$
- Tensor B : shape $(3,)$: $B = [1 \ 2 \ 3]$

B1: Căn lề B thành $(1, 3)$.

B2: So sánh $(3, 1)$ và $(1, 3)$:

Các chiều đều $\neq$ nhau và đều có 1 thàng $= 1 \Rightarrow$ kéo dẫn $(3, 3)$:

$$\begin{aligned} \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} &\longrightarrow \begin{bmatrix} 10 & 10 & 10 \\ 20 & 20 & 20 \\ 30 & 30 & 30 \end{bmatrix} \\ [1 \ 2 \ 3] &\longrightarrow \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} \\ \Rightarrow A + B &= \begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \\ 31 & 32 & 33 \end{bmatrix} \end{aligned}$$

Nhưng thật ra mục tiêu là: lấy $10 + 1 = 11$, $20 + 2 = 22$, $30 + 3 = 33$.

VD 3.2: Nếu

$A = \text{torch.randn}(100000, 1)$ tensor 2D

$B = \text{torch.randn}(100000)$ tensor 1D

$C = A + B$ bị kéo thành $(100\,000, 100\,000)$

$\Rightarrow$ Số phần tử: $100\,000^2 \times 4 \text{ bytes} \approx 40 \text{ GB}$.

④ Khắc phục

★ $\Rightarrow \text{tensor_mới} = \text{tensor_cũ.view}(d_0, d_1, d_2, \dots, d_n)$

Trong đó $d_0 \rightarrow d_n$ là kích thước mà các chiều mới mà ta muốn nắn thành.

$\Rightarrow$ Quy tắc: $\prod_{i=0}^n d_i$ (trong vở ghi $\sum_{i=0}^n d_i$) = tổng số phần tử của tensor ban đầu.

⇒ Quy tắc đặc biệt: số -1

Ta được đặt duy nhất **MÔT** chiều mang giá trị -1 . Khi đó tại vị trí mà d_i ta đặt là -1 sẽ được tính:

$$d_{\text{tại chỗ } -1} = \frac{\text{Tổng phần tử}}{\text{Tích các chiều còn lại}}$$

VD: Tensor gốc $(32, 28, 28)$

Giả sử 1 batch 32 ảnh, mỗi ảnh kích thước 28×28 .

- Nếu muốn giữ nguyên 32 ảnh còn 28×28 duỗi thành 1 hàng dài các pixel:

$$\mathbf{x} = \mathbf{x}.\text{view}(32, -1) \quad \text{Máy tự tính chiều sau} = \frac{32 \times 28 \times 28}{32} = 784$$

⇒ Shape mới: $(32, 784)$.

★ `squeeze()`: quét qua các chiều của tensor, chiều nào kích thước $= 1$ thì xóa bỏ hoàn toàn.

Cách dùng 1:

VD: $(1, 4, 1, 10, 1) \xrightarrow{\text{squeeze}()} (4, 10)$ (trong vở ghi $(4, 5)$)

Cách dùng 2:

- ⎧
- Chọn chính xác chiều muốn bóp $= \text{dim}$
 - Shape $(1, 10, 1)$
 - `A.squeeze()` → $(10,)$
 - `B = A.squeeze(dim=2)` hoặc `A.squeeze(dim=-1)` chỉ bóp chiều cuối
- shape mới $(1, 10)$

III. Lưu ý và Backpropagation

- Trong production AI, ta không bao giờ dùng công thức

$$A^{-1} = \frac{1}{\det A} \cdot \text{adj}(A)$$

vì việc tính $\text{adj}(A)$ ta phải che hàng và cột để tính $\det$ liên tục.

- ★ Tính $\det$ cấp n cần gọi n lần $\det$ cấp $n-1$, $\det$ cấp $n-1$ gọi $n-1$ lần $\det$ cấp $n-2$ ⇒ Độ phức tạp $n!$.

1. Backpropagation là gì?

- Là thuật toán giúp mạng nơ-ron học hỏi từ sai lầm. Nó lần theo lỗi sai từ đầu ra quay ngược về từng đầu vào để đo xem mỗi trọng số W phải chịu bao nhiêu % trách nhiệm $\langle \text{gradient} \rangle$, từ đó giúp mô hình tự điều chỉnh để dự đoán chuẩn xác hơn.

2. Vấn đề:

a) **Chiều xuôi:** $d_{\text{in}} \rightarrow d_{\text{out}}$

- Đầu vào $x : (d_{\text{in}} \times 1)$ ta chỉ xét 1 đối tượng.
- Weight: $W : (d_{\text{out}} \times d_{\text{in}})$

$$\begin{cases} d_{\text{out}} : \text{đại diện cho các nơ-ron} \\ d_{\text{in}} : \text{đại diện cho các đặc trưng} \end{cases}$$

- z đầu ra: $(d_{\text{out}} \times 1)$

$$\boxed{z_{d_{\text{out}} \times 1} = W_{d_{\text{out}} \times d_{\text{in}}} @ x_{d_{\text{in}} \times 1}}$$

b) **Chiều ngược:** $d_{\text{out}} \rightarrow d_{\text{in}}$ **⟨Kéo lỗi từ $d_{\text{out}} \rightarrow d_{\text{in}}$ ⟩**

- Tầng sau báo về vectơ lỗi của các nơ-ron:

$$g_z : (d_{\text{out}} \times 1)$$

- Mục tiêu: Kéo lỗi này dội ngược về cho đầu vào x

↓

Vectơ lỗi r bắt buộc có kích thước $(d_{\text{in}} \times 1)$.

Vì VD: $x = \begin{bmatrix} x_1 \\ \vdots \\ x_4 \end{bmatrix}$ (4×1) thì mục tiêu là quy trách nhiệm đặc tính x_1 gây ra ? lỗi, $x_2 \dots$ ra các con số:

$$r = \begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ r_4 \end{bmatrix}$$

⇒ Vấn đề kích thước: nếu nhân trực tiếp W và g_z :

$$(d_{\text{out}} \times d_{\text{in}}) @ (d_{\text{out}} \times 1) \implies \text{Không nhân được}$$

⇒ Giải pháp: $W^T (d_{\text{in}} \times d_{\text{out}})$

$$\Rightarrow r_{(d_{\text{in}} \times 1)} = W_{(d_{\text{in}} \times d_{\text{out}})}^T @ g_{z(d_{\text{out}} \times 1)}$$

3. Ví dụ

Bước 1:

- Cho VD:

$$\begin{aligned} d_{\text{in}} &= 2 & x &= (x_1, x_2) \\ d_{\text{out}} &= 3 & z &= (z_1, z_2, z_3) \\ x_1 &: \text{Diện tích} & z_1 &: \text{Điểm tiện nghi} \\ x_2 &: \text{Số phòng} & z_2 &: \text{Điểm phù hợp} \\ & & z_3 &: \text{Điểm sang trọng} \end{aligned}$$

- Ma trận $W = \begin{bmatrix} 2 & 1 \\ 1 & 4 \\ 3 & 2 \end{bmatrix}$ quy định cách 3 chuyên gia chấm điểm:

$$\left\{ \begin{array}{l} \text{Chuyên gia 1 : coi trọng diện tích gấp đôi số phòng: } z_1 = 2x_1 + 1x_2 \\ \text{Chuyên gia 2 : } z_2 = 1x_1 + 4x_2 \\ \text{Chuyên gia 3 : } z_3 = 3x_1 + 2x_2 \end{array} \right.$$

$$\Rightarrow \text{Toàn bộ tầm ảnh hưởng của } x_1 \text{ nằm ở cột 1: } \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}$$

$$\Rightarrow \text{Toàn bộ tầm ảnh hưởng của } x_2 \text{ nằm ở cột 2: } \begin{bmatrix} 1 \\ 4 \\ 2 \end{bmatrix}$$

Bước 2:

Từ 3 điểm số z_1, z_2, z_3 , mô hình dự đoán giá nhà là 10 tỷ, nhưng thực tế chỉ 6 tỷ $\Rightarrow$ hớ 4 tỷ. Cấp trên kiểm tra và gửi giấy phạt:

$$g_z = \begin{bmatrix} 10 \text{ triệu} \\ 20 \\ 30 \end{bmatrix} \quad \begin{array}{l} \text{chuyên gia 1 bị phạt 10 củ} \\ \text{tức chuyên gia 2 bị phạt 20} \\ \text{chuyên gia 3 bị phạt 30} \end{array}$$

Bước 3: Chiều ngược - Truy cứu trách nhiệm

★ Anh diện tích x_1 đã nhúng tay vào cả 3 chuyên gia theo tỉ lệ cột 1 $[2, 1, 3]$:

- Góp 2 phần vào chuyên gia 1: chịu lỗi $2 \times 10 = 20$
- Góp 1 phần vào chuyên gia 2: chịu lỗi $1 \times 20 = 20$
- Góp 3 phần vào chuyên gia 3: chịu lỗi $3 \times 30 = 90$

$\Rightarrow$ Tổng lỗi do x_1 gây ra:

$$\text{lỗi } x_1 = 20 + 20 + 90 = 130 \text{ triệu}$$

★ Tương tự anh số phòng gây:

$$\text{lỗi } x_2 = 1 \times 10 + 4 \times 20 + 2 \times 30 = 150 \text{ tr}$$

Về mặt toán chính là:

$$W^T @ g_z = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \end{bmatrix} \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix} = \begin{bmatrix} 130 \\ 150 \end{bmatrix}$$

$(2 \times 3) \quad (3 \times 1)$

4. Khi không tính A^{-1} ($n!$) ta làm thế nào

⇒ Khử Gauss và các phép phân rã ($O(N^3)$).

a) Khử Gauss và phân rã LU

- **Ý tưởng:** Lấy hàng trừ cho nhau đưa về ma trận Δ trên (U), đồng thời lưu lại các hệ số nhân vào ma trận Δ dưới (L).

Khi đó:

$$\begin{aligned}Ax = b &\iff L \cdot Ux = b \\&\iff L \cdot (Ux) = b \\Đặt &Ux = y \iff Ly = b\end{aligned}$$

- Vì L là MT Δ dưới $\Rightarrow$ thế từ trên $\downarrow$ là tìm được $Ux = g$ (giả sử $Ux = c$).
- Vì U là ma trận Δ trên $\Rightarrow$ thế từ dưới $\uparrow$ là tìm được x .

★ Ví dụ:

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 7 & 4 \\ -1 & 7 & 9 \end{bmatrix}$$

Bước 1: Khử cột đầu:

+) Hàng 2 — nhân hàng 1 với 2 = hàng 2

+) Hàng 2 mới = Hàng ② cũ - ② $\times$ hàng ① = $[0 \quad 3 \quad 2]$
 $\Rightarrow L[2][1] = 2$ (hàng 2 cột 1 là số 2) ①

+) $H_{3 \text{ new}} = H_{\textcircled{3}} \underbrace{-}_{\text{luôn là dấu trừ}} (-1) \cdot H_{\textcircled{1}}$
 $\Rightarrow$ hàng 3 cột 1 là số -1 ②

$$\Rightarrow A_{\text{new}} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 3 & 2 \\ 0 & 9 & 10 \end{bmatrix}$$

$$H_{3 \text{ new}} = H_{\textcircled{3}} - \textcircled{3} \cdot H_{\textcircled{2}}$$

$\Rightarrow$ Hàng 3 cột 2 là số 3 ③

$$\Rightarrow A_{\text{new}} = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 3 & 2 \\ 0 & 0 & 4 \end{bmatrix} = U$$

Từ ①, ②, ③ và tính chất đường chéo luôn = 1:

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 3 & 1 \end{bmatrix} = L$$

Nếu nhân lại đúng $L @ U = A$. Lưu ý $LU \neq UL$.

b) Phân rã Cholesky ($A = L \cdot L^T$)

L : ma trận $\triangle$ dưới.

- **Ý tưởng:** Khi ma trận A đối xứng thì ma trận trên chính là chuyển vị của ma trận dưới.
- **Ưu:**
 - Nhanh $\times 2$ so với LU .
 - Tiết kiệm 50% RAM vì chỉ lưu 1 ma trận.
- **Nhược:** Chỉ áp dụng khi A xác định dương (trong vở ghi các phần tử > 0 , chuẩn đại số là $x^T A x > 0 \forall x \neq 0$ hay mọi trị riêng/định thức con chính > 0).

Cách tìm L : Đặt

$$L = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \Rightarrow L^T = \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

$$\Rightarrow L \cdot L^T = \begin{bmatrix} l_{11}^2 & l_{21}l_{11} & l_{31}l_{11} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 & l_{21}l_{31} + l_{22}l_{32} \\ l_{31}l_{11} & l_{31}l_{21} + l_{32}l_{22} & l_{31}^2 + l_{32}^2 + l_{33}^2 \end{bmatrix}$$

Giờ ta cho:

$$= \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = A$$

Giờ chỉ cần giải từng vị trí 1 cứ dần dần là tìm được full l_{ij} .

c) Phân rã QR: đã nói ở Lesson 1.

IV. Phép nhân Hadamard vs Nhân ma trận & Công thức đếm tham số Mạng Nơ-ron

1. Phân biệt Phép nhân Hadamard và Phép nhân ma trận

Tiêu chí	Element-wise / Hadamard (* / $\odot$)	Matrix Multiplication (@)
Điều kiện Shape	Cùng kích thước ($m \times n$) (hoặc broadcast được)	Số cột MT trước = Số hàng MT sau: $(m \times n) @ (n \times p) = (m \times p)$
Công thức	$C_{ij} = A_{ij} \cdot B_{ij}$	$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$
Ứng dụng AI	Cổng lọc trong LSTM/GRU, mặt nạ Dropout mask, áp dụng Gradient mask	Lan truyền tiến tầng Dense ($W @ x$), tính điểm Attention ($Q @ K^T$)

2. Công thức tổng quát đếm tham số trong mạng ANN / MLP

① Với 1 tầng kết nối đầy đủ (Fully Connected / Dense Layer):

Giả sử tầng có d_{in} nút đầu vào và d_{out} nút đầu ra:

$$\begin{cases} \text{Số trọng số (Weights } W) = d_{\text{in}} \times d_{\text{out}} \\ \text{Số độ lệch (Biases } b) = d_{\text{out}} \quad (\text{nếu không bias: } 0) \end{cases} \implies \boxed{\text{Tổng tham số} = d_{\text{in}}d_{\text{out}} + d_{\text{out}}}$$

② Với mạng nhiều tầng liên tiếp $[n_0, n_1, n_2, \dots, n_L]$:

$$\boxed{\text{Tổng trọng số} = \sum_{l=1}^L n_{l-1} \cdot n_l \quad , \quad \text{Tổng tham số (có bias)} = \sum_{l=1}^L (n_{l-1} + 1) \cdot n_l}$$

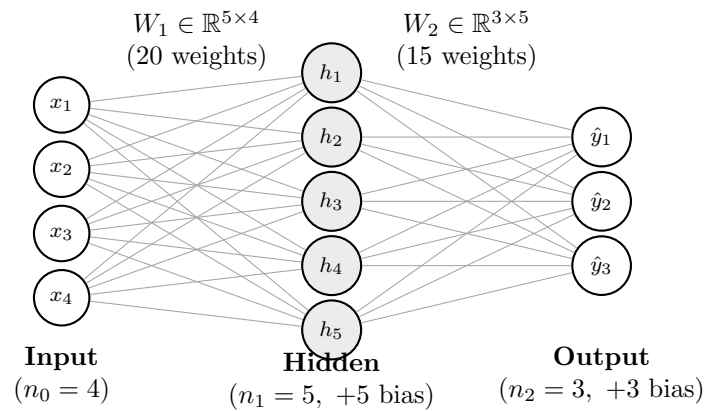
VD: Mạng ANN kết nối đầy đủ có tầng đầu vào 4 nút, 1 tầng ẩn 5 nút, tầng đầu ra 3 nút:

- Trường hợp không dùng độ lệch (No bias):

$$\text{Số trọng số} = (4 \times 5) + (5 \times 3) = 20 + 15 = 35 \text{ trọng số}$$

- Trường hợp có độ lệch (With bias ở cả tầng ẩn và đầu ra):

$$\text{Số tham số} = (4 \times 5 + 5) + (5 \times 3 + 3) = 25 + 18 = 43 \text{ tham số}$$



3. Chuyển vị Tích ma trận, Tích ngoài & Các Chuẩn (Norms)

- Quy tắc Chuyển vị đảo thứ tự: $(A_{m \times n})^T$ đổi hàng thành cột ($n \times m$), và với tích hai ma trận:

$$\boxed{(A @ B)^T = B^T @ A^T}$$

- Tích nội (Inner/Dot Product) vs Tích ngoài (Outer Product): Với $\vec{u} \in \mathbb{R}^{m \times 1}$, $\vec{v} \in \mathbb{R}^{n \times 1}$:

- Tích nội ($m = n$): $\vec{u}^T @ \vec{v} \in \mathbb{R}$ (1 số vô hướng).
- Tích ngoài: $\vec{u} @ \vec{v}^T \in \mathbb{R}^{m \times n}$ tạo ra **ma trận hạng 1 (Rank-1 Matrix)**, xuất hiện trực tiếp khi tính đạo hàm trọng số $\frac{\partial L}{\partial W} = g_z @ x^T$ và trong phân rã SVD:

Tích nội (Inner Product): $\vec{u}^T @ \vec{v}$ **Tích ngoài (Outer Product):** $\vec{u} @ \vec{v}^T$

$$\boxed{(1 \times m)} \times \boxed{\begin{matrix} (1) \\ \times \\ m \\ \times \\ 1 \end{matrix}} = \boxed{} \text{ vô hướng } (1 \times 1) \quad \boxed{\begin{matrix} (1) \\ \times \\ m \end{matrix}} \times \boxed{(1 \times n)} = \boxed{\begin{matrix} & & & \\ & & & \\ & & & \\ & & & \end{matrix}} \text{ Ma trận hạng 1 } (m \times n)$$

- Các chuẩn đo độ lớn (Norms) và Vết (Trace):

$$\|x\|_1 = \sum_i |x_i| \text{ (Lasso)}, \quad \|x\|_2 = \sqrt{\sum_i x_i^2} \text{ (Ridge)}, \quad \|x\|_\infty = \max_i |x_i|$$

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\text{Tr}(A^T A)} \quad (\text{Chuẩn Frobenius của ma trận})$$

V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 2

- **Tầng Tuyến tính (Dense Layer):** Lưu $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ để truy xuất từng nơ-ron liên tục trong bộ nhớ $O(1)$. Khi nhân theo lô $X \in \mathbb{R}^{B \times d_{\text{in}}}$:

$$\text{Forward: } \boxed{Z = X @ W^T + b} \quad , \quad \text{Backward về } x: \boxed{r = W^T @ g_z}$$

- **Quy tắc Broadcasting (PyTorch / NumPy):** So khớp shape từ **phải sang trái**; chỉ tương thích khi 2 chiều bằng nhau hoặc có 1 chiều bằng 1. Cảnh giác lỗi cộng nhầm $(N, 1) + (N,) \rightarrow (N, N)$ gây tràn bộ nhớ RAM/VRAM (dùng `.view()` hoặc `.squeeze()`).
- **Giải hệ $Ax = b$ trong Production:** Không bao giờ tính $A^{-1} = \frac{1}{\det A} \text{adj}(A)$ ($O(n!)$). Dùng **Phân rã LU** ($O(n^3)$) cho ma trận vuông tổng quát hoặc **Phân rã Cholesky** $A = LL^T$ (nhanh gấp đôi, tiết kiệm 50% RAM cho ma trận đối xứng xác định dương).
- **Đếm tham số MLP:** Một tầng Dense có $d_{\text{in}} \cdot d_{\text{out}} + d_{\text{out}}$ tham số (nếu có bias).

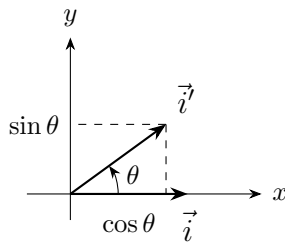
Lesson 3: Matrix Transformations and Eigenvalues

I. Ma trận là cỗ máy biến đổi không gian

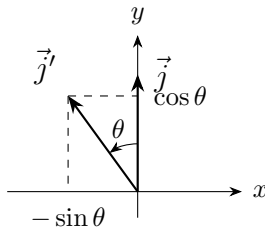
⟨4 phép biến đổi không gian cơ bản⟩

a) Phép xoay (Rotation)

- Xoay bộ dữ liệu 1 góc θ ngược chiều kim đồng hồ mà không làm méo hình dạng:



Vectơ $\vec{i} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ khi xoay góc θ sẽ tới $\vec{i}' = \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}$



Vectơ $\vec{j} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ khi xoay góc θ sẽ tới $\vec{j}' = \begin{bmatrix} -\sin \theta \\ \cos \theta \end{bmatrix}$

Gộp lại ta có ma trận xoay chuẩn:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

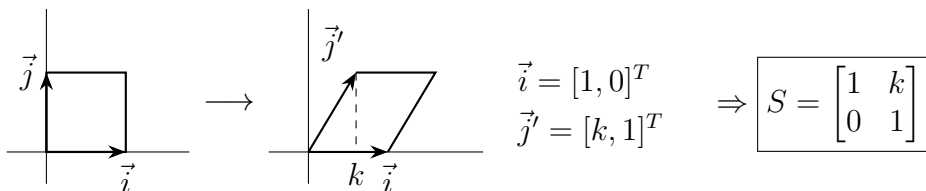
b) Phép co giãn (Scaling)

- Kéo trục x lên S_x lần, trục y lên S_y lần:

$$\begin{array}{ll} \vec{i} & \text{mới thành } [S_x, 0]^T \\ \vec{j} & \text{mới thành } [0, S_y]^T \end{array} \implies S = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix}$$

c) Phép trượt méo (Shearing)

- Biến 1 hình vuông thành hình bình hành mà không đổi chiều cao:



$$\begin{array}{ll} \vec{i} = [1, 0]^T \\ \vec{j} = [k, 1]^T \end{array} \implies S = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$$

d) Phép phản xạ / đối xứng (Reflection)

- Lật ngược dữ liệu qua trục tọa độ.
- Lật qua trục y (y giữ nguyên, đổi dấu x — trong vở ghi lật qua trục x , y giữ nguyên):

$$\text{Ref}_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \text{Ref}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

II. Composition — Thứ tự phép biến đổi: $A @ B \neq B @ A$

- Áp dụng biến đổi A lên x trước rồi đến B được viết là:

$$y = B @ (A @ x) = (B @ A) @ x$$

- **Quy ước:** Biến đổi nào xảy ra trước thì đứng gần x hơn.

❓ Tại sao tác dụng biến đổi A lên x lại là Ax mà không phải xA ?

- Nếu viết xA thì trước hết vectơ x phải đặt ngang $x = [x_1 \ x_2 \ \dots \ x_n]$, ma trận A cũng phải xếp ngang $\begin{bmatrix} - & i & - \\ - & j & - \end{bmatrix}$.
- Nó không sai nhưng đa số ta xếp dọc và dùng Ax cũng như ta viết hàm số $f(x)$ thay vì $(x)f$.

III. Định thức $\det(A)$: Thước đo tỷ lệ co giãn thể tích

- Diện tích sau biến đổi $= |\det(A)| \times$ Diện tích trước.
- **Các trạng thái của $\det(A)$:**
 - 1) $\det(A) = 1$: Diện tích S không đổi (phép xoay, trượt).
 - 2) $\det(A) = 2$: Mọi diện tích S trong không gian phóng đại $\times 2$.
 - 3) $\det(A) = 0$: Ma trận suy biến (singular) $\Rightarrow$ **nguy hiểm!**
 $\Rightarrow$ **Hệ quả:** Toàn bộ mặt phẳng 2D bị ép dẹp thành 1 đường thẳng hoặc 1 điểm $\Rightarrow$ thông tin bị nén mất chiều. Ta không thể từ 1 đường thẳng khôi phục lại mặt phẳng 2D ban đầu $\Rightarrow$ **Ma trận không thể nghịch đảo.**
 - 4) $\det(A) < 0$: Không gian bị lật (đổi định hướng bàn tay) và bị co giãn về diện tích theo tỷ lệ $|\det(A)|$.

IV. Eigenvalues & Eigenvectors (Trị riêng & Vectơ riêng)

- **Vấn đề:** Khi 1 ma trận nhào nặn không gian, hầu hết mọi vectơ đều bị quay sang hướng $\neq$, liệu có $\exists$ những vectơ không bị xoay hướng không?

⇒ Những vectơ đó chính là **Eigenvectors** $\vec{v}$:

- Khi nhân ma trận A vào vectơ riêng $\vec{v}$, nó không làm đổi phương của $\vec{v}$, mà chỉ kéo dài hoặc co ngắn lại theo 1 tỷ số vô hướng λ (scalar):

$$\boxed{A\vec{v} = \lambda\vec{v}} \quad \begin{cases} \vec{v} : \text{eigenvector} \text{ — hướng trục không bị quay } (\vec{v} \neq \vec{0}) \\ \lambda : \text{eigenvalue} \text{ — hệ số co giãn dọc theo trục đó} \end{cases}$$

★ Cách tìm:

B1: $Av = \lambda v \iff (A - \lambda I)v = 0$

⟨Cách 1⟩ Vì $\vec{v} \neq \vec{0} \implies \det(A - \lambda I) = 0$ (phương trình đặc trưng).

VD:

$$A = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \implies A - \lambda I = \begin{bmatrix} 4 - \lambda & 2 \\ 1 & 3 - \lambda \end{bmatrix}$$

$$\det(A - \lambda I) = 0 \implies (4 - \lambda)(3 - \lambda) - 2 = 0 \iff \lambda^2 - 7\lambda + 10 = 0 \iff \begin{cases} \lambda_1 = 2 \\ \lambda_2 = 5 \end{cases}$$

⟨Cách 2: Dùng định lý Viète với Vết (Trace) và Định thức⟩

$$\begin{cases} \lambda_1 + \lambda_2 = \text{Trace}(A) = 4 + 3 = 7 \\ \lambda_1 \cdot \lambda_2 = \det(A) = 12 - 2 = 10 \end{cases} \iff \begin{cases} \lambda_1 = 2 \\ \lambda_2 = 5 \end{cases}$$

Bước 2: Tìm eigenvector tương ứng (trong vở ghi eigenvalue tương ứng):

- Với $\lambda_2 = 5$: $Av = \lambda v$

$$\implies \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 5 \begin{bmatrix} x \\ y \end{bmatrix} \iff \begin{cases} 4x + 2y = 5x \\ x + 3y = 5y \end{cases} \iff x = 2y$$

Chọn $y = 1 \implies x = 2 \implies$ vectơ riêng tương ứng là $\underline{\vec{v}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}}$ $v_2 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$.

- Với $\lambda_1 = 2$: (bổ sung chi tiết phần ... trong vở)

$$\begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 2 \begin{bmatrix} x \\ y \end{bmatrix} \iff \begin{cases} 4x + 2y = 2x \\ x + 3y = 2y \end{cases} \iff x = -y$$

Chọn $y = 1 \implies x = -1 \implies$ vectơ riêng tương ứng là $v_1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$.

V. Vì sao Eigenvalues là trọng tâm trong ML

a) PCA (Principal Component Analysis) — Giảm chiều dữ liệu

- **Mục tiêu**: Ví dụ nén từ 100 cột ↓ 2, 3 cột sao cho mất ít thông tin nhất.

- **Khái niệm thông tin:** Phương sai (độ phân tán) càng lớn thì thông tin càng nhiều. Trục nào mà dữ liệu phân bố hẹp lép (phương sai gần = 0) thì chứa toàn nhiễu $\Rightarrow$ nên bỏ đi.

❓ Vì sao phương sai = 0 lại là thông tin nhiễu?

VD: Giả sử gắn cảm biến GPS 3D (x, y, z) lên 1 ô tô chạy trên sân phẳng:

1) **Chuyển động thật:**

- Khoảng cách xe chạy trên mặt sân là từ 0 – 50 mét.
- $\Rightarrow$ Độ phân tán lớn: $\lambda_1 = 50, \lambda_2 = 30$.
- $\Rightarrow$ Đây là thông tin thật về chuyển động xe.

2) **Độ cao xe:**

- Xe luôn bám đất, không bay, chiều cao thật là 0.
 - Nhưng do cảm biến nhiễu lúc 0,001, lúc $-0,001$.
- $\Rightarrow$ Dữ liệu trục z đẹp như tờ giấy, phương sai trên trục z bé: $\lambda_3 = 0,0001$.
- $\Rightarrow$ Dữ liệu không quan trọng.

❓ Vì sao tín hiệu hẹp lép lại bị coi là “chứa nhiễu”?

$$\text{Dữ liệu đo được} = \underset{\langle \text{Mạnh} \rangle}{\text{Tín hiệu thật}} + \underset{\langle \text{Yếu} \rangle}{\text{Nhiều ngẫu nhiên}}$$

- Tín hiệu thật tạo ra các trục biến thiên rất rộng (λ lớn).
- Nhiều ngẫu nhiên làm dữ liệu rung rinh rất ít.

$\Rightarrow$ Trục hẹp lép không phải là không có nhiễu mà là **KHÔNG CÓ TÍN HIỆU THẬT NÀO**.

b) Cơ chế giải quyết bằng Eigenvalue và Eigenvector cho bài toán PCA

- Ta tính ma trận hiệp phương sai của dữ liệu (Covariance Matrix):

$$\Sigma = \frac{1}{N} X^T \cdot X \quad (\text{hoặc } \frac{1}{N-1} X^T X \text{ với dữ liệu đã trừ mean})$$

- Các λ nói ở trên đều là các λ tìm từ ma trận Σ chứ không phải ma trận X gốc.

VD: 1 tập giá trị gồm 3 trị riêng $\lambda_1 = 6, \lambda_2 = 3, \lambda_3 = 1$.

Nếu nén 3 chiều $\downarrow$ 2 chiều (giữ trục 1, 2):

$$\Rightarrow \text{Tỷ lệ thông tin giữ lại} = \frac{\lambda_1 + \lambda_2}{\sum \lambda_i} = \frac{6 + 3}{10} = 90\%$$

$\Rightarrow$ Vứt trục 3 chỉ làm mất 10% thông tin nhưng giảm được $\frac{1}{3}$ chi phí tính toán.

c) Sự ổn định của mạng hồi quy (RNN stability) & Chéo hóa ma trận

- Ở các phần trước ta viết riêng lẻ $\begin{cases} Av_1 = \lambda_1 v_1 \\ Av_2 = \lambda_2 v_2 \end{cases}$.
- Có thể gộp thành:

$$A \cdot V = V \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \underline{V} \quad \text{với } V = [v_1 \ v_2], \ D = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

$$\begin{aligned} &\iff AV = VD \iff A \cdot V \cdot V^{-1} = VDV^{-1} \iff A = VDV^{-1} \\ &\iff \boxed{A = V \cdot \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} V^{-1}} \quad \langle \text{Phân rã trị riêng — Eigendecomposition} \rangle \end{aligned}$$

Nếu muốn tính A^{100} :

$$\begin{aligned} \implies A^{100} &= (VDV^{-1})(VDV^{-1}) \dots (VDV^{-1}) \\ &= VD(V^{-1}V)D(V^{-1}V) \dots (V^{-1}V)DV^{-1} \\ &= V \cdot D^{100}V^{-1} \end{aligned}$$

$$\iff \boxed{A^{100} = V \cdot \begin{bmatrix} \lambda_1^{100} & 0 \\ 0 & \lambda_2^{100} \end{bmatrix} V^{-1}}$$

vì $\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} = \begin{bmatrix} \lambda_1^2 & 0 \\ 0 & \lambda_2^2 \end{bmatrix} \dots$ cứ như vậy.

$\Rightarrow$ **Đánh giá sự ổn định mạng hồi quy RNN:**

- Nếu $|\lambda| > 1$: VD: 1,2 thì $1,2^{50} \approx 9100 \implies$ tràn bộ nhớ (Exploding Gradients) và mô hình sập (NaN).
- Nếu $|\lambda| < 1$: VD: 0,9 thì $0,9^{50} \approx 0,00515 \implies$ đạo hàm triệt tiêu về 0 (Vanishing Gradients), mô hình quên sạch các từ ở đầu câu.

$\Rightarrow$ **Để ổn định:** $|\lambda_i| \approx 1$.

VI. Biểu diễn Vectơ qua Phép biến đổi tuyến tính & Định lý Phổ (Spectral Theorem)

1. Tìm ảnh của một vectơ qua phép biến đổi tuyến tính

① **Quy tắc cốt lõi:** Mọi phép biến đổi tuyến tính $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ đều được đặc trưng duy nhất bởi một ma trận $A \in \mathbb{R}^{m \times n}$. Ảnh của vectơ $\vec{u} \in \mathbb{R}^{n \times 1}$ qua phép biến đổi T chính là tích ma trận:

$$\boxed{\vec{v} = T(\vec{u}) = A @ \vec{u}}$$

Về mặt hình học, nếu $\vec{u} = [u_1, u_2, u_3]^T$ và $A = [\vec{c}_1 \ \vec{c}_2 \ \vec{c}_3]$ thì ảnh $\vec{v}$ là tổ hợp tuyến tính các cột của A :

$$A @ \vec{u} = u_1 \vec{c}_1 + u_2 \vec{c}_2 + u_3 \vec{c}_3$$

VD: Cho ma trận A kích thước 3×3 và vectơ u kích thước (3×1) :

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 1 & 2 & 1 \end{bmatrix}, \quad u = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}$$

Biểu diễn của vectơ u qua phép biến đổi tuyến tính đặc trưng bởi A là:

$$A @ u = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 0 + 1 \cdot 2 \\ 0 \cdot 1 + 1 \cdot 0 + 3 \cdot 2 \\ 1 \cdot 1 + 2 \cdot 0 + 1 \cdot 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \\ 3 \end{bmatrix}$$

★ **Nhận xét nhanh:** Hàng 1 và hàng 3 của A giống hệt nhau nên tọa độ thứ 1 và thứ 3 của ảnh bắt buộc phải bằng nhau ($= 3$), đồng thời $\det(A) = 0$ ($\text{rank}(A) = 2 < 3$).

2. Định lý Phổ (Spectral Theorem) cho Ma trận đối xứng thực

❓ Khi nào ma trận A có các vectơ riêng **vuông góc đôi một** ($\vec{v}_i \perp \vec{v}_j$) để thay $V^{-1} = V^T$?

- Nếu $A \in \mathbb{R}^{n \times n}$ là **ma trận đối xứng** ($A = A^T$, ví dụ điển hình là ma trận hiệp phương sai $\Sigma = \frac{1}{N} X^T X$ và ma trận Hessian H), thì:

- Toàn bộ trị riêng $\lambda_1, \dots, \lambda_n$ đều là **số thực**.
- Các vectơ riêng ứng với các trị riêng khác nhau **trực giao đôi một**, sau khi chuẩn hóa độ dài bằng 1 tạo thành ma trận trực giao Q ($Q^T Q = I \iff Q^{-1} = Q^T$).

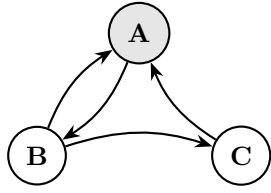
$$\implies A = Q \Lambda Q^T = \sum_{i=1}^n \lambda_i \vec{q}_i \vec{q}_i^T$$

Nhờ tính chất $Q^{-1} = Q^T$ của ma trận đối xứng mà trong PCA (Lesson 10), khi chiếu xuống k chiều bằng $Z = XW_k$ thì lúc tái tạo ngược về d chiều chỉ cần nhân chuyển vị $\hat{X} = ZW_k^T$ mà không tốn công nghịch đảo!

3. Ứng dụng Trị riêng trong PageRank ($\lambda = 1$) và Kết nối tắt ResNet

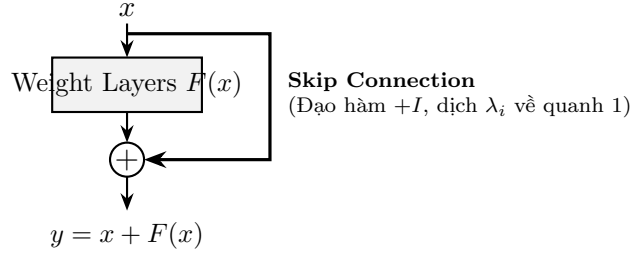
- Thuật toán Google PageRank:** Mô hình hóa mạng lưới liên kết web bằng ma trận chuyển trạng thái Markov P (tổng mỗi cột = 1). Điểm uy tín hội tụ lâu dài của các trang web chính là **vectơ riêng trạng thái dừng** $\vec{\pi}$ ứng với **trị riêng** $\lambda = 1$: $P\vec{\pi} = 1 \cdot \vec{\pi}$.
- Kết nối tắt trong ResNet (Residual Connection):** Thay vì học trực tiếp $y = F(x)$, ResNet thiết kế $y = x + F(x)$. Khi lấy đạo hàm qua tầng, ma trận Jacobian trở thành $I + \frac{\partial F}{\partial x}$, giúp dịch chuyển các trị riêng xoay quanh 1 (thay vì 0), ngăn chặn triệt tiêu đạo hàm ở các mạng sâu hàng trăm tầng:

Đồ thị Web PageRank ($P\vec{\pi} = 1 \cdot \vec{\pi}$)



Trạng thái dừng $\vec{\pi}$ là vectơ riêng ứng với $\lambda = 1$

Khối ResNet ($y = x + F(x)$)



VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 3

- **Ma trận là Phép biến đổi Không gian:** Các cột của ma trận A chính là tọa độ mới của các vectơ cơ sở $\vec{i}, \vec{j}$ sau biến đổi. Ghép nhiều phép biến đổi: biến đổi nào làm trước thì nhân gần $\vec{x}$ hơn ($y = B @ (A @ x) = (B @ A) @ x$, không có tính giao hoán $AB \neq BA$).
- **Định thức $\det(A)$:** Đo tỷ lệ phóng đại diện tích/thể tích $|\det(A)|$. Nếu $\det(A) = 0$: không gian bị bóp dẹt mất chiều (suy biến), thông tin bị mất vĩnh viễn $\Rightarrow \nexists A^{-1}$.
- **Trị riêng & Vectơ riêng:** $A\vec{v} = \lambda\vec{v}$ ($\vec{v} \neq \vec{0}$ giữ nguyên phương, chỉ co giãn λ lần). Tìm λ qua phương trình đặc trưng $\det(A - \lambda I) = 0$, với tổng $\sum \lambda_i = \text{Trace}(A)$ và tích $\prod \lambda_i = \det(A)$.
- **Phân rã Trị riêng & Ứng dụng AI:** $A = VDV^{-1} \Rightarrow A^k = VD^kV^{-1}$. Trong PCA, λ_i của ma trận hiệp phương sai Σ là phương sai (lượng thông tin) trên trục v_i ; trong RNN, $|\lambda_{\max}| > 1$ gây nổ đạo hàm (Exploding), $|\lambda_{\max}| < 1$ gây triệt tiêu đạo hàm (Vanishing).

Lesson 4: Calculus for Machine Learning

I. Bản chất và cơ chế

a) Đạo hàm 1 biến $f'(x)$: về hình học là độ dốc (slope) của tiếp tuyến tại điểm x .

- Nếu $f'(x) > 0$: Dốc lên sang phải, muốn $f(x)$ giảm $\implies$ phải lùi sang trái.
- Nếu $f'(x) < 0$: Dốc xuống, muốn $f(x)$ giảm $\implies$ đi sang phải.
- Nếu $f'(x) = 0$: Điểm bằng phẳng $\implies$ **critical point**: cực đại, cực tiểu, điểm uốn (hoặc điểm yên ngựa).

b) Vector Gradient (∇f)

- Là vectơ gom tất cả đạo hàm riêng của hàm nhiều biến $f(x_1, x_2, \dots, x_n)$:

$$\nabla f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T$$

$\Rightarrow$ **Định lý**: ∇f luôn chỉ theo **hướng tăng nhanh nhất** (*steepest ascent*) của hàm số và $\perp$ với đường mức⁽¹⁾ (*contour line*) tại điểm đó.

$\Rightarrow$ Hướng giảm nhanh nhất (*steepest descent*) là $-\nabla f$.

c) Chain Rule & Computation Graph (Nền tảng Backpropagation)

- Một mô hình Deep Learning chỉ là 1 hàm hợp khổng lồ:

$$x \longrightarrow z_1 \longrightarrow a_1 \longrightarrow z_2 \longrightarrow \hat{y} \longrightarrow \mathcal{L}$$

- Chain Rule nói rằng:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdots \frac{\partial z_1}{\partial w} \quad (\text{trong vở ghi mắt xích cuối là } \frac{\partial z_1}{\partial x})$$

⁽¹⁾ **Đường mức (Contour line)**: là tập hợp các điểm (w_1, w_2) trên mặt phẳng mà tại đó hàm loss có cùng 1 giá trị không đổi $f(w_1, w_2) = C$. VD: $f(x, y) = x^2 + y^2$ tại độ cao $C = 4$ có đường mức là đường tròn $x^2 + y^2 = 4$.

- **Forward Pass** (Lan truyền xuôi): truyền giá trị từ trái $\rightarrow$ phải để tính $\mathcal{L}$.
- **Backward Pass** (Lan truyền ngược): truyền đạo hàm từ phải $\rightarrow$ trái = phép $\times$ ma trận.

II. Thuật toán Gradient Descent and Learning Rate

a) Công thức cập nhật và vai trò của Learning Rate

- **Vấn đề:** Đã biết hướng xuống dốc là $-\nabla L$, vậy công thức toán học để máy tính tự động cập nhật trọng số w qua từng vòng lặp là gì? Và ta nên bước mỗi bước dài bao nhiêu?

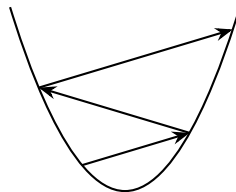
⇒ Công thức cập nhật kinh điển — **Gradient Descent**:

$$w_{\text{new}} = w_{\text{old}} - lr \times \frac{\partial L}{\partial w}$$

- lr (*Learning rate* — tốc độ học, thường ký hiệu η): là 1 số dương nhỏ (0,01; 0,001) đóng vai trò kiểm soát độ dài sải chân.
- Có dấu trừ ($-$) vì $\frac{\partial L}{\partial w}$ chỉ hướng làm hàm loss tăng, trừ đi giúp đi về hướng hàm loss giảm.

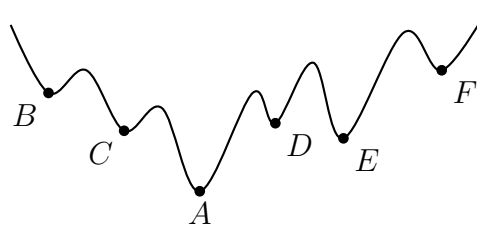
⇒ **Hệ quả khi chọn lr :**

- **lr quá nhỏ:** Mỗi bước nhích quá ít $\Rightarrow$ mô hình bò rất chậm (*crawl*), tốn nhiều thời gian và chi phí GPU.
- **lr quá lớn:** Bước quá dài. VD đang ở bên sườn núi trái, bước 1 phát văng sang sườn núi phải ở vị trí cao hơn cả lúc đầu (*overshooting*); cứ thế bước sau cao hơn bước trước, hàm loss vọt lên vô cùng $\Rightarrow$ **Divergence (Phân kỳ)**.



Hiện tượng Overshooting & Divergence khi lr quá lớn

b) Bản đồ địa hình Loss (Loss Landscape)



$\left\{ \begin{array}{l} \text{Điểm A : Global minimum (Cực tiểu toàn cục)} \\ \text{Điểm B, C, E, D, F : Local minimum (Cực tiểu cục bộ)} \end{array} \right.$

⇒ **Vấn đề:** Trong không gian 1 hoặc 2 chiều, GD⁽²⁾ rất dễ rơi xuống hố local minimum và bị kẹt tại đó (vì tại đó đạo hàm cũng $= 0$).

⇒ **Thực tế:** Nhưng trong mạng Neural hàng triệu chiều, 1 điểm muốn là local minimum thực sự thì cả 1 triệu chiều phải đồng loạt cong vòng lên cùng 1 lúc $\Rightarrow$ xác suất $\approx \left(\frac{1}{2}\right)^{1\,000\,000} \approx 0$.

⇒ Hầu hết các chỗ có đạo hàm = 0 trong không gian nhiều chiều là **điểm yên ngựa (saddle point)**: nó có chiều cong lên nhưng vẫn có chiều khác thoải xuống để mô hình thoát ra được.

III. Đạo hàm số và Đạo hàm giải tích

❓ **Vấn đề:** Lập trình AI tính $f'(x)$ có mấy cách?

Cách 1: Analytical Derivative **⟨Đạo hàm giải tích⟩**

- Dùng quy tắc toán học: VD $f(x) = x^2 \rightarrow f'(x) = 2x$.
- $\left\{ \begin{array}{l} \text{Ưu : Chính xác tuyệt đối, chạy nhanh.} \\ \text{Nhược : Mạng 100 tầng mà tự ngồi viết tay công thức đạo hàm} \Rightarrow \text{rất dễ sai sót.} \end{array} \right.$

(2) **GD:** Gradient Descent.

Cách 2: Numerical Derivative **⟨Đạo hàm số⟩**: xấp xỉ = sai phân trung tâm

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad (h \approx 0,000 \dots 1)$$

Phát biểu: Nhích sang trái và phải 1 chút, lấy chênh lệch chia khoảng cách $2h$.

❓ **Sao không dùng sai phân tiến** $\frac{f(x+h) - f(x)}{h}$?

⇒ Vì độ cong bậc 2 dư ra ở bên phải $(+\frac{1}{2}f''(x)h^2)$ tự động triệt tiêu với phần bên trái khi lấy $f(x+h) - f(x)$, giúp sai số giảm từ $O(h)$ xuống $O(h^2)$.

⇒ **Nhược:**

- Chậm — cực chậm!
- Nếu mạng có $N = 1$ triệu weights, cần chạy forward pass 2 triệu lần cho đúng 1 bước cập nhật.

⇒ Trong thực tế, Numerical Derivative chỉ dùng để **kiểm thử (Gradient Checking)** xem công thức đạo hàm giải tích viết tay có bị lỗi (bug) hay không.

Cách 3: Automatic Differentiation (Autodiff)

- Đây là cơ chế lõi của PyTorch và TensorFlow: kết hợp ưu điểm của cả 2 — máy tính tự áp dụng quy tắc đạo hàm giải tích trên từng phép toán nguyên tử thông qua Chain Rule tốn $O(1)$ cho mỗi nút và chính xác tuyệt đối.

b) 5 viên gạch cơ sở về đạo hàm trong Machine Learning

$$1) f(x) = x^2 \implies f'(x) = 2x$$

$$2) y = wx + b \implies \frac{\partial y}{\partial w} = x, \frac{\partial y}{\partial x} = w, \frac{\partial y}{\partial b} = 1$$

$$3) f(x) = e^x \implies f'(x) = e^x$$

$$4) f(x) = \ln(x) \implies f'(x) = \frac{1}{x}$$

$$5) \text{Hàm kích hoạt Sigmoid: } \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}} \implies \boxed{\sigma'(x) = \sigma(x)(1 - \sigma(x))}$$

★ Chứng minh nhanh:

$$\sigma'(x) = -\frac{-e^{-x}}{(1 + e^{-x})^2} = \frac{1}{1 + e^{-x}} \cdot \frac{e^{-x}}{1 + e^{-x}} = \frac{1}{1 + e^{-x}} \cdot \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) = \sigma(x)(1 - \sigma(x))$$

c) Chain Rule (Quy tắc dây chuyền)

- **Cơ chế:** Nếu $y = f(g(x))$ thì tốc độ thay đổi tổng cộng = tích các tốc độ thay đổi qua từng mắt xích:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx} = f'(g(x)) \cdot g'(x)$$

VD: $y = (3x + 1)^2$, tính $\frac{dy}{dx}$:

$$y = u^2 \implies f'(u) = \underline{2u} \quad 2u = 2(3x + 1)$$

$$u' = (3x + 1)' = 3$$

$$\implies \frac{dy}{dx} = f'(u) \cdot u' = 6(3x + 1)$$

IV. Ma trận Hessian — Đo độ cong không gian

a) Vấn đề

- Đạo hàm bậc 1 = 0 $\implies \begin{cases} \text{local max (cực đại)} \\ \text{local min (cực tiểu)} \\ \text{saddle point (điểm yên ngựa)} \end{cases}$

$\implies$ Làm sao biết mình đang đứng ở đâu?

$\implies$ Ta cần 1 công cụ đo **độ cong (curvature)** — đó là đạo hàm bậc 2 (**Ma trận Hessian**).

b) Bản chất và cấu trúc Ma trận Hessian

- Với hàm 1 biến:

– $f''(x) > 0$: Độ dốc tăng từ âm $\rightarrow$ dương $\Downarrow$ Mặt cong ngửa lên $\implies$ **Đây là cực tiểu.**



– $f''(x) < 0$: Độ dốc giảm từ dương $\rightarrow$ âm, mặt cong úp $\Downarrow \implies$ **Đỉnh là cực đại.**

• **Với hàm nhiều biến $f(x_1, x_2, \dots, x_n)$:**

$\Rightarrow$ Gom $n \times n$ đạo hàm riêng bậc 2 lại ta được ma trận Hessian $H \in \mathbb{R}^{n \times n}$:

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

VD: Với hàm 2 biến $f(x, y)$:

$$\implies H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

c) Trị riêng của H

- Trị riêng cho biết hệ số co giãn dọc theo trục chính.
- Với ma trận Hessian H , mỗi trị riêng λ_i là **độ cong** (đạo hàm bậc 2) dọc theo hướng chính thứ i tại điểm dừng:

$$\begin{cases} \text{Mọi } \lambda_i > 0 \text{ (} H \text{ xác định dương)} \implies \text{Điểm cực tiểu (Local Minimum)} \\ \text{Mọi } \lambda_i < 0 \text{ (} H \text{ xác định âm)} \implies \text{Điểm cực đại (Local Maximum)} \\ \text{Có cả } \lambda_i > 0 \text{ lẫn } \lambda_j < 0 \implies \text{Điểm yên ngựa (Saddle Point)} \end{cases}$$

d) Vì sao Hessian quan trọng trong Machine Learning

• **Phương pháp Newton (Newton's Method):**

– Gradient Descent: $w_{\text{new}} = w_{\text{old}} - lr \cdot \nabla f$

– Newton's Update: $w_{\text{new}} = w_{\text{old}} - H^{-1} \cdot \nabla f$

Newton hội tụ nhanh hơn vì H^{-1} giúp “co giãn” lại bước nhảy của gradient theo độ cong $\frac{1}{\lambda_i}$:

$$\begin{cases} \text{Hướng nào dốc đứng (steep, } \lambda_i \text{ lớn)} : \text{bước nhảy tự thu nhỏ lại} \\ \text{Hướng nào bằng phẳng (flat, } \lambda_i \text{ nhỏ)} : \text{bước nhảy tự kéo dài ra} \end{cases}$$

$\Rightarrow$ **Vấn đề:** Với mạng có N tham số, H có shape $N \times N$ (VD $N = 10^9 \implies 10^{18}$ phần tử) nên không thể lưu trữ hay nghịch đảo trực tiếp được!

$\Rightarrow$ Cần các thuật toán xấp xỉ bậc 2 hoặc tốc độ học thích nghi

Phương pháp	Dùng thông tin gì	Chi phí	Tốc độ hội tụ
Gradient Descent	Chỉ đạo hàm bậc 1 (∇f)	$O(N)$	Chậm (tuyến tính)
Newton's Method	Toàn bộ ma trận H và H^{-1}	$O(N^3)$	Nhanh (bậc 2)
L-BFGS	Xấp xỉ Hessian từ lịch sử gradient	$O(N)$	Trung bình (siêu tuyến tính)
Adam	Tốc độ học thích nghi cho TỪNG parameter	$O(N)$	Trung bình — nhanh
Natural Gradient	Ma trận thông tin Fisher (Hessian thống kê)	$O(N^2)$	Nhanh

⇒ ADAM là trình tối ưu mặc định cho Deep Learning.

V. Xấp xỉ chuỗi Taylor ⟨Taylor Series Approximation⟩

$$f(x+h) = f(x) + \frac{f'(x)}{1!} \cdot h + \frac{f''(x)}{2!} h^2 + \frac{f'''(x)}{3!} h^3 + \dots$$

❓ Vì sao chuỗi Taylor quan trọng với Machine Learning?

- Taylor bậc 1 = Gradient Descent:

$$f(x+h) \approx f(x) + f'(x) \cdot h \quad (\text{hoặc đa biến: } f(w+\Delta w) \approx f(w) + \nabla f(w)^T \Delta w)$$

- Taylor bậc 2 = Newton's Method:

$$f(x+h) \approx f(x) + f'(x) \cdot h + \frac{f''(x)h^2}{2} \quad (\text{đa biến: } f(w) + \nabla f(w)^T \Delta w + \frac{1}{2} \Delta w^T H \Delta w)$$

VI. Tích phân trong Học máy ⟨Integrals in ML⟩

- Đạo hàm đo tốc độ thay đổi tức thời.
- Tích phân tính sự tích lũy, xuất hiện ở 4 bài toán trụ cột:

1. Xác suất: Biến ngẫu nhiên liên tục có hàm mật độ $p(x)$

$$P(a < X < b) = \int_a^b p(x) dx$$

2. Kỳ vọng (Expected Value)

$$E[f(x)] = \int_{-\infty}^{+\infty} f(x)p(x) dx$$

3. KL Divergence: Đo sự khác biệt giữa 2 phân phối p và q

$$\text{KL}(p \parallel q) = \int_{-\infty}^{+\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

4. Hằng số chuẩn hóa trong Suy luận Bayes (Marginal Likelihood)

$$\text{Bayes: } p(w \mid \text{data}) = \frac{p(\text{data} \mid w) \cdot p(w)}{\int p(\text{data} \mid w) p(w) dw}$$

VII. Quy tắc chuỗi đa biến trong Đồ thị tính toán

• Lượt truyền xuôi (Forward):

$$x \xrightarrow{\times w} z_1 = wx \xrightarrow{+b} z_2 = wx + b \xrightarrow{\text{sigmoid}} a = \sigma(z_2) \xrightarrow{\text{loss}} L = -(y \log(a) + (1-y) \log(1-a))$$

• Lượt truyền ngược (Backward):

– Đạo hàm loss theo a :

$$\frac{dL}{da} = \frac{-y}{a} + \frac{1-y}{1-a} = \frac{a-y}{a(1-a)}$$

– Qua cổng Sigmoid:

$$\frac{dL}{dz_2} = \frac{dL}{da} \cdot a(1-a) = a-y \quad (\text{triệt tiêu mẫu số cực đẹp!})$$

– Tách ra 2 nhánh về w và b :

$$\begin{cases} \frac{dL}{dw} = \frac{dL}{dz_2} \cdot x = (a-y)x \\ \frac{dL}{db} = \frac{dL}{dz_2} \cdot 1 = a-y \end{cases}$$

VIII. Ma trận Jacobian & Bảng đạo hàm các Hàm kích hoạt hiện đại

1. Ma trận Jacobian $J \in \mathbb{R}^{m \times n}$ cho hàm vectơ $\vec{f}: \mathbb{R}^n \rightarrow \mathbb{R}^m$

① Phân biệt Gradient, Jacobian và Hessian:

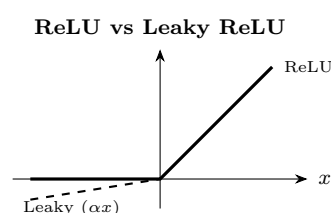
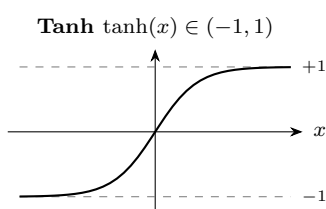
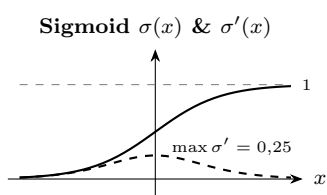
- Khi hàm số trả về 1 số vô hướng $L: \mathbb{R}^n \rightarrow \mathbb{R}$ (như hàm Loss), đạo hàm bậc 1 là vectơ Gradient $\nabla L \in \mathbb{R}^{n \times 1}$.

- Khi hàm số biến đổi từ vectơ $\vec{x} \in \mathbb{R}^n$ sang vectơ $\vec{y} = \vec{f}(\vec{x}) \in \mathbb{R}^m$ (như một tầng mạng nơ-ron hay hàm Softmax), đạo hàm bậc 1 là **Ma trận Jacobian** $J \in \mathbb{R}^{m \times n}$, trong đó hàng i cột j là:

$$J_{ij} = \frac{\partial f_i}{\partial x_j} \iff J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}_{(m \times n)}$$

2. Tổng hợp các Hàm kích hoạt (Activation Functions) và Đạo hàm

Hàm kích hoạt	Công thức $f(x)$	Đạo hàm $f'(x)$	Đặc tính trong Deep Learning
Sigmoid $\sigma(x)$	$\frac{1}{1 + e^{-x}}$	$\sigma(x)(1 - \sigma(x))$	Đầu ra $(0, 1)$. Dễ gây <i>Vanishing Gradient</i> vì $\max \sigma'(x) = 0,25$ tại $x = 0$.
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$1 - \tanh^2(x)$	Đầu ra $(-1, 1)$, đối xứng qua 0 (<i>zero-centered</i>). $\max f'(x) = 1$.
ReLU	$\max(0, x)$	$\begin{cases} 1 & x > 0 \\ 0 & x < 0 \end{cases}$	Chống triệt tiêu đạo hàm vùng $x > 0$, tính toán nhanh; có thể gây <i>Dying ReLU</i> khi $x < 0$.
Leaky ReLU	$\max(\alpha x, x)$	$\begin{cases} 1 & x > 0 \\ \alpha & x < 0 \end{cases}$	Khắc phục <i>Dying ReLU</i> bằng độ dốc nhỏ $\alpha \approx 0,01$ ở miền âm.
GELU	$x \cdot \Phi(x)$	$\Phi(x) + x\phi(x)$	Trơn theo xác suất Gaussian $\Phi(x)$, chuẩn trong BERT, GPT, ViT.
SiLU / Swish	$x \cdot \sigma(x)$	$\sigma(x) + x\sigma'(x)$	Hàm kích hoạt trơn không đơn điệu, dùng trong kiến trúc LLaMA / YOLOv8.



IX. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 4

- **Gradient & Hướng Tối ưu:** Vectơ $\nabla f = \left[\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right]^T$ luôn vuông góc với đường mức (*contour line*) và chỉ hướng tăng nhanh nhất. Quy tắc cập nhật Gradient Descent: $w_{\text{new}} = w_{\text{old}} - lr \cdot \nabla L$.
- **Sai phân Trung tâm (Gradient Checking):** $f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$ có sai số $O(h^2)$ (chính xác hơn sai phân tiến $O(h)$ nhờ triệt tiêu số hạng bậc hai $\frac{1}{2}f''(x)h^2$), dùng để kiểm thử code đạo hàm giải tích.
- **Ma trận Hessian** $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$: Đo độ cong bậc 2. Mọi $\lambda_i(H) > 0 \implies$ cực tiểu; mọi $\lambda_i(H) < 0 \implies$ cực đại; trái dấu $\implies$ điểm yên ngựa (*saddle point* — phổ biến nhất trong không gian triệu chiều).
- **Phương pháp Newton vs Adam:** Bước nhảy Newton $\Delta w = -H^{-1} \nabla f$ tự co giãn theo độ cong nhưng tốn $O(N^3)$ không khả thi cho Deep Learning $\implies$ dùng **Adam** ($O(N)$) xấp xỉ tốc độ học thích nghi cho từng tham số.

Lesson 5: Chain Rule and Autodiff

I. Vấn đề

- Một mạng nơ-ron thực chất là 1 hàm hợp khổng lồ:
+ Nhân ma trận (@) → cộng bias (+b) → hàm kích hoạt (σ) → lặp hàng trăm lần → hàm loss.

$$\implies L = f_n(f_{n-1}(\dots f_1(x, w_1) \dots))$$

⇒ **Vấn đề:** Để cập nhật hàng triệu trọng số w , ta cần tính đạo hàm riêng $\frac{\partial L}{\partial w_i}$ cho từng trọng số:

Giải đạo hàm bằng tay → bất khả thi.
TÍNH $\frac{f(x+h) - f(x-h)}{2h}$ (sai phân hữu hạn)
⇒ với N tham số cần chạy forward pass $2N$ lần cho mỗi bước → quá chậm.

⇒ **Solution: Chain Rule & Automatic Differentiation (Autodiff).**

II. The Concept

2.1. The Chain Rule

a) Vấn đề:

- Làm sao biết nhích nhẹ 1 trọng số x ở đầu thì hàm loss ở cuối thay đổi bao nhiêu?

b) Bản chất:

- Nếu $y = f(g(x)) \implies \boxed{\frac{dy}{dx} = f'(g(x)) \cdot g'(x)}$

2.2. Computational Graphs (Đồ thị tính toán)

a) Vấn đề: Máy tính phải bẻ nhỏ biểu thức phức tạp thành các phép toán nguyên tử (+, -, ×, exp, ReLU ...).

b) Nguyên lý Upstream × Local Gradient:

- Mỗi phép toán là 1 node: chiều **forward pass** truyền dữ liệu, chiều **backward pass** truyền đạo hàm từ output về input.

= Cách làm:

$$\text{Downstream Gradient} = \text{Upstream Gradient} \times \text{Local Gradient}$$

$$\begin{array}{ccccc} \frac{\partial L}{\partial x_1} & = & \frac{\partial L}{\partial a} & \times & \frac{\partial a}{\partial x_1} \\ \uparrow & & \uparrow & & \uparrow \\ \text{Downstream} & & \text{Upstream} & & \text{Local} \end{array}$$

VD 1: $L = (x_1x_2 + 1)^2$, $x_1 = 2$, $x_2 = 3$

$$L = (a + 1)^2 = b^2$$

① Forward:

$$\begin{cases} a = x_1x_2 = 6 \\ b = a + 1 = 7 \\ L = b^2 = 49 \end{cases}$$

② Backward: Xét tại nút nhân $a = x_1 \cdot x_2$:

- Từ phía sau (thượng nguồn) truyền về bảo là: “Ê nút a , cứ khi a tăng 1 đơn vị thì loss tăng 14 đơn vị nhé”.

$$\Rightarrow \text{Upstream gradient: } \frac{\partial L}{\partial a} = 14 \quad \left(\text{vì } \frac{\partial L}{\partial a} = 2(a + 1) = 2(6 + 1) = 14 \right)$$

$\uparrow$
 $\frac{d(a+1)^2}{da}$

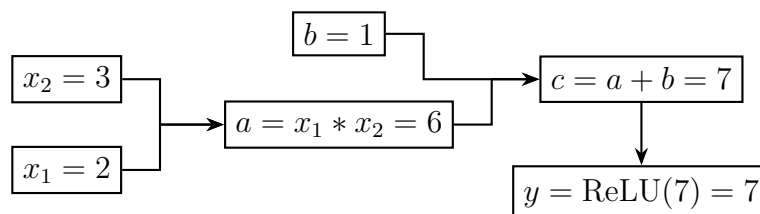
- Bản thân nút $a = x_1x_2$ tự biết: “Cứ khi x_1 tăng 1 đơn vị thì a tăng $x_2 = 3$ đơn vị” (vì $x_1 \times x_2 = x_1 \cdot 3$ nên $x_1 \uparrow 1$ thì $a \uparrow 3$):

$$\Rightarrow \text{Local gradient: } \frac{\partial a}{\partial x_1} = x_2 = 3$$

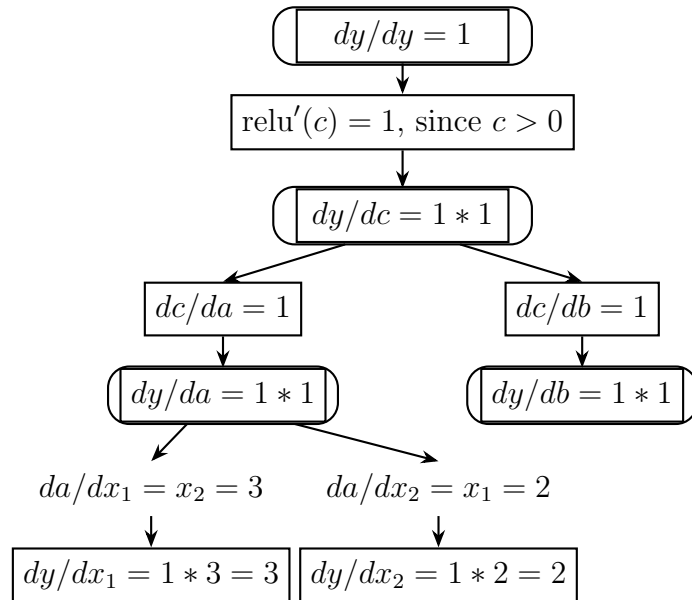
$$\Rightarrow \text{Khớp lại: } \frac{\partial L}{\partial x_1} = \underset{\uparrow \text{up}}{14} \times \underset{\uparrow \text{local}}{3} = 42$$

VD 2: $y = \text{ReLU}(x_1x_2 + b)$, $x_1 = 2$, $x_2 = 3$, $b = 1$

1. Forward pass



2. Backward pass



c) Forward Mode vs Reverse Mode & Dual Numbers

- **Vấn đề:** Nhân Chain Rule từ trái $\rightarrow$ phải hay ngược lại cho cùng 1 kết quả toán học, nhưng vì sao mọi thư viện Deep Learning đều chọn chiều ngược (Reverse Mode)?

VD: Với $y = \sin(x^2)$ tại $x = 2$:

$$\begin{aligned}
 \text{(Forward Mode)} \quad & \begin{cases} x = 2 & dx/dx = 1 \\ a = x^2 = 4 & da/dx = 2x = 4 \\ y = \sin(a) & dy/dx = \frac{dy}{da} \cdot \frac{da}{dx} = \cos(a) \cdot 4 = 4 \cos(4) \end{cases} \\
 \text{(Reverse Mode)} \quad & \begin{cases} y = \sin(a) & dy/dy = 1 \\ a = x^2 & dy/da = \cos a = \cos 4 \\ x = 2 & dy/dx = \frac{dy}{da} \cdot \frac{da}{dx} = \cos 4 \cdot (2x) = 4 \cos 4 \end{cases}
 \end{aligned}$$

Bản chất: Giả sử hàm $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (n inputs, m outputs):

Mode	Khởi tạo	Hướng lan	Chi phí toàn bộ	Phù hợp khi
Forward	$\frac{\partial x_i}{\partial x_i} = 1$	input $\rightarrow$ output	$O(n)$ lượt chạy (1 lượt cho mỗi input)	Ít input, nhiều output ($n \ll m$)
Reverse	$\frac{\partial L}{\partial L} = 1$	out $\rightarrow$ in	$O(m)$ lượt chạy (1 lượt cho mỗi output)	Nhiều input, ít output ($n \gg m$, mạng Neural có $m = 1$ hàm Loss!)

★ Dual Numbers (Số kép cho Forward Mode Autodiff):

Biểu diễn dạng $\begin{cases} \text{cặp (giá trị, đạo hàm)} \\ a + a'\varepsilon \end{cases} \quad (\text{với quy ước } \varepsilon^2 = 0, \varepsilon \neq 0)$

⇒ Khi thực hiện phép nhân hai số kép:

$$(a + a'\varepsilon)(b + b'\varepsilon) = ab + (a'b + ab')\varepsilon + \underbrace{a'b'\varepsilon^2}_{=0} = ab + (a'b + ab')\varepsilon$$

- Phần thực ab chính là giá trị phép nhân.
- Phần gắn với ε là $(a'b + ab')$ tự động chính là đạo hàm của tích $(uv)' = u'v + uv'$.

Lưu ý về ε : $\varepsilon^2 = 0$ là quy ước đại số (nilpotent), nó không có nghĩa $\varepsilon = 0$.

d) Cơ chế trong Autograd⁽¹⁾ Engine và PyTorch

- **Vấn đề:** Khi 1 biến x rẽ nhánh vào nhiều phép toán khác nhau (VD $y = f(x) + x$), làm sao lan truyền ngược đạo hàm mà không bị tính thiếu hoặc sai thứ tự?
- **Autograd dựa trên 3 trụ cột:**

- 1) **Topological Sort (Sắp xếp Tô-pô):** Trước khi chạy ngược, đồ thị phải sắp xếp các nút sao cho mọi nút chỉ được truyền đạo hàm sau khi các nút con phía sau đã gom đủ đạo hàm về cho nó.
- 2) **Gradient Accumulation (Cộng dồn đạo hàm):** Khi x đi vào nhiều nhánh $u(x), v(x)$, đạo hàm tổng theo x là **tổng** các nhánh (Multivariable Chain Rule):

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x} + \frac{\partial y}{\partial v} \cdot \frac{\partial v}{\partial x} + \dots$$

- 3) **Dynamic Graph (Đồ thị động — Define-by-Run):** Đồ thị được dựng mới mỗi lần chạy forward, cho phép dùng `if/else`, vòng lặp `for` thoải mái trong hàm forward.
- **Mục đích Autograd:** Tự tìm đạo hàm cho bạn — chỉ cần viết phép tính xuôi **forward**, Autograd tự tính đạo hàm ngược **backward**.
 - **Cách hoạt động:** Mỗi biến (Tensor) trong Autograd là cục dữ liệu có 2 ngăn:
 - `.data` (chứa giá trị thực)
 - `.grad` (chứa đạo hàm $\frac{\partial L}{\partial x}$)
 - **Chiều xuôi:** VD khi bạn viết `L = (x * w) + 1`, Autograd vừa tính ra kết quả lưu vào `.data`, vừa lưu lại con trỏ hàm ngược (`grad_fn`) nhớ lại các bước đã đi (ai nhân với ai, ...).

⁽¹⁾ **Autograd:** Automatic Gradient (hệ thống tính đạo hàm tự động).

- **Chiều ngược (`L.backward()`):** Khi gọi `backward()`, Autograd sẽ lần ngược lại từng bước đi theo thứ tự Topological Sort ngược, dùng Chain Rule để điền (cộng dồn) kết quả đạo hàm vào ngăn `.grad` của từng biến.

Lưu ý: Biến nào cần lấy đạo hàm thì phải bật công tắc (`requires_grad = True`), Autograd mới theo dõi nó; còn khi suy luận (*Inference / Evaluation*), bọc trong `with torch.no_grad()` để tắt dựng đồ thị và tiết kiệm VRAM.

III. Tại sao phải gọi `optimizer.zero_grad()` & Vấn đề Đạo hàm trong RNN / LSTM

1. Cơ chế cộng dồn `.grad` trong PyTorch và `zero_grad()`

❓ Vì sao khi huấn luyện PyTorch bắt buộc phải gọi `optimizer.zero_grad()` trước mỗi lần `loss.backward()`?

- Do trụ cột số 2 (**Gradient Accumulation**), mỗi khi gọi `backward()`, PyTorch thực hiện phép cộng dồn:

$$\mathbf{w}.\text{grad} \leftarrow \mathbf{w}.\text{grad} + \frac{\partial L_{\text{batch hiện tại}}}{\partial \mathbf{w}}$$

- Thiết kế này rất hữu ích khi một trọng số được dùng lại nhiều lần trong cùng 1 lượt forward (như trọng số chia sẻ trong RNN) hoặc khi ta muốn gộp nhiều mini-batch nhỏ thành 1 batch lớn (Gradient Accumulation khi thiếu VRAM).
- Tuy nhiên, khi sang bước cập nhật mới, nếu quên gọi `optimizer.zero_grad()`, đạo hàm của batch mới sẽ bị cộng chồng lên đạo hàm của batch cũ làm sai lệch hoàn toàn hướng đi của Gradient Descent!

2. Backpropagation Through Time (BPTT) trong RNN và Cơ chế Cổng của LSTM / GRU

① Vì sao mạng RNN truyền thống thất bại với chuỗi phụ thuộc dài?

Xét trạng thái ẩn của RNN qua T bước thời gian: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$.

Khi lan truyền ngược đạo hàm từ bước cuối T về bước đầu 1 theo quy tắc dây chuyền (Chain Rule):

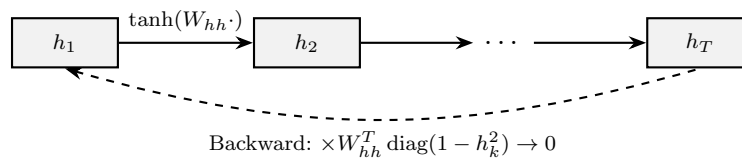
$$\frac{\partial L_T}{\partial h_1} = \frac{\partial L_T}{\partial h_T} \cdot \prod_{k=2}^T \frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial L_T}{\partial h_T} \cdot \prod_{k=2}^T \text{diag}(1 - h_k^2) W_{hh}^T$$

- Vì đạo hàm của $\tanh$ bị chặn bởi $0 \leq 1 - h_k^2 \leq 1$, tích liên tiếp T ma trận sẽ làm đạo hàm **triệt tiêu về 0 (Vanishing Gradient)** nếu trị riêng lớn nhất của W_{hh} nhỏ hơn 1, hoặc **bùng nổ ra ∞ (Exploding Gradient)** nếu trị riêng lớn hơn 1.
- Hệ quả:** Mạng RNN không thể học được các phụ thuộc xa về không gian và thời gian do vấn đề đạo hàm (Gradient problems).

② Giải pháp của LSTM / GRU và Kết nối tắt ResNet:

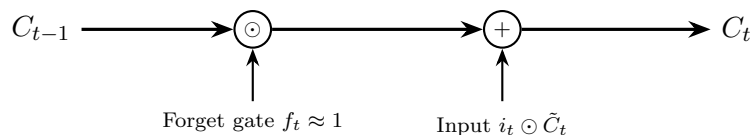
- LSTM** tạo ra trạng thái tế bào $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ (với cổng quên f_t , cổng nạp i_t), thay phép nhân ma trận liên tiếp bằng **phép cộng tuyến tính** trên đường cao tốc thông tin (*Cell state highway*).
- ResNet / Transformer** sử dụng kết nối tắt $y = f(x) + x \implies \frac{\partial y}{\partial x} = \frac{\partial f(x)}{\partial x} + I$. Số hạng $+I$ đảm bảo đạo hàm luôn có đường truyền thẳng bằng 1 về tận các tầng đầu tiên mà không bị triệt tiêu:

RNN chuẩn (Nhân liên tiếp $W_{hh}^T \Rightarrow$ Đạo hàm tắt dần):



Đường cao tốc LSTM Cell State ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$):

Truyền thẳng đạo hàm qua phép cộng (+)



3. Bảng Quy tắc Đạo hàm Cục bộ của các Cổng Toán học

- **Cổng Cộng** ($z = x + y$ — Bộ phân phối đạo hàm): $\frac{\partial z}{\partial x} = 1, \frac{\partial z}{\partial y} = 1 \Rightarrow$ truyền nguyên vẹn Upstream Gradient về cả hai nhánh.
- **Cổng Nhân** ($z = x \cdot y$ — Bộ tráo đổi hệ số): $\frac{\partial z}{\partial x} = y, \frac{\partial z}{\partial y} = x \Rightarrow$ đạo hàm theo biến này bằng giá trị forward của biến kia (nên Forward pass bắt buộc phải lưu bộ nhớ đệm *cache* các giá trị kích hoạt).
- **Cổng Nhân Ma trận** ($Z = X @ W$): Với Upstream Gradient $G = \frac{\partial L}{\partial Z}$, ta luôn có cặp công thức khớp chiều:

$$\boxed{\frac{\partial L}{\partial X} = G @ W^T \quad , \quad \frac{\partial L}{\partial W} = X^T @ G}$$

IV. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 5

- Nguyên lý Lan truyền ngược (Backpropagation): Tại mỗi nút trên Đồ thị Tính toán:

$$\text{Downstream Gradient} = \text{Upstream Gradient} \times \text{Local Gradient}$$

Nếu 1 biến phân nhánh đi vào nhiều nút, đạo hàm tổng bằng **tổng đạo hàm từ các nhánh** (`grad += ...`).

- Vì sao Deep Learning dùng Reverse-Mode Autodiff? Với mạng $f : \mathbb{R}^n \rightarrow \mathbb{R}^1$ ($n \sim 10^9$ trọng số, $m = 1$ hàm Loss), Forward-mode tốn $O(n)$ lượt chạy còn **Reverse-mode chỉ tốn đúng 1 lượt chạy ngược** để lấy đạo hàm cho toàn bộ n trọng số!
- 3 Trụ cột của PyTorch Autograd: (1) Sắp xếp Tô-pô ngược (*Reverse Topological Sort*), (2) Cộng dồn đạo hàm (`+=`, do đó phải gọi `optimizer.zero_grad()` mỗi step), (3) Đồ thị động (*Define-by-Run*).

Lesson 6: Probability and Distributions

I. Concept

a) Độc lập vs Xung khắc

- **Độc lập (Independent):** Không ảnh hưởng đến nhau:

$$P(A \cap B) = P(A) \cdot P(B)$$

- **Xung khắc (Mutually Exclusive):** Không thể cùng xảy ra trong 1 phép thử:

$$P(A \cap B) = 0 \implies P(A \cup B) = P(A) + P(B)$$

b) Probability Mass Functions (PMF) vs Probability Density Functions (PDF)

- **PMF (Hàm khối xác suất):** Dùng cho biến ngẫu nhiên rời rạc — đọc trực tiếp giá trị xác suất tại từng điểm $P(X = k)$, với $\sum_k P(X = k) = 1$.
- **PDF (Hàm mật độ xác suất):** Dùng cho biến ngẫu nhiên liên tục — giá trị $f(x)$ tại 1 điểm là **mật độ (density)** chứ không phải xác suất (vì tại đúng 1 điểm đơn lẻ $P(X = x) = 0$). Muốn xác định xác suất phải lấy tích phân trên 1 khoảng $[a, b]$:

$$P(a \leq X \leq b) = \int_a^b f(x) dx \quad , \quad \text{và} \quad \int_{-\infty}^{+\infty} f(x) dx = 1$$

Lưu ý: Giá trị hàm mật độ xác suất $f(x)$ hoàn toàn có thể > 1 (miễn là diện tích dưới đường cong bằng 1).

II. Common Distributions (5 phân phối xác suất cốt lõi)

① Bernoulli: 1 phép thử, 2 kết quả 0 hoặc 1

$\implies$ Mô hình hóa bài toán **Binary Classification**

- $P(X = 1) = p$, $P(X = 0) = 1 - p$
- $\text{Mean}^{(1)} = E[X] = p$ (Kỳ vọng, giá trị trung bình)
- $\text{Variance} = \text{Var}(X) = p(1 - p)$ (Phương sai σ^2)

② Categorical: 1 phép thử, k kết quả loại trừ nhau

$\implies$ Mô hình hóa bài toán **Multi-class Classification**

$$P(X = i) = p_i \quad \left(\text{với} \quad \sum_{i=1}^k p_i = 1 \right)$$

⁽¹⁾ **Mean**: giá trị trung bình (kỳ vọng).

③ **Uniform (Phân phối đều)**: mọi kết quả có khả năng xảy ra ngang nhau

⇒ Dùng để **khởi tạo trọng số ngẫu nhiên (Weight Initialization)** và lấy mẫu:

- Rời rạc (n giá trị): $P(X = k) = \frac{1}{n}$
- Liên tục trên đoạn $[a, b]$:

$$f(x) = \frac{1}{b-a}, \quad E[X] = \frac{a+b}{2}, \quad \text{Var}(X) = \frac{(b-a)^2}{12}$$

④ **Normal / Gaussian (Phân phối chuẩn hình chuông)**:

- Đặc trưng bởi trung bình μ và phương sai σ^2 , ký hiệu $X \sim \mathcal{N}(\mu, \sigma^2)$:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (2)$$

- **Chuẩn tắc (Standard Normal $\mu = 0, \sigma = 1$)** — Quy tắc thực nghiệm 68 – 95 – 99,7:
 - 68% dữ liệu nằm trong khoảng $\mu \pm 1\sigma$
 - 95% dữ liệu nằm trong khoảng $\mu \pm 2\sigma$
 - 99,7% dữ liệu nằm trong khoảng $\mu \pm 3\sigma$

⑤ **Poisson**: Đếm số sự kiện hiếm xảy ra trong một khoảng thời gian/không gian cố định

$$P(X = k) = \frac{\lambda^k \cdot e^{-\lambda}}{k!} \quad \left(\begin{array}{l} \text{Mean} = \lambda \\ \text{Variance} = \lambda \end{array} \right)$$

III. Expected Value & Variance (Kỳ vọng và Phương sai)

1. Vấn đề:

Làm sao tóm tắt cả 1 phân phối xác suất phức tạp thành 2 con số đại diện cho “tâm điểm” (hàm loss trung bình) và “độ bất ổn định” (độ phân tán của dữ liệu quanh giá trị trung bình).

⁽²⁾ $\exp(u) = e^u$ | $f(x)$ càng cao $\rightarrow$ dữ liệu rơi vào vùng quanh x càng dày.

$f(x)$ là độ cao của đường cong mật độ tại điểm x (vì biến liên tục không đếm rời rạc từng số được).

2. Concept

a) **Expected Value** $E(X)$ — Trung bình có trọng số theo xác suất:

$$\left\{ \begin{array}{l} \text{Rời rạc:} \quad E(X) = \mu = \sum x_i \cdot P(X = x_i) \\ \text{Liên tục:} \quad E(X) = \mu = \int_{-\infty}^{+\infty} x \cdot f(x) dx \end{array} \right.$$

b) **Variance** ($\text{Var}(X)$, $V(X)$) — độ phân tán / độ lệch bình phương trung bình của dữ liệu quanh giá trị trung bình μ):

$$\text{Var}(X) = E[(X - E(X))^2] = E[X^2] - [E(X)]^2$$

$$\text{Standard Deviation (Độ lệch chuẩn } \sigma) = \sqrt{\text{Var}(X)}$$

c) **Quy tắc biến đổi tuyến tính:**

$$E[aX + b] = aE(X) + b, \quad \text{Var}(aX + b) = a^2\text{Var}(X)$$

⟨Lưu ý: cộng thêm hằng số b chỉ dịch chuyển cả đám mây dữ liệu chứ không làm thay đổi độ phân tán!⟩

IV. Joint and Marginal Distributions ⟨Phân phối đồng thời & biên⟩

1. Vấn đề:

Dữ liệu thực tế luôn gồm nhiều biến đi cùng nhau. Ví dụ: Thời tiết X và việc mang ô Y . Khi có bảng xác suất chung của cả 2, làm sao để trích lại phân phối của riêng 1 biến?

2. Concept

- **Joint Distribution** $P(X, Y)$: Xác suất 2 biến đồng thời cùng xảy ra.
- **Marginal Distribution** $P(X)$: Cộng dồn (marginalize out) toàn bộ các trường hợp của biến kia:

$$P(X = x) = \sum_y P(X = x, Y = y)$$

VD: $P(\text{Nắng}) = P(\text{Nắng, mang ô}) + P(\text{Nắng, không ô})$.

⇒ **Cách kiểm tra tính độc lập**: VD $P(\text{Nắng, Mang ô}) = 0,4$.

Nếu $P(\text{Nắng}) \cdot P(\text{Mang ô}) \neq 0,4 \Rightarrow$ chứng tỏ không độc lập $\Rightarrow$ Nắng và mang ô phụ thuộc nhau.

V. Why the Normal Distribution Shows Up Everywhere

1. Concept

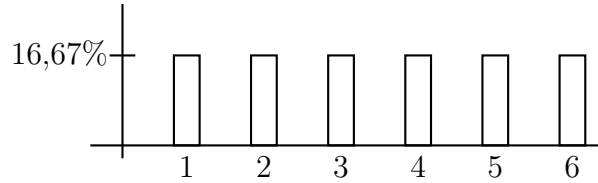
- **Central Limit Theorem (CLT — Định lý Giới hạn Trung tâm)**: Tổng hoặc trung bình cộng của nhiều biến ngẫu nhiên độc lập sẽ hội tụ về phân phối chuẩn (Gaussian), bất kể phân phối gốc ban đầu có hình dạng gì.

VD:

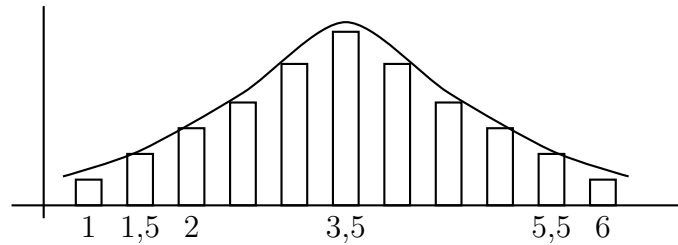
Tung 1 viên xúc xắc : phân phối đều (Uniform)

Tung 2 viên xúc xắc : phân phối hình tam giác $\triangle$

→ **Tung 1 viên** ($n = 1$): Mỗi mặt từ 1 → 6 có 1 cách ra (16,67%):



→ **Tung 2 viên** ($n = 2$): Ta đo giá trị trung bình, VD $(1, 2) \rightarrow 1,5$; $(2, 3) \rightarrow 2,5 \dots$; $(1, 6) \rightarrow 3,5$:



→ **Tung 30 viên** ($n = 30$): Khi n đủ lớn, hình dạng phân phối trung bình mẫu sẽ Gauss hóa hoàn toàn!

VI. Log Probabilities ⟨Xác suất Logarit⟩

1. Vấn đề:

- Trong NLP / LLM, xác suất của 1 câu gồm n từ là tích $P(w_1) \cdot P(w_2) \dots P(w_n) = 0,01 \times 0,003 \times 0,02 \dots$. Chỉ sau khoảng 30 từ, con số nhỏ quá mức lưu trữ dấu phẩy động của máy tính (*Floating-point Underflow*) và bị làm tròn thành 0,0.

2. Giải pháp:

- Lấy log để biến phép nhân ($\times$) thành phép cộng (+):

$$\log \left(\prod_{i=1}^n P(w_i) \right) = \sum_{i=1}^n \log P(w_i)$$

Quy tắc:

- 1) $\log(ab) = \log(a) + \log(b)$
- 2) $0 < P \leq 1 \implies \log P \text{ luôn } \leq 0$

3) Càng âm nặng $\implies$ xác suất P càng thấp.

4) Cross-Entropy Loss chính là trừ log của lớp đúng ($-\log P_{\text{true_class}}$).

a) Giả sử dạy AI nhận diện 1 bức ảnh con chó trong 3 loài [Mèo, Chó, Chim]:

- Vì đây là ảnh chó $\implies$ nhãn đúng (one-hot) là $y = [0, 1, 0]$ (0%, 100%, 0%).
- Bây giờ, mô hình AI của bạn nhìn ảnh và đoán xác suất: $p = [0,2; 0,7; 0,1]$.

$\Rightarrow$ (?) Làm sao chấm điểm phạt (loss) cho AI?

a.1) Công thức gốc của Cross-Entropy:

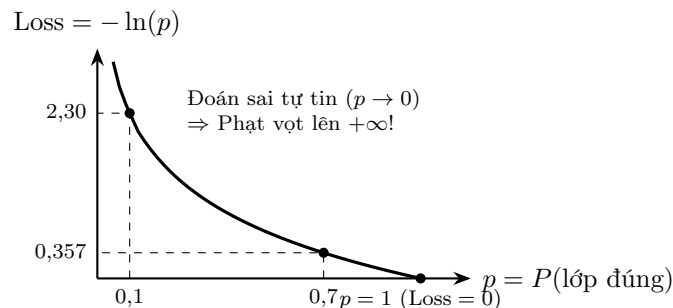
“Lấy từng nhãn thật nhân với log xác suất AI đoán, rồi đổi dấu”:

$$\text{loss} = -(0 \cdot \ln(0,2) + 1 \cdot \ln(0,7) + 0 \cdot \ln(0,1))$$

- Vì Mèo, Chim có nhãn thật = 0 nên nhân gì cũng = 0. Cuối cùng cả công thức chỉ còn lại đúng 1 số hạng của lớp đúng:

$$\text{loss} = -\ln(0,7) \approx 0,357$$

Bản chất: Loss càng gần 0 chứng tỏ xác suất AI đoán cho lớp đúng càng gần 1 (AI đoán càng đúng thì hàm loss càng nhỏ). Nếu $P_{\text{AI đoán}}$ càng nhỏ ($\rightarrow 0$) thì loss (hình phạt $-\ln P$) vọt lên càng lớn ($\rightarrow +\infty$).



VII. Softmax Function (trong vở đánh số VI)

- Khi mạng nơ-ron tính toán xong bức ảnh [Mèo, Chó, Chim], kết quả ở tầng cuối gọi là **logits** chỉ là các phép nhân cộng bình thường nên kết quả có thể âm, dương, lớn hơn 1, cộng lại không ra 100%.
- **VD:** Mạng tính ra điểm thô (logits) là: Mèo: $-1,0$ điểm; Chó: $2,0$ điểm; Chim: $1,0$ điểm.
- Ta không thể nói với người dùng hoặc đưa vào hàm $\text{loss} = -\ln p$ ở mục trước rằng xác suất con mèo là $-1,0$!

⇒ Cần cái phễu tên là **Softmax** để ép 3 con số $[-1, 2, 1]$ thành phân phối xác suất chuẩn chỉnh qua 2 bước:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

1) **Mũ hóa** (e^{z_i}) để biến hết số âm $\rightarrow$ số dương:

$$\begin{aligned} \text{Mèo: } e^{-1} &\approx 0,37 \\ \text{Chó: } e^2 &\approx 7,39 \\ \text{Chim: } e^1 &\approx 2,72 \end{aligned} \quad \Rightarrow \quad \sum 3 \text{ thành} = 10,48$$

2) **Chia cho tổng** để quy về tỷ lệ %:

$$\begin{aligned} \text{Mèo: } \frac{0,37}{10,48} &\approx 3,5\% (\approx 4\%) \\ \text{Chó: } \frac{7,39}{10,48} &\approx 70,5\% (\approx 70\%) \\ \text{Chim: } \frac{2,72}{10,48} &\approx 26,0\% (\approx 26\%) \end{aligned}$$

⇒ Đó là tác dụng của Softmax: đưa vào 1 dãy số thực bất kỳ, nó trả về 1 dãy xác suất thuộc $(0, 1)$ có tổng = 100%.

★ **TRICK: Mẹo trừ $\max(z)$ để máy tính không bị tràn số (Overflow)**

VD: Khi làm bước 1 tính e^z , nếu mạng nơ-ron tính ra điểm quá to là [999, 1000, 1001]:

- Nếu tính trực tiếp $e^{1000} \rightarrow \text{inf} / \text{NaN}$ làm sập chương trình ngay!

⇒ **Mẹo chuẩn công nghiệp:** Trước khi lấy mũ, ta lấy cả dãy trừ đi số lớn nhất $M = \max(z)$:

$$\text{Softmax}(z_i) = \frac{e^{z_i - M}}{\sum_{j=1}^K e^{z_j - M}} \quad \left(\text{với } M = \max_k z_k \right)$$

Khi đó $[999, 1000, 1001] - 1001 \rightarrow [-2, -1, 0]$, mũ lớn nhất chỉ là $e^0 = 1$, tính toán an toàn tuyệt đối mà kết quả toán học không đổi (vì nhân cả tử và mẫu với e^{-M})!

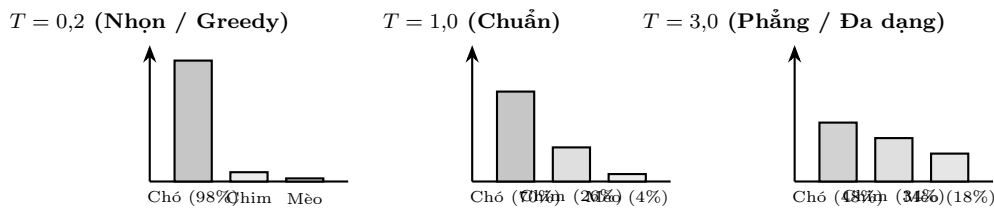
VIII. Sampling ⟨Lấy mẫu ngẫu nhiên⟩

- Là rút ra các giá trị ngẫu nhiên từ 1 phân phối xác suất (nghĩa là rút ra 1 phần tử dựa theo xác suất — phần tử nào có xác suất cao thì dễ trúng hơn).

IX. Các chiến lược Sampling trong LLM, Tổ hợp xác suất & Kiểm định giả thuyết

1. Chiến lược sinh văn bản trong LLM: Temperature, Top- k , Top- p

- **Temperature Scaling** ($T > 0$): Chia logits cho T trước khi qua Softmax: $P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$.
 - $T \rightarrow 0$: Phân phối nhọn hoắt tại cực đại $\rightarrow$ trở thành **Greedy Decoding** (luôn chọn từ có xác suất cao nhất, văn bản lặp và an toàn).
 - $T > 1$: Làm phẳng phân phối xác suất $\rightarrow$ tăng tính đa dạng, sáng tạo nhưng dễ sinh ảo giác (*hallucination*).
- **Top- k Sampling**: Chỉ giữ lại đúng k tokens có xác suất cao nhất, gán xác suất các token còn lại bằng 0, chuẩn hóa lại tổng xác suất của k tokens đó về 1 rồi mới lấy mẫu.
- **Top- p (Nucleus) Sampling**: Sắp xếp các token theo xác suất giảm dần và lấy tập nhỏ nhất các token đầu tiên sao cho **tổng xác suất tích lũy** $\geq p$ (VD $p = 0,9$), giúp số lượng ứng viên tự động co giãn linh hoạt theo độ chắc chắn của ngữ cảnh.



2. Phương pháp Biến cố đối trong Tổ hợp Xác suất (Bài toán Trùng lặp)

① **Quy tắc**: Khi bài toán hỏi xác suất xuất hiện “ít nhất 2 phần tử trùng nhau” khi chọn độc lập k lần từ N lựa chọn có hoàn lại:

$$P(\text{ít nhất 2 người trùng số}) = 1 - P(\text{cả } k \text{ người khác nhau}) = 1 - \frac{N(N-1) \dots (N-k+1)}{N^k}$$

VD: Bốn người bạn A, B, C, D ($k = 4$), mỗi người chọn ngẫu nhiên 1 số trong 5 số $\{1, 2, 3, 4, 5\}$ ($N = 5$). Xác suất để có ít nhất 2 người chọn cùng 1 số là:

- Không gian mẫu: $5^4 = 625$.
- Số cách để cả 4 người chọn 4 số khác nhau hoàn toàn: $5 \times 4 \times 3 \times 2 = 120$.
- Xác suất cần tìm:

$$P = 1 - \frac{5 \times 4 \times 3 \times 2}{5^4} = 1 - \frac{24}{125} = \frac{101}{125}$$

3. Kiểm định Giả thuyết Thống kê: Sai lầm Loại I, Loại II & Power

Trong kiểm định thống kê với **Giả thuyết không** H_0 (*Null Hypothesis* — mặc định không có hiệu ứng/khác biệt) và **Giả thuyết đối** H_1 :

Quyết định thống kê	Thực tế H_0 ĐÚNG	Thực tế H_0 SAI (H_1 Đúng)
Bác bỏ H_0 (Chọn H_1)	Sai lầm Loại I (α — FP) (Bác bỏ H_0 khi H_0 đúng)	Quyết định đúng (Power = $1 - \beta$)
Không bác bỏ H_0	Quyết định đúng ($1 - \alpha$)	Sai lầm Loại II (β — FN) (Thừa nhận H_0 khi H_0 sai)

- ★ **Phân tích sức mạnh thống kê (Statistical Power Analysis):** Nhằm xác định kích cỡ mẫu tối ưu (**optimal sample size**) cần thu thập để đảm bảo phát hiện được hiệu ứng thực sự (đạt độ mạnh $1 - \beta \geq 0,8$) với mức ý nghĩa α cho trước (thường $\alpha = 0,05$).

4. Vì sao Nhiều Gaussian sinh ra hàm MSE, còn Bernoulli sinh ra BCE?

- **Hồi quy (Regression) với giả định nhiễu Gaussian** $y_i = \hat{y}_i + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$: Khi đó $p(y_i | x_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \hat{y}_i)^2}{2\sigma^2}\right)$. Lấy Negative Log-Likelihood (NLL):

$$-\ln \prod_{i=1}^N p(y_i | x_i) = \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \text{const} \quad \Longleftrightarrow \quad \text{Tối thiểu hóa MSE!}$$

- **Phân loại nhị phân (Binary Classification)** với phân phối Bernoulli $y_i \in \{0, 1\}$, $P(y_i = 1) = \hat{y}_i$: Xác suất gộp là $P(y_i | x_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$. Lấy Negative Log-Likelihood:

$$-\ln \prod_{i=1}^N P(y_i | x_i) = -\sum_{i=1}^N [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)] \quad \Longleftrightarrow \quad \text{Tối thiểu hóa BCE!}$$

X. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 6

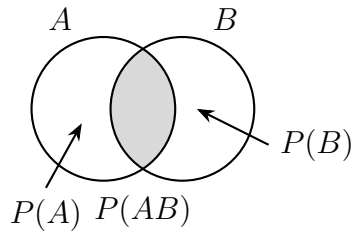
- **Kỳ vọng & Phương sai:** $E[aX + b] = aE[X] + b$; $\text{Var}(X) = E[X^2] - (E[X])^2$
và $\text{Var}(aX + b) = a^2\text{Var}(X)$ (dịch chuyển hằng số b không làm đổi độ phân tán).
- **PMF vs PDF & Định lý Giới hạn Trung tâm (CLT):** Với biến liên tục, xác suất tại 1 điểm lẻ bằng 0, phải lấy tích phân $P(a \leq X \leq b) = \int_a^b f(x) dx$. Theo CLT, tổng/trung bình của nhiều biến ngẫu nhiên độc lập luôn hội tụ về phân phối Chuẩn $\mathcal{N}(\mu, \sigma^2)$.
- **Log-Probabilities & Mẹo ổn định số học Softmax:** Lấy log biến phép nhân chuỗi xác suất thành phép cộng chống tràn số dưới (*underflow*). Khi tính Softmax, luôn trừ đi $M = \max_k z_k$ trước khi lấy mũ: $\text{Softmax}(z_i) = \frac{e^{z_i - M}}{\sum_j e^{z_j - M}}$ để chống tràn số trên (*overflow*).

Lesson 7: Bayes Theorem

I. The Concept

$$P(AB) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

$$\Rightarrow \text{Bayes' Theorem: } \boxed{P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)} = \frac{P(AB)}{P(B)}}$$



VD 1: 1 bệnh có tỷ lệ mắc trong dân số là $\frac{1}{10\,000}$:

$$\Rightarrow P(\text{sick}) = 0,0001 \quad , \quad P(\text{healthy}) = 0,9999$$

Máy xét nghiệm có độ chính xác 99%: tức nếu dương tính thật (sick) thì đoán đúng 99% ($P(+ | \text{sick}) = 0,99$) và chỉ 1% người khỏe bị báo nhầm dương tính ($P(+ | \text{healthy}) = 0,01$).

$\Rightarrow$ **Tổng xác suất dương tính** (Theo công thức xác suất đầy đủ): $P(+)=0,99$

$$\begin{aligned} P(+) &= P(\text{healthy}) \cdot P(+ | \text{healthy}) + P(\text{sick}) \cdot P(+ | \text{sick}) \\ &= 0,9999 \cdot 0,01 + 0,99 \cdot 0,0001 \\ &= 0,010098 \end{aligned}$$

$\Rightarrow$ **Xác suất thực sự bị bệnh khi đã test ra dương tính:**

$$\boxed{P(\text{sick} | +) = \frac{P(+ | \text{sick}) \cdot P(\text{sick})}{P(+)} = \frac{0,99 \times 0,0001}{0,010098} \approx 0,0098 \approx 0,98\%}$$

- Tức dù test ra + tính nhưng xác suất ốm thật chỉ có 0,98%!

$\Rightarrow$ (?) **Vì sao máy chính xác 99% mà kết quả chỉ ~ 1%?** Vì bệnh quá hiếm (Base Rate Fallacy), số người khỏe bị báo nhầm (99,99 người trên 10 000) áp đảo hoàn toàn số người bị bệnh thật (0,99 người).

Thuật ngữ cần nhớ:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

- $P(A | B)$: **Posterior** (Xác suất hậu nghiệm — sau khi có bằng chứng B)
- $P(B | A)$: **Likelihood** (Hàm hợp lý — độ khớp của bằng chứng B nếu A đúng)
- $P(A)$: **Prior** (Xác suất tiên nghiệm — niềm tin ban đầu trước khi quan sát B)
- $P(B)$: **Evidence** (Bằng chứng / Hằng số chuẩn hóa)

II. Naive Bayes, MLE and Laplace Smoothing

1. Vấn đề:

- Một email không chỉ có 1 từ “lottery” mà có hàng trăm từ $(w_1, w_2, \dots, w_n)$. Nếu tính xác suất đồng thời

$$P(w_1, w_2, \dots, w_n \mid \text{class})$$

có xét đến thứ tự phụ thuộc giữa mọi cặp từ, số lượng tham số sẽ bùng nổ theo hàm mũ $(|V|^n)$; và nếu trong tập từ đó có 1 từ mới tình chưa gặp trong lúc train, xác suất từ đó $= 0$ và nó kéo cả tích xác suất về 0.

2. The Concept

a) Giả định “Ngây thơ” (Naive Conditional Independence):

Thuật toán Naive Bayes giả định các đặc trưng **độc lập có điều kiện** với nhau khi đã biết nhãn **class**. Nhờ đó, xác suất của cả câu tách thành tích xác suất từng từ:

$$\text{score}(\text{class}) = P(\text{class}) \times \prod_{i=1}^n P(\text{feature}_i \mid \text{class})$$

- Ta bỏ qua mẫu số Evidence $P(w_1, \dots, w_n)$ vì nó giống nhau cho mọi class, chỉ cần so sánh tử số xem class nào lớn hơn.

b) Maximum Likelihood Estimation (MLE):

- Ước lượng xác suất trực tiếp = cách đếm tần suất từ tập đã train:

$$P(\text{“free”} \mid \text{spam}) = \frac{\text{count}(\text{“free”, spam})^{(1)}}{\text{total_words_in_spam}}$$

c) Laplace Smoothing (Add-1 Smoothing):

- Để chống lỗi **Zero Probability** khi gặp từ mới chưa từng xuất hiện ở 1 class (count = 0 làm toàn bộ tích = 0), ta cộng thêm 1 vào tử số và cộng kích thước từ điển $|V|$ (*vocabulary size*) vào mẫu số:

$$P(\text{word} \mid \text{class}) = \frac{\text{count}(\text{word, class}) + 1}{\text{total_words_in_class} + |V|}$$

III. Maximum A Posteriori (MAP) vs MLE và Tư duy Bayesian trong ML

1. Câu hỏi:

- Khi huấn luyện mô hình ML, thứ ta có trong tay là **data** (tập train) và thứ ta cần tìm là bộ trọng số parameters (θ hay w).

MLE (Maximum Likelihood Estimation) hỏi: “Bộ tham số θ nào làm cho data hiện tại dễ xảy ra nhất?”

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} P(\text{data} \mid \theta)$$

MAP (Maximum A Posteriori) hỏi: “Sau khi nhìn thấy data, bộ tham số θ nào có xác suất đúng cao nhất (vừa khớp data, vừa hợp lý với kiến thức nền Prior)?”

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} P(\theta \mid \text{data})$$

⁽¹⁾ count(“free”, spam) là số lần từ “free” xuất hiện trong nhóm các email được gán nhãn là spam.

⇒ **Sự khác biệt:**

$\left\{ \begin{array}{l} \text{MLE: tối đa hóa } P(\text{data} \mid \theta) \rightarrow \text{chỉ nhìn vào dữ liệu.} \\ \text{MAP: tối đa hóa } P(\theta \mid \text{data}) \rightarrow \text{kết hợp dữ liệu với niềm tin ban đầu (Prior) về tham số.} \end{array} \right.$

$$\Rightarrow \begin{array}{ccc} P(\theta \mid \text{data}) & \propto & P(\text{data} \mid \theta) \cdot P(\theta) \\ & \uparrow & \\ & \text{Proportional to (tỉ lệ thuận)} & \end{array}$$

2. Bayesian vs Frequentist: The Practical Difference

- **Frequentist (Trường phái Tần suất):** Coi tham số θ là 1 con số cố định chưa biết. Hỏi: “Nếu lặp đi lặp lại thí nghiệm nhiều lần thì chuyện gì xảy ra?”
- **Bayesian (Trường phái Bayes):** Coi tham số θ là 1 biến ngẫu nhiên có phân phối xác suất (*distribution*). Hỏi: “Dựa trên những gì đã quan sát, tôi TIN gì về tham số này?”

Aspect	Frequentist	Bayesian
Output	1 con số cố định (Point estimate)	Cả 1 dải phân phối xác suất
Uncertainty (Độ bất định)	Khoảng tin cậy (<i>Confidence Interval</i>)	Khoảng khả tín (<i>Credible Interval</i>)
Small Data	Dễ bị Overfit (học vẹt)	Prior đóng vai trò Regularization chống Overfit
Computation	Nhanh hơn (tối ưu điểm)	Chậm hơn, thường cần lấy mẫu (MCMC / Variational Inference)

3. Bài tập thực hành (Ex): Cập nhật Bayes tuần tự qua 2 lần xét nghiệm

- Gọi D : thực sự mắc bệnh; $\bar{D}$: không mắc bệnh.
- $+_1$: xét nghiệm lần 1 dương tính.

- $+_2$: xét nghiệm lần 2 dương tính.

Tóm tắt: $P(D) = 0,0001$, độ chính xác máy xét nghiệm 0,99:

$$\Rightarrow \begin{cases} P(+_1 | D) = 0,99 = P(+_2 | D) \\ P(+_1 | \bar{D}) = 0,01 = P(+_2 | \bar{D}) \end{cases}$$

a) **Xác suất mắc bệnh sau khi xét nghiệm lần 1 dương tính:**

$$\begin{aligned} P(D | +_1) &= \frac{P(+_1 | D) \cdot P(D)}{P(+_1)} \\ &= \frac{P(+_1 | D) \cdot P(D)}{P(+_1 | D) \cdot P(D) + P(+_1 | \bar{D}) \cdot P(\bar{D})} \approx 0,98\% \end{aligned}$$

b) **Xác suất mắc bệnh sau khi xét nghiệm cả 2 lần độc lập đều dương tính:**

- Xác suất hậu nghiệm sau lần 1 trở thành **xác suất tiên nghiệm mới (New Prior)** cho lần 2:

$$P_{\text{new}}(D) = P(D | +_1) = 0,98\% = 0,0098$$

Độ chính xác máy ở lần 2 vẫn là 0,99:

$$\Rightarrow \begin{cases} P(+_2 | D) = 0,99 \\ P(+_2 | \bar{D}) = 0,01 \\ P(+_2) = P(+_2 | D) \cdot P_{\text{new}}(D) + P(+_2 | \bar{D}) \cdot P_{\text{new}}(\bar{D}) \end{cases}$$

$$\Rightarrow \boxed{P(D | +_1, +_2) = \frac{P(+_2 | D) P_{\text{new}}(D)}{P(+_2)} = \frac{0,99 \cdot 0,0098}{0,99 \cdot 0,0098 + 0,01 \cdot 0,9902} \approx 49,5\%}$$

★ **Bản chất:** Ý (b) làm y hệt ý (a), chẳng qua việc xét nghiệm lần 1 dương tính đã cập nhật niềm tin tiên nghiệm $P(D)$ từ 0,01% lên 0,98% trước khi bước vào lần xét nghiệm thứ 2!

IV. Bản chất Bayes của Regularization (L1/L2) & Ma trận Nhầm lẫn (Confusion Matrix)

1. Bản chất Bayes của L2 (Ridge) và L1 (Lasso) Regularization

Xét bài toán tối ưu $\hat{w}_{\text{MAP}} = \arg \max_w [\ln P(\text{data} | w) + \ln P(w)] = \arg \min_w [-\ln P(\text{data} | w) - \ln P(w)]$:

- **Trường hợp 1 — Trọng số có Prior phân phối Chuẩn (Gaussian) $w_i \sim \mathcal{N}(0, \tau^2)$:**

$$P(w_i) \propto \exp\left(-\frac{w_i^2}{2\tau^2}\right) \Rightarrow -\ln P(w) = \frac{1}{2\tau^2} \sum_i w_i^2 + C = \lambda \|w\|_2^2 + C$$

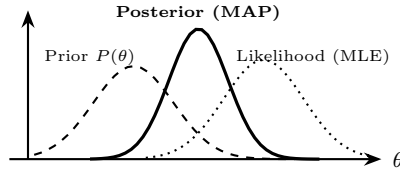
$\Rightarrow$ Chính là số hạng phạt **L2 Regularization (Ridge / Weight Decay)**!

- Trường hợp 2 — Trọng số có Prior phân phối Laplace $w_i \sim \text{Laplace}(0, b)$:

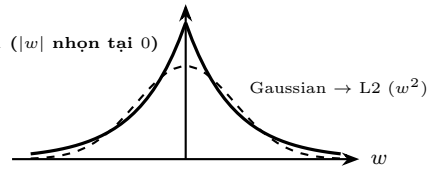
$$P(w_i) \propto \exp\left(-\frac{|w_i|}{b}\right) \Rightarrow -\ln P(w) = \frac{1}{b} \sum_i |w_i| + C = \lambda \|w\|_1 + C$$

$\Rightarrow$ Chính là số hạng phạt **L1 Regularization (Lasso — ép các trọng số dư thừa về đúng 0)**!

Cập nhật Bayes (MAP vs MLE)



Gaussian (L2) vs Laplace (L1)



2. Hệ thống Chỉ số Đánh giá Phân loại: Precision, Recall, F1-Score

Với bài toán phân loại nhị phân, gọi:

- **TP (True Positive):** Thực tế Dương tính, mô hình đoán Dương tính.
- **FP (False Positive — Báo động giả):** Thực tế Âm tính, mô hình đoán nhầm là Dương tính.
- **FN (False Negative — Bỏ sót):** Thực tế Dương tính, mô hình đoán nhầm là Âm tính.
- **TN (True Negative):** Thực tế Âm tính, mô hình đoán Âm tính.

		DỰ ĐOÁN CỦA MÔ HÌNH (Predicted)		
		Dương tính (1)	Âm tính (0)	
THỰC TẾ (Actual)	Dương tính (1)	TP (Đoán đúng +) FN (Bỏ sót - Loại II)	FP (Báo động giả - Loại I)	Recall = $\frac{TP}{TP+FN}$ (hàng +)
	Âm tính (0)	FN (Báo động giả - Loại I)	TN (Đoán đúng -)	
		Precision = $\frac{TP}{TP+FP}$ (cột +)		

- **Precision (Độ chuẩn xác)** = $\frac{TP}{TP + FP}$ (Trong số mẫu đoán Dương, bao nhiêu % là Dương thật?)
- **Recall / Sensitivity (Độ nhạy)** = $\frac{TP}{TP + FN}$ (Trong số mẫu Dương thật, bắt trúng bao nhiêu %?)
- **F1-Score** = $\frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$ (Trung bình điều hòa cân bằng giữa Precision và Recall)
- **FPR** = $\frac{FP}{FP + TN}$, **TNR (Specificity)** = $\frac{TN}{TN + FP} = 1 - \text{FPR}$

★ Nguyên tắc chọn Metric thực chiến:

- Khi cần **hạn chế tối đa việc bỏ sót mẫu dương tính thật** ($FN \rightarrow 0$, ví dụ: sàng lọc bệnh nhân nghi nhiễm bệnh truyền nhiễm, phát hiện ung thư, phát hiện gian lận tài chính): bắt buộc ưu tiên tối đa hóa **Recall**!
- Khi cần **tránh báo nhầm mẫu âm tính thành dương tính** ($FP \rightarrow 0$, ví dụ: lọc email Spam không được xóa nhầm email công việc quan trọng, gợi ý nội dung cho trẻ em): ưu tiên tối đa hóa **Precision**!

3. Tiên nghiệm Liên hợp (Conjugate Priors) & A/B Testing Bayes

- Khi phân phối Tiên nghiệm (*Prior*) và phân phối Hậu nghiệm (*Posterior*) thuộc **cùng một họ phân phối**, ta gọi đó là **Conjugate Prior** (giúp cập nhật nghiệm giải tích tức thì không cần tích phân số MCMC).
- **Cặp liên hợp Beta – Binomial kinh điển**: Nếu tỷ lệ thành công p có tiên nghiệm $\text{Beta}(\alpha, \beta)$ và ta quan sát thêm h lần thành công (ngửa) cùng t lần thất bại (sấp), phân phối hậu nghiệm cập nhật đơn giản bằng phép cộng:

$$\text{Posterior} = \text{Beta}(\alpha + h, \beta + t)$$

VD: Khởi đầu với $\text{Beta}(1, 1)$ (phân phối đều) và quan sát 7 lần ngửa, 3 lần sấp $\Rightarrow$ Hậu nghiệm là $\text{Beta}(1 + 7, 1 + 3) = \text{Beta}(8, 4)$ (kỳ vọng $\frac{8}{8+4} = \frac{2}{3}$).

V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 7

- **Định lý Bayes**: $P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$ (Posterior $\propto$ Likelihood $\times$ Prior).

Trong bài toán bệnh hiểm (*Base Rate Fallacy*), $P(A)$ cực nhỏ khiến tỷ lệ dương tính giả áp đảo dương tính thật.

- **Naive Bayes & Laplace Smoothing**: Giả định các từ độc lập có điều kiện theo nhãn: $\text{score}(c) = \log P(c) + \sum_{i=1}^n \log P(w_i | c)$, với $P(w | c) = \frac{\text{count}(w, c) + 1}{\text{total_words}_c + |V|}$ để chống lỗi nhân với 0 (*Zero Probability*).

- **MLE vs MAP**: MLE tối đa hóa $P(\text{data} | \theta)$ (dễ Overfit khi ít dữ liệu); MAP tối đa hóa $P(\text{data} | \theta)P(\theta)$, trong đó Prior Gaussian tương đương phạt **L2 (Ridge)**, Prior Laplace tương đương phạt **L1 (Lasso)**.

Lesson 8: Optimization — Gradient Descent Family

I. The Problem & The Concept

1. The Problem

- Mọi thuật toán tối ưu trong Deep Learning đều sinh ra để trả lời 1 câu hỏi:
“Làm sao chạm tới đáy thung lũng hàm Loss nhanh và đáng tin cậy nhất?”

2. The Concept

- Thuật toán cơ bản nhất:

$$w = w - lr \cdot \nabla f$$

VD: Giả sử hàm mất mát là $L(w) = w^2$ (đáy tại $w = 0$).
Ta đang đứng ở $w = 4$, chọn $lr = 0,1$:

- Đạo hàm tại $w = 4$ là gradient $= 2w = 8 > 0 \implies$ hàm đang đi lên $\implies$ phải lùi sang trái.
- Bước cập nhật: $w_{\text{new}} = w - lr \cdot \nabla f = 4 - 0,1 \cdot 8 = 3,2$.

$$\implies \text{Loss giảm từ } 4^2 = 16 \downarrow 3,2^2 = 10,24$$

II. SGD vs Batch vs Mini-batch Gradient Descent

- Các biến thể của GD khác nhau ở số lượng mẫu dữ liệu (*batch size*) được dùng để tính gradient trong mỗi bước cập nhật (*update step*):
- Batch GD:** Tính gradient trên toàn bộ tập dữ liệu rồi mới bước 1 bước:

$$1 \text{ bước} \begin{cases} \rightarrow \text{Đường đi chính xác, không nhiễu} \\ \searrow \text{Rất chậm, tốn bộ nhớ RAM/VRAM} \end{cases}$$

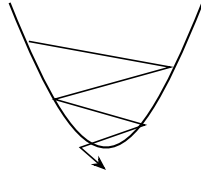
- Stochastic Gradient Descent (SGD):** Tính gradient trên 1 mẫu ngẫu nhiên duy nhất và bước ngay lập tức:

$$1 \text{ mẫu} \begin{cases} \rightarrow \text{Tốc độ mỗi bước cực nhanh} \\ \searrow \text{Đường đi nhiễu mạnh (giật cục)} \end{cases}$$

- Mini-batch GD:** Cân bằng lại — tính gradient trên 1 cụm nhỏ (32, 64, 128, ... mẫu) rồi cập nhật $\implies$ nhiễu vừa phải (giúp thoát hố nông), tốc độ tính song song trên GPU tối ưu.

III. Momentum (Quán tính)

- Vanilla GD chỉ nhìn vào đạo hàm tại thời điểm hiện tại nên khi gặp hẻm núi hẹp và dốc đứng 2 bên, nó đập qua đập lại giữa 2 vách núi và tiến về đáy rất chậm:



- Momentum (Quán tính)** khắc phục điều này bằng cách tích lũy (*accumulating*) các gradient quá khứ vào một đại lượng vận tốc v , giống như quả bóng lăn xuống dốc: hướng nào nhất quán (luôn + hoặc luôn -) thì bóng tăng tốc, hướng nào đổi chiều liên tục (dao động ngang vách núi) thì tự triệt tiêu nhau:

$$v_t = \beta v_{t-1} + g_t \quad (\text{với } g_t = \nabla f(w_{t-1}))$$

$$w_t = w_{t-1} - lr \cdot v_t$$

- Hệ số β (thường chọn 0,9) kiểm soát lượng lịch sử được giữ lại (tức lấy 0,9 vận tốc cũ cộng thêm đạo hàm mới).

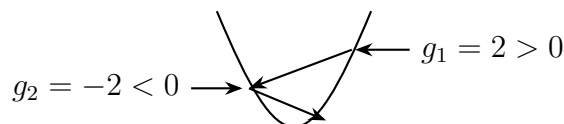
$\Rightarrow \beta$ càng lớn thì quán tính càng mạnh và quỹ đạo càng mượt, nhưng phản ứng chậm hơn khi gặp khúc cua đổi hướng gấp (vì khi đổi hướng gấp, dấu đạo hàm ngược lại mà βv_{t-1} còn lớn nên mất nhiều bước hơn để triệt tiêu).

VD: Cho $v_0 = 0$, $\beta = 0,9$, $lr = 0,1$:

- TH₁ — Đi thẳng xuống dốc:**

- Bước 1: $g_1 = 2 \rightarrow v_1 = 0,9 \cdot 0 + 2 = 2$.
- Bước 2 vẫn cùng chiều dốc: $g_2 = 2 \rightarrow v_2 = 0,9 \cdot 2 + 2 = 3,8$ (vận tốc tăng vọt từ 2 lên 3,8).

- TH₂ — Zig-zag đập trái phải:**



Bước 1: $g_1 = 2 \Rightarrow v_1 = 2$.

Bước 2: $g_2 = -2 \Rightarrow v_2 = 0,9 \cdot 2 - 2 = -0,2$ (hai chiều ngược dấu tự triệt tiêu nhau!).

IV. ADAM: Adaptive Learning Rates

- Các trọng số khác nhau trong mạng nơ-ron cần tốc độ học khác nhau:

$$\left\{ \begin{array}{l} \text{Trọng số hiếm khi nhận được đạo hàm lớn (đặc trưng thưa) cần bước} \\ \text{nhảy lớn hơn khi nó có tín hiệu.} \\ \text{Trọng số liên tục nhận đạo hàm khổng lồ cần bước nhảy nhỏ hơn} \\ \text{để tránh bị văng (overshooting).} \end{array} \right.$$

- **Adam (Adaptive Moment Estimation)** theo dõi 2 đại lượng riêng biệt cho từng trọng số tại bước thứ t :

- 1) **First moment (m_t)**: Trung bình trượt hàm mũ của gradient — giống Momentum (giữ hướng đi).
- 2) **Second moment (v_t)**: Trung bình trượt hàm mũ của bình phương gradient — đo độ lớn của đạo hàm.

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1)g_t \\ v_t &= \beta_2 \cdot v_{t-1} + (1 - \beta_2)g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \quad , \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \langle \text{Bias Correction} \rangle \\ w_t &= w_{t-1} - lr \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \end{aligned}$$

❓ Vì sao chia cho $\sqrt{\hat{v}_t}$?

- Trọng số có đạo hàm lớn $\implies g_t$ lớn $\implies \hat{v}_t$ lớn $\implies$ chia cho $\sqrt{\hat{v}_t}$ (1 số lớn) làm cho bước nhảy thực tế nhỏ lại.
- Trọng số có đạo hàm nhỏ $\implies$ chia cho $\sqrt{\hat{v}_t}$ (số nhỏ) làm bước nhảy thực tế được phóng to lên.

❓ Vì sao cần Bias Correction ($1 - \beta^t$)?

VD: Tại bước đầu $t = 1$ với $v_0 = 0$, $g_1 = 10$, $\beta_2 = 0,999$:

- Nếu không có Bias Correction:

$$v_1 = 0,999 \cdot 0 + (1 - 0,999) \cdot 10^2 = 0,1 \quad \langle \text{quá nhỏ so với } g_1^2 = 100 \text{ do bị kéo về } v_0 = 0 \rangle$$

- Nhờ có Bias Correction:

$$\hat{v}_1 = \frac{0,1}{1 - 0,999^1} = \frac{0,1}{0,001} = 100 \quad \langle \text{khôi phục chuẩn xác độ lớn } 10^2 = 100! \rangle$$

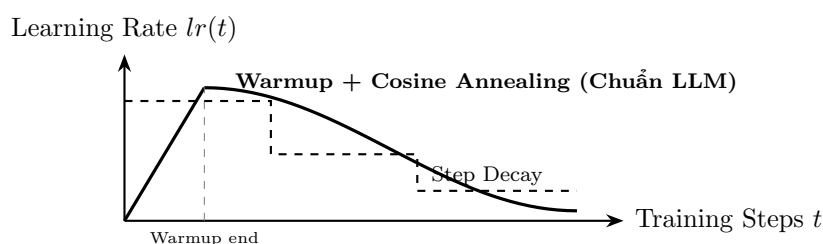
Adam's default hyperparameters:

$$lr = 0,001 \quad , \quad \beta_1 = 0,9 \quad , \quad \beta_2 = 0,999 \quad , \quad \varepsilon = 10^{-8} \quad (\varepsilon \text{ nhỏ để chống chia cho } 0)$$

V. Learning Rate Schedules ‹Lịch trình tốc độ học›

- Dùng 1 lr cố định suốt quá trình train là không tối ưu: giai đoạn đầu cần bước nhảy lớn để đi nhanh và thoát vùng trũng nông, nhưng giai đoạn cuối cần bước nhảy nhỏ để tinh chỉnh (*fine-tune*) chậm đúng đây mà không bị văng qua lại.

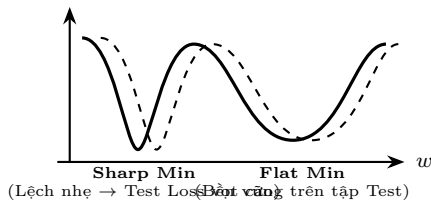
Schedule	Formula	Use case (Ứng dụng thực tế)
Step decay	$lr = lr \times \text{factor}$ sau mỗi N epochs ⁽¹⁾	Đơn giản, kiểm soát thủ công (ResNet cổ điển)
Exponential decay	$lr = lr_0 \cdot \text{decay}^t$	Giảm mượt theo hàm mũ
Cosine annealing	$lr = lr_{\min} + 0,5(lr_{\max} - lr_{\min}) \left(1 + \cos \frac{\pi t}{T}\right)$	Giảm mượt theo hình cos, chuẩn công nghiệp cho Transformer
Warmup + decay	Tăng tuyến tính từ $0 \rightarrow lr_{\max}$ trong các bước đầu rồi giảm dần	Dùng cho Large Models (LLMs), chống nổ đạo hàm ở những bước đầu khi trọng số còn ngẫu nhiên



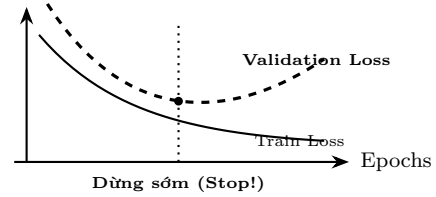
VI. Convex vs Non-Convex & Loss Landscape

- **Hàm lồi (Convex function):** VD $f(x) = x^2$ chỉ có duy nhất 1 đáy (cực tiểu cục bộ cũng chính là cực tiểu toàn cục) nên Gradient Descent luôn tìm được nghiệm tối ưu.
- **Hàm không lồi (Non-convex function):** Chính là hàm mất mát của mạng nơ-ron sâu — nằm trên không gian hàng triệu chiều, chứa vô số cực tiểu cục bộ, điểm yên ngựa (*saddle points*) và vùng bằng phẳng (*plateaus*).
- **Vì sao nhiều của Mini-batch SGD lại có lợi?** Nhiễu ngẫu nhiên giữa các mini-batch đóng vai trò “nhiệt năng” giúp mô hình bật ra khỏi các điểm yên ngựa và các hố cực tiểu nhọn (*Sharp Minima* — dễ Overfit) để rơi vào các vùng **cực tiểu phẳng (Flat Minima)** có khả năng tổng quát hóa trên tập Test tốt hơn nhiều!

Sharp vs Flat Minima



Early Stopping



(1) **Epoch**: Số lần **toàn bộ tập dữ liệu huấn luyện** được đưa qua mô hình trọn vẹn (1 vòng forward + backward trên toàn tập train). Còn **Iteration / Step** là 1 lần cập nhật trọng số trên 1 mini-batch.

VII. Các Kỹ thuật Chống Overfitting & Chiến lược Tìm kiếm Siêu tham số

1. Chẩn đoán và Xử lý Overfitting (Học vẹt)

Tình huống thực tế: Một mô hình phân loại sau khi huấn luyện đạt độ chính xác trên tập Train là 99%, nhưng trên tập Test chỉ đạt 51% $\Rightarrow$ Mô hình đang bị **Overfitting nặng** (Phương sai cao — High Variance).

① Các phương pháp GIÚP cải thiện độ chính xác trên tập Test:

- 1) **Data Augmentation (Tăng cường dữ liệu)**: Xoay, lật, cắt ảnh, thêm nhiễu nhẹ để tăng sự đa dạng của tập huấn luyện.
- 2) **Dropout (Tăng hệ số p — tỉ lệ ngẫu nhiên tắt các nút mạng trong lúc train)**: Buộc mạng nơ-ron không được phụ thuộc độc quyền vào bất kỳ nơ-ron đơn lẻ nào, hoạt động như một tổ hợp (ensemble) các mạng con. (Lưu ý: Khi suy luận / Test, phải tắt Dropout và nhân tỷ lệ bù).
- 3) **Tăng hệ số phạt Regularization λ (L1 / L2 Weight Decay)**: Ép các trọng số w phải nhỏ, làm hàm quyết định trơn mượt hơn.
- 4) **Early Stopping (Dừng sớm)**: Theo dõi Validation Loss và dừng huấn luyện ngay khi Validation Loss bắt đầu tăng trở lại dù Train Loss vẫn giảm.

★ **Bẫy thường gặp**: Giảm giá trị tham số regularization λ (hoặc giảm tỉ lệ Dropout p) sẽ làm mô hình tự do hơn nữa $\Rightarrow$ **càng làm Overfitting nặng thêm** chứ KHÔNG giúp cải thiện độ chính xác trên tập Test!

2. Tìm kiếm Siêu tham số: Grid Search vs Random Search

Tiêu chí	Grid Search (Tìm kiếm lưới)	Random Search (Tìm kiếm ngẫu nhiên)
Cơ chế	Thử toàn bộ mọi tổ hợp (combinations) trên lưới giá trị rời rạc đã định trước.	Lấy mẫu ngẫu nhiên các tổ hợp siêu tham số từ không gian phân phối xác suất.
Chi phí & Hiệu quả	Bùng nổ tổ hợp $\prod_{i=1}^d m_i$; lãng phí nhiều lần chạy vào các siêu tham số ít ảnh hưởng.	Khám phá được nhiều giá trị khác nhau hơn cho các siêu tham số quan trọng (như lr) với cùng ngân sách tính toán.

Nguyên tắc bất di bất dịch: Mọi quá trình chọn siêu tham số (*Hyperparameter Tuning*) phải đánh giá trên **Validation Set** (hoặc Cross-Validation), tuyệt đối **không được dùng Test Set để tinh chỉnh siêu tham số!**

3. Sự khác biệt giữa Adam và AdamW (Decoupled Weight Decay)

- Trong **Adam cổ điển kèm L2**, đạo hàm phạt λw_{t-1} bị cộng trực tiếp vào g_t nên bị chia cho $\sqrt{\hat{v}_t}$ (khiến các trọng số có gradient lớn lại bị phạt nhẹ đi).
- AdamW** tách riêng bước phạt trọng số (*Decoupled Weight Decay*) ra khỏi bước cập nhật thích nghi của Adam, trở thành trình tối ưu chuẩn mực để huấn luyện mọi mô hình Transformer / LLM:

$$w_t = w_{t-1} - lr \cdot \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} + \lambda w_{t-1} \right)$$

VIII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 8

- Batch vs SGD vs Mini-batch**: Mini-batch GD ($B = 32..256$) cân bằng tốc độ song song GPU và giữ lại lượng nhiễu vừa đủ để thoát hố nông, hội tụ về **Flat Minima**.
- Momentum (Quán tính)**: $v_t = \beta v_{t-1} + g_t$, $w_t = w_{t-1} - lr \cdot v_t$ giúp tăng tốc dọc đáy hẻm núi (*ravine*) và triệt tiêu dao động zig-zag hai bên vách núi.
- Adam & Bias Correction**: Kết hợp Momentum (m_t) và bình phương đạo hàm (v_t) kèm hiệu chỉnh độ lệch $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$, $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$ (để chống thiên lệch về 0 ở các bước đầu):
$$w_t = w_{t-1} - lr \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$
- Lịch trình Tốc độ học & Chống Overfit**: Dùng **Linear Warmup + Cosine Annealing** cho Transformer; chống Overfitting bằng Data Augmentation, tăng Dropout p , tăng Weight Decay λ , và Early Stopping.

Lesson 9: Information Theory

I. The Concept

1. Information Content (Lượng thông tin tự thân)

- Lượng thông tin (hay độ bất ngờ — *surprise*) của 1 sự kiện x có xác suất $p(x)$ được tính bằng công thức:

$$I(x) = -\log(p(x)) = \log\left(\frac{1}{p(x)}\right)$$

- Sử dụng $\log_2$ cho đơn vị **bits**, $\ln$ ($\log_e$) cho đơn vị **nats** — cùng 1 ý tưởng, chỉ khác đơn vị đo.
- Những sự kiện chắc chắn xảy ra tức $p(x) = 1$ mang lượng thông tin $I(x) = -\log(1) = 0$; ngược lại, 1 điều hiếm/khó xảy ra ($p(x) \ll 1$) khi xảy ra sẽ mang rất nhiều thông tin.

2. Entropy (Average Surprise) — Độ bất ngờ trung bình

a) Vấn đề:

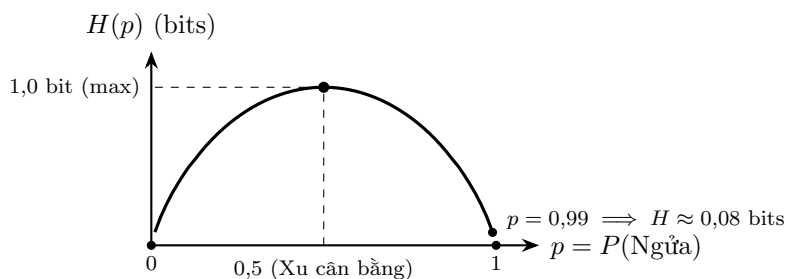
- Information content chỉ đo độ bất ngờ của 1 kết quả đơn lẻ x , nhưng output của 1 bài toán Machine Learning là cả 1 phân phối xác suất P gồm nhiều kịch bản. Ta cần 1 thước đo độ hỗn loạn / độ bất định trung bình (*uncertainty*) của toàn bộ phân phối:

$$H(P) = -\sum_x p(x) \cdot \log[p(x)] = E_{x \sim P}[I(x)]$$

$H(P)$: Shannon Entropy.

VD:

- Đồng xu cân bằng** (50%/50%): $H = -(0,5 \log_2 0,5 + 0,5 \log_2 0,5) = 1$ bit
⟨Độ bất ngờ max = 1 vì xu cân bằng hoàn toàn, không thể đoán trước ra mặt nào⟩
- Đồng xu bịp** (99% ngửa): $H = -(0,99 \log_2 0,99 + 0,01 \log_2 0,01) \approx 0,08$ bits
⟨Độ bất ngờ rất thấp vì gần như biết chắc chắn sẽ ra mặt ngửa⟩



3. Cross-Entropy (Entropy chéo)

a) Vấn đề:

- Trong bài toán phân loại, dữ liệu thật tuân theo phân phối chuẩn P , còn mô hình của mình đưa ra phân phối dự đoán Q . Làm sao đo được sai số khi dùng Q để xấp xỉ P nhằm cập nhật trọng số?

b) Cross-Entropy:

- Đo độ bất ngờ trung bình (số bits trung bình) khi dùng bộ mã tối ưu cho phân phối dự đoán Q để mã hóa các sự kiện thực chất được sinh ra từ phân phối thật P :

$$H(P, Q) = - \sum_x p(x) \log(q(x))$$

- Nếu Q khớp hoàn hảo với P ($Q = P$) thì $H(P, Q) = H(P)$ (đạt giá trị nhỏ nhất).
- Bất kỳ sự lệch pha nào giữa Q và P cũng sẽ làm $H(P, Q) > H(P)$.

★ Ở bài toán phân loại (Classification), nhãn thật P là 1 **one-hot vector** (lớp đúng có xác suất = 1, các lớp khác = 0):

- Nên $p(x) = 1$ tại đúng true_class và bằng 0 tại mọi lớp còn lại:

$$\Rightarrow \begin{aligned} H(P, Q) &= -P(\text{true_class}) \cdot \log(q(\text{true_class})) \\ &= -\log(q(\text{true_class})) \end{aligned}$$

4. KL Divergence — Độ phân kỳ Kullback-Leibler

- Kullback-Leibler Divergence đo **độ bất ngờ dôi dư ra** (số bits bị lãng phí thêm) khi dùng phân phối Q thay vì phân phối thật P :

$$D_{\text{KL}}(P \parallel Q) = \sum_x p(x) \log \left[\frac{p(x)}{q(x)} \right] = H(P, Q) - H(P)$$

$$\Rightarrow \text{Cross-Entropy} = \text{Entropy} + \text{KL Divergence}$$

$$\Leftrightarrow H(P, Q) = H(P) + D_{\text{KL}}(P \parallel Q)$$

⟨Vì $H(P)$ của nhãn thật là hằng số đối với trọng số mô hình, nên tối thiểu hóa $H(P, Q)$ hoàn toàn tương đương với tối thiểu hóa $D_{\text{KL}}(P \parallel Q)$!⟩

5. Mutual Information — Thông tin tương hỗ

- Mutual Information $I(X; Y)$ đo lường lượng thông tin mà việc biết 1 biến cung cấp cho biến còn lại:

$$\begin{aligned} I(X; Y) &= H(X) - H(X | Y) \\ &= H(X) + H(Y) - H(X, Y) \end{aligned}$$

- Nếu X, Y độc lập: $I(X; Y) = 0$.
- Nếu X, Y xác định hoàn toàn lẫn nhau: $I(X; Y) = H(X) = H(Y)$.

6. Conditional Entropy (Entropy có điều kiện)

- $H(Y | X)$ đo độ bất ngờ còn sót lại về Y sau khi bạn đã quan sát biết X :

$$H(Y | X) = H(X, Y) - H(X) \quad (\text{với } H(X, Y) = H(Y, X))$$

Lưu ý: $H(X, Y)$ ở đây là **Joint Entropy** (mục 7) của 2 biến ngẫu nhiên khác nhau X và Y , không phải Cross-Entropy $H(P, Q)$ ở mục 3 (vốn là 2 phân phối xác suất khác nhau trên cùng 1 biến ngẫu nhiên).

$$\left\{ \begin{array}{l} \text{Nếu } X \text{ xác định hoàn toàn } Y \text{ (VD: } X \text{ là độ C, } Y \text{ là độ F)} \\ \text{thì } H(Y | X) = 0 \text{ vì biết } X \text{ xóa sạch mọi sự bất định về } Y. \\ \text{Nếu } X \text{ không cho biết gì về } Y \text{ (VD: } X \text{ là tung đồng xu, } Y \text{ là thời tiết ngày mai)} \\ \text{thì } H(Y | X) = H(Y). \end{array} \right.$$

$$\implies 0 \leq H(Y | X) \leq H(Y)$$

7. Joint Entropy — Entropy đồng thời

a) **Vấn đề:** Khi ghép 2 biến X, Y thành 1 hệ thống chung $(X, Y) \implies$ tính tổng độ bất định của cả cặp biến này và hiểu được phần thông tin giao thoa giữa chúng.

$$H(X, Y) = - \sum_x \sum_y p(x, y) \log(p(x, y))$$

Tính chất: $H(X, Y) \leq H(X) + H(Y)$

- Dấu “=” xảy ra khi X, Y are mutually-exclusive
- Dấu “=” xảy ra khi và chỉ khi X, Y **độc lập** ($p(x, y) = p(x)p(y)$). Nếu chúng có chia sẻ thông tin với nhau, Joint Entropy sẽ nhỏ hơn tổng các Entropy riêng lẻ.
- Phần bị khuyết đi (phần giao nhau) chính là Mutual Information:

$$I(X; Y) = H(X) + H(Y) - H(X, Y)$$

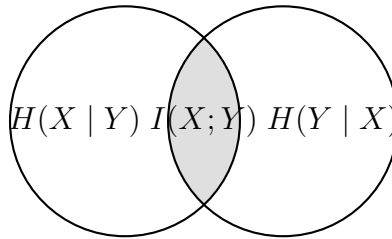
★ Các công thức, hệ thức liên hệ (Quy tắc dây chuyền của Entropy):

$$H(X, Y) = H(X) + H(Y | X) = H(Y) + H(X | Y)$$

$$I(X; Y) = H(X) - H(X | Y) = H(Y) - H(Y | X)$$

$$H(X, Y) = H(X) + H(Y) - I(X; Y)$$

Vòng trái: $H(X)$ Vòng phải: $H(Y)$



Tổng diện tích cả 2 hình tròn: $H(X, Y)$

8. Mutual Information (Deep Dive) — Đi sâu hơn với mục 5

- Mutual Information $I(X; Y)$ đo mức độ giảm sự bất định về 1 biến thông qua biến kia (và chính là KL Divergence giữa phân phối đồng thời $p(x, y)$ với tích hai phân phối biên $p(x)p(y)$):

$$\begin{aligned} I(X; Y) &= H(X) - H(X | Y) = H(X) + H(Y) - H(X, Y) \\ &= H(Y) - H(Y | X) = \sum_x \sum_y p(x, y) \log \left[\frac{p(x, y)}{p(x)p(y)} \right] \\ &= D_{\text{KL}}(P(X, Y) \parallel P(X)P(Y)) \end{aligned}$$

- Properties (Các tính chất cốt lõi):

- $I(X; Y) \geq 0$: Bạn không bao giờ mất thông tin (trung bình) khi quan sát thêm dữ kiện, hay việc biết thêm thông tin Y không bao giờ làm tăng độ bất định trung bình của X .
- $I(X; Y) = 0 \iff X, Y$ độc lập thống kê.
- $I(X; Y) = I(Y; X)$ (tính đối xứng — khác với KL Divergence vốn bất đối xứng).
- $I(X; X) = H(X)$ (tự thông tin tương hỗ chính là Entropy).

9. Label Smoothing and Cross-Entropy

a) **Vấn đề:** Phân loại tiêu chuẩn (*standard classification*) dùng **hard targets**: $[0, 0, 1, 0]$ có Entropy = 0 (mục tiêu tuyệt đối 100% khiến mô hình đẩy logits lên ∞ và trở nên quá tự tin — *overconfident*).

b) **The Concept:**

- **Label Smoothing** thay nhãn cứng = nhãn mềm (*soft targets*) để chủ động tăng nhẹ entropy của phân phối đích lên lớn hơn 0, ngăn mô hình quá tự tin:

$$\text{soft_target} = (1 - \varepsilon) \cdot \text{hard_target} + \frac{\varepsilon}{\text{num_classes}}$$

Trong đó num_classes là tổng số lớp phân loại.

VD: Với hard_target = [0, 0, 1, 0] $\implies$ num_classes = 4, chọn $\varepsilon = 0,1$:

$$\text{soft_target} = (1 - 0,1)[0, 0, 1, 0] + \frac{0,1}{4} = [0,025; 0,025; 0,925; 0,025]$$

- Hàm mất mát Cross-Entropy khi có Label Smoothing tách thành 2 thành phần:

$$L = (1 - \varepsilon) \cdot \text{CE}(\text{hard_target}, \text{prediction}) + \varepsilon \cdot H_{\text{uniform}}(\text{prediction})$$

Trong đó: CE là Cross-Entropy Loss chuẩn, còn H_{uniform} là Cross-Entropy giữa phân phối đều U và dự đoán Q (đóng vai trò phạt nếu dự đoán quá nhọn).

10. Why Cross-Entropy is the Classification Loss

a) **Góc nhìn Lý thuyết Thông tin:** Cross-Entropy đo số lượng bits bị lãng phí khi sử dụng phân phối của mô hình thay vì phân phối thực tế. Việc tối thiểu hóa nó biến mô hình thành bộ nén/mã hóa thực tế hiệu quả nhất.

b) **Góc nhìn Thống kê (Maximum Likelihood Estimation):** Đối với N mẫu huấn luyện độc lập có các lớp thực tế là y_i :

$$\text{Likelihood} = \prod_{i=1}^N q(y_i) \iff \text{Negative Log-Likelihood (NLL)} = - \sum_{i=1}^N \log(q(y_i))$$

$\Rightarrow$ Tối đa hóa hàm hợp lý (MLE) hoàn toàn tương đương với tối thiểu hóa hàm mất mát Cross-Entropy!

c) **Góc nhìn Đạo hàm (Gradient View):**

- Đạo hàm của Cross-Entropy L kết hợp với Softmax theo giá trị logits⁽¹⁾ z_i có dạng tuyến tính cực kỳ đẹp và chống bão hòa đạo hàm:

$$\frac{\partial L}{\partial z_i} = q_i - y_i = \text{predict}_i - \text{true}_i$$

⁽¹⁾ **Logits:** vectơ điểm số thô do tầng tuyến tính cuối cùng của mạng nơ-ron tạo ra trước khi đi qua hàm kích hoạt Softmax để chuẩn hóa thành xác suất.

11. Đơn vị đo thông tin: bits, nats và hartleys

$\log_2 \rightarrow$ **bits** (shannons)

$\log_e (\ln) \rightarrow$ **nats** (natural units — dùng mặc định trong PyTorch)

$\log_{10} \rightarrow$ **hartleys** (bans)

$$\left\{ \begin{array}{l} 1 \text{ nat} = \ln(e) = 1 \\ \text{Đổi cơ số: } \log_2(e) = \frac{\ln(e)}{\ln(2)} \approx 1,4427 \text{ bits} \end{array} \right. \Rightarrow \boxed{1 \text{ nat} = \frac{1}{\ln 2} \text{ bits} \approx 1,4427 \text{ bits}}$$

12. Perplexity — Độ bối rối (Thước đo chuẩn cho Language Models)

- **Perplexity (PPL)** là hàm mũ của Cross-Entropy trung bình trên mỗi token $H(P, Q) \Rightarrow$ Nó cho biết **số lượng lựa chọn hiệu dụng (branching factor)** có khả năng xảy ra ngang nhau mà mô hình ngôn ngữ đang phân vân ở mỗi bước dự đoán từ tiếp theo:

Perplexity $= 2^{H(P, Q)}$ (nếu $H(P, Q)$ tính bằng $\log_2$ — bits)

Perplexity $= e^{H(P, Q)} = \exp\left(-\frac{1}{T} \sum_{t=1}^T \ln q(w_t | w_{<t})\right)$ (nếu tính bằng $\ln$ — nats)

- **Ý nghĩa:** 1 mô hình ngôn ngữ (Language Model) có Perplexity = 50 thì xét về mặt trung bình, nó đang bối rối như thể phải tung xúc xắc đồng đều giữa 50 từ vựng tiếp theo có thể xảy ra. **Perplexity càng thấp thì mô hình ngôn ngữ càng tốt** (tối thiểu lý thuyết là PPL = 1 khi đoán chắc chắn đúng 100%).

II. Mô hình Ngôn ngữ N-grams & Các Chỉ số Đánh giá trong NLP

1. Công thức đếm số lượng n -grams trong văn bản

① **Định nghĩa:** Một n -gram là một cụm gồm n từ (hoặc n tokens) liên tiếp nhau trong câu:

- $n = 1$: **Unigram** (1 từ đơn).
- $n = 2$: **Bigram** (cặp 2 từ liên tiếp).
- $n = 3$: **Trigram** (cụm 3 từ liên tiếp).

Nếu một câu sau khi tách từ (loại bỏ dấu câu không tính là từ) có tổng cộng **N từ**, thì số lượng n -grams liên tiếp tạo thành (khi trượt cửa sổ độ dài n từ đầu câu đến cuối câu, không thêm padding) là:

$$\boxed{\text{Số lượng } n\text{-grams} = N - n + 1} \Rightarrow \boxed{\text{Số lượng Bigrams } (n = 2) = N - 1}$$

VD: Cho câu văn: “Tập đoàn Công nghiệp - Viễn thông quân đội Viettel”.
 Khi tách thành các từ đơn (bỏ dấu gạch ngang), ta có $N = 9$ từ:

[Tập, đoàn, Công, nghiệp, Viễn, thông, quân, đội, Viettel] ($N = 9$ từ)

Số lượng **bigrams** (các cặp 2 từ liên kề) tạo thành là:

$$N - 1 = 9 - 1 = 8 \text{ bigrams}$$

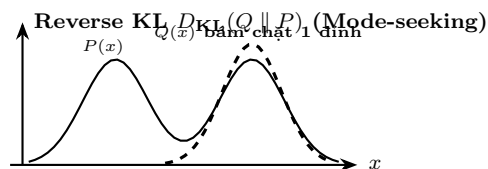
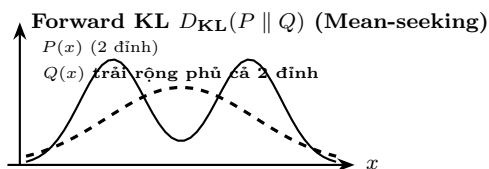
(gồm: Tập đoàn, đoàn Công, Công nghiệp, nghiệp Viễn, Viễn thông, thông quân, quân đội, đội Viettel).

2. Phân biệt các Chỉ số Đánh giá (Metrics) trong NLP

Chỉ số (Metric)	Bài toán áp dụng	Bản chất Toán học
Perplexity (PPL)	Language Modeling (Mô hình ngôn ngữ)	$PPL = e^{\text{Cross-Entropy}}$. Đo mức độ bất ngờ của mô hình khi dự đoán từ tiếp theo; càng thấp càng tốt.
BLEU (Bilingual Evaluation Understudy)	Machine Translation (Dịch máy)	Dựa trên Precision (Độ chính xác) của việc so khớp các n -grams (từ 1-gram đến 4-gram) giữa bản dịch của máy và bản dịch tham chiếu, nhân kèm hệ số phạt câu quá ngắn (<i>Brevity Penalty</i>).
ROUGE (ROUGE-1, ROUGE-2, ROUGE-L)	Text Summarization (Tóm tắt văn bản)	Tập trung vào Recall (Độ bao phủ) của các unigrams (ROUGE-1), bigrams (ROUGE-2) hoặc chuỗi con chung dài nhất (ROUGE-L) để kiểm tra bản tóm tắt có bỏ sót ý chính của văn bản gốc hay không.

3. Tính Bất đối xứng của KL Divergence & Mutual Information vs Pearson

- **Forward KL vs Reverse KL** ($D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P)$):
 - **Forward KL** $D_{\text{KL}}(P \parallel Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$ (*Mean-seeking / Zero-avoiding*): Bất cứ nơi nào $P(x) > 0$, mô hình $Q(x)$ bắt buộc phải > 0 (nếu $Q(x) \rightarrow 0$ thì $\log \frac{P}{Q} \rightarrow +\infty$), khiến Q trải rộng phủ kín mọi đỉnh của P (dùng trong Supervised Learning / MLE).
 - **Reverse KL** $D_{\text{KL}}(Q \parallel P) = \sum Q(x) \log \frac{Q(x)}{P(x)}$ (*Mode-seeking / Zero-forcing*): Bất cứ nơi nào $P(x) = 0$ thì $Q(x)$ bị ép về 0, khiến Q co cụm bám chặt vào 1 đỉnh cao nhất (mode) của P (dùng trong Variational Inference / RLHF).
- **Mutual Information $I(X; Y)$ vs Hệ số tương quan Pearson r** : Hệ số Pearson chỉ đo được mối quan hệ **tuyến tính** ($y = ax + b$), trong khi Mutual Information $I(X; Y) = D_{\text{KL}}(P(X, Y) \parallel P(X)P(Y))$ bắt được **mọi sự phụ thuộc phi tuyến** bất kỳ giữa đặc trưng X và nhãn Y .



III. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 9

- **Lượng tin & Entropy:** $I(x) = -\log p(x)$; Entropy Shannon đo độ bất định trung bình $H(P) = -\sum_x p(x) \log p(x)$ (đạt cực đại khi phân phối đều).

- **Cross-Entropy & KL Divergence:**

$$H(P, Q) = -\sum_x p(x) \log q(x) = H(P) + D_{\text{KL}}(P \parallel Q)$$

Vì $H(P)$ của nhãn thật cố định nên tối thiểu hóa Cross-Entropy hoàn toàn tương đương với tối thiểu hóa $D_{\text{KL}}(P \parallel Q)$ (và tương đương với MLE).

- **Đạo hàm Softmax + Cross-Entropy:** $\frac{\partial L}{\partial z_i} = q_i - y_i = \text{predict}_i - \text{true}_i$ (tuyến tính, chống bão hòa đạo hàm).
- **Perplexity & N-grams:** $\text{PPL} = e^{H(P, Q)}$ đo số lựa chọn từ vựng hiệu dụng mà Language Model đang phân vân; câu N từ có $N - n + 1$ cụm n -grams ($N - 1$ bigrams).

Lesson 10: Dimensionality Reduction (Giảm chiều dữ liệu)

I. The Problem

- Một tập dữ liệu có 784 features (ví dụ ảnh 28×28) nhưng phần lớn là dư thừa. **Dimensionality Reduction** giúp tìm ra bề mặt thông tin cốt lõi (*Manifold*), nén dữ liệu từ 784 chiều $\downarrow$ 2, 10 hoặc 50 chiều mà vẫn giữ được cấu trúc quan trọng nhất.

II. The Concept: Curse of Dimensionality (Lời nguyền chiều cao)

- Có 3 thứ bị phá vỡ hoàn toàn khi số chiều d tăng lên:

1. Distance becomes meaningless (Khoảng cách mất đi ý nghĩa phân biệt)

- Khoảng cách giữa 2 điểm bất kỳ ngẫu nhiên hội tụ về cùng 1 giá trị (điểm gần nhất và điểm xa nhất cách nhau gần như bằng nhau):

Dimension (d)	Avg distance ratio $\left(\frac{\text{khoảng cách max}}{\text{khoảng cách min}} \right)$
2	$\sim 5,0$
10	$\sim 1,8$
100	$\sim 1,2$
1000	$\sim 1,02$

2. Volume concentrates in corners (Thể tích dồn hết ra các góc và vỏ ngoài)

- 1 khối siêu lập phương đơn vị trong không gian d chiều có 2^d góc; ở 100 chiều, gần như toàn bộ thể tích nằm ở phần rìa và các góc xa tâm, khiến các điểm dữ liệu dạt hết ra biên và mô hình bị “đói” dữ liệu ở bên trong.

3. You need exponentially more data (Cần lượng dữ liệu tăng theo hàm mũ)

- Để duy trì cùng mật độ mẫu trong không gian khi tăng từ 2D $\rightarrow$ 20D, cần tăng số lượng dữ liệu lên gấp n^{18} lần (với VD 2D cần n^2 mẫu thì 20D cần n^{20} mẫu).

$\Rightarrow$ Giảm chiều giúp đưa mật độ dữ liệu về mức cô đọng có thể huấn luyện được.

III. PCA: Find the Directions That Matter

- **Principal Component Analysis (PCA)** tìm các trục trực giao mà dọc theo đó dữ liệu biến thiên mạnh nhất: nó xoay hệ trục tọa độ sao cho trục đầu tiên (PC_1) nắm giữ nhiều phương sai nhất, trục thứ 2 (PC_2) vuông góc với trục 1 và nắm giữ phương sai nhiều thứ nhì, và cứ thế.
- **Algorithm (Thuật toán 5 bước chuẩn):**
 - **Bước 1 — Center the data:** Đưa dữ liệu về tâm O (trừ đi giá trị trung bình μ của từng đặc trưng: $X_c = X - \mu$).
 - **Bước 2 — Compute covariance:** Tính ma trận hiệp phương sai $\Sigma = \frac{1}{n-1} X_c^T X_c$.
 - **Bước 3 — Eigendecomposition:** Phân rã trị riêng $\Sigma v_i = \lambda_i v_i$ (tìm các hướng chính).
 - **Bước 4 — Sort by eigenvalue:** Sắp xếp các cặp (λ_i, v_i) giảm dần theo trị riêng $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$ (phương sai lớn nhất xếp trước).
 - **Bước 5 — Project:** Chiếu dữ liệu lên ma trận $W_k = [v_1, \dots, v_k]$ gồm top k eigenvectors: $Z = X_c @ W_k$, bỏ phần còn lại.

❓ Vì sao trục có phương sai lớn nhất lại chính là Vectơ riêng của Σ ?

Phương sai của dữ liệu X_c khi chiếu lên hướng đơn vị u ($\|u\|^2 = u^T u = 1$) là $\text{Var}(X_c u) = u^T \Sigma u$. Dùng nhân tử Lagrange để tối đa hóa $u^T \Sigma u$ với ràng buộc $u^T u = 1$:

$$\mathcal{L}(u, \lambda) = u^T \Sigma u - \lambda(u^T u - 1) \implies \frac{\partial \mathcal{L}}{\partial u} = 2\Sigma u - 2\lambda u = 0 \iff \boxed{\Sigma u = \lambda u}$$

Khi đó phương sai đạt được trên trục u là $u^T \Sigma u = u^T (\lambda u) = \lambda(u^T u) = \lambda$ (chính là trị riêng λ !).

IV. t-SNE⁽¹⁾: Bảo toàn Láng giềng Cục bộ

a) Nguyên lý hoạt động:

- Ánh xạ dữ liệu nhiều chiều $\downarrow$ 2D, 3D trong khi vẫn giữ những điểm nào nằm gần nhau: những điểm từng là hàng xóm trong không gian 784 chiều vẫn là hàng xóm trong mặt phẳng 2D.
- Trong không gian nhiều chiều ban đầu, t-SNE chuyển khoảng cách Euclid giữa các cặp điểm thành 1 phân phối xác suất có điều kiện (theo phân phối Gaussian), sau đó tìm cách sắp xếp các điểm trong không gian 2D (theo phân phối Student- t bậc tự do 1 có đuôi nặng để chống hiện tượng dồn cục — *crowding problem*) sao cho 2 phân phối xác suất khớp nhau nhất (tối thiểu hóa KL Divergence).

$\Rightarrow$ Nói là phân phối xác suất nhưng thực chất nó là 1 ma trận vuông kích thước $n \times n$, trong đó $P_{j|i}$ (trong vở viết P_{ij}) đo độ thân thiết giữa điểm i và điểm j :

$$P_{j|i} = \frac{\exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|x_i - x_k\|^2}{2\sigma_i^2}\right)}$$

	A	B	C	D	
A	0	0,7000	0,2999	0,0001	→ Tổng hàng = 1
B	0,6500	0	0,3499	0,0001	→ Tổng hàng = 1
C	.	.	0	.	
D	.	.	.	0	

Gọi mỗi hàng là 1 phân phối xác suất của các láng giềng quanh điểm đó.

b) 5 đặc tính quan trọng của t-SNE:

- 1) **Phi tuyến (Non-linear):** Có khả năng trải phẳng các đa tạp (*manifolds*)⁽²⁾ xoắn phức tạp mà PCA tuyến tính bó tay.
- 2) **Ngẫu nhiên (Stochastic):** Hàm mục tiêu của t-SNE là non-convex và khởi tạo ngẫu nhiên, nên mỗi lần chạy sẽ cho ra 1 hình vẽ xoay/lật khác nhau.
- 3) **Siêu tham số Perplexity (thường từ 5 đến 50):** Điều chỉnh số lượng láng giềng hiệu dụng mà mỗi điểm cần quan tâm.
- 4) **Khoảng cách giữa các cụm xa nhau là vô nghĩa:** Nó chỉ bảo toàn cấu trúc cục bộ, không phản ánh khoảng cách toàn cục thật sự trong không gian gốc.
- 5) **Độ phức tạp tính toán cao & Không có ánh xạ tham số:** $O(n^2)$ (hoặc $O(n \log n)$ với Barnes-Hut), và vì t-SNE tối ưu trực tiếp tọa độ 2D chứ không học ma trận chiếu W , ta **tuyệt đối KHÔNG dùng t-SNE làm bước tiền xử lý giảm chiều trước khi huấn luyện bộ phân loại (Classifier)** (vì không thể chiếu mẫu Test mới và khoảng cách toàn cục đã bị bóp méo).

⁽¹⁾ **t-SNE:** t-Distributed Stochastic Neighbor Embedding.

⁽²⁾ **Manifold (Đa tạp):** là 1 không gian hình học tổng thể bị uốn cong, cuộn tròn trong không gian nhiều chiều, nhưng khi zoom lại cực gần quanh 1 điểm bất kỳ thì phẳng lì (tương tự bề mặt Trái Đất 3D khi đứng gần trông như mặt phẳng 2D).

V. Thuật toán UMAP

UMAP (*Uniform Manifold Approximation & Projection*) hoạt động tương tự t-SNE nhưng vượt trội hơn ở:

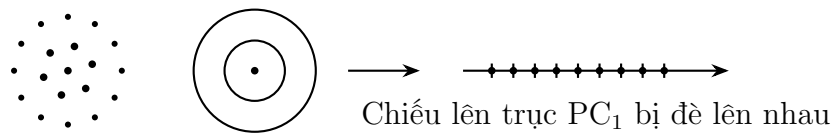
- **Nhanh hơn rất nhiều:** Sử dụng đồ thị láng giềng gần nhất xấp xỉ (*Approximate Nearest Neighbors*) thay vì tính khoảng cách từng cặp $O(n^2)$ trên toàn bộ tập dữ liệu.
- **Bảo toàn cấu trúc toàn cục tốt hơn:** Vị trí tương đối giữa các cụm (*clusters*) trên output của UMAP mang nhiều ý nghĩa hình học thực tế hơn so với t-SNE nhờ hàm mục tiêu Cross-Entropy mờ (*Fuzzy Simplicial Set Cross-Entropy*) phạt cả những điểm ở xa bị kéo lại gần.
- **Ý nghĩa 2 siêu tham số chính:**
 - **n_neighbors:** Số lượng láng giềng (tương tự **perplexity** của t-SNE; nhỏ → chi tiết cục bộ, lớn → bức tranh toàn cục).
 - **min_dist:** Kiểm soát mức độ co cụm chặt chẽ của các điểm trên không gian đầu ra (**min_dist** nhỏ tạo các cụm đặc khít).

VI. When to Use Which ⟨Khi nào dùng thuật toán nào?⟩

- **PCA:** Tiền xử lý giảm chiều trước khi đưa vào mô hình huấn luyện (nén đặc trưng, khử đa cộng tuyến, tăng tốc ML) $\implies$ **bảo toàn phương sai và cấu trúc tuyến tính toàn cục.**
- **t-SNE:** Cần vẽ biểu đồ trực quan hóa 2D/3D cho tập dữ liệu vừa và nhỏ ($< 10\,000$ mẫu) để quan sát tách cụm cục bộ $\implies$ **bảo toàn láng giềng cục bộ.**
- **UMAP:** Trực quan hóa 2D/3D cho dữ liệu lớn, vừa muốn giữ cấu trúc cụm cục bộ vừa muốn giữ tương quan vị trí toàn cục.

VII. Kernel PCA

- PCA tiêu chuẩn chỉ tìm được các không gian con tuyến tính (*linear subspaces*): nó chỉ xoay hệ trục tọa độ và bỏ bớt trục, nên nếu dữ liệu nằm trên 1 đa tạp phi tuyến (như 2 đường tròn đồng tâm 2D), mọi đường chiếu thẳng $\downarrow$ 1D đều làm 2 đường tròn đè lên nhau:



- **Kernel PCA:** Áp dụng **Kernel Trick** để ngầm ánh xạ dữ liệu $\phi(x)$ lên 1 không gian đặc trưng nhiều chiều hơn (nơi dữ liệu trở nên phân tách tuyến tính) thông qua ma trận nhân $K \in \mathbb{R}^{n \times n}$ với $K_{ij} = k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ mà **không cần tính trực tiếp tọa độ $\phi(x)$** trong không gian đó.
- Các hàm nhân (*Kernel functions*) phổ biến:
-) **RBF (Gaussian) Kernel:**

$$k(x, y) = \exp(-\gamma \|x - y\|^2)$$

Tốt cho hầu hết dữ liệu phi tuyến và đa tạp trơn (*Smooth Manifold* — đa tạp không có góc nhọn hay gấp khúc, khả vi vô hạn lần).

-) **Polynomial Kernel:**

$$k(x, y) = (x \cdot y + c)^d \quad \langle \text{Phù hợp cho các quan hệ đa thức bậc } d \rangle$$

-) **Sigmoid Kernel:**

$$k(x, y) = \tanh(\alpha x \cdot y + c)$$

Bảng so sánh Kernel PCA và Standard PCA:

Phương pháp	Kích thước Ma trận & Big-O	Khả năng Tái tạo (Reconstruction)
Kernel PCA	Ma trận $K : (n \times n) \rightarrow O(n^2)$ lưu trữ, $O(n^3)$ phân rã	Không có biến đổi ngược trực tiếp, phải học thêm hàm xấp xỉ nghịch ảnh (<i>pre-image</i>)
Standard PCA	Ma trận $\Sigma : (d \times d) \rightarrow O(d^2)$ lưu trữ, $O(nd^2 + d^3)$	Có biến đổi ngược chính xác về không gian gốc ($\hat{X} = ZW_k^T + \mu$)

Trong đó: d là số chiều gốc ban đầu của dữ liệu, n là tổng số mẫu dữ liệu (*sample size*).

VIII. Reconstruction Error ⟨Sai số Tái tạo⟩

- Khi nén dữ liệu từ $d = 784$ D xuống $k = 50$ D bằng ma trận $W_k \in \mathbb{R}^{d \times k}$, làm sao đo lường thông tin bị mất đi?

⇒ Tái tạo dữ liệu (*reconstruct*) về lại d chiều và tính sai số trung bình bình phương:

- 1) **Chiếu $\downarrow k$ chiều (Encode):** $X_{\text{reduced}} = X_{\text{centered}} \cdot W_k \quad (n \times k)$
- 2) **Tái tạo lại d chiều (Decode):** $\hat{X}_{\text{centered}} = X_{\text{reduced}} \cdot W_k^T \quad (n \times d)$
- 3) **Tính sai số tái tạo:** $\text{MSE} = \frac{1}{n} \|X_{\text{centered}} - \hat{X}_{\text{centered}}\|_F^2$

Lưu ý quan trọng:

1. Công thức tính Sai số Tái tạo và Tỷ lệ Phương sai Giữ lại

- Tổng phương sai bị mất đi (Reconstruction Error) trên toàn bộ các chiều bằng đúng **tổng các trị riêng bị loại bỏ** (từ thứ $k+1 \rightarrow d$):

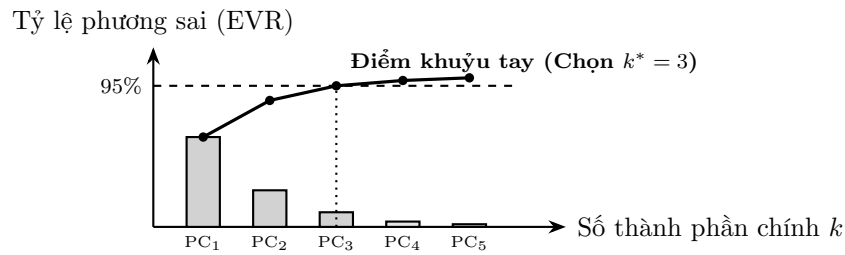
$$\text{Reconstruction Error} = \sum_{i=k+1}^d \lambda_i$$

- **Tỷ lệ phương sai giải thích tích lũy (Cumulative Explained Variance Ratio — EVR):** Tỷ lệ % thông tin được k thành phần chính đầu tiên giữ lại:

$$\text{Cumulative EVR}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

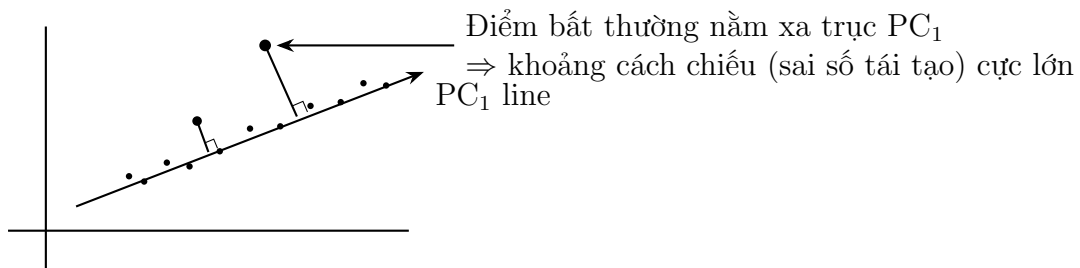
- **Tỷ lệ thông tin mất mát (Fraction of Variance Lost):**

$$\text{Fraction Lost} = \frac{\sum_{i=k+1}^d \lambda_i}{\sum_{i=1}^d \lambda_i} = 1 - \text{Cumulative EVR}(k)$$



2. Ứng dụng Phát hiện Bất thường (Anomaly Detection)

- Nếu huấn luyện PCA (hoặc Autoencoder) trên tập dữ liệu bình thường, các mẫu bình thường nằm sát mặt phẳng con k chiều nên khi chiếu xuống rồi tái tạo lại sẽ có **sai số tái tạo (Reconstruction Error) rất thấp**.
- Ngược lại, các điểm dị biệt / gian lận (*anomalies*) không tuân theo quy luật tương quan chung sẽ nằm lệch hẳn khỏi mặt phẳng PC₁, nên khi chiếu xuống rồi tái tạo lại sẽ có **Reconstruction Error cực cao**:



IX. Các Kỹ thuật Chuẩn hóa Dữ liệu (Feature Scaling), Chống Rò rỉ Dữ liệu & K-Means / Kernel SVM

1. So sánh 4 Kỹ thuật Chuẩn hóa Đặc trưng (Feature Scaling)

Kỹ thuật	Công thức Toán học	Đặc điểm & Phạm vi đầu ra
Min-Max Normalization	$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$	Ép toàn bộ đặc trưng về đúng đoạn $[0, 1]$. Nhạy cảm với điểm ngoại lai (<i>outliers</i>).
Z-score Standardization	$x' = \frac{x - \mu}{\sigma}$	Đưa dữ liệu về trung bình $\mu = 0$, độ lệch chuẩn $\sigma = 1$ (không bị giới hạn trong $[0, 1]$). Bắt buộc cho PCA, SVM, Logistic Regression.
Robust Scaling	$x' = \frac{x - \text{Median}}{\text{IQR}}$	Với $\text{IQR} = Q_3 - Q_1$. Dùng Trung vị và Khoảng tứ phân vị nên chống chịu cực tốt với Outliers .
Log Transformation	$x' = \ln(1 + x)$	Thu hẹp các phân phối lệch phải nặng (<i>right-skewed</i> như thu nhập, giá nhà, lượt xem).

2. Chống Rò rỉ Dữ liệu (Data Leakage) & Chuỗi thời gian

- Quy tắc chuẩn đối với Tập Test:

- Phân phối của tập Test phải phản ánh trung thực phân phối dữ liệu thực tế khi triển khai (*production distribution*). Tuyệt đối không đưa tập Test vào huấn luyện hay dùng tập Test để chọn siêu tham số (*hyperparameter tuning*).
- Mọi tham số tiền xử lý (như μ, σ của Z-score, $x_{\min}, x_{\max}$ của Min-Max, ma trận W_k của PCA, hay giá trị mean/median để điền dữ liệu khuyết thiếu — *imputation*) **chỉ được tính (fit) trên riêng tập Train**, sau đó mới áp dụng (**transform**) sang tập Validation và tập Test! Tính mean/median trên toàn bộ dữ liệu trước khi chia Train/Test sẽ gây **Data Leakage**.

- Quy tắc riêng cho Dữ liệu Chuỗi thời gian (Time Series):

- Bắt buộc **chia tập Train và Test theo khung thời gian tách biệt** (quá khứ làm Train, tương lai làm Test); tuyệt đối không chia ngẫu nhiên (*random split*) vì sẽ làm lộ thông tin tương lai vào quá khứ.
- Khi dữ liệu mục tiêu thay đổi phân phối theo thời gian gọi là **Concept Drift** (còn phân phối đầu vào $P(X)$ thay đổi gọi là **Data / Covariate Drift**).

3. Học không giám sát với K-Means & Bản chất Kernel SVM

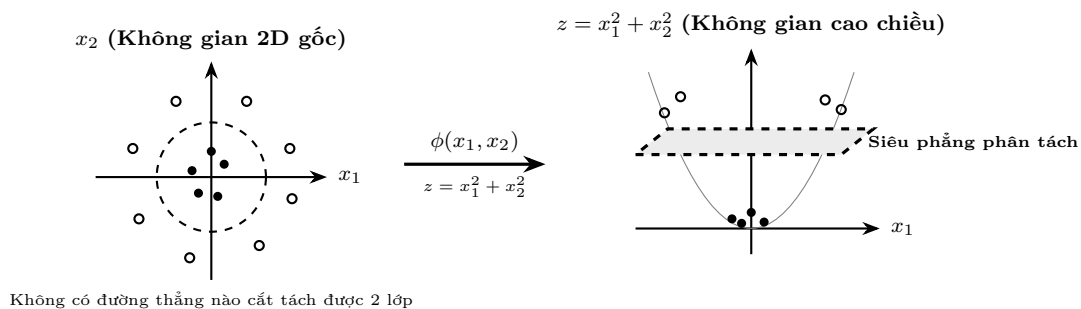
- Học không giám sát (Unsupervised Learning — làm việc hoàn toàn với dữ liệu không nhãn): Thuật toán **K-Means Clustering** lặp lại 2 bước (gán cụm gần nhất và

cập nhật tâm cụm bằng trung bình cộng) cho đến khi đạt **1 trong 3 điều kiện dừng**:

- 1) Các điểm dữ liệu không còn thay đổi cụm được gán sau vòng lặp.
- 2) Các tâm cụm (*centroids*) không thay đổi vị trí (hoặc độ dịch chuyển $< \varepsilon$) sau vòng lặp.
- 3) Quá trình huấn luyện chạm đến số vòng lặp tối đa (`max_iter`).

(Lưu ý: Số cụm K là siêu tham số cố định truyền vào từ đầu nên “ K không đổi” không phải là điều kiện dừng!).

- **Kernel SVM (Support Vector Machine với hàm nhân)**: Kết hợp cả 2 bản chất:
 - (1) Tìm hàm ánh xạ dữ liệu từ không gian đặc trưng ban đầu sang không gian mới có chiều cao hơn, nhờ đó (2) Phân loại được 2 lớp dữ liệu vốn hoàn toàn **không khả tách tuyến tính (non-linearly separable)** ở không gian gốc:



X. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 10

- **Lời nguyền Chiều cao (Curse of Dimensionality)**: Khi $d \uparrow$, tỷ số khoảng cách $\frac{d_{\max}}{d_{\min}} \rightarrow 1$ (mất ý nghĩa phân biệt), thể tích dồn hết ra 2^d góc rìa, và lượng mẫu cần thiết tăng theo hàm mũ.
- **Thuật toán PCA & EVR**: Đưa dữ liệu về tâm ($X_c = X - \mu$), phân rã $\Sigma = \frac{1}{n-1} X_c^T X_c$, chiếu $Z = X_c W_k$ và tái tạo $\hat{X} = Z W_k^T + \mu$. Tỷ lệ thông tin giữ lại:

$$\text{EVR}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i}$$

 ; sai số tái tạo $= \sum_{i=k+1}^d \lambda_i$ (dùng phát hiện bất thường — *Anomaly Detection*).
- **PCA vs t-SNE vs UMAP**: PCA tuyến tính dùng làm bước tiền xử lý ML; t-SNE ($O(n^2)$, bảo toàn láng giềng cục bộ) và UMAP (nhanh, giữ cả cấu trúc toàn cục) chỉ dùng để trực quan hóa 2D/3D.
- **Chuẩn hóa & Chống Data Leakage**: Min-Max ép về $[0, 1]$, Z-score đưa về $(\mu = 0, \sigma = 1)$, Robust Scaling dùng IQR chống Outliers. Mọi phép chuẩn hóa/imputation/PCA chỉ được *fit* trên tập Train!

Lesson 11: Singular Value Decomposition (SVD)

I. The Problem

- Có ma trận dữ liệu 1000×2000 cần nén, khử nhiễu, tìm giả nghịch đảo ... Công cụ Eigendecomposition (Lesson 3) bó tay vì đây không phải ma trận vuông; mà kể cả ma trận vuông cũng không phải lúc nào cũng có đủ bộ vectơ riêng độc lập tuyến tính.

$\Rightarrow$ SVD (**Phân rã Giá trị Kỳ dị**) giải quyết triệt để vì hoạt động trên **mọi ma trận, mọi shape** ($m \times n$), **mọi rank**, không cần bất kỳ điều kiện ràng buộc nào!

II. The Concept

1. Nó làm gì về mặt hình học?

- Mọi phép biến đổi tuyến tính A bất kỳ đều có thể phân tách thành chuỗi 3 thao tác hình học cơ bản:

$$\text{Xoay } (V^T) \longrightarrow \text{Co giãn dọc trục } (\Sigma) \longrightarrow \text{Xoay } (U)$$

Công thức phân rã tổng quát:

$$A = U \Sigma V^T$$

Và khi nhân với 1 vectơ x đầu vào ($Ax = U \Sigma V^T x$), thứ tự tác động từ **phải** $\rightarrow$ **trái**:

- V^T ($n \times n$): Thực hiện phép xoay (hoặc phản xạ) các vectơ trong không gian đầu vào $\mathbb{R}^n$ để căn chỉnh trùng với các trục chính đầu vào.
- Σ ($m \times n$): Thực hiện phép co giãn (kéo / nén) dọc theo từng trục tương ứng với **giá trị kỳ dị** σ_i nằm trên đường chéo chính $\Rightarrow$ biến 1 mặt cầu đơn vị thành hình elipxôit (*hyper-ellipsoid*).
- U ($m \times m$): Xoay kết quả sang hướng định vị cuối cùng trong không gian đầu ra $\mathbb{R}^m$.

2. Phân rã SVD đầy đủ (Full SVD)

$$\begin{matrix} A & = & U & \Sigma & V^T \\ (m \times n) & & (m \times m) & (m \times n) & (n \times n) \end{matrix} \implies A^T = V \Sigma^T U^T$$

- $U \in \mathbb{R}^{m \times m}$: **Orthogonal matrix** thỏa $U^T \cdot U = U \cdot U^T = I$. Các cột của U là các vectơ riêng trực chuẩn của AA^T , được gọi là **Left Singular Vectors**.
- $\Sigma \in \mathbb{R}^{m \times n}$: Ma trận đường chéo hình chữ nhật kích thước $m \times n$; các phần tử trên đường chéo chính $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ ($r = \text{rank}(A) \leq \min(m, n)$) được gọi là **Singular Values** (Các giá trị kỳ dị):

$$\begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}_{(m > n)} \quad \text{hoặc} \quad \begin{bmatrix} \sigma_1 & 0 & 0 & 0 \\ 0 & \sigma_2 & 0 & 0 \end{bmatrix}_{(m < n)}$$

với $\sigma_i = \sqrt{\lambda_i}$, trong đó λ_i là trị riêng khác 0 của ma trận đối xứng bán xác định dương $A^T A$ (và cũng là trị riêng của AA^T).

- $V \in \mathbb{R}^{n \times n}$: **Orthogonal matrix** thỏa $V^T \cdot V = V \cdot V^T = I$. Các cột của V là các vectơ riêng trực chuẩn của $A^T A$, được gọi là **Right Singular Vectors**.

3. Ý nghĩa của Left Singular Vectors, Singular Values, Right Singular Vectors

Từ $A = U \Sigma V^T$, nhân hai vế với V (vì $V^T V = I$) ta có hệ thức ghép cặp:

$$Av_i = \sigma_i u_i$$

- **Right Singular Vectors** (v_i — các cột của V): Tạo 1 hệ cơ sở trực chuẩn cho không gian đầu vào $\mathbb{R}^n$.
- **Singular Values** (σ_i — đường chéo của Σ): Là hệ số phóng đại cho biết ma trận A kéo dãn vectơ dọc theo hướng v_i bao nhiêu lần ($\sigma_i = 0$ nghĩa là bóp hướng đó xẹp về 0).
- **Left Singular Vectors** (u_i — các cột của U): Tạo hệ cơ sở trực chuẩn cho không gian đầu ra $\mathbb{R}^m$, chỉ ra hướng đích trong không gian đầu ra mà vectơ v_i đáp xuống sau khi bị biến đổi bởi A .

4. Outer Product Form (Dạng tổng các tích ngoài hạng 1)

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T$$

Vì $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0 \implies$ ma trận hạng 1 đầu tiên ($\sigma_1 u_1 v_1^T$) nắm giữ cấu trúc năng lượng quan trọng nhất của A , số hạng thứ hai ($\sigma_2 u_2 v_2^T$) nắm giữ phần quan trọng thứ nhì, ...

5. Truncated SVD (Xấp xỉ hạng thấp tối ưu — Định lý Eckart-Young-Mirsky)

- Khi chỉ giữ lại k giá trị kỳ dị lớn nhất ($k < r$, với $r = \text{rank}(A)$), ta thu được ma trận xấp xỉ hạng k tốt nhất:

$$A_k = U_k \Sigma_k V_k^T = \sum_{i=1}^k \sigma_i u_i v_i^T \quad \langle u_i, v_i \text{ là vectơ cột, } \sigma_i \text{ là hằng số vô hướng} \rangle$$

a) Công thức tính Sai số Xấp xỉ và Tỷ lệ Năng lượng Giữ lại:

- 1) Theo chuẩn phổ (Spectral Norm / Operator 2-Norm):

$$\|A - A_k\|_2 = \sigma_{k+1} \quad \langle \text{Bằng đúng giá trị kỳ dị lớn nhất bị cắt bỏ!} \rangle$$

2) Theo chuẩn Frobenius (Frobenius Norm):

$$\|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \sigma_{k+2}^2 + \dots + \sigma_r^2} \quad (\text{Căn bậc hai tổng } \sigma_i^2 \text{ bị cắt})$$

3) Tỷ lệ phương sai / năng lượng giữ lại (Energy Retained):

$$\text{Energy Retained}(k) = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$$

4) Tỷ lệ nén dung lượng (Image Compression Ratio): Thay vì lưu $m \times n$ số của bức ảnh gốc A , Truncated SVD hạng k chỉ cần lưu k cột của U_k (mk số), k giá trị σ_i (k số), và k hàng của V_k^T (nk số):

$$\text{Số phần tử lưu trữ} = k(m + n + 1) \implies \text{Tỷ lệ dung lượng} = \frac{k(m + n + 1)}{m \cdot n}$$

VD: Ma trận A có các giá trị kỳ dị $[10, 8, 4, 3, 0]$. Nếu xấp xỉ ở hạng $k = 2$ (giữ 2 giá trị lớn nhất là 10 và 8):

- Sai số chuẩn phổ: $\|A - A_2\|_2 = \sigma_3 = 4$.
- Sai số chuẩn Frobenius: $\|A - A_2\|_F = \sqrt{4^2 + 3^2} = \sqrt{25} = 5$.
- Tỷ lệ năng lượng giữ lại: $\frac{10^2 + 8^2}{10^2 + 8^2 + 4^2 + 3^2} = \frac{164}{189} \approx 86,77\%$.

6. Pseudoinverse via SVD (Ma trận Giả nghịch đảo Moore-Penrose)

- Nghịch đảo thường A^{-1} chỉ $\exists$ khi ma trận A vuông và $\det(A) \neq 0$.
- Với ma trận **không vuông** hoặc **suy biến (thiếu rank)**, ta dùng ma trận giả nghịch đảo A^+ :

$$A^+ = V\Sigma^+U^T$$

★ Cách lập Σ^+ ($n \times m$) từ Σ ($m \times n$):

- **Bước 1:** Chuyển vị ma trận Σ từ $(m \times n) \rightarrow (n \times m)$.
- **Bước 2:** Nghịch đảo tất cả các giá trị kỳ dị khác 0 trên đường chéo: thay $\sigma_i \rightarrow \frac{1}{\sigma_i}$.
- **Bước 3:** Giữ nguyên tất cả các số 0.

Ứng dụng giải hệ phương trình vô nghiệm (Bình phương tối thiểu):

$$Ax = b \quad \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ 6 \end{bmatrix} \iff \begin{cases} x_1 + x_2 = 3 \\ 2x_1 + x_2 = 5 \\ 3x_1 + x_2 = 6 \end{cases}$$

2 phương trình đầu cho ra $x_1 = 2, x_2 = 1$ nhưng thay vào phương trình 3 thì $3(2) + 1 = 7 \neq 6$.

- Vì không có nghiệm thỏa mãn cả 3 phương trình $\implies$ mục tiêu là tìm x sao cho sai số bình phương nhỏ nhất: $\min_x \|Ax - b\|_2^2$:

$$\boxed{x^* = A^+b = V\Sigma^+U^Tb} \quad \text{là nghiệm tối ưu (và có chuẩn } \|x^*\|_2 \text{ nhỏ nhất).}$$

7. Mối liên hệ sống còn giữa SVD và PCA

- Trong thực tế công nghiệp (`scikit-learn`), **PCA được tính trực tiếp bằng SVD** trên ma trận dữ liệu đã chuẩn hóa trung tâm về 0 ($X = U\Sigma V^T$) thay vì tính ma trận hiệp phương sai $X^T X$:

$$C = \frac{1}{n-1} X^T X = \frac{1}{n-1} (V\Sigma^T U^T)(U\Sigma V^T) = \boxed{\frac{1}{n-1} V\Sigma^2 V^T}$$

- Các cột của V chính là các trục thành phần chính (Principal Components W), và trị riêng của ma trận hiệp phương sai liên hệ với giá trị kỳ dị qua:

$$\boxed{\lambda_i = \frac{\sigma_i^2}{n-1}}$$

- **❓ Vì sao tính PCA qua SVD ổn định số học hơn qua $X^T X$?** Vì hệ số điều kiện $\kappa(X) = \frac{\sigma_{\max}}{\sigma_{\min}}$ khi nhân $X^T X$ sẽ bị **bình phương**: $\kappa(X^T X) = \kappa(X)^2$ (VD $\kappa(X) = 10^8 \implies \kappa(X^T X) = 10^{16}$ gây mất sạch độ chính xác dấu phẩy động 64-bit!).

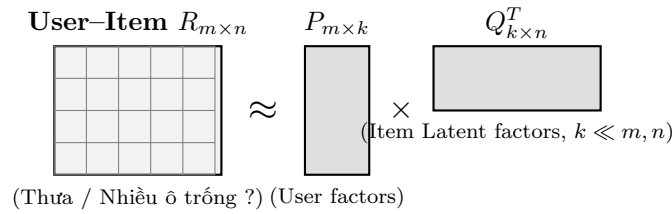
III. Matrix Factorization trong Hệ Gợi ý (Recommender Systems) & Toán học trong CNN / Transformer

1. Matrix Factorization trong Hệ thống Gợi ý & LSA trong NLP

① **Bản chất và Mục đích:** Trong các hệ thống gợi ý (như Netflix, Shopee, TikTok), ma trận tương tác User-Item $R \in \mathbb{R}^{m \times n}$ rất lớn và thưa. Phương pháp **Matrix Factorization** (dựa trên Truncated SVD / ALS) phân rã R thành tích của 2 ma trận hạng thấp $k \ll \min(m, n)$:

$$\boxed{R_{(m \times n)} \approx P_{(m \times k)} @ Q_{(k \times n)}^T \iff \hat{r}_{u,i} = \vec{p}_u^T \vec{q}_i}$$

- **Mục đích cốt lõi: Giảm chiều dữ liệu (Dimensionality Reduction) và học các đặc trưng ẩn (Latent Features)** của người dùng ($\vec{p}_u \in \mathbb{R}^k$) và sản phẩm ($\vec{q}_i \in \mathbb{R}^k$), từ đó dự đoán chính xác điểm đánh giá cho các ô còn trống!
- **Latent Semantic Analysis (LSA) trong NLP & Khử nhiễu (Noise Reduction):**
Áp dụng Truncated SVD lên ma trận Term-Document giúp gom các từ đồng nghĩa vào cùng không gian chủ đề ẩn (k chiều), đồng thời loại bỏ các giá trị kỳ dị nhỏ ($\sigma_{k+1}, \dots, \sigma_r$ đại diện cho nhiễu ngẫu nhiên).



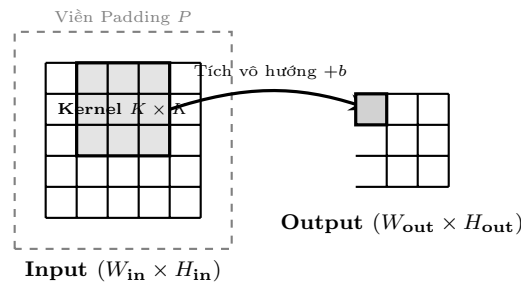
2. Toán học tính Kích thước Đầu ra của CNN & Khối 1×1 Conv

① Công thức tính kích thước Feature Map đầu ra của lớp Conv2D / Pooling:

Cho đầu vào có kích thước $(W_{\text{in}} \times H_{\text{in}} \times C_{\text{in}})$, đi qua bộ lọc gồm **F** kernels có kích thước $(K \times K \times C_{\text{in}})$, bước trượt **Stride** = S , và đệm viền **Padding** = P :

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1, \quad H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K + 2P}{S} \right\rfloor + 1, \quad C_{\text{out}} = F$$

- Để giữ nguyên kích thước không gian ($W_{\text{out}} = W_{\text{in}}, H_{\text{out}} = H_{\text{in}}$) với $S = 1$: ta dùng **Same Padding** $P = \frac{K-1}{2}$ hoặc dùng phép tích chập 1×1 convolution ($K = 1, P = 0, S = 1$).



VD: Đầu vào có kích thước $32 \times 32 \times 3$ (rộng 32, cao 32, 3 kênh màu), sử dụng $F = 10$ kernels kích thước 5×5 , stride $S = 1$, không có padding ($P = 0$):

$$W_{\text{out}} = H_{\text{out}} = \frac{32 - 5 + 2(0)}{1} + 1 = 28, \quad C_{\text{out}} = 10 \implies \text{Kích thước đầu ra: } 28 \times 28 \times 10$$

② 3 Mục đích cốt lõi của khối 1×1 Convolution (Pointwise Conv trong Inception / ResNet):

- 1) Tổng hợp (tổ hợp tuyến tính) các đặc trưng theo chiều sâu (cross-channel pooling).
- 2) Giảm (hoặc tăng) số lượng kênh (feature maps / bottleneck) để giảm mạnh chi phí tính toán trước các lớp $3 \times 3, 5 \times 5$ đắt đỏ.
- 3) Giúp mô hình học được các đặc trưng phi tuyến phức tạp hơn khi đi kèm hàm kích hoạt (như ReLU) sau lớp 1×1 conv.

3. Độ phức tạp của Self-Attention & Cơ chế PagedAttention

- **Self-Attention:** Với chuỗi đầu vào gồm N tokens và chiều ẩn d :

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{Q @ K^T}{\sqrt{d}} \right) @ V$$

$\begin{matrix} Q & @ & K^T \\ (N \times d) & & (d \times N) \end{matrix}$
 $\begin{matrix} @ \\ (N \times d) \end{matrix}$

Vì phải nhân $Q @ K^T$ để tạo ma trận tương quan $(N \times N)$ giữa mọi cặp token, độ phức tạp thời gian và bộ nhớ của Self-Attention theo độ dài chuỗi N là $O(N^2)$.

- **Cơ chế PagedAttention (trong vLLM):** Lấy cảm hứng từ bộ nhớ ảo phân trang (*Virtual Memory Paging*) của hệ điều hành, chia bộ nhớ đệm **KV Cache** thành các **block có kích thước cố định** và dùng **Block Table** để ánh xạ chỉ mục logic của KV Cache sang các vị trí bộ nhớ GPU vật lý không cần nằm liên tiếp nhau theo thời gian thực, giúp triệt tiêu hiện tượng phân mảnh bộ nhớ GPU khi phục vụ nhiều luồng request có độ dài khác nhau!

IV. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 11

- **Phân rã SVD Tổng quát:** Mọi ma trận $A \in \mathbb{R}^{m \times n}$ đều phân rã được thành

$$A = U \Sigma V^T = \sum_{i=1}^r \sigma_i u_i v_i^T \quad (\text{Xoay } V^T \rightarrow \text{Co giãn } \Sigma \rightarrow \text{Xoay } U), \text{ với } \sigma_i = \sqrt{\lambda_i(A^T A)}.$$

- **Xấp xỉ Hạng thấp Tối ưu (Truncated SVD A_k):** Giữ k giá trị kỳ dị lớn nhất cho sai số $\|A - A_k\|_2 = \sigma_{k+1}$, $\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}$, và tỷ lệ năng lượng giữ lại $\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$.

- **Giải nghịch đảo Moore-Penrose & PCA qua SVD:** $A^+ = V \Sigma^+ U^T$ giải hệ bình phương tối thiểu $x^* = A^+ b$ cho mọi ma trận; tính PCA qua SVD của X_c ($\lambda_i = \frac{\sigma_i^2}{n-1}$) giúp tránh bình phương hệ số điều kiện $\kappa(X^T X) = \kappa(X)^2$.

- **CNN & Transformer:** Kích thước đầu ra CNN: $W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1$;
độ phức tạp Self-Attention theo độ dài chuỗi N là $O(N^2)$.

Lesson 12: Tensor Operations

I. Vấn đề (The Problem)

Khi xây dựng một mô hình Transformer hay mạng CNN thực tế, luồng tính toán trông rất gọn trên giấy nhưng khi chạy lại lập tức báo lỗi:

- `RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x768 and 512x768)`
- Bạn thử thêm `.transpose(1, 2)`, rồi gọi `.view(B, T, -1)` thì gặp ngay lỗi tiếp theo: `RuntimeError: view size is not compatible with input tensor's size and stride`
- Nguy hiểm hơn: có những lỗi **không hề vắng Exception** mà âm thầm cộng nhăm hai tensor theo cơ chế Broadcasting dọc sai chiều, làm mô hình vẫn hội tụ ra số nhưng dự đoán toàn rác hoặc nổ bộ nhớ VRAM lên hàng chục GB.

❓ Vì sao ma trận 2 chiều (2D Matrix) là không đủ cho Deep Learning?

- Ma trận 2D chỉ biểu diễn được quan hệ cặp giữa 2 tập đối tượng (VD: Batch $\times$ Features).
- Dữ liệu thực tế luôn có cấu trúc nhiều chiều hơn: một lô 32 ảnh màu RGB kích thước 224×224 là mảng 4 chiều (32, 3, 224, 224); cơ chế Multi-Head Self-Attention với 12 đầu (heads) xử lý chuỗi 128 token là mảng 4 chiều $(B, H, T, d_k) = (16, 12, 128, 64)$.
- Để làm chủ mọi kiến trúc Deep Learning hiện đại mà không bao giờ bị kẹt ở lỗi Shape hay Stride, ta phải hiểu cấu trúc vật lý của **Tensor** từ tầng bộ nhớ RAM 1D cho tới ký hiệu rút gọn **Einstein Summation** (`einsum`).

II. Bản chất Tensor & Quy ước Shape trong AI

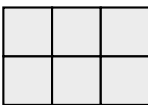
1. Định nghĩa Tensor theo Bậc (Rank / Order)

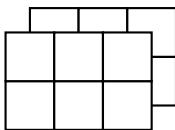
① Khái niệm:

- **Tensor** là mảng đa chiều chứa các con số có cùng kiểu dữ liệu (`dtype`), được đặc trưng bởi **Bậc (Rank / Order — số trục ndim)**, **Kích thước (shape)** và **Bước nhảy bộ nhớ (strides)**.
- **Lưu ý thuật ngữ:** Trong lập trình Deep Learning, “Rank của Tensor” là **số chiều (số trục axis/dim)**, hoàn toàn khác với “Rank của ma trận” (số cột độc lập tuyến tính ở Lesson 1).


0D Scalar
Rank 0
shape: ()


1D Vector
Rank 1
shape: (3,)


2D Matrix
Rank 2
shape: (2, 3)


3D Tensor
Rank 3
shape: (2, 2, 3)

Tổng số phần tử (numel) = $\prod_{k=0}^{r-1} \text{shape}[k]$

 VD: shape (2, 3, 4) $\implies 2 \times 3 \times 4 = 24$ phần tử.

2. Quy ước Shape chuẩn trong các bài toán Deep Learning

Lĩnh vực / Tầng	Ký hiệu Shape	Ý nghĩa từng trục (Axis / Dimension)
Dữ liệu bảng / MLP	(B, D)	B : kích thước lô (batch_size), D : số đặc trưng (features).
Chuỗi / NLP / Audio	(B, T, D)	B : batch, T : độ dài chuỗi (seq_len), D : chiều nhúng (embed_dim).
Thị giác (PyTorch)	(B, C, H, W) [NCHW]	C : kênh màu (RGB = 3) đứng trước chiều cao H , rộng W (Channels-first).
Thị giác (TF / Khác)	(B, H, W, C) [NHWC]	Kênh màu C đứng cuối (Channels-last). Nhầm NCHW với NHWC gây lỗi ngầm nghiêm trọng!
Multi-Head Attention	(B, H, T, d_k)	B : batch, H : số đầu chú ý (num_heads), T : độ dài chuỗi, d_k : chiều mỗi đầu ($d_{\text{model}} = H \cdot d_k$).

III. Cấu trúc Bộ nhớ Vật lý 1D & Bước nhảy (Strides)

1. Ánh xạ Tensor nhiều chiều xuống RAM 1 chiều

① Bản chất vật lý (Vì sao cần Stride & Offset?):

Trong toán học, ta hình dung Ma trận 2D là một cái bảng (hàng, cột) và Tensor 3D là một khối hộp (như khối Rubik). Nhưng về mặt vật lý, **thanh RAM hay VRAM trên GPU không hề có “hàng”, “cột” hay “khối 3D”** — nó chỉ là một dải ô nhớ 1 chiều xếp thành 1 hàng ngang duy nhất (1D flat buffer), đánh số từ 0, 1, 2, ...

Khi lưu ma trận $A = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$ có shape = (2, 3) vào RAM, máy tính buộc phải “đuối thẳng” (flatten) thành 1 hàng ngang theo 2 chuẩn:

- **Row-major (Ưu tiên hàng — chuẩn C / PyTorch / NumPy mặc định):** Xếp hết **hàng 0** rồi mới tới **hàng 1** (các phần tử cùng hàng nằm sát nhau).

RAM 1D: [10, 20, 30, 40, 50, 60] (chỉ số từ 0 đến 5)

- **Column-major (Ưu tiên cột — chuẩn Fortran / Julia / MATLAB):** Xếp hết **cột 0** rồi mới tới **cột 1**, **cột 2** (các phần tử cùng cột nằm sát nhau).

RAM 1D: [10, 40, 20, 50, 30, 60]

⇒ **Bài toán:** Khi ta gọi $A[1, 2]$ (hàng 1, cột 2 — số 60), làm sao máy tính biết số 60 đang nằm ở ô thứ mấy trong dải RAM 1D? Đó là lúc cần **Stride (Bước nhảy)** và **Offset (Vị trí thực tế)**.

② Công thức Bước nhảy (Strides) và Địa chỉ Ô nhớ (Memory Offset):

Hãy hiểu đơn giản: Vector stride = $(s_0, s_1, \dots, s_{r-1})$ là số ô nhớ trên dải RAM 1D mà ta phải nhảy qua khi muốn nhích thêm 1 đơn vị ở trục thứ k .

- Với một tensor liên tục chuẩn Row-major (Contiguous) có shape = $(d_0, d_1, \dots, d_{r-1})$:

$$s_{r-1} = 1 \quad , \quad s_k = s_{k+1} \cdot d_{k+1} = \prod_{m=k+1}^{r-1} d_m$$

Mẹo tính nhẩm bằng tay (luôn tính ngược từ phải sang trái):

- Chiều cuối cùng (bên phải nhất) luôn có stride = 1:** Vì các phần tử ở chiều cuối nằm sát vách nhau trong RAM, nhích 1 bước chỉ tốn 1 ô nhớ.
 - Stride của chiều đứng trước = (Stride chiều ngay sau nó) × (Shape chiều ngay sau nó):** Muốn nhích 1 đơn vị ở trục lớn, ta phải nhảy qua toàn bộ các phần tử của trục nhỏ bên trong nó.
- Vị trí thực tế (offset) trên mảng 1D của phần tử có tọa độ $(i_0, i_1, \dots, i_{r-1})$ chỉ là phép nhân tọa độ với bước nhảy tương ứng:

$$\text{offset}(i_0, i_1, \dots, i_{r-1}) = \text{storage_offset} + \sum_{k=0}^{r-1} i_k \cdot \text{stride}[k]$$

(Trong đó *storage_offset* là vị trí bắt đầu của Tensor trong dải RAM; mặc định khi tạo mới thì *storage_offset* = 0).

VD 1: Tính Strides và Offset cho Tensor 2D và 3D

- Trường hợp 2D:** Cho ma trận $A = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$ có shape = $(d_0, d_1) = (2, 3)$, lưu trên RAM 1D là $[10, 20, 30, 40, 50, 60]$. Giả sử đang đứng ở $A[0, 0] = 10$ (ô số 0):

– **Bước 1 (Tính strides từ phải sang trái):**

- * Sang cột kế tiếp (s_1): Từ 10 sang 20 nằm sát nhau trong RAM ⇒ bước 1 ô ($s_1 = 1$).
- * Xuống hàng tiếp theo (s_0): Từ 10 (hàng 0) xuống 40 (hàng 1) phải nhảy qua trọn vẹn 3 cột của hàng đầu ⇒ bước $s_0 = s_1 \times d_1 = 1 \times 3 = 3$ ô.
- * Kết quả: $\boxed{\text{stride} = (3, 1)}$.

– **Bước 2 (Tìm vị trí $A[1, 2] = 60$):** Đi tới hàng 1 (nhảy $1 \times 3 = 3$ ô) + đi tới cột 2 (nhảy thêm $2 \times 1 = 2$ ô):

$$\text{offset}(1, 2) = 0 + 1 \times 3 + 2 \times 1 = \boxed{5} \quad (\text{ô chỉ số 5 trên RAM 1D, đúng bằng số 60}).$$

- Trường hợp 3D:** Cho Tensor T có shape = $(d_0, d_1, d_2) = (2, 3, 4)$ lưu liên tục (*storage_offset* = 0). Hãy hình dung T như một tòa nhà có $d_0 = 2$ tầng, mỗi tầng có $d_1 = 3$ hàng, mỗi hàng có $d_2 = 4$ cột (phòng):

– **Bước 1 (Tính strides từ phải sang trái):**

- * *Trục cuối* (s_2 — cột/phòng): Sang phòng bên cạnh chỉ bước **1 ô** $\implies s_2 = 1$.
- * *Trục giữa* (s_1 — hàng): Xuống 1 hàng phải nhảy qua hết 4 phòng của hàng hiện tại $\implies s_1 = s_2 \times d_2 = 1 \times 4 = 4$.
- * *Trục đầu* (s_0 — tầng): Xuống 1 tầng phải nhảy qua hết 3 hàng của tầng hiện tại (mỗi hàng 4 ô) $\implies s_0 = s_1 \times d_1 = 4 \times 3 = 12$.

$$\implies \boxed{\text{stride} = (12, 4, 1)}$$

– **Bước 2 (Tìm vị trí phần tử $T[1, 2, 3]$ — tầng 1, hàng 2, cột 3 trên mảng 24 số):**

$$\text{offset}(1, 2, 3) = \underbrace{1 \times 12}_{\text{qua 1 tầng}} + \underbrace{2 \times 4}_{\text{qua 2 hàng}} + \underbrace{3 \times 1}_{\text{qua 3 cột}} = 12 + 8 + 3 = \boxed{23}$$

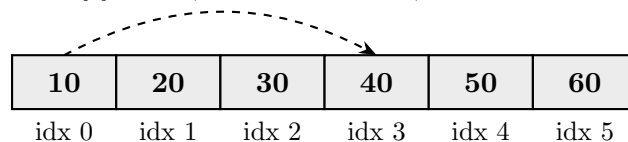
2. Phép biến đổi Zero-Copy (View) vs Sao chép Vật lý (Copy)

① Vì sao `transpose()` không tốn thời gian copy dữ liệu?

Khi gọi $B = A.\text{transpose}(0, 1)$ trên ma trận A có shape = (2, 3) và stride = (3, 1), PyTorch giữ nguyên vùng nhớ 1D [10, 20, 30, 40, 50, 60] và chỉ hoán đổi metadata:

$$A : \begin{cases} \text{shape} = (2, 3) \\ \text{stride} = (3, 1) \end{cases} \xrightarrow{. \text{transpose}(0, 1)} B = A^T : \begin{cases} \text{shape} = (3, 2) \\ \text{stride} = (1, 3) \end{cases} \quad \langle \text{Zero-copy } O(1) \rangle$$

Bộ nhớ RAM 1D vật lý (dùng chung cho cả A và A^T):
 $\text{stride}[1] = +3$ (nhảy cột trong A^T)



Góc nhìn Logic A (2×3)
 $\text{stride} = (3, 1) \implies$ **Contiguous**

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{bmatrix}$$

Góc nhìn Logic A^T (3×2)
 $\text{stride} = (1, 3) \implies$ **Non-contiguous**

$$\begin{bmatrix} 10 & 40 \\ 20 & 50 \\ 30 & 60 \end{bmatrix}$$

② Vì sao gọi `A.transpose(0, 1).view(6)` lập tức báo lỗi `RuntimeError`?

- Khi duyệt A^T theo thứ tự hàng (Row-major): ta cần chuỗi [10, 40, 20, 50, 30, 60].
- Nhưng trên RAM 1D, dữ liệu vẫn đang nằm theo thứ tự [10, 20, 30, 40, 50, 60]. Khoảng cách từ 10 $\rightarrow$ 40 là +3, nhưng từ 40 $\rightarrow$ 20 lại là -2 (bước nhảy không đều, **Non-contiguous!**).
- Do đó không tồn tại bộ **stride** đơn lẻ nào cho tensor 1D (6,) đọc đúng thứ tự của A^T mà không chép lại RAM.

WRONG vs RIGHT KHI DÙNG VIEW / RESHAPE

- (⊗) Sai: `x.transpose(1, 2).view(B, T, -1)` → Lỗi `RuntimeError` vì sau `transpose`, tensor bị mất tính liên tục (`is_contiguous() == False`).
- (✓) Đúng 1 (Trường minh): `x.transpose(1, 2).contiguous().view(B, T, -1)` → Hàm `.contiguous()` cấp phát vùng nhớ mới và sắp xếp lại các phần tử đúng thứ tự hàng trước khi `view`.
- (✓) Đúng 2 (Tự động): `x.transpose(1, 2).reshape(B, T, -1)` → Trả về `view` nếu liên tục, hoặc tự động gọi `.contiguous().view()` nếu không liên tục.

② Bảng phân loại các phép toán Zero-copy (View) và Copy:

Phép toán	Loại bộ nhớ	Cơ chế thay đổi Metadata (Shape & Strides)
<code>view(new_shape)</code>	Zero-copy	Giữ nguyên RAM, tính lại <code>stride</code> . Đòi hỏi tensor phải <code>contiguous</code> .
<code>transpose(d0, d1), permute(...)</code>	Zero-copy	Hoán vị trực tiếp các phần tử trong tuple <code>shape</code> và tuple <code>stride</code> . Làm tensor trở thành <code>non-contiguous</code> .
<code>squeeze()</code> , <code>unsqueeze(d)</code>	Zero-copy	Xóa hoặc chèn chiều có kích thước = 1 vào <code>shape</code> và <code>stride</code> (không chạm tới RAM).
<code>expand(...)</code> , <code>Broadcasting</code>	Zero-copy	Đặt <code>stride[k] = 0</code> tại chiều có kích thước = 1 được kéo dài thành N (đọc lặp lại cùng 1 ô nhớ N lần mà không tốn thêm RAM).
<code>contiguous()</code> , <code>clone()</code>	Copy ($O(N)$)	Cấp phát bộ nhớ vật lý mới trên RAM/VRAM và chép toàn bộ phần tử sang vùng nhớ mới.

IV. Thủ thuật `stride = 0` trong Broadcasting & Nhân Ma trận theo Lô

1. Cơ chế `stride = 0` và Quy tắc Căn lề Phải của Broadcasting

① Tại sao Broadcasting không tốn thêm bộ nhớ?

Khi cộng ma trận Z có shape $(2, 3)$ với vector b có shape $(3,)$:

- **Bước 1 (Pad 1 bên trái):** b có ít chiều hơn nên được chèn chiều 1 vào bên trái: $(3,) \rightarrow (1, 3)$ với `stride = (3, 1)`.
- **Bước 2 (Kéo dài ảo bằng `stride = 0`):** Chiều đầu của b kéo từ $1 \rightarrow 2$ bằng cách gán `stride[0] = 0`:

$$b_{\text{broadcast}} : \quad \text{shape} = (2, 3), \quad \boxed{\text{stride} = (0, 1)}$$

- Khi truy xuất $b_{\text{broadcast}}[i, j]$, công thức địa chỉ cho:

$$\text{offset}(i, j) = i \times \underbrace{0}_{\text{stride}[0]} + j \times 1 = j$$

$\Rightarrow$ Mọi hàng $i = 0, 1, \dots$ đều tự động trở về đúng phần tử thứ j của vector b gốc mà không nhân bản dữ liệu!

VD 2: Kiểm tra tương thích Broadcasting 4D

Cho Tensor A có shape $(8, 1, 6, 1)$ và Tensor B có shape $(7, 1, 5)$. Tìm shape của $A + B$:

$$\begin{array}{rcl} \text{Tensor } A : & (& 8, \quad 1, \quad 6, \quad 1 \quad) \\ \text{Tensor } B \text{ (đệm 1 bên trái)} : & (& 1, \quad 7, \quad 1, \quad 5 \quad) \\ \hline \text{Kết quả } \max(d_A, d_B) : & (& 8, \quad 7, \quad 6, \quad 5 \quad) \quad \checkmark \end{array}$$

2. Nhân Ma trận theo Lô (Batched MatMul — bmm / @)

① Nguyên tắc cốt lõi:

Máy tính **không bao giờ** nhân trực tiếp hai khối 3D hay 4D với nhau — nó **chỉ biết nhân hai Ma trận 2D cơ bản**: $\underset{(M \times K)}{A} @ \underset{(K \times N)}{B} = \underset{(M \times N)}{C}$ (hai số K ở giữa phải bằng nhau và tự triệt tiêu).

Các chiều đứng ở phía trước (như Batch, Head) thực chất chỉ là “**số lượng cặp ma trận 2D**” được xếp chồng lên nhau để GPU nhân song song cùng một lúc!

② Hai công cụ nhân theo lô trong PyTorch:

- Hàm `torch.bmm(A, B)` (Cứng nhắc — chỉ nhận đúng Tensor 3D):

Tưởng tượng A và B là hai chồng sách cùng có B cuốn. Máy tính bắt cặp từng cuốn sách thứ b của hai chồng rồi nhân ma trận 2D $(M \times K)$ với $(K \times N)$:

$$\underset{(B \times M \times K)}{A} @ \underset{(B \times K \times N)}{B} = \underset{(B \times M \times N)}{C} \quad \text{với} \quad C[b, i, j] = \sum_{k=0}^{K-1} A[b, i, k] \cdot B[b, k, j]$$

(Hạn chế: Nếu đưa Tensor 4D hoặc hai bên khác kích thước lô B vào, `bmm` sẽ báo lỗi ngay).

- Toán tử tổng quát $A @ B$ (`torch.matmul` — Linh hoạt cho mọi số chiều):

Mẹo đọc dấu `@` trong 3 giây: Hãy lấy tay che các chiều phía trước lại, chỉ nhìn vào **ĐÚNG 2 TRỰC CUỐI CÙNG**:

1. **2 trực cuối cùng**: Luôn thực hiện phép nhân ma trận 2D $(..., M, K) @ (..., K, N) \rightarrow (..., M, N)$.
2. **Toàn bộ các trục đứng trước (batch dimensions)**: Tự động bắt cặp và tuân theo quy tắc **Broadcasting** (nếu một bên thiếu chiều Batch hoặc có chiều Batch = 1, nó tự động nhân bản ảo qua `stride = 0` cho khớp!).

VD 3: Giải chi tiết phép nhân Tensor 4D và 3D bằng quy tắc “Che phần đầu, nhân 2 số cuối”

- Trường hợp 1 (Nhân hai Tensor 4D trong Multi-Head Attention):

$$\underbrace{(16, 12, 128, 64)}_{\text{Batch}}^Q @ \underbrace{(16, 12, 64, 128)}_{\text{Batch}}^{K^T} \longrightarrow \text{Shape kết quả: } \boxed{(16, 12, 128, 128)}$$

- Các trục đầu (Batch): Cả hai đều có (16, 12) (16 mẫu, mỗi mẫu 12 đầu chú ý $\implies$ 192 cặp ma trận) $\longrightarrow$ **Giữ nguyên** (16, 12).
- 2 trục cuối (Nhân ma trận 2D): $(128 \times 64) @ (64 \times 128) \longrightarrow$ hai số **64** ở giữa triệt tiêu, còn (128, 128).

- Trường hợp 2 (Nhân Tensor 3D với Ma trận trọng số 2D — có Broadcasting):

$$\underbrace{(16, 128, 768)}_{\text{Batch}}^X @ (768, 3072)^W \longrightarrow \text{Shape kết quả: } \boxed{(16, 128, 3072)}$$

- Ý nghĩa: Lô X gồm 16 câu văn (mỗi câu 128 từ, mỗi từ 768 đặc trưng) đi qua cùng 1 bộ trọng số W duy nhất của tầng Linear.
- 2 trục cuối (Nhân ma trận 2D): $(128 \times 768) @ (768 \times 3072) \longrightarrow$ hai số **768** ở giữa triệt tiêu, ra (128, 3072).
- Trục đầu (Broadcasting): X có chiều lô là 16, còn W không có chiều lô $\longrightarrow$ PyTorch tự động broadcast (dùng chung) ma trận W cho cả 16 mẫu trong lô mà không tốn thêm RAM, giữ lại chiều 16 ở đầu!

V. Ký hiệu Tổng Einstein $\langle$ Einstein Summation — einsum $\rangle$

1. Ba Quy tắc Vàng để đọc và viết einsum trong 5 giây

① Ý tưởng cốt lõi (Kết nối trực tiếp từ Phép nhân Ma trận):

Nhìn lại phép nhân 2 ma trận 2D vừa học ở mục IV.2:

$$\underbrace{A}_{(M \times K)} @ \underbrace{B}_{(K \times N)} = \underbrace{C}_{(M \times N)}$$

- Hai chiều ở giữa đều là **K** (phải giống nhau) $\longrightarrow$ nhân tương ứng với nhau rồi **cộng gộp lại (triệt tiêu mất chữ K)**.
- Còn lại hai chiều ở hai đầu là M và $N \longrightarrow$ **giữ lại** làm kích thước của kết quả C .

Ý tưởng của einsum: Thay vì phải nhớ hàng chục tên hàm (dot, outer, trace, diag, bmm, permute, sum), hãy “Đặt chữ cái cho từng chiều”!

Ta đổi tên kích thước (M, K) và (K, N) thành các chữ cái thường cho gọn:

- Ma trận A có 2 chiều: đặt hàng là i , cột là $k \longrightarrow$ viết là “ ik ”.
- Ma trận B có 2 chiều: đặt hàng là k , cột là $j \longrightarrow$ viết là “ kj ”.
- Kết quả C giữ lại hàng i , cột j (triệt tiêu k) $\longrightarrow$ viết sau mũi tên là $\rightarrow ij$.

Ghép lại thành chuỗi duy nhất:

$$\text{np.einsum}(\text{"ik, kj} \rightarrow \text{ij"}, A, B) \iff C_{ij} = \sum_k A_{ik} B_{kj}$$

(Trong đó: **vế trái** mũi tên ik, kj là các trục **đầu vào**; **vế phải** mũi tên ij là các trục **đầu ra**).

② Bóc tách “3 Quy tắc Vàng” qua chuỗi "ik, kj -> ij":

3 QUY TẮC CỐT LÕI CỦA CHUỖI "input_axes -> output_axes"

1. Quy tắc 1 — Chữ cái LẶP LẠI ở vế trái → NHÂN tương ứng với nhau (*):

- Nhìn bên trái mũi tên (ik, kj): Chữ k xuất hiện ở cả A lẫn B .
- Báo cho máy tính biết: “*Chiều k của A và chiều k của B phải có cùng độ dài. Hãy bắt cặp từng phần tử thứ k của hai bên rồi nhân với nhau: $A_{ik} \times B_{kj}$.*”

2. Quy tắc 2 — Chữ cái BIẾN MẤT ở vế phải → CỘNG DỒN / Triệt tiêu (sum):

- Nhìn sang bên phải mũi tên ($\rightarrow ij$): Chữ k có ở bên trái nhưng sang bên phải đã biến mất!
- Chữ cái nào không được viết lại ở vế phải nghĩa là ta muốn **cộng tổng tất cả các tích dọc theo chiều đó** ($\sum_k$) để làm chiều đó biến mất (giống hết hai số K ở giữa triệt tiêu nhau khi nhân ma trận).
- **Kết hợp Quy tắc 1 + 2 trên chữ k :** Nhân theo k rồi Cộng dồn theo k
 $\implies \sum_k A_{ik} B_{kj}$.

3. Quy tắc 3 — Chữ cái GIỮ LẠI ở vế phải → HÌNH DÁNG ĐẦU RA (shape):

- Bên phải mũi tên viết ij → Tensor kết quả C có 2 chiều: chiều thứ nhất là i , chiều thứ hai là j .
- *Điều kỳ diệu:* Nếu đổi vế phải thành " $ik, kj \rightarrow ji$ " (viết j trước i), sau khi nhân xong máy tính **tự động chuyển vị** (C^T) luôn mà không cần gọi `.T` hay `.transpose()`!

Tóm tắt 3 từ khóa: **Lặp ở vế trái = NHÂN** •

Mất ở vế phải = CỘNG TỔNG • **Còn ở vế phải = SHAPE RA**

③ Thử tài đọc einsum trong 5 giây (Áp dụng đúng 3 quy tắc):

1. " $ij \rightarrow ji$ " (Chỉ có 1 ma trận A đầu vào):

Không có chữ lặp lại ($\rightarrow$ không nhân); không có chữ biến mất ở vế phải ($\rightarrow$ không cộng dồn); vế phải đảo ij thành ji (Quy tắc 3) → **Đảo hàng thành cột (Chuyển vị A^T)!**

2. " $ij \rightarrow i$ " (Chỉ có 1 ma trận A đầu vào):

Bên trái có i và j , nhưng bên phải chỉ còn i (chữ j biến mất!). Theo Quy tắc 2:

chữ j (cột) biến mất nghĩa là **cộng dồn hết các cột lại** $\rightarrow$ Ra tổng của từng hàng ($A.sum(dim=1)$).

3. "i, i $\rightarrow$ " (2 vector 1D u và v):

Chữ i **lặp lại** ở 2 đầu vào (Quy tắc 1 $\rightarrow$ nhân từng cặp $u_i \cdot v_i$); bên phải mũi tên **để trống** (chữ i **biến mất** — Quy tắc 2 $\rightarrow$ cộng dồn tất cả lại $\sum_i u_i v_i$) $\rightarrow$ Chính là **Tích vô hướng (Dot product)** ra 1 con số!

4. "bij, bjk $\rightarrow$ bik" (Nhân ma trận theo lô `torch.bmm` vừa học ở mục IV.2):

Tương ứng với $\underset{(B,M,K)}{A} @ \underset{(B,K,N)}{B} = \underset{(B,M,N)}{C}$ khi đặt tên: lô là b, hàng là i, chiều chung ở giữa là j, cột là k. Chữ b và j đều lặp lại ở 2 bên, nhưng ở vế phải $\rightarrow$ bik:

- Chữ b được **giữ lại** (giữ nguyên chiều Batch b).
- Chữ j ở giữa **biến mất** (nhân rồi cộng triệt tiêu chiều ở giữa j), còn lại bik!

2. Từ điển tra cứu các mẫu einsum kinh điển trong AI

Phép toán	Chuỗi einsum	Công thức Toán học	Tương đương
Chuyển vị	"ij $\rightarrow$ ji"	$B_{ji} = A_{ij}$	<code>A.T</code>
Vết ma trận (Trace)	"ii $\rightarrow$ "	$s = \sum_i A_{ii}$	<code>torch.trace(A)</code>
Đường chéo lô	"bii $\rightarrow$ bi"	$D_{bi} = A_{bii}$	<code>torch.diagonal</code>
Tích vô hướng	"i,i $\rightarrow$ "	$s = \sum_i u_i v_i$	<code>torch.dot(u, v)</code>
Tích ngoài	"i,j $\rightarrow$ ij"	$M_{ij} = u_i v_j$	<code>torch.outer(u, v)</code>
Nhân ma trận	"ik,kj $\rightarrow$ ij"	$C_{ij} = \sum_k A_{ik} B_{kj}$	<code>A @ B</code>
Batched MatMul	"bij,bjk $\rightarrow$ bik"	$C_{bik} = \sum_j A_{bij} B_{bjk}$	<code>torch.bmm(A, B)</code>
Điểm Attention	"bhqd,bhkd $\rightarrow$ bhqk"	$S_{bhqk} = \sum_d Q_{bhqd} K_{bhkd}$	<code>Q @ K.mT</code>

★ Công thức tính chi phí tính toán (FLOPs / Multiply-Adds) của einsum:

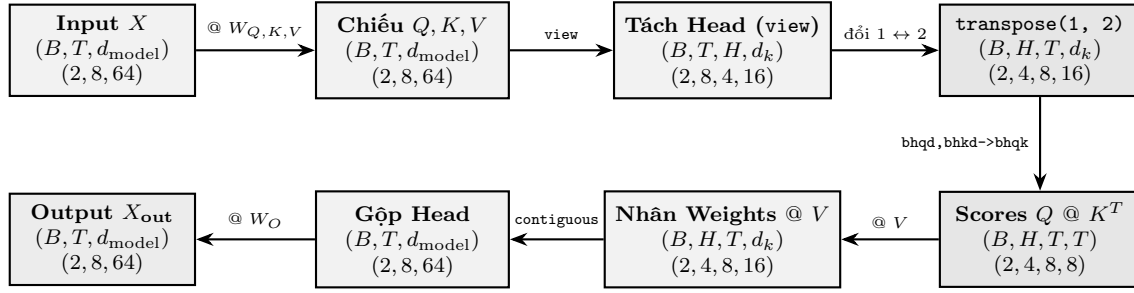
Số phép nhân-cộng của một biểu thức einsum bằng **tích kích thước của TẤT CẢ các chữ cái xuất hiện ở vế trái** (bao gồm cả trục giữ lại lần trục bị cộng rút gọn):

$$\text{VD: "bhqd,bhkd} \rightarrow \text{bhqk"} \implies \text{Số phép nhân-cộng} = B \times H \times T_q \times T_k \times d_k$$

VI. Ứng dụng : Truy vết Shape trong Multi-Head Attention & KV-Cache

1. Sơ đồ Truy vết Tensor Shape qua tầng Multi-Head Attention

Xét đầu vào $X \in \mathbb{R}^{B \times T \times d_{\text{model}}}$ với cấu hình chuẩn: $B = 2$ (batch), $T = 8$ (tokens), $d_{\text{model}} = 64$, chia làm $H = 4$ đầu chú ý $\implies$ mỗi đầu có chiều $d_k = \frac{d_{\text{model}}}{H} = \frac{64}{4} = 16$.



❓ Vì sao sau khi `view(B, T, H, d_k)` phải gọi `transpose(1, 2)` về (B, H, T, d_k) ?

- Vì phép nhân ma trận theo lô $Q @ K$. `transpose(-2, -1)` luôn nhân trên **2 trục cuối cùng**.
- Để mỗi đầu chú ý (b, h) tự tính ma trận tương quan $(T \times T)$ độc lập giữa T token, hai trục cuối bắt buộc phải là (T, d_k) , còn (B, H) phải đứng ở đầu làm các chiều batch!
- Khi gộp các head lại, ta bắt buộc phải gọi:
`attn_out.transpose(1, 2).contiguous().view(B, T, d_model)`
 Nếu quên `.contiguous()` trước `.view()`, chương trình sẽ lập tức dừng vì lỗi stride!

2. Công thức tính dung lượng VRAM của KV-Cache trong LLM

Khi sinh văn bản tự hồi quy (Autoregressive Generation), để không phải tính lại các vector Key (K) và Value (V) của các token trước đó, mô hình lưu lại hai Tensor K, V có shape = (B, H_{kv}, T, d_k) tại mỗi tầng trong tổng số L tầng Transformer:

$$\text{Dung lượng KV-Cache (Bytes)} = \underbrace{2}_{(K \text{ và } V)} \times L \times B \times T \times (H_{kv} \cdot d_k) \times \text{bytes_per_param}$$

VD 4: Tính KV-Cache cho mô hình có $L = 32$ tầng, $H_{kv} = 32$ đầu, $d_k = 128$ ($d_{\text{model}} = 4096$), chạy ở chuẩn FP16/BF16 (2 bytes/số), với $B = 1$ và độ dài ngữ cảnh $T = 4096$ tokens:

$$\text{KV-Cache} = 2 \times 32 \times 1 \times 4096 \times (32 \times 128) \times 2 \text{ bytes} = 2\,147\,483\,648 \text{ bytes} = \boxed{2,0 \text{ GB VRAM}}$$

VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 12

- **Bộ nhớ 1D & Strides:** Tensor đa chiều được lưu phẳng trên RAM 1D. Địa chỉ phần tử $(i_0, \dots, i_{r-1})$ là $\text{offset} = \sum_k i_k \cdot \text{stride}[k]$. Với tensor liên tục Row-major: $\text{stride}[k] = \prod_{m>k} \text{shape}[m]$.
- **Contiguous vs View:** `transpose()` và `permute()` là các phép toán Zero-copy chỉ đổi `stride`, làm tensor mất tính liên tục (`non-contiguous`). Phải gọi `.contiguous()` trước `.view()`, hoặc dùng `.reshape()`.
- **Broadcasting & stride = 0:** Căn lề shape từ phải sang trái; chiều có kích thước = 1 được kéo dài ảo thành N bằng cách gán `stride = 0` (không tốn thêm bộ nhớ vật lý).
- **Einstein Summation (einsum):** Chữ cái chung giữa các đầu vào thì nhân với nhau; chữ cái vắng mặt ở vế phải $\rightarrow$ thì cộng rút gọn. Điểm chú ý Multi-Head Attention viết gọn là "`bhqd, bhkd  $\rightarrow$  bhqk`".
- **Chuỗi Shape trong MHA:**

$$\begin{array}{lcl}
 (B, T, d_{\text{model}}) & \xrightarrow{\text{view}} & (B, T, H, d_k) \xrightarrow{\text{transpose}(1,2)} \\
 (B, H, T, d_k) & \xrightarrow{QK^T} & (B, H, T, T) \xrightarrow{@V} (B, H, T, d_k) \xrightarrow{\text{transpose+contiguous+view}} \\
 (B, T, d_{\text{model}}). & &
 \end{array}$$

Lesson 13: Numerical Stability

I. Vấn đề (The Problem)

Trong toán học thuần túy, tập số thực $\mathbb{R}$ có độ chính xác vô hạn và phạm vi vô tận. Nhưng trên máy tính (CPU/GPU), mọi số thực phải nén vào 32 bits, 16 bits hoặc 8 bits hữu hạn. Sự lệch pha giữa toán lý thuyết và số học dấu phẩy động (Floating-Point Arithmetic) gây ra 3 thảm họa kinh điển khi huấn luyện AI:

- **Nổ NaN / Inf giữa chừng:** Mô hình huấn luyện mượt suốt 3 tiếng (9000 bước), đến bước 9001 một logit vọt lên 90 $\implies e^{90}$ tràn số thành Inf trong FP32 $\implies$ hàm Softmax lấy $\frac{\text{Inf}}{\text{Inf}} = \text{NaN}$. Chỉ cần **1 phần tử NaN** xuất hiện trong gradient, toàn bộ ma trận trọng số sẽ nhiễm NaN vĩnh viễn ở bước tiếp theo!
- **Chết đứng Gradient (Silent Underflow):** Khi chuyển từ FP32 sang FP16 để tăng tốc trên GPU, các đạo hàm nhỏ cỡ 10^{-8} bị làm tròn thẳng về 0.0, làm mô hình ngừng học hoàn toàn hoặc tụt 2% – 3% độ chính xác mà không hề báo lỗi.
- **Triệt tiêu thảm khốc (Catastrophic Cancellation):** Tính phương sai bằng công thức $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$ trong BatchNorm/LayerNorm có thể trả về phương sai **âm** do sai số làm tròn, khiến căn bậc hai $\sqrt{\text{Var}} = \text{NaN}$.

II. Chuẩn IEEE 754: Cấu trúc Số Dấu phẩy động (FP32, FP16, BF16, FP8)

1. Giải phẫu một số Floating-Point theo chuẩn IEEE 754

Mọi số thực dấu phẩy động được máy tính lưu trữ qua 3 trường bit:

$$\text{Giá trị} = (-1)^S \times 2^{E-\text{bias}} \times \left(1 + \frac{M}{2^m}\right)$$

- **Bit dấu S (Sign — 1 bit):** $S = 0$ là số dương (+), $S = 1$ là số âm (−).
- **Phần mũ E (Exponent — e bits):** Quyết định **dải động (Dynamic Range)** — số lớn nhất và nhỏ nhất có thể biểu diễn (với $\text{bias} = 2^{e-1} - 1$).
- **Phần định trị M (Mantissa / Fraction — m bits):** Quyết định **độ chính xác (Precision)** — khoảng cách giữa hai số kế tiếp nhau (số chữ số thập phân có nghĩa).

FP32 (32-bit):

S (1)	Exponent (8 bits)	Mantissa / Fraction (23 bits) $\rightarrow \sim 7$ chữ số thập phân
---------	--------------------------	---

FP16 (16-bit):

S (1)	Exp (5 bits)	Mantissa (10 bits)
---------	--------------	--------------------

$\leftarrow$ Dải hẹp ($\text{max} = 65504$), dễ tràn!

BF16 (16-bit):

S (1)	Exponent (8 bits)	Mantissa (7 bits)
---------	--------------------------	-------------------

$\leftarrow$ Cùng dải động với FP32 ($\sim 3,4 \times 10^{38}$)!

FP8 (8-bit):

S (1)	Exp (4/5)	Man (3/2)
---------	-----------	-----------

$\leftarrow$ E4M3 (Forward) & E5M2 (Backward trên H100)

2. Bảng So sánh Định dạng Số trong Huấn luyện & Suy luận LLM

Chuẩn	Số Bit	Mũ E (Bias)	Định trị M	Cực đại (Max)	Vai trò trong AI hiện đại
FP64	64	11 (1023)	52 bits	$\approx 1,80 \times 10^{308}$	Chuẩn kiểm thử đạo hàm số học, tính toán khoa học.
FP32	32	8 (127)	23 bits	$\approx 3,40 \times 10^{38}$	Lưu Master Weights , hàm Loss, tích lũy Optimizer.
FP16	16	5 (15)	10 bits	65 504	Inference nhanh; nếu Train bắt buộc phải có Loss Scaling .
BF16	16	8 (127)	7 bits	$\approx 3,40 \times 10^{38}$	Vua của LLM Training : không sợ tràn số như FP16!
FP8	8	4 (7)	3 bits	448 (E4M3)	Tăng tốc $\times 2$ trên GPU Hopper/Blackwell (cần per-tensor scale).
INT8	8	0 (Nguyên)	7 bits + dấu	$[-128, 127]$	Lượng tử hóa (Quantization) trọng số khi triển khai Inference.

❓ Vì sao Google tạo ra BF16 (Brain Float 16) và tại sao nó đánh bại FP16 khi huấn luyện LLM?

- Mạng nơ-ron **chịu đựng tốt nhiều làm tròn** (7 bits Mantissa của BF16 là đủ để hướng gradient đi đúng), nhưng lại **cực kỳ nhạy cảm với phạm vi độ lớn (Dynamic Range)**.
- Trong FP16 (chỉ có 5 bits Exponent), bất kỳ kích hoạt hay tổng bình phương nào vượt quá 65 504 đều thành Inf, và bất kỳ gradient nào nhỏ hơn $\approx 6 \times 10^{-8}$ đều tụt về 0.
- BF16 giữ nguyên **đúng 8 bits Exponent của FP32** (chỉ cắt bỏ 16 bits đuôi Mantissa của FP32). Nhờ vậy, BF16 có dải động lên tới $3,4 \times 10^{38}$ y hệt FP32, chuyển đổi qua lại với FP32 chỉ bằng phép dịch bit và hầu như không cần Loss Scaling!

III. Machine Epsilon, Swamping & Triệt tiêu Thảm khốc

1. Machine Epsilon (ϵ_{mach}) và Hiện tượng Nuốt số (Swamping)

① Machine Epsilon (ϵ_{mach}):

Là khoảng cách giữa số 1.0 và số đầu phẩy động tiếp theo lớn hơn 1.0 (tức số dương nhỏ nhất

ở bit cuối cùng của Mantissa 2^{-m}):

$$\boxed{\epsilon_{\text{mach}} = 2^{-m}} \implies \begin{cases} \text{FP32 } (m = 23) : \epsilon_{\text{mach}} = 2^{-23} \approx \mathbf{1,19 \times 10^{-7}} \\ \text{FP16 } (m = 10) : \epsilon_{\text{mach}} = 2^{-10} \approx \mathbf{9,77 \times 10^{-4}} \\ \text{BF16 } (m = 7) : \epsilon_{\text{mach}} = 2^{-7} \approx \mathbf{7,81 \times 10^{-3}} \end{cases}$$

★ **Hệ quả (Vĩ sao $0.1 + 0.2 \neq 0.3$):** Số $0.1_{10} = 0.0001100110011 \dots_2$ là phân số nhị phân tuần hoàn vô hạn. Khi cắt ở m bits, máy tính lưu giá trị xấp xỉ $\implies$ **Không bao giờ dùng $a == b$ cho số thực**, phải dùng $|a - b| < \text{tol}$!

② Hiện tượng Nuốt số (Swamping / Absorption):

Khi cộng một số rất lớn A với một số rất nhỏ b sao cho $\frac{|b|}{|A|} < \frac{1}{2}\epsilon_{\text{mach}}$, để căn chỉnh số mũ (exponent alignment), các bit có nghĩa của b bị dịch phải vượt quá 23 bits Mantissa của FP32 và biến mất hoàn toàn:

$$\text{Trong FP32: } \boxed{10^8 + 10^{-8} = 10^8} \quad \text{và thậm chí} \quad \boxed{10^8 + 1.0 = 10^8}$$

- **Tác hại trong AI:** Khi trọng số $w \sim 1.0$ (hoặc trong BF16 với $\epsilon_{\text{mach}} \approx 0,0078$), nếu bản cập nhật $-\eta \cdot g$ quá nhỏ (VD 10^{-5}) và ta cộng trực tiếp vào trọng số BF16/FP16, phép cộng $w \leftarrow w - \eta g$ sẽ trả về nguyên xi w cũ! Đó là lý do **Master Weights bắt buộc phải lưu ở FP32**.

2. Triệt tiêu Thảm khốc (Catastrophic Cancellation)

① Bản chất:

Khi lấy hiệu của hai số thực xấp xỉ bằng nhau ($a \approx b$), toàn bộ các chữ số có nghĩa ở phần đầu triệt tiêu nhau, chỉ còn lại phần **nhỏ làm tròn ở các bit cuối** bị đôn lên làm chữ số đứng đầu, khiến sai số tương đối vọt lên hàng chục hoặc hàng trăm phần trăm!

VD 1: Phép trừ làm mất sạch chữ số có nghĩa

- Giả sử trong FP32, $a = 1.0000001$ (lưu thực tế là 1.00000011920929) và $b = 1.0000000$.
- Hiệu đúng: $a - b = 10^{-7}$. Hiệu máy tính trả về: $1.1920929 \times 10^{-7} \implies$ **Sai số tương đối 19,2% chỉ sau 1 phép trừ!**

② Hai bài toán kinh điển bị Catastrophic Cancellation và Cách khử:

Bài toán 1 — Khử căn thức $\sqrt{x+1} - \sqrt{x}$ khi x rất lớn (VD $x = 10^8$ in FP32):

- ⊗ **Tính ngây thơ:** Trong FP32, $\sqrt{10^8+1} = \sqrt{10^8} = 10^4 \implies \sqrt{x+1} - \sqrt{x} = 10^4 - 10^4 = 0.0$ (sai hoàn toàn so với giá trị thực $\approx 5 \times 10^{-5}$).
- ✓ **Nhân liên hợp (chuyển hiệu thành tổng):**

$$\sqrt{x+1} - \sqrt{x} = \frac{(\sqrt{x+1} - \sqrt{x})(\sqrt{x+1} + \sqrt{x})}{\sqrt{x+1} + \sqrt{x}} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

$$\text{Với } x = 10^8: \frac{1}{\sqrt{10^8+1} + \sqrt{10^8}} \approx \frac{1}{20\,000} = 5 \times 10^{-5} \text{ ✓}.$$

Bài toán 2 — Tính Phương sai (Variance) trong BatchNorm / LayerNorm:

Cho tập dữ liệu $X = [10^6, 10^6 + 1, 10^6 + 2]$ (kỳ vọng $\mu = 10^6 + 1$, phương sai thực $\sigma^2 = \frac{(-1)^2 + 0^2 + 1^2}{3} = \frac{2}{3} \approx 0,6667$).

- ⊗ **Công thức 1-pass ngây thơ $\text{Var}(X) = \frac{1}{N} \sum x_i^2 - \bar{x}^2$:**
Vì $x_i^2 \approx 10^{12}$ vượt quá độ chính xác 7 chữ số của FP32, cả $\frac{1}{N} \sum x_i^2$ và $\bar{x}^2$ đều bị làm tròn ở hàng trăm nghìn $\implies$ hiệu hai số khổng lồ bằng 0 hoặc **ra số âm**, làm $\sqrt{\text{Var} + \varepsilon}$ nổ NaN!
- ✓ **Giải pháp 1 (Trừ trung bình trước — Shifted Variance):** Tính $\mu = \bar{x}$ trước, rồi tính $\frac{1}{N} \sum (x_i - \mu)^2$.
- ✓ **Giải pháp 2 (Thuật toán Welford chạy 1 vòng lặp ổn định tuyệt đối):**

$$\delta = x_k - \mu_{k-1}, \quad \mu_k = \mu_{k-1} + \frac{\delta}{k}, \quad M_{2,k} = M_{2,k-1} + \delta \cdot (x_k - \mu_k), \quad \sigma^2 = \frac{M_{2,N}}{N}$$

IV. Tràn số, Shift-Invariant Softmax & Thủ thuật Log-Sum-Exp

1. Vì sao hàm $\exp(z)$ dễ gây tràn số trong Deep Learning?

Trong chuẩn FP32 (Max $\approx 3,40 \times 10^{38}$ và Min Subnormal $\approx 1,40 \times 10^{-45}$):

$$\begin{cases} z > 88,72 \implies e^z > 3,40 \times 10^{38} \longrightarrow \text{Overflow thành +Inf} & (\text{VD: } e^{1000} = \text{Inf}) \\ z < -103,97 \implies e^z < 1,40 \times 10^{-45} \longrightarrow \text{Underflow về 0.0} & (\text{VD: } e^{-1000} = 0.0) \end{cases}$$

Khi đưa vector logits $z = [1000, 1001, 1002]$ vào hàm Softmax ngây thơ:

$$\text{Softmax}_{\text{naive}}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{e^{1000}}{e^{1000} + e^{1001} + e^{1002}} = \frac{\text{Inf}}{\text{Inf}} = \boxed{\text{NaN}} \quad (\otimes)$$

2. Bất biến Tịnh tiến của Softmax (Shift-Invariant Softmax)

① Chứng minh tính chất Bất biến Tịnh tiến:

Với mọi hằng số thực $c \in \mathbb{R}$, nếu ta trừ c khỏi mọi phần tử của vector logits z :

$$\text{Softmax}(z_i - c) = \frac{e^{z_i - c}}{\sum_{j=1}^K e^{z_j - c}} = \frac{e^{z_i} \cdot e^{-c}}{\left(\sum_{j=1}^K e^{z_j}\right) \cdot e^{-c}} = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} = \text{Softmax}(z_i)$$

② Chọn $c = \max_k z_k$ để triệt tiêu hoàn toàn Overflow:

$$\boxed{\text{Softmax}_{\text{stable}}(z_i) = \frac{e^{z_i - c}}{\sum_{j=1}^K e^{z_j - c}} \quad \text{với } c = \max_k z_k}$$

- **Chống Overflow tuyệt đối:** Vì $z_i - c \leq 0 \forall i$, số mũ lớn nhất chỉ là $e^0 = 1 \implies$ không bao giờ vượt quá 1, bất kể logits lớn tới 10^6 !
- **Chống chia cho 0 do Underflow:** Luôn có ít nhất 1 phần tử đạt cực đại $z_{k^*} = c \implies e^{z_{k^*} - c} = e^0 = 1$, nên mẫu số luôn $\geq 1.0 > 0$!

VD 2: Tính Softmax ổn định cho $z = [1000, 1001, 1002]$

- **Bước 1:** Tìm $c = \max(1000, 1001, 1002) = 1002$.
- **Bước 2:** Dịch vector: $z - c = [-2, -1, 0]$.
- **Bước 3:** Lấy mũ: $e^{z-c} = [e^{-2}, e^{-1}, e^0] \approx [0,1353, 0,3679, 1,0000]$, tổng = 1,5032.
- **Bước 4:** Chia chuẩn hóa: $p \approx \left[\frac{0,1353}{1,5032}, \frac{0,3679}{1,5032}, \frac{1,0000}{1,5032} \right] = \boxed{[0,0900, 0,2447, 0,6652]} \quad (\checkmark)$.

3. Thủ thuật Log-Sum-Exp (LSE) & Hàm Fused Cross-Entropy

① Vì sao $\log(\text{softmax_stable}(z))$ vẫn chết vì Underflow?

Giả sử $z = [0, 1000]$ và nhãn đúng là lớp 0. Dù dùng $\text{Softmax}_{\text{stable}}$ với $c = 1000$:

$$p_0 = \frac{e^{0-1000}}{e^{-1000} + e^0} = \frac{0.0}{0.0 + 1.0} = 0.0 \implies \mathcal{L}_{\text{CE}} = -\ln(p_0) = -\ln(0.0) = \boxed{+\text{Inf}} \quad (\otimes)$$

Trong khi về mặt đại số, $-\ln\left(\frac{e^0}{e^0 + e^{1000}}\right) \approx 1000$ là một số hữu hạn hoàn toàn bình thường!

② Chứng minh Công thức Log-Sum-Exp (LSE) và Fused Log-Softmax:

Với $c = \max_k z_k$, ta biến đổi trực tiếp hàm $\text{LSE}(z) = \ln \sum_{j=1}^K e^{z_j}$:

$$\text{LSE}(z) = \ln \sum_{j=1}^K e^{(z_j - c) + c} = \ln \left(e^c \sum_{j=1}^K e^{z_j - c} \right) \implies \boxed{\text{LSE}(z) = c + \ln \sum_{j=1}^K e^{z_j - c}}$$

Thay vào công thức $\ln \text{Softmax}(z_y) = z_y - \text{LSE}(z)$ (hàm `F.log_softmax` và `F.cross_entropy`):

$$\boxed{\ln p_y = \ln \left(\frac{e^{z_y}}{\sum_j e^{z_j}} \right) = (z_y - c) - \ln \sum_{j=1}^K e^{z_j - c}}$$

- Với $z = [0, 1000]$ và $y = 0$ ($c = 1000$), công thức gộp tính thẳng trong không gian log:

$$\mathcal{L}_{\text{CE}} = -(0 - 1000) + \ln(e^{-1000} + e^0) = 1000 + \ln(0 + 1) = \mathbf{1000} \quad (\checkmark)$$

- Nếu bắt buộc phải nhận đầu vào là xác suất $p \in [0, 1]$ (VD trong Binary Cross-Entropy): Phải kẹp biên (clamp) trước khi lấy log:

$$\boxed{p_{\text{safe}} = \text{clip}(p, \varepsilon, 1 - \varepsilon)} \quad (\text{với } \varepsilon = 10^{-7}) \quad \text{để tránh } \ln(0) \text{ và } \ln(1 - 1).$$

V. Kiểm soát Lỗi Số học & Ổn định Huấn luyện

1. NaN và Inf: Cơ chế Phát sinh, Phát hiện & Phòng ngừa

Một giá trị NaN (Not a Number) hoặc Inf (Vô cực) duy nhất xuất hiện ở đạo hàm sẽ lấy nhiệm toàn bộ trọng số chỉ sau **1 bước cập nhật** và làm chết quá trình huấn luyện ngay lập tức.

- Nguồn gốc sinh ra Inf:** (1) Hàm $\exp(x)$ với số dương lớn ($x > 88,7$ trong FP32); (2) Chia một số khác 0 cho 0 ($1.0 / 0.0$); (3) Tràn số (Overflow) khi cộng dồn nhiều số lớn.
- Nguồn gốc sinh ra NaN:** Các phép toán vô định: $0.0 / 0.0$, $\text{Inf} - \text{Inf}$, $\text{Inf} * 0$, căn bậc hai số âm $\sqrt{-1}$, lấy log số âm $\ln(-1)$, hoặc mọi phép tính có mặt sẵn một số NaN.
- Phát hiện & 6 Chiến lược Phòng ngừa trong Code:**

- *Phát hiện*: Dùng `math.isnan(x)` / `torch.isnan(x)`, `torch.isinf(x)`, hoặc kiểm tra an toàn tổng hợp bằng `torch.isfinite(x)` (đúng khi không phải NaN cũng không phải Inf).
- *Phòng ngừa*: (1) Kẹp biên trước khi lấy mũ: $\exp(\text{clamp}(x, -80, 80))$; (2) Cộng $\varepsilon = 10^{-8}$ vào mẫu số: $\frac{x}{y + 10^{-8}}$; (3) Cộng ε bên trong $\ln(x + 10^{-8})$; (4) Luôn dùng hàm ổn định (`logsumexp`, `F.cross_entropy`); (5) Cắt ngưỡng đạo hàm (Gradient Clipping); (6) Kiểm tra `isfinite` sau mỗi pha Forward khi debug.

2. Kiểm thử Đạo hàm Số học (Numerical Gradient Checking)

Khi tự cài đặt một tầng mạng hoặc hàm Loss mới, đạo hàm giải tích (g_{ana} từ Backprop) rất dễ bị viết sai công thức. Ta kiểm chứng bằng cách đối chiếu với đạo hàm xấp xỉ qua **Sai phân Trung tâm (Centered Finite Differences)**:

$$g_{\text{num},i} = \frac{f(x + he_i) - f(x - he_i)}{2h} + \mathcal{O}(h^2)$$

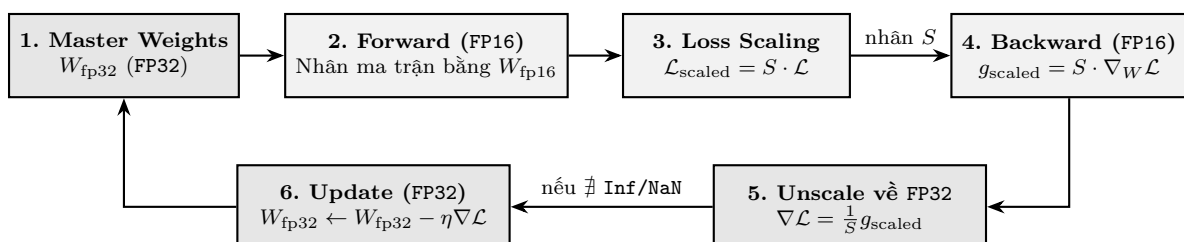
(Chính xác bậc $\mathcal{O}(h^2)$, tốt hơn hẳn công thức sai phân tiến $\frac{f(x+h) - f(x)}{h}$ chỉ đạt bậc $\mathcal{O}(h)$).

- **Cách chọn bước nhảy h** : Quá lớn thì sai số xấp xỉ Taylor lớn; quá nhỏ (như 10^{-15}) thì $f(x+h) \approx f(x-h)$ bị **Catastrophic Cancellation** phá hỏng kết quả! Giá trị chuẩn là $h \in [10^{-7}, 10^{-5}]$.
- Công thức Sai số Tương đối (Relative Error) và Thang đánh giá (gradcheck):

$$\text{rel_err} = \frac{|g_{\text{ana}} - g_{\text{num}}|}{\max(|g_{\text{ana}}|, |g_{\text{num}}|, 10^{-8})} \implies \begin{cases} < 10^{-7} : \text{Đúng tuyệt đối} \\ < 10^{-5} : \text{Chấp nhận được} \\ > 10^{-3} : \text{Có lỗi cài đặt!} \\ > 1.0 : \text{Sai hoàn toàn!} \end{cases}$$

3. Huấn luyện Độ chính xác Hỗn hợp (Mixed Precision Training & Loss Scaling)

Để tận dụng nhân Tensor Cores trên GPU (nhanh gấp $2 \times - 8 \times$ và giảm 50% VRAM), quy trình huấn luyện Mixed Precision gồm **6 bước chuẩn**: (1) Giữ bản gốc **Master Weights** ở FP32 $\rightarrow$ (2) Ép sang FP16/BF16 để chạy Forward nhanh $\rightarrow$ (3) Tính Loss ở FP32 $\rightarrow$ (4) Chạy Backward ở FP16/BF16 $\rightarrow$ (5) Đảo đạo hàm về FP32 $\rightarrow$ (6) Cập nhật vào Master Weights FP32.



- Vì sao FP16 bắt buộc phải có Loss Scaling (nhân hệ số S , VD $S = 1024$ hoặc 65 536)?
 Vì đạo hàm thường rất nhỏ ($\leq 10^{-8}$), mà mọi số dưới $\approx 6 \times 10^{-8}$ trong FP16 đều tụt về 0 (Underflow). Nhân Loss với S giúp phóng đại toàn bộ đạo hàm lên S lần để vượt qua ngưỡng Underflow của FP16, sau đó chia trả lại cho S ở FP32 trước khi cập nhật trọng số. **Dynamic Loss Scaling** khởi đầu với $S = 65\,536$: nếu gặp Inf thì giảm nửa ($S \leftarrow S/2$), nếu qua N bước không tràn thì nhân đôi ($S \leftarrow 2S$).
- bfloat16 vs float16: FP16 (1-5-10) chính xác hơn nên hợp cho **Inference**; BF16 (1-8-7) có cùng 8 bit mũ với FP32 ($\max \approx 3,4 \times 10^{38}$) nên không sợ nổ Inf khi logits/activations tăng vọt và hầu như không cần Loss Scaling khi **Training**!

4. Cắt ngưỡng Đạo hàm (Gradient Clipping)

Khi đạo hàm tăng theo cấp số nhân qua nhiều tầng (Exploding Gradients trong RNN, mạng sâu, Transformer), một batch nhiều duy nhất có thể phá hủy toàn bộ trọng số. Có 2 cách cắt ngưỡng:

- **Cách 1 — Kẹp theo giá trị (Clip by Value):** $g_i \leftarrow \text{clamp}(g_i, -\text{max_val}, \text{max_val})$. Đơn giản nhưng làm lệch hướng của vector đạo hàm.
- **Cách 2 — Cắt theo Chuẩn (Clip by Norm — Chuẩn mực trong AI, hàm clip_grad_norm_):** Co nhỏ toàn bộ vector đạo hàm theo cùng tỉ lệ nếu độ dài $\|g\|_2$ vượt quá max_norm , giúp giữ nguyên hướng đi của đạo hàm:

$$\text{Nếu } \|g\|_2 > \text{max_norm} : \quad g \leftarrow g \cdot \frac{\text{max_norm}}{\|g\|_2}$$

(Giá trị chuẩn thường dùng: $\text{max_norm} = 1.0$ cho Transformer, 0.5 cho RL, và 5.0 cho mạng đơn giản).

5. Các tầng Chuẩn hóa (Normalization Layers) như Bộ ổn định Số học

BatchNorm, LayerNorm và RMSNorm không chỉ giúp hội tụ nhanh mà còn là lá chắn chống tràn số. Nếu không có chuẩn hóa, giá trị kích hoạt (activations) nhân dồn qua các tầng sẽ phình to theo cấp số nhân (Tầng 1: $[0, 1] \rightarrow$ Tầng 5: $[0, 100] \rightarrow$ Tầng 10: $[0, 10\,000] \rightarrow$ Tầng 50: Inf!). Tầng LayerNorm kéo mọi kích hoạt về tâm 0, phương sai 1 ở mỗi tầng:

$$\text{LayerNorm}(x) = \frac{x - \text{mean}(x)}{\text{std}(x) + \varepsilon} \odot \gamma + \beta$$

(Trong đó $\varepsilon = 10^{-5}$ ngăn chia cho 0 khi mọi phần tử bằng nhau; γ, β là tham số học được).

6. Bảng Chẩn đoán 7 Lỗi Số học khi Huấn luyện Mô hình Thực tế

Triệu chứng Lỗi (Bug)	Nguyên nhân Số học (Cause)	Cách khắc phục (Fix)
1. Loss nổ NaN sau vài epoch	Logits quá lớn làm Softmax tràn số (Overflow), hoặc Learning Rate quá cao.	Dùng Stable Softmax (trừ max), giảm Learning Rate, bật Gradient Clipping.
2. Loss đứng im ở $\ln(K)$	Đầu ra gần như phân phối đều do đạo hàm triệt tiêu (Vanishing) hoặc ReLU chết.	Kiểm tra nhãn dữ liệu, hàm Loss, đổi sang LeakyReLU / GELU.
3. Accuracy thấp hơn bài báo 1–3%	Dùng FP16 không có Loss Scaling làm các bản cập nhật nhỏ bị tụt về 0 (Underflow).	Bật Dynamic Loss Scaling (GradScaler) hoặc chuyển sang BF16.
4. Chuẩn đạo hàm bằng 0.0 ở một số tầng	Nơ-ron ReLU chết (toàn bộ đầu vào âm) hoặc đạo hàm bị Underflow trong FP16.	Dùng GELU / LeakyReLU, bật Loss Scaling, kiểm tra khởi tạo trọng số.
5. Khác kết quả giữa 2 GPU khác nhau	Phép cộng số thực không có tính kết hợp: $(a + b) + c \neq a + (b + c)$ khi cộng song song.	Chấp nhận sai số $\sim 10^{-6}$, hoặc bật cờ <code>use_deterministic_algorithms(True)</code> .
6. Hàm <code>exp()</code> trả về Inf trong Loss	Truyền thẳng logits thô vào <code>exp()</code> mà không trừ max.	Dùng <code>F.log_softmax()</code> hoặc <code>F.cross_entropy()</code> (có sẵn Log-Sum-Exp).
7. Hội tụ tốt ở FP32, vỡ trận khi sang FP16	FP16 không biểu diễn được đạo hàm $< 6 \times 10^{-8}$ hoặc kích hoạt > 65504 .	Dùng Mixed Precision (AMP) kèm Loss Scaling, hoặc dùng BF16.

VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 13

- **IEEE 754 & Định dạng số:** Exponent quyết định **dải động** (Range), Mantissa quyết định **độ chính xác** (Precision). BF16 (1-8-7) có cùng 8 bits Exponent với FP32 (dải động $\sim 3,4 \times 10^{38}$) nên ổn định vượt trội so với FP16 (1-5-10, max = 65 504) khi huấn luyện LLM.

- **Swamping & Catastrophic Cancellation:** Cộng số quá nhỏ vào số quá lớn làm mất sạch số nhỏ ($10^8 + 1 = 10^8$ trong FP32); trừ hai số gần bằng nhau làm mất các chữ số có nghĩa (phải dùng thuật toán Welford hoặc trừ kỳ vọng trước khi tính phương sai).

- **Shift-Invariant Softmax & Log-Sum-Exp (LSE):** Luôn trừ $c = \max_k z_k$ trước khi lấy mũ:

$$\text{Softmax}(z_i) = \frac{e^{z_i - c}}{\sum_j e^{z_j - c}}, \quad \text{LSE}(z) = c + \ln \sum_j e^{z_j - c}, \quad \ln p_y = (z_y - c) - \ln \sum_j e^{z_j - c}$$

Bắt buộc dùng hàm gộp `F.cross_entropy(logits, y)` thay vì tách rời `log(softmax(z))`.

- **GradCheck, AMP & Clipping:** Kiểm thử đạo hàm bằng sai phân trung tâm $\frac{f(x+h) - f(x-h)}{2h}$ ($h \in [10^{-7}, 10^{-5}]$); giữ **Master Weights ở FP32** kèm Loss Scaling khi dùng FP16; cắt đạo hàm theo chuẩn `clip_grad_norm_` để bảo toàn hướng vector đạo hàm.

Lesson 14: Norms and Distances

I. Vấn đề (The Problem)

Giả sử bạn có hai vector biểu diễn hai từ vựng (word embeddings), hai hồ sơ khách hàng hoặc hai mảng điểm ảnh. Câu hỏi cốt lõi của mọi thuật toán Machine Learning (KNN, K-Means, Hệ thống gợi ý, Vector Database trong RAG, và chính các Hàm Loss) là: **“Hai đối tượng này gần nhau đến mức nào?”**

Không tồn tại một thước đo khoảng cách “vạn năng” cho mọi bài toán. Hai điểm dữ liệu có thể là hàng xóm sát vách dưới thước đo này nhưng lại xa nhau vời vợi dưới thước đo khác:

- **Dữ liệu không gian vật lý & ảnh mức pixel:** Khoảng cách đường thẳng Euclid (L_2) hoạt động tốt khi các chiều có cùng thang đo.
- **Xử lý ngôn ngữ tự nhiên (NLP) & Embeddings:** Một bài báo dài gấp đôi bài báo kia nhưng viết cùng một chủ đề vẫn phải được coi là “giống nhau”. Độ tương đồng Cosine (*Cosine Similarity*) bỏ qua độ dài và chỉ đo góc hướng ngữ nghĩa.
- **Dữ liệu tương quan & đa thang đo:** Chiều cao và cân nặng đi đôi với nhau. Khoảng cách *Mahalanobis* tự động khử tương quan và chuẩn hóa phương sai để bắt đúng điểm dị biệt (Outlier).
- **Tập hợp, Chuỗi ký tự & Phân phối xác suất:** Tập thẻ (tags) cần *Jaccard*, chuỗi văn bản sai chính tả cần *Edit Distance (Levenshtein)*, còn hai phân phối xác suất tách rời nhau cần *Wasserstein Distance* để không bị mất đạo hàm.

II. Chuẩn Vector (Vector Norms) & Họ Khoảng cách L_p

1. Định nghĩa Chuẩn (Norm) và Khoảng cách Cầm sinh

Chuẩn (Norm) là hàm đo “độ lớn” (chiều dài) của một vector $\vec{x} \in \mathbb{R}^n$, ký hiệu là $\|\vec{x}\|$. Mọi hàm khoảng cách giữa hai vector $\vec{a}$ và $\vec{b}$ đều được suy ra từ chuẩn của hiệu hai vector đó:

$$d(\vec{a}, \vec{b}) = \|\vec{a} - \vec{b}\|$$

Để một hàm $\|\cdot\|$ được công nhận là một **Chuẩn hợp lệ** (và tạo ra một không gian Metric chuẩn), nó phải thỏa mãn **3 tiên đề hiển nhiên của thước đo độ dài**:

$$\left\{ \begin{array}{ll} \text{① Tính xác định dương (Non-negativity):} & \|\vec{x}\| \geq 0 \text{ và } \|\vec{x}\| = 0 \iff \vec{x} = \vec{0} \\ \text{② Tính thuần nhất tuyệt đối (Homogeneity):} & \|\alpha \cdot \vec{x}\| = |\alpha| \cdot \|\vec{x}\| \\ \text{③ Bất đẳng thức tam giác (Triangle Ineq.):} & \|\vec{x} + \vec{y}\| \leq \|\vec{x}\| + \|\vec{y}\| \end{array} \right.$$

2. Họ Chuẩn Tổng quát L_p (L_1, L_2, L_p, L_∞)

Với tham số $p \geq 1$, chuẩn L_p (Minkowski Norm) của vector $\vec{x} = [x_1, x_2, \dots, x_n]^T$ được định nghĩa là:

$$\|\vec{x}\|_p = (|x_1|^p + |x_2|^p + \dots + |x_n|^p)^{\frac{1}{p}} = \left(\sum_{i=1}^n |x_i|^p \right)^{\frac{1}{p}}$$

Tên Chuẩn	Công thức Toán học	Ý nghĩa Hình học & Khi nào sử dụng?
Chuẩn L_1 (Manhattan)	$\ \vec{x}\ _1 = \sum_{i=1}^n x_i $	Tổng trị tuyệt đối (đi bộ dọc theo lưới ô vuông bàn cờ, không đi chéo). Bền vững trước nhiễu ngoại lai (Outliers) ; dùng cho dữ liệu nhiều chiều thưa, MAE Loss và Lasso (L_1).
Chuẩn L_2 (Euclidean)	$\ \vec{x}\ _2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\vec{x}^T \vec{x}}$	Độ dài đường chim bay (định lý Pythagoras). Phạt nặng các sai số lớn do bình phương; dùng cho dữ liệu liên tục cùng thang đo, MSE Loss và Ridge (L_2).
Chuẩn L_∞ (Chebyshev)	$\ \vec{x}\ _\infty = \lim_{p \rightarrow \infty} \ \vec{x}\ _p = \max_{1 \leq i \leq n} x_i $	Chỉ lấy thành phần lệch lớn nhất (Worst-case), bỏ qua mọi chiều còn lại. Dùng trong kiểm soát dung sai sản xuất, nước đi vua cờ vua, và nhiễu đối kháng (ϵ -ball).

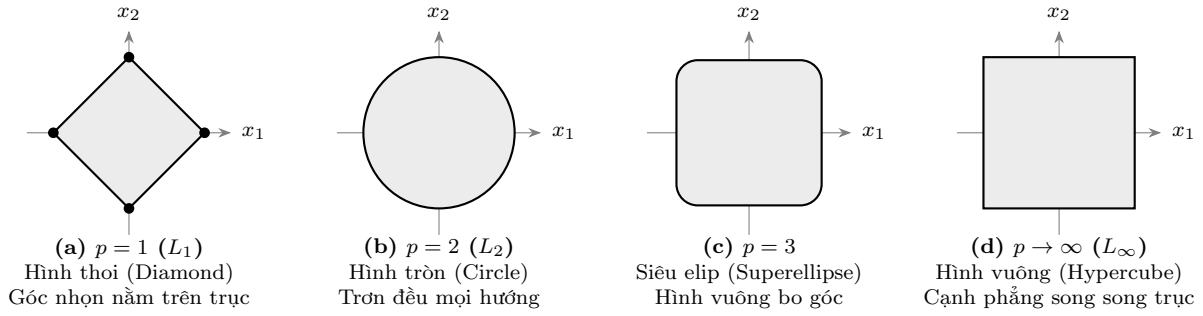
VD 1: Tính khoảng cách L_1, L_2, L_∞ giữa hai điểm $A = (1, 1)$ và $B = (4, 5)$

Ta có vector hiệu $\vec{x} = B - A = (4 - 1, 5 - 1) = (3, 4)$:

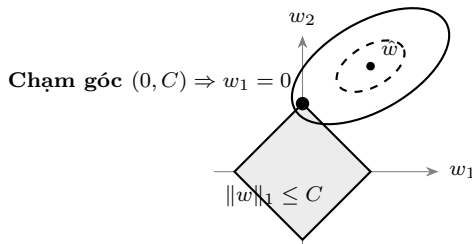
- **Khoảng cách L_1 (Manhattan):** Đi ngang 3 ô rồi đi lên 4 ô: $d_{L_1}(A, B) = |4-1| + |5-1| = 3 + 4 = \boxed{7}$.
- **Khoảng cách L_2 (Euclidean):** Đường chéo thẳng: $d_{L_2}(A, B) = \sqrt{(4-1)^2 + (5-1)^2} = \sqrt{9 + 16} = \boxed{5,0}$.
- **Khoảng cách L_∞ (Chebyshev):** Chiều lệch lớn nhất: $d_{L_\infty}(A, B) = \max(|4-1|, |5-1|) = \max(3, 4) = \boxed{4}$.

3. Mặt cầu Đơn vị (Unit Balls) & Liên hệ với Regularization

Tập hợp mọi điểm cách gốc tọa độ một khoảng bằng 1 ($\|\vec{x}\|_p = 1$) tạo thành **Mặt cầu đơn vị (Unit Ball)** có hình dáng biến đổi theo p :



Tại sao L_1 (Lasso) triệt tiêu trọng số về 0 còn L_2 (Ridge) thì không?



- **Phạt L_1 (Lasso — Cộng $\lambda\|w\|_1$):**
Tạo miền ràng buộc **hình thoi** có các góc nhọn nằm đúng trên trục tọa độ ($w_1 = 0$). Đường đồng mức Loss dễ dàng chạm vào góc nhọn này trước, ép trọng số thừa về đúng 0 (tự động chọn lọc đặc trưng).
- **Phạt L_2 (Ridge — Cộng $\lambda\|w\|_2^2$):**
Tạo miền ràng buộc **hình tròn** không có góc nhọn trên trục, nên chỉ thu nhỏ đồng đều mọi trọng số về sát 0 chứ hiếm khi bằng 0 tuyệt đối.

III. Khoảng cách Euclid (L_2) vs Độ tương đồng Cosine <Cosine Similarity>

1. Đẳng thức Liên hệ giữa L_2 và Cosine trên Mặt cầu Đơn vị

Cho hai vector bất kỳ $\vec{u}, \vec{v} \in \mathbb{R}^d$. Công thức khoảng cách Euclid và Cosine Similarity/Distance là:

$$d_{L_2}(\vec{u}, \vec{v}) = \|\vec{u} - \vec{v}\|_2 = \sqrt{\sum_{i=1}^d (u_i - v_i)^2} \quad , \quad \cos \theta = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|_2 \|\vec{v}\|_2} \quad , \quad d_{\cos}(\vec{u}, \vec{v}) = 1 - \cos \theta \in [0, 2]$$

① Khi nào L_2 và Cosine hoàn toàn tương đương? (Chứng minh chi tiết):

Giả sử cả hai vector đã được **chuẩn hóa L_2 về độ dài bằng 1** ($\|\vec{u}\|_2 = 1$ và $\|\vec{v}\|_2 = 1$, tức nằm trên mặt cầu đơn vị). Ta khai triển bình phương khoảng cách Euclid:

$$\begin{aligned} \|\vec{u} - \vec{v}\|_2^2 &= (\vec{u} - \vec{v})^T (\vec{u} - \vec{v}) \\ &= \|\vec{u}\|_2^2 + \|\vec{v}\|_2^2 - 2(\vec{u} \cdot \vec{v}) \\ &= 1 + 1 - 2(\|\vec{u}\|_2 \|\vec{v}\|_2 \cos \theta) \quad (\text{thay } \|\vec{u}\|_2 = \|\vec{v}\|_2 = 1) \end{aligned}$$

$$\Rightarrow \boxed{\|\vec{u} - \vec{v}\|_2^2 = 2(1 - \cos \theta) = 2 d_{\cos}(\vec{u}, \vec{v}) \iff \|\vec{u} - \vec{v}\|_2 = \sqrt{2 - 2 \cos \theta}}$$

★ **Kết luận vàng cho Vector Database (FAISS / Milvus / Qdrant):** Khi các vector nhúng (Embeddings) đã được chuẩn hóa $\|\vec{x}\|_2 = 1$, việc tìm láng giềng gần nhất theo **khoảng cách Euclid nhỏ nhất** ($\min \|\vec{u} - \vec{v}\|_2$), **Cosine Similarity lớn nhất** ($\max \cos \theta$) và **Tích vô hướng lớn nhất** ($\max \vec{u} \cdot \vec{v}$) cho ra cùng một thứ tự xếp hạng 100%!

2. Khi nào L_2 và Cosine Mâu thuẫn? (Thiên lệch Độ dài trong RAG)

Khi các vector **chưa chuẩn hóa độ dài** ($\|\vec{u}\|_2 \neq \|\vec{v}\|_2$), L_2 bị chi phối bởi độ lớn (magnitude), còn Cosine chỉ đo thuần túy hướng (angle):

VD 2: So sánh $\vec{q} = (2, 1)$ với $\vec{d}_A = (6, 3)$ và $\vec{d}_B = (1, 2.2)$:

- **Theo Cosine Similarity:**
 $\cos(\vec{q}, \vec{d}_A) = \frac{12 + 3}{\sqrt{5}\sqrt{45}} = 1,00$ (trùng hướng!), còn $\cos(\vec{q}, \vec{d}_B) \approx 0,78$.
 $\Rightarrow$ Cosine chọn đúng $\vec{d}_A$ (cùng tỷ lệ từ khóa 2 : 1).
- **Theo Khoảng cách Euclid L_2 :**
 $\|\vec{q} - \vec{d}_A\|_2 = \sqrt{20} \approx 4,47$ (xa!), còn $\|\vec{q} - \vec{d}_B\|_2 \approx 1,56$ (gần!).
 $\Rightarrow L_2$ chọn nhầm $\vec{d}_B$ chỉ vì bài $\vec{d}_A$ dài gấp 3 lần bài $\vec{q}$!

IV. Khoảng cách Mahalanobis (Mahalanobis Distance)

1. Vì sao Euclid (L_2) thất bại khi Đặc trưng Tương quan?

Khoảng cách Euclid đối xử với mọi chiều hoàn toàn bình đẳng. Nhưng nếu các đặc trưng **có tương quan với nhau** hoặc **khác đơn vị đo**, L_2 sẽ đưa ra kết luận sai lầm:

- **Ví dụ trực quan:** Chiều cao và cân nặng có tương quan thuận rất chặt. Một người cao 1m88, nặng 82kg là hoàn toàn cân đối. Nhưng một người cao 1m52 mà nặng 82kg là bất thường (Outlier).
- Khoảng cách Euclid có thể báo hai người này cách điểm trung bình một khoảng bằng nhau, nhưng **Khoảng cách Mahalanobis** phát hiện chính xác người thứ hai là Outlier vì nó tính đến cấu trúc hiệp phương sai của dữ liệu!

2. Công thức Mahalanobis & Giải mã Ma trận S , S^{-1}

Với ma trận hiệp phương sai của dữ liệu là $S \in \mathbb{R}^{n \times n}$, khoảng cách Mahalanobis giữa hai vector $\vec{x}$ và $\vec{y}$ là:

$$d_M(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T S^{-1} (\vec{x} - \vec{y})}$$

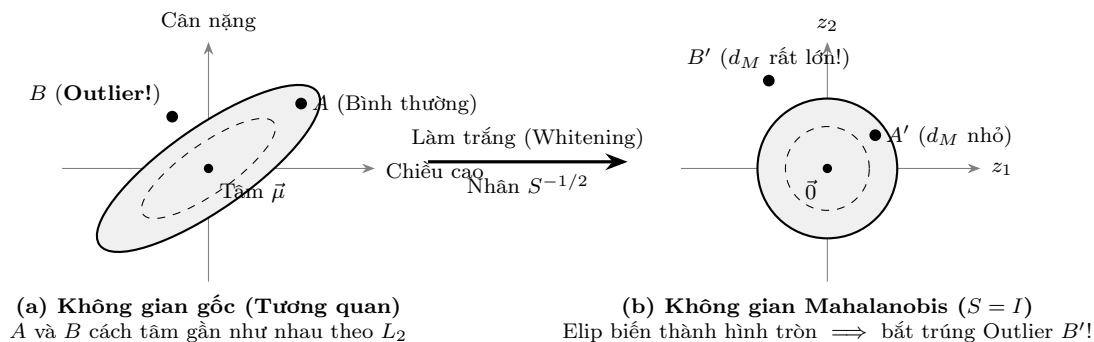
① Ma trận Hiệp phương sai S và Nghịch đảo S^{-1} là gì?

- **Cấu trúc của S (Covariance Matrix):** Là ma trận thống kê đo **độ phình to và độ nghiêng của đám mây dữ liệu** (không phải ma trận đạo hàm bậc 2 Hessian!). Ví dụ với 2 cột dữ liệu X_1, X_2 :

$$S = \begin{bmatrix} \text{Var}(\mathbf{X}_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Var}(\mathbf{X}_2) \end{bmatrix}$$

- **Đường chéo chính $\text{Var}(X_i)$:** Phương sai (độ dao động riêng) của từng cột.
- **Ngoài đường chéo $\text{Cov}(X_i, X_j)$:** Độ tương quan đi đôi giữa 2 cột (quyết định độ nghiêng của đám mây elip).
- **Tại sao lại nhân với S^{-1} (Ma trận nghịch đảo của S)?**

Ở 1 chiều, phép chia phương sai là $\sqrt{(x-y)^2/\sigma^2} = \sqrt{(x-y)(\sigma^2)^{-1}(x-y)}$. Sang nhiều chiều, vì ma trận không có phép chia $\div$, nên nhân với S^{-1} **ở giữa chính là phép “chia cho phương sai của từng chiều”**: nó tự động **xoay trục khử tương quan và chia chuẩn hóa phương sai từng chiều về 1 (Whitening)**, rồi mới đo khoảng cách Euclid L_2 ! Nếu dữ liệu vốn độc lập và có phương sai bằng 1 ($S = I$), Mahalanobis suy biến về đúng L_2 .



VD 3: Thấy tận mắt cách S^{-1} “chia cho phương sai” bằng số cụ thể:

Giả sử tâm $\vec{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, cột 1 dao động mạnh ($\text{Var} = 4$) còn cột 2 dao động ít ($\text{Var} = 1$):

$$S = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix} \Rightarrow S^{-1} = \begin{bmatrix} 1/4 & 0 \\ 0 & 1 \end{bmatrix}. \text{ Xét 2 điểm } A = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \text{ và } B = \begin{bmatrix} 0 \\ 1.5 \end{bmatrix}:$$

- Theo Euclid L_2 : $\|A - \vec{\mu}\|_2 = 2.0 > \|B - \vec{\mu}\|_2 = 1.5$ (L_2 tưởng lầm A xa tâm hơn B).
- Theo Mahalanobis: $d_M(A, \vec{\mu}) = \sqrt{2^2 \times \frac{1}{4} + 0} = 1.0$ độ lệch chuẩn; còn $d_M(B, \vec{\mu}) = \sqrt{0 + 1.5^2 \times 1} = 1.5$ độ lệch chuẩn $\Rightarrow$ Điểm B mới là điểm lệch bất thường hơn!

V. Khoảng cách trên Tập hợp, Chuỗi ký tự & Phân phối Xác suất

1. Độ tương đồng Jaccard (Jaccard Similarity) (Cho Tập hợp / Sets)

Khi dữ liệu không phải là vector số thực mà là **các tập hợp rời rạc** (tập từ khóa, thẻ phân loại, mặt hàng trong giỏ), **Jaccard Similarity** đo tỷ lệ phần tử chung trên tổng số phần tử

duy nhất của cả hai tập hợp:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1] \quad , \quad d_{\text{Jaccard}}(A, B) = 1 - J(A, B) \in [0, 1]$$

- **VD 4**: Cho $A = \{\text{cat}, \text{dog}, \text{fish}\}$ và $B = \{\text{cat}, \text{bird}, \text{fish}, \text{snake}\}$:
 - Phần giao (chung): $A \cap B = \{\text{cat}, \text{fish}\} \implies |A \cap B| = 2$.
 - Phần hợp (tất cả): $A \cup B = \{\text{cat}, \text{dog}, \text{fish}, \text{bird}, \text{snake}\} \implies |A \cup B| = 5$.
 - $\implies \text{Jaccard Similarity} = \frac{2}{5} = 0,4$ và $\text{Jaccard Distance} = 1 - 0,4 = 0,6$.
- **Ứng dụng thực tế**: So khớp tập thẻ (tags/categories), lọc văn bản trùng lặp ở quy mô lớn (dùng thuật toán băm **MinHash** để xấp xỉ Jaccard), và chấm điểm mô hình nhận diện/phân vùng ảnh (**IoU — Intersection over Union** chính là Jaccard Similarity giữa khung dự đoán và khung thật!).

2. Khoảng cách Chỉnh sửa — Edit Distance (Levenshtein Distance)

Edit Distance (Khoảng cách Levenshtein) đếm số thao tác ký tự đơn tối thiểu gồm Thêm (Insert), Xóa (Delete) hoặc Thay thế (Substitute) để biến chuỗi A thành chuỗi B .

VD 5: Biến chuỗi kitten → sitting:

1. kitten → sitten (Thay k → s)
2. sitten → sittin (Thay e → i)
3. sittin → sitting (Thêm g vào cuối)

$\implies \text{Edit Distance} = 3$.

Ứng dụng: Kiểm tra & sửa lỗi chính tả, tìm kiếm chuỗi mờ (*Fuzzy string matching*), làm sạch dữ liệu văn bản nhiễu, và giống hàng chuỗi gen DNA.

Ma trận Quy hoạch động (Dynamic Programming):

Ô (i, j) lưu khoảng cách chỉnh sửa giữa i ký tự đầu của chuỗi dọc và j ký tự đầu của chuỗi ngang:

	∅	s	i	t	t	i	n	g
∅	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

3. So sánh Phân phối: KL Divergence vs Wasserstein

Khi đối tượng cần so sánh không phải là 2 điểm dữ liệu, mà là **2 phân phối xác suất** P và Q (ví dụ: phân phối ảnh thật P và phân phối ảnh do AI sinh ra Q), ta có hai công cụ cốt lõi:

① KL Divergence (Độ phân kỳ Kullback–Leibler — Ôn lại từ Bài 09):

Đo lượng thông tin thất thoát khi dùng phân phối xấp xỉ Q để biểu diễn phân phối thật P :

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

- **Lưu ý sống còn:** Dù hay được dùng làm hàm Loss để kéo Q về gần P , KL Divergence **KHÔNG phải là một khoảng cách (Metric)** vì nó **bất đối xứng**: $D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$ và không thỏa mãn bất đẳng thức tam giác.
- **Forward KL** $D_{KL}(P \parallel Q)$ là *Mean-seeking* (Q cố gắng trải rộng để phủ mọi vùng có $P(x) > 0$); còn **Reverse KL** $D_{KL}(Q \parallel P)$ là *Mode-seeking* (Q co cụm vào một đỉnh duy nhất của P).
- **Điểm yếu chí mạng:** Nếu có vùng nào $P(x) > 0$ mà $Q(x) = 0$ (hai phân phối **không chồng lấp nhau**), phép chia $\log(P(x)/0)$ sẽ nổ ngay thành $+\infty$, làm mô hình mất sạch đạo hàm để học!

② Wasserstein Distance (Khoảng cách Xúc đất — Earth Mover's Distance):

- **Hình ảnh trực quan (Bài toán Xúc đất):** Hãy tưởng tượng phân phối P là một **đống đất** và phân phối Q là một **cái khuôn hồ** ở cách đó một đoạn. Muốn biến đống đất P thành hình dáng của Q , bạn phải xúc từng xẻng đất mang sang lấp vào hồ. **Công sức bỏ ra** được tính bằng:

$$\text{Công sức} = (\text{Khối lượng đất di chuyển}) \times (\text{Quãng đường phải bê đi})$$

Khoảng cách Wasserstein chính là **tổng công sức tối thiểu** trong mọi kế hoạch vận chuyển γ có thể:

$$W(P, Q) = \inf_{\gamma \in \Pi(P, Q)} \mathbb{E}_{(x, y) \sim \gamma} [d(x, y)]$$

- **Công thức 1 chiều (1D) qua Hàm phân phối tích lũy (CDF):**
Với phân phối 1 chiều, gọi $\text{CDF}_P(x) = \mathbb{P}(X \leq x)$ là **Hàm phân phối tích lũy** (tổng xác suất cộng dồn từ trái sang phải tính đến vị trí x). Khi đó, bài toán tìm kế hoạch xúc đất tối ưu rút gọn thành một công thức cực kỳ đẹp — chỉ cần lấy **diện tích chênh lệch giữa hai đường cộng dồn CDF**:

$$W_1(P, Q) = \int_{-\infty}^{+\infty} |\text{CDF}_P(x) - \text{CDF}_Q(x)| dx, \text{ Rồi rạc: } W_1 = \sum_i |\text{CDF}_P(i) - \text{CDF}_Q(i)|$$

Thuốc đo	Công thức Toán học	Tính chất & Ứng dụng trong AI
KL Divergence (Không phải Metric)	$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$	Bất đối xứng ($D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$). Nổ $+\infty$ khi P, Q không chồng lấp. Dùng trong VAE , Distillation , và phạt KL trong RLHF .
Wasserstein (Earth Mover's)	$W_1(P, Q) = \int_{-\infty}^{+\infty} \text{CDF}_P(x) - \text{CDF}_Q(x) dx$	Là Metric đối xứng thực sự . Vẫn cung cấp đạo hàm mượt khi 2 phân phối tách rời nhau. Cứu cánh cho huấn luyện mạng WGAN !

VD 6: Tính từng bước trên 2 phân phối không chồng lấp: Cho $P = [1, 0, 0, 0, 0]$ (toàn bộ đất ở ô 1) và $Q = [0, 0, 0, 0, 1]$ (cần đổ đất vào ô 5, cách đó 4 bước):

- Theo KL Divergence: Ngay tại ô 1, $P(1) = 1$ nhưng $Q(1) = 0 \implies 1 \cdot \log(1/0) = \boxed{+\infty}$ (nổ số, không biết Q đang cách P bao xa!).
- Theo Wasserstein (W_1):
 - Cách 1 (Nhắm theo bài toán xúc đất): Bê khối lượng đất bằng 1 đi qua 4 ô sang phải $\implies \text{Công} = 1 \times 4 = 4$.
 - Cách 2 (Dùng công thức cộng dồn CDF): Cộng dồn từ trái sang phải ta được $\text{CDF}_P = [1, 1, 1, 1, 1]$ và $\text{CDF}_Q = [0, 0, 0, 0, 1]$. Lấy trị tuyệt đối hiệu từng ô cộng lại:

$$W_1(P, Q) = |1 - 0| + |1 - 0| + |1 - 0| + |1 - 0| + |1 - 1| = 1 + 1 + 1 + 1 + 0 = \boxed{4}$$

VI. Bảng Chọn Khoảng cách theo Bài toán & Liên hệ với các Hàm Loss

1. Bảng Quyết định Chọn Metric cho 12 Bài toán Thực tế

Bài toán Thực tế (Task)	Thước đo Tốt nhất	Lý do Toán học (Vì sao chọn thước đo này?)
1. Độ tương đồng văn bản (NLP)	Cosine Similarity	Độ dài văn bản là nhiều; hướng của vector mới mang ngữ nghĩa.
2. So sánh ảnh ở mức Pixel	Chuẩn L_2 (Euclidean)	Quan hệ không gian trực tiếp; các pixel có cùng thang đo $[0, 255]$.
3. Đặc trưng nhiều chiều thưa	Chuẩn L_1 (Manhattan)	Bền vững, không phóng đại bình phương các sai lệch ngoại lai hiếm gặp.
4. Độ trùng lặp tập hợp (Tags)	Jaccard Similarity	Dữ liệu bản chất là tập hợp rời rạc, không phải tọa độ vector.
5. So khớp chuỗi / Sửa chính tả	Edit Distance	Đếm trực tiếp số thao tác gõ phím (thêm, xóa, sửa ký tự).
6. Phát hiện điểm dị biệt (Outlier)	Mahalanobis	Tự động tính đến tương quan chéo và sự lệch thang đo giữa các cột.
7. So sánh phân phối (VAE, RLHF)	KL Divergence	Đo lượng thông tin thất thoát khi dùng phân phối Q xấp xỉ P .
8. Huấn luyện mạng sinh GAN	Wasserstein (W_1)	Vẫn cung cấp đạo hàm có nghĩa ngay cả khi 2 phân phối tách rời nhau.
9. Tìm kiếm Vector DB (RAG)	Cosine / Dot Product	Các mô hình Embedding được huấn luyện để mã hóa ngữ nghĩa vào hướng.
10. Hệ thống Gợi ý (RecSys)	Dot Product	Độ lớn của vector giúp mã hóa thêm độ phổ biến hoặc độ tự tin.
11. Giống hàng chuỗi DNA	Weighted Edit Dist.	Chi phí đột biến thay thế khác nhau tùy thuộc vào từng cặp Nucleotide.
12. Kiểm định Chất lượng (QC)	Chuẩn L_∞ (Max Norm)	Mọi kích thước cơ khí đều phải nằm trong ngưỡng; sai số tệ nhất quyết định.

2. Liên hệ với các Hàm Loss: Hàm Loss là Hàm Khoảng cách!

Khi huấn luyện mô hình, **Hàm mục tiêu (Loss Function)** thực chất chính là một hàm khoảng cách đo độ lệch giữa dự đoán $\hat{y}$ và nhãn thật y :

Hàm Loss (Loss Function)	Khoảng cách Cơ sở	Hành vi Toán học & Đạo hàm khi Huấn luyện
MSE Loss	Bình phương L_2 ($\ y - \hat{y}\ _2^2$)	Phạt cực nặng các lỗi lớn (nhạy cảm với Outliers), đạo hàm trơn tại 0.
MAE Loss	Chuẩn L_1 ($\ y - \hat{y}\ _1$)	Phạt mọi lỗi tuyến tính như nhau; cực kỳ bền vững trước dữ liệu nhiễu.
Huber Loss	Kết hợp L_2 (khi lỗi nhỏ) và L_1 (khi lỗi lớn)	Kết hợp ưu điểm của cả hai: đạo hàm mượt quanh 0 như L_2 nhưng không bị nổ trước Outliers như L_1 .
Cross-Entropy Loss	KL Divergence	Đo độ lệch giữa phân phối nhãn thật P và phân phối dự đoán $Q = \text{Softmax}(z)$.
Hinge Loss (SVM)	$\max(0, \text{margin} - d)$	Chỉ phạt khi điểm dữ liệu vi phạm lề an toàn (<i>margin</i>); bằng 0 khi đã phân lớp đúng đủ xa.
Triplet / Contrastive Loss	Khoảng cách L_2 hoặc Cosine	Kéo cặp cùng loại (<i>Anchor - Positive</i>) lại gần nhau và đẩy cặp khác loại (<i>Negative</i>) ra xa quá ngưỡng <i>margin</i> .

VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 14
Họ chuẩn L_p & Regularization: $d(\vec{a}, \vec{b}) = \ \vec{a} - \vec{b}\ _p$. Chuẩn L_1 tạo mặt cầu hình thoi có đỉnh nhọn trên trục tọa độ (giúp Lasso ép trọng số thừa về đúng 0 và giúp MAE bền vững trước Outliers); chuẩn L_2 tạo mặt cầu hình tròn (dùng trong Ridge và MSE). Cosine vs Dot Product vs L_2: Cosine chỉ đo góc hướng ($\cos \theta \in [-1, 1]$), khử hoàn toàn nhiễu độ dài văn bản. Khi các vector đã chuẩn hóa $\ \vec{a}\ _2 = \ \vec{b}\ _2 = 1$, cả ba thước đo cho thứ tự xếp hạng giống hệt nhau vì $\ \vec{a} - \vec{b}\ _2^2 = 2(1 - \cos \theta)$. Mahalanobis Distance: $d_M(\vec{x}, \vec{y}) = \sqrt{(\vec{x} - \vec{y})^T S^{-1} (\vec{x} - \vec{y})}$ làm trắng dữ liệu (khử tương quan + chuẩn hóa phương sai) trước khi đo L_2, giúp phát hiện Outliers chuẩn xác. Dữ liệu Phi Vector & Phân phối: Dùng Jaccard ($A \cap B / A \cup B$) cho tập hợp; Edit Distance cho chuỗi ký tự; KL Divergence (bất đối xứng) và Wasserstein (metric đối xứng không mất đạo hàm khi tách rời) cho các phân phối xác suất.

Lesson 15: Statistics for Machine Learning

I. Vấn đề (The Problem)

- Giả sử bạn huấn luyện 2 mô hình phân loại trên cùng tập dữ liệu kiểm thử (*test set*) gồm $n = 100$ mẫu:

Model A đạt Accuracy = 87% , Model B đạt Accuracy = 89%

- Bạn vội vàng kết luận Model B tốt hơn 2% và triển khai (*deploy*) lên hệ thống Production. Ba tuần sau, chỉ số thực tế tụt dốc thảm hại. **Chuyện gì đã xảy ra?**
- Bản chất:** Khoảng chênh lệch 2% trên 100 mẫu chỉ là 2 mẫu dự đoán khác nhau — hoàn toàn có thể sinh ra do **hiệu ngẫu nhiên (random noise)** khi lấy mẫu tập test chứ không phải năng lực thực sự của mô hình!
- Tương tự, khi đánh giá độ trễ (*Inference Latency*) của API LLM hay tính phương sai theo lô (*Batch Normalization*), nếu dùng sai công thức ước lượng mẫu hoặc bỏ qua kiểm định thống kê, toàn bộ quyết định kỹ thuật đều chỉ là **đoán mò được nguy trạng bằng con số**.

WRONG — Sai lầm kinh điển khi đánh giá mô hình AI

- So sánh trực tiếp 2 con số điểm đơn lẻ (*point estimates*): $0,89 > 0,87 \implies$ Chọn B $\otimes$
- Chia cho n thay vì $n - 1$ khi ước lượng phương sai từ mẫu nhỏ (*biased estimator*) $\otimes$
- Tưởng nhầm $p\text{-value} = 0,03$ nghĩa là “xác suất giả thuyết H_0 đúng bằng 3%” $\otimes$

$\implies$ **Giải pháp chuẩn:** Dùng Thống kê suy diễn (*Inferential Statistics*): hiệu chỉnh Bessel $\frac{1}{n-1}$, khoảng tin cậy (*Confidence Interval*), lấy mẫu lại **Bootstrap**, và kiểm định giả thuyết có kiểm soát sai lầm đa kiểm định $\checkmark$.

II. Thống kê Mô tả & Hiệu chỉnh Bessel (Descriptive Statistics)

1. Trung bình vs Trung vị (Mean vs Median & Outliers)

① **Công thức & Bản chất hình học:** Cho tập mẫu $X_{(n \times 1)} = [x_1, x_2, \dots, x_n]^T$ đã sắp xếp tăng dần $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\text{Med}(X) = \begin{cases} x_{(\frac{n+1}{2})} & \text{nếu } n \text{ lẻ} \\ \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2} & \text{nếu } n \text{ chẵn} \end{cases}$$

❓ Vì sao khi giám sát Latency của hệ thống AI ta luôn dùng phân vị P_{50} (Median), P_{95} , P_{99} thay vì Mean?

- **Mean** là điểm cân bằng trọng lực (tối thiểu hóa tổng bình phương $\sum (x_i - c)^2$), nên chỉ cần **1 điểm ngoại lai (outlier)** cực lớn sẽ kéo lệch toàn bộ giá trị trung bình.
- **Median** (P_{50}) và **Khoảng tứ phân vị IQR** $= Q_3 - Q_1 = P_{75} - P_{25}$ tối thiểu hóa khoảng cách tuyệt đối $\sum |x_i - c|$, hoàn toàn **miễn nhiễm (robust)** với giá trị cực đoan ở hai đầu đuôi.

VD 1: Độ trễ phản hồi (ms) của 5 request vào LLM server là $X = [10, 12, 14, 16, 948]$.

- **Bước 1 (Tính Mean):** $\bar{x} = \frac{10 + 12 + 14 + 16 + 948}{5} = \frac{1000}{5} = 200$ ms (Bị méo nặng bởi 1 request nghẽn mạng!)
- **Bước 2 (Tính Median):** Dữ liệu đã xếp tăng dần, phần tử chính giữa ($i = 3$) là $\text{Med}(X) = 14$ ms (Phản ánh đúng trải nghiệm thực của đa số người dùng).

2. Phương sai Mẫu & Hiệu chỉnh Bessel ($\frac{1}{n-1}$ vs $\frac{1}{n}$)

① Sự khác biệt giữa Quần thể (Population) và Mẫu (Sample):

Phương sai Quần thể (biết μ thật): $\sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2$

Phương sai Mẫu chưa hiệu chỉnh (chệch): $s_{\text{biased}}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \quad \otimes$

Phương sai Mẫu có Hiệu chỉnh Bessel (không chệch): $s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad \odot$

❓ Vì sao chia cho n lại luôn đánh giá thấp (*underestimate*) phương sai thực σ^2 của quần thể?

Chứng minh đại số & Bậc tự do (Degrees of Freedom):

- Trung bình mẫu $\bar{x}$ chính là điểm làm cực tiểu hóa hàm $\sum_{i=1}^n (x_i - c)^2$. Do đó, với mọi $\mu \neq \bar{x}$, ta luôn có bất đẳng thức:

$$\sum_{i=1}^n (x_i - \bar{x})^2 < \sum_{i=1}^n (x_i - \mu)^2$$

- Khai triển chính xác:

$$\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 + (\bar{x} - \mu)^2$$

Lấy kỳ vọng hai vế (biết $E[(\bar{x} - \mu)^2] = \text{Var}(\bar{X}) = \frac{\sigma^2}{n}$):

$$\sigma^2 = E[s_{\text{biased}}^2] + \frac{\sigma^2}{n}$$

$$\Rightarrow E[s_{\text{biased}}^2] = \frac{n-1}{n} \sigma^2 < \sigma^2$$

$\Rightarrow$ Nhân với $\frac{n}{n-1}$ (tức chia cho $n-1$) ta được ước lượng không chệch:

$$E[s^2] = \sigma^2.$$

★ **Giải thích theo Bậc tự do (*Degrees of Freedom*):** Để tính s^2 , ta phải dùng $\bar{x}$. Mà tổng n độ lệch luôn bị ràng buộc bởi 1 phương trình $\sum_{i=1}^n (x_i - \bar{x}) = 0$. Khi biết $n - 1$ độ lệch đầu tiên, độ lệch thứ n bị khóa cứng hoàn toàn! Vậy chỉ có đúng **$n - 1$ thành phần độc lập** mang thông tin về độ phân tán.

VD 2: Cho một quần thể nhỏ gồm đúng $N = 3$ số: $\{2, 4, 6\}$.

- **Bước 1 (Thông số thật):** Trung bình $\mu = 4$, phương sai thật:

$$\sigma^2 = \frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{3} = \frac{8}{3} \approx 2,67$$

- **Bước 2 (Rút mọi mẫu kích thước $n = 2$ có hoàn lại — có $3^2 = 9$ mẫu):**

- 3 mẫu trùng nhau $(2, 2), (4, 4), (6, 6)$ có $\sum (x_i - \bar{x})^2 = 0$.
- 4 mẫu lệch 2 đơn vị $(2, 4), (4, 2), (4, 6), (6, 4)$ có $\bar{x} \in \{3, 5\} \implies \sum (x_i - \bar{x})^2 = 1^2 + (-1)^2 = 2$.
- 2 mẫu lệch 4 đơn vị $(2, 6), (6, 2)$ có $\bar{x} = 4 \implies \sum (x_i - \bar{x})^2 = (-2)^2 + 2^2 = 8$.

- **Bước 3 (So sánh kỳ vọng của 2 công thức):** Tổng $\sum (x_i - \bar{x})^2$ trên cả 9 mẫu là $3(0) + 4(2) + 2(8) = 24$.

$$\text{Chia cho } n = 2 : E[s_{\text{biased}}^2] = \frac{1}{9} \times \frac{24}{2} = \frac{4}{3} \neq \sigma^2 \otimes \langle \text{Hụt mất 50\%!} \rangle$$

$$\text{Chia cho } n - 1 = 1 : E[s^2] = \frac{1}{9} \times \frac{24}{1} = \frac{8}{3} = \sigma^2 \odot \langle \text{Khớp chính xác!} \rangle$$

3. Ma trận Hiệp phương sai & Tương quan Pearson vs Spearman

Với ma trận dữ liệu đã trừ trung bình $\tilde{X}_{(n \times d)} = X - \mathbf{1}\bar{x}^T$, ma trận hiệp phương sai mẫu $\Sigma_{(d \times d)}$ được tính bằng phép nhân ma trận:

$$\Sigma_{(d \times d)} = \frac{1}{n-1} \tilde{X}_{(d \times n)}^T @ \tilde{X}_{(n \times d)}, \quad \text{Cov}(X_j, X_k) = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k)$$

- **Hệ số tương quan tuyến tính Pearson:** Chuẩn hóa hiệp phương sai về đoạn $[-1, 1]$:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{s_X \cdot s_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \in [-1, 1]$$

- **Tương quan hạng Spearman (ρ_s):** Thay giá trị thực x_i, y_i bằng **thứ hạng (rank)** $R(x_i), R(y_i) \in \{1, \dots, n\}$ rồi tính Pearson trên hạng. Spearman bắt được mọi quan hệ **đơn điệu phi tuyến** (VD $y = x^3$ có $r_{\text{Pearson}} < 1$ nhưng $\rho_{\text{Spearman}} = 1$) và không bị phá hỏng bởi outliers.

III. Luật Số Lớn (LLN) vs Định lý Giới hạn Trung tâm (CLT)

① **Phân biệt rạch ròi 2 định lý nền tảng của Học máy:** Cho các biến ngẫu nhiên độc lập cùng phân phối (*i.i.d.*) $X_1, X_2, \dots, X_n$ có kỳ vọng μ và phương sai hữu hạn σ^2 :

- **Luật Số Lớn (Law of Large Numbers — LLN):** Khi $n \rightarrow \infty$, trung bình mẫu hội tụ theo xác suất về đúng hằng số kỳ vọng thật (giải thích vì sao Empirical Risk $\hat{L}_n$ hội tụ về True Risk L):

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{p} \mu \quad \text{khi } n \rightarrow \infty$$

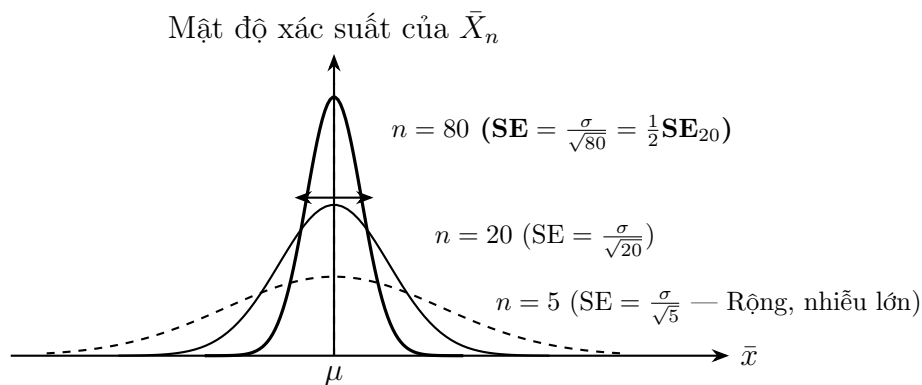
- **Định lý Giới hạn Trung tâm (Central Limit Theorem — CLT):** Bất kể phân phối gốc của X_i méo mó ra sao, khi n đủ lớn (thường $n \geq 30$), phân phối của **trung bình mẫu** $\bar{X}_n$ sẽ hội tụ về phân phối chuẩn:

$$\bar{X}_n \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right) \iff Z = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} \mathcal{N}(0, 1)$$

② **Sai số Chuẩn của Trung bình (Standard Error of the Mean — SE):**

$$\text{SE}(\bar{X}_n) = \sigma_{\bar{X}} = \frac{\sigma}{\sqrt{n}} \approx \frac{s}{\sqrt{n}}$$

★ **Quy luật căn bậc hai nghiệt ngã trong AI:** Vì SE tỉ lệ nghịch với $\sqrt{n}$ chứ không phải n , nên muốn giảm một nửa sai số ước lượng (hoặc giảm một nửa nhiễu Gradient trong Mini-batch SGD), bạn bắt buộc phải thu thập **GẤP 4 LẦN** số lượng mẫu ($4 \times n$)! Muốn giảm sai số đi 10 lần, cần gấp 100 lần dữ liệu!



IV. Khoảng Tin Cậy (CI) & Lấy mẫu lại Bootstrap

1. Khoảng Tin Cậy Giải tích (Analytical Confidence Interval)

Từ CLT, với mức tin cậy $1 - \alpha = 95\%$ ($\alpha = 0,05 \implies z_{\alpha/2} = z_{0,025} = 1,96$), khoảng tin cậy 95% cho kỳ vọng μ là:

$$CI_{95\%} = \bar{x} \pm 1,96 \cdot \frac{s}{\sqrt{n}} = \left[\bar{x} - 1,96 \frac{s}{\sqrt{n}}, \bar{x} + 1,96 \frac{s}{\sqrt{n}} \right]$$

Lưu ý bản chất: Không được hiểu “xác suất μ nằm trong khoảng này là 95%” (vì μ là hằng số cố định, không phải biến ngẫu nhiên!). Hiểu đúng: *Nếu lặp lại thí nghiệm lấy mẫu 100 lần và tính 100 khoảng CI khác nhau, sẽ có khoảng 95 khoảng chứa đúng giá trị μ thật.*

2. Phương pháp Bootstrap Phi tham số (Non-parametric Bootstrap)

❓ Vì sao trong thực tế ML ta thường xuyên phải dùng Bootstrap thay cho công thức $\bar{x} \pm 1,96 \frac{s}{\sqrt{n}}$?

- Công thức giải tích chỉ dùng được cho **trung bình cộng (Mean)**. Trong khi đó, các chỉ số sống còn của AI như **F1-Score**, **ROC-AUC**, **Precision@K**, **Median Latency** là các hàm phi tuyến phức tạp không có công thức phương sai đóng và không tuân theo phân phối chuẩn khi mẫu nhỏ!

① **Thuật toán Bootstrap (Lấy mẫu có hoàn lại B lần)**: Giả sử tập test gốc $\mathcal{D}$ có n phần tử. Để tìm khoảng tin cậy 95% cho bất kỳ chỉ số θ nào (VD: $\theta = \text{F1}$ hoặc $\Delta\text{AUC} = \text{AUC}_B - \text{AUC}_A$):

- 1) **Bước 1**: Từ n mẫu gốc, rút ngẫu nhiên n lần **có hoàn lại (with replacement)** để tạo thành tập giả lập $\mathcal{D}^{*(b)}$ (một số mẫu xuất hiện 2–3 lần, khoảng 36,8% mẫu không xuất hiện vì $(1 - \frac{1}{n})^n \rightarrow e^{-1} \approx 0,368$).
- 2) **Bước 2**: Tính chỉ số $\hat{\theta}^{*(b)}$ trên tập $\mathcal{D}^{*(b)}$ vừa rút.
- 3) **Bước 3**: Lặp lại quy trình trên B lần độc lập (thường $B = 1000 \rightarrow 10000$), thu được dãy $\{\hat{\theta}^{*(1)}, \hat{\theta}^{*(2)}, \dots, \hat{\theta}^{*(B)}\}$.
- 4) **Bước 4 (Percentile CI)**: Sắp xếp B giá trị tăng dần và cắt hai đầu ở phân vị 2,5% và 97,5%:

$$CI_{95\%}^{\text{Bootstrap}} = \left[\hat{\theta}_{(0,025 \cdot B)}^*, \hat{\theta}_{(0,975 \cdot B)}^* \right]$$

Nếu khoảng tin cậy của hiệu $\Delta = \text{Metric}_B - \text{Metric}_A$ **không chứa số 0**, ta kết luận sự khác biệt có ý nghĩa thống kê!

V. Kiểm định Giả thuyết, A/B Testing & Đa kiểm định

1. Quy trình Kiểm định, Thống kê t -test / z -test và Bản chất p -value

- **Giả thuyết không (H_0 — Null Hypothesis):** Mặc định “không có sự khác biệt” ($\mu_B - \mu_A = 0$, kết quả quan sát được chỉ là do may rủi ngẫu nhiên).
- **Giả thuyết đối (H_1 — Alternative Hypothesis):** Điều ta muốn chứng minh ($\mu_B - \mu_A \neq 0$ hoặc $\mu_B > \mu_A$).
- **Thống kê kiểm định (Test Statistic — đo độ lệch chuẩn hóa bằng đơn vị Standard Error):**

$$\text{One-sample } t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

,

$$\text{Welch's Two-sample } t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

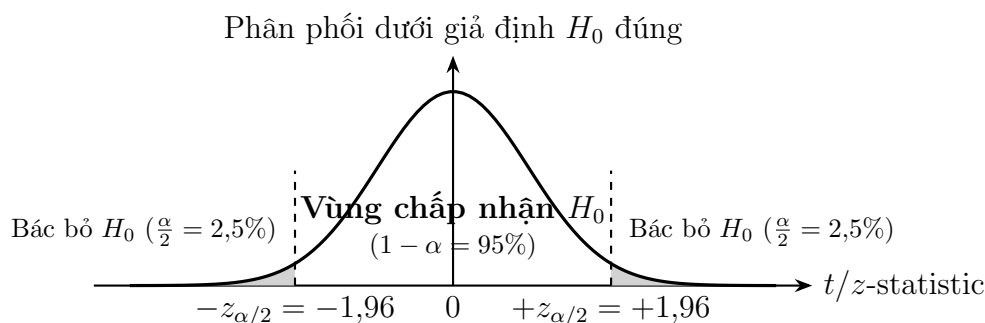
- **Định nghĩa chính xác của p -value:**

$$p\text{-value} = P(\text{Quan sát được dữ liệu cực đoan bằng hoặc hơn thực tế} \mid H_0 \text{ ĐÚNG})$$

CẢNH BÁO: 3 Ngộ nhận chí mạng về p -value trong Nghiên cứu & Kỹ thuật AI

- 1) **⊗ Sai:** “ $p = 0,03$ nghĩa là xác suất H_0 đúng bằng 3%”.
✓ Đúng: p -value là $P(\text{Data} \mid H_0)$, **KHÔNG PHẢI** $P(H_0 \mid \text{Data})$! Muốn tính $P(H_0 \mid \text{Data})$ phải dùng định lý Bayes với xác suất tiên nghiệm $P(H_0)$.
- 2) **⊗ Sai:** “ $p \geq 0,05$ chứng tỏ 2 mô hình giống hệt nhau (H_0 đúng)”.
✓ Đúng: Không đủ bằng chứng để bác bỏ H_0 (*Fail to reject H_0*) thường chỉ vì kích thước mẫu n quá nhỏ (thiếu Statistical Power).
- 3) **⊗ Sai:** “ $p = 0,0001$ chứng tỏ mô hình mới vượt trội hoàn toàn về mặt thực tiễn”.
✓ Đúng: Khi $n = 10^6$, chênh lệch vô nghĩa 0,03% (VD 92,37% vs 92,34%) vẫn cho $p < 0,001$ vì $SE = \frac{s}{\sqrt{n}} \rightarrow 0$. Bắt buộc phải đo thêm **Độ lớn Hiệu ứng (Effect Size — Cohen's d)**:

$$d = \frac{\bar{x}_1 - \bar{x}_2}{s_{\text{pooled}}} \quad (d \approx 0,2 : \text{nhỏ}; d \approx 0,5 : \text{vừa}; d \geq 0,8 : \text{ lớn})$$



2. Sai lầm Loại I (α), Loại II (β) & Hiệu chỉnh Bonferroni

- **Sai lầm Loại I (α — False Positive):** Bác bỏ H_0 khi H_0 thực sự đúng (báo động giả: triển khai mô hình vô dụng).
- **Sai lầm Loại II (β — False Negative):** Không bác bỏ H_0 khi H_1 thực sự đúng (bỏ lỡ một mô hình thực sự tốt).
- **Sức mạnh Thống kê (Statistical Power = $1 - \beta$):** Xác suất phát hiện đúng hiệu ứng có thật (tiêu chuẩn công nghiệp: $1 - \beta \geq 0,80$).
- **Bẫy Đa kiểm định (Multiple Testing Problem / p -hacking):** Nếu bạn thử nghiệm $m = 20$ bộ siêu tham số (*hyperparameters*) hoàn toàn vô tác dụng (H_0 đều đúng) với mức ý nghĩa $\alpha = 0,05$, xác suất xuất hiện ít nhất 1 dương tính giả vọt lên tới:

$$P(\text{ít nhất 1 False Positive}) = 1 - (1 - \alpha)^m = 1 - (0,95)^{20} \approx \mathbf{64,15\%}!$$

⇒ **Hiệu chỉnh Bonferroni (Bonferroni Correction):** Chia nhỏ ngưỡng ý nghĩa cho tổng số phép thử m :

$$\alpha_{\text{adjusted}} = \frac{\alpha}{m} \quad \left(\text{VD: } m = 20 \implies \alpha_{\text{adjusted}} = \frac{0,05}{20} = 0,0025 \right)$$

Chỉ bác bỏ H_0 cho cấu hình thứ i nếu $p_i < \frac{\alpha}{m}$, giúp bảo toàn tổng tỷ lệ dương tính giả (FWER) $\leq \alpha$.

VI. Nghịch lý Simpson (Simpson's Paradox)

① **Bản chất hiện tượng:** Một mô hình AI (hoặc phương pháp điều trị) có thể **chiến thắng** trên **TỪNG** nhóm con (subgroup) riêng lẻ, nhưng khi gộp chung toàn bộ dữ liệu lại thì **thua cuộc hoàn toàn!** Nguyên nhân là do tồn tại một **biến gây nhiễu ẩn** (Confounding Variable Z) vừa ảnh hưởng đến tỷ lệ phân bổ mẫu, vừa ảnh hưởng đến độ khó của bài toán.

VD 3: So sánh độ chính xác (*Accuracy*) của **Model A** (hiện tại) và **Model B** (mới) trên 1 000 truy vấn người dùng, chia thành 2 nhóm độ khó: **Truy vấn Dễ (Easy)** và **Truy vấn Khó (Hard)**:

Phân nhóm dữ liệu	Model A (Baseline)	Model B (Model mới)	Model thắng
Nhóm 1: Truy vấn Dễ (<i>Easy Queries</i>)	$\frac{720}{800} = 90,0\%$	$\frac{190}{200} = 95,0\%$	Model B ✓ (+5,0%)
Nhóm 2: Truy vấn Khó (<i>Hard Queries</i>)	$\frac{100}{200} = 50,0\%$	$\frac{480}{800} = 60,0\%$	Model B ✓ (+10,0%)
TỔNG HỢP (Gộp chung) (<i>Aggregated Total</i>)	$\frac{720 + 100}{800 + 200} = \frac{820}{1000} = 82,0\%$	$\frac{190 + 480}{200 + 800} = \frac{670}{1000} = 67,0\%$	Model A ⊗ (+15,0%!)

❓ Vì sao xảy ra nghịch lý toán học $\frac{a_1}{b_1} < \frac{c_1}{d_1}$ và $\frac{a_2}{b_2} < \frac{c_2}{d_2}$ nhưng $\frac{a_1+a_2}{b_1+b_2} > \frac{c_1+c_2}{d_1+d_2}$?

- **Bước 1 (Phân tích trọng số ẩn):** Độ chính xác tổng hợp là trung bình có trọng số theo tỷ lệ phân bổ lưu lượng (*traffic split*):

$$\begin{aligned} Acc_{total}(A) &= 0,8 \times 90\% + 0,2 \times 50\% = 72\% + 10\% = 82\% \\ Acc_{total}(B) &= 0,2 \times 95\% + 0,8 \times 60\% = 19\% + 48\% = 67\% \end{aligned}$$

- **Bước 2 (Kết luận kỹ thuật):** Model B vượt trội Model A ở cả câu dễ lẫn câu khó, nhưng trong đợt test đã bị định tuyến (*routed*) tới 80% câu hỏi khó (trong khi Model A nhận 80% câu hỏi dễ).

★ **Bài học bắt buộc trong AI Evaluation:** Không bao giờ chỉ nhìn con số tổng hợp (*overall metric*). Bắt buộc phải thực hiện **Lấy mẫu phân tầng (Stratified Sampling / Stratified Evaluation)** để cố định phân phối biến gây nhiễu $P(Z)$ giống hệt nhau giữa các mô hình!

VII. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 15

- **Thống kê Mô tả & Hiệu chỉnh Bessel:** Median và $IQR = Q_3 - Q_1$ bền vững trước Outliers hơn Mean. Khi ước lượng phương sai từ mẫu, luôn chia cho $n - 1$:
$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$
 vì ràng buộc $\sum (x_i - \bar{x}) = 0$ làm mất đi 1 bậc tự do, giúp $E[s^2] = \sigma^2$.
- **LLN vs CLT & Sai số chuẩn (Standard Error):** LLN đảm bảo $\bar{X}_n \xrightarrow{p} \mu$; CLT đảm bảo $\bar{X}_n \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right)$ với $SE = \frac{\sigma}{\sqrt{n}}$. Muốn giảm một nửa sai số ước lượng cần tăng kích thước mẫu lên **gấp 4 lần**.
- **Khoảng tin cậy (CI) & Bootstrap:** Giải tích cho trung bình: $\bar{x} \pm 1,96 \frac{s}{\sqrt{n}}$. Với các chỉ số phi tuyến (F1, AUC, P_{99}), rút mẫu có hoàn lại B lần (**Bootstrap**) và lấy phân vị [2,5%, 97,5%].
- **Kiểm định Giả thuyết & Bonferroni:** $p\text{-value} = P(\text{Data cực đoan} \mid H_0 \text{ đúng}) \neq P(H_0 \text{ đúng})$. Luôn báo cáo kèm **Effect Size (Cohen's d)** và chia ngưỡng $\alpha_{\text{adj}} = \alpha/m$ khi thực hiện m phép kiểm định.
- **Nghịch lý Simpson:** Biến gây nhiễu (*confounder*) làm đảo chiều kết luận khi gộp dữ liệu $\implies$ luôn đánh giá phân tầng (*Stratified Evaluation*) và dùng cùng một tập test cố định.

Lesson 16: Sampling Methods

I. Vấn đề (The Problem)

- Mọi hệ thống AI tạo sinh (*Generative AI*), học tăng cường (*Reinforcement Learning*) và suy diễn Bayes đều xoay quanh bài toán cốt lõi: **làm sao rút mẫu ngẫu nhiên $x \sim p(x)$ và tính kỳ vọng trên không gian nhiều chiều?**

$$\mathbb{E}_{x \sim p}[f(x)] = \int_{\mathbb{R}^d} f(x) p(x) dx \approx \frac{1}{N} \sum_{i=1}^N f(x_i) \quad \text{với } x_i \stackrel{\text{i.i.d.}}{\sim} p(x)$$

- Tại sao tích phân giải tích và lưới rời rạc thất bại?** Trong không gian $d = 100$ chiều (VD không gian tiềm ẩn của VAE / Diffusion), nếu chia mỗi chiều thành 10 điểm lưới thì cần 10^{100} phép tính — bất khả thi! Trong khi đó, **Ước lượng Monte Carlo (Monte Carlo Estimation)** có sai số chuẩn $\text{SE} = \frac{\sigma_f}{\sqrt{N}} = \mathcal{O}\left(\frac{1}{\sqrt{N}}\right)$ hoàn toàn độc lập với số chiều d .
- Tương tự, khi LLM dự đoán từ tiếp theo trên bộ từ vựng $V = 50\,000$ tokens: nếu luôn chọn $\arg \max_i P_i$ (*Greedy*), văn bản bị lặp vô tận; nếu chọn hoàn toàn ngẫu nhiên đều, văn bản trở thành vô nghĩa. **Chiến lược lấy mẫu (Sampling Strategy)** chính là bộ điều khiển chất lượng và sự sáng tạo của AI.

WRONG — Những ngộ nhận phổ biến về Sampling trong AI

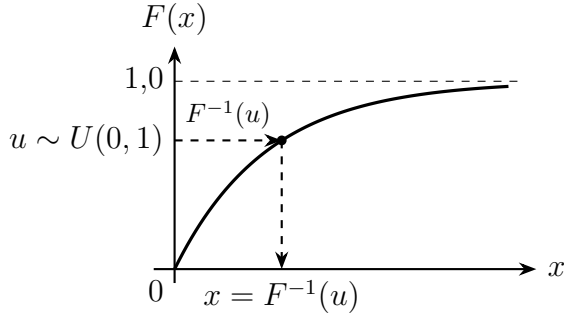
- Tưởng rằng có thể lấy mẫu trực tiếp từ mọi hàm mật độ $p(x)$ bất kỳ mà không biết hằng số chuẩn hóa $Z = \int \tilde{p}(x) dx$ $\otimes$
- Lan truyền ngược (*backprop*) trực tiếp qua lệnh lấy mẫu ngẫu nhiên $z \sim \mathcal{N}(\mu, \sigma^2)$ khi huấn luyện mạng nơ-ron $\Rightarrow$ **Gradient bị đứt gãy hoàn toàn!** $\otimes$
- Dùng Rejection Sampling trong không gian hàng trăm chiều $\Rightarrow$ tỷ lệ chấp nhận $\rightarrow 0$ $\otimes$

II. Biến đổi Ngược CDF & Reparameterization

1. Phương pháp Biến đổi Ngược (Inverse Transform Sampling)

① **Định lý & Thuật toán:** Máy tính chỉ sinh được trực tiếp biến ngẫu nhiên đều $U \sim \text{Uniform}(0, 1)$. Cho biến ngẫu nhiên mục tiêu X có hàm phân phối tích lũy $F(x) = P(X \leq x)$ liên tục và tăng ngặt. Khi đó:

$$U \sim \text{Uniform}(0, 1) \quad \Rightarrow \quad X = F^{-1}(U) \quad \text{tuân theo đúng phân phối } F(x)$$



Chứng minh 1 dòng cực gọn:

Vì F là hàm đơn điệu tăng trên $[0, 1]$, và với $U \sim \text{Uniform}(0, 1)$ ta có $P(U \leq a) = a, \forall a \in [0, 1]$:

$$\begin{aligned} P(X \leq x) &= P(F^{-1}(U) \leq x) \\ &= P(U \leq F(x)) \\ &= F(x) \quad (\text{DPCM}) \quad \checkmark \end{aligned}$$

VD 1: Sinh mẫu từ **Phân phối Mũ** $\text{Exponential}(\lambda)$ có PDF $f(x) = \lambda e^{-\lambda x}$ ($x \geq 0$).

- **Bước 1 (Tìm CDF):** $F(x) = \int_0^x \lambda e^{-\lambda t} dt = 1 - e^{-\lambda x}$.
- **Bước 2 (Giải phương trình ngược $F(x) = u$):**

$$1 - e^{-\lambda x} = u \iff e^{-\lambda x} = 1 - u \iff x = F^{-1}(u) = -\frac{\ln(1-u)}{\lambda} \stackrel{d}{=} -\frac{\ln(u)}{\lambda}$$

(Do $U \sim \text{Uniform}(0, 1)$ nên $1 - U$ cũng tuân theo $\text{Uniform}(0, 1)$). Ví dụ với $\lambda = 2$ và rút được $u = 0,5 \implies x = -\ln(0,5)/2 \approx 0,3466$.

2. Biến đổi Box-Muller & Mẹo Tái tham số hóa (Reparameterization)

① **Biến đổi Box-Muller** (Sinh phân phối Chuẩn từ Uniform): Phân phối Gaussian $\mathcal{N}(0, 1)$ không có công thức sơ cấp cho $F^{-1}(u)$. Ta rút 2 số độc lập $U_1, U_2 \sim \text{Uniform}(0, 1)$ và đổi sang tọa độ cực:

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2) \quad , \quad Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2) \implies Z_1, Z_2 \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$$

② **Reparameterization Trick** (Kingma & Welling — dùng trong VAE): Khi huấn luyện mạng VAE, bộ mã hóa (*Encoder*) xuất ra vector $\mu_{(d \times 1)}$ và $\sigma_{(d \times 1)}$, sau đó cần lấy mẫu $z \sim \mathcal{N}(\mu, \text{diag}(\sigma^2))$ đưa vào Decoder.

- Nếu lấy mẫu trực tiếp $z \sim \mathcal{N}(\mu, \sigma^2)$: Toán tử ngẫu nhiên chặn đứng đạo hàm $\frac{\partial z}{\partial \mu} \otimes$.
- **Giải pháp Tái tham số hóa:** Tách riêng phần nhiễu ngẫu nhiên không chứa tham số $\varepsilon \sim \mathcal{N}(0, I)$ ra ngoài nhánh tính toán:

$$z = \mu + \sigma \odot \varepsilon \quad \text{với } \varepsilon \sim \mathcal{N}(0, I) \implies \frac{\partial z}{\partial \mu} = 1, \quad \frac{\partial z}{\partial \sigma} = \varepsilon \quad \checkmark$$

- **Mở rộng cho biến rời rạc (Gumbel-Softmax):** Thay $\arg \max_i (\ln p_i + g_i)$ (với nhiễu Gumbel $g_i = -\ln(-\ln u_i)$) bằng hàm khả vi $y_i = \frac{\exp((\ln p_i + g_i)/\tau)}{\sum_j \exp((\ln p_j + g_j)/\tau)}$.

III. Lấy mẫu Bác bỏ (Rejection) & Lấy mẫu Quan trọng (Importance)

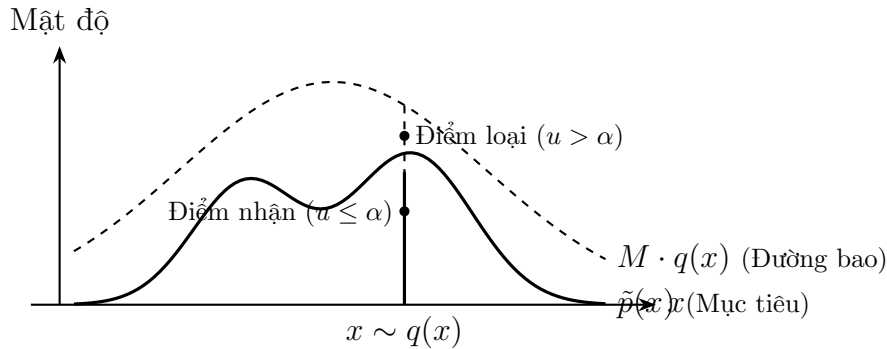
1. Lấy mẫu Bác bỏ <Rejection Sampling>

① Bài toán & Thuật toán: Ta cần lấy mẫu từ phân phối mục tiêu $p(x) = \frac{\tilde{p}(x)}{Z}$ (biết hàm $\tilde{p}(x)$ nhưng không tính được hằng số chuẩn hóa Z). Chọn một phân phối đề xuất (*proposal distribution*) $q(x)$ dễ lấy mẫu và hằng số bao $M > 0$ sao cho đường cong $Mq(x) \geq \tilde{p}(x)$, $\forall x$:

1) **Bước 1**: Rút ứng viên $x \sim q(x)$ và rút $u \sim \text{Uniform}(0, 1)$.

2) **Bước 2 (Kiểm tra chấp nhận)**: Tính tỷ lệ chấp nhận $\alpha(x)$ và quyết định:

$$\alpha(x) = \frac{\tilde{p}(x)}{M \cdot q(x)} \in [0, 1] \implies \text{Nếu } u \leq \alpha(x) \text{ thì **Chấp nhận } x**; ngược lại **Bác bỏ**}$$



❓ Vì sao Rejection Sampling sụp đổ hoàn toàn trong không gian nhiều chiều (*Curse of Dimensionality*)?

- Tỷ lệ chấp nhận trung bình bằng tỷ số thể tích dưới hai mặt cong:

$$P(\text{Accept}) = \int \frac{\tilde{p}(x)}{Mq(x)} q(x) dx = \frac{Z}{M}$$

- Giả sử trong d chiều, $p(x) = \mathcal{N}(0, \sigma_p^2 I_d)$ và $q(x) = \mathcal{N}(0, \sigma_q^2 I_d)$ với $\sigma_q/\sigma_p = 1,1$ (chỉ rộng hơn 10% ở mỗi chiều). Khi đó hệ số bao $M = \left(\frac{\sigma_q}{\sigma_p}\right)^d = (1,1)^d$. Với $d = 100$ chiều, $P(\text{Accept}) = (1,1)^{-100} \approx 0,0072\%$ (cứ 14 000 mẫu mới nhận được 1 mẫu!).

2. Lấy mẫu Quan trọng <Importance Sampling & PPO>

Khi chỉ cần tính kỳ vọng $\mathbb{E}_{x \sim p}[f(x)]$ (hoặc tập mẫu được thu thập từ phân phối cũ $q(x)$ khác với phân phối hiện tại $p(x)$), ta nhân và chia cho $q(x) > 0$:

$$\mathbb{E}_{x \sim p}[f(x)] = \int f(x) \frac{p(x)}{q(x)} q(x) dx = \mathbb{E}_{x \sim q}[f(x) \cdot w(x)] \approx \frac{1}{N} \sum_{i=1}^N w(x_i) f(x_i)$$

trong đó $w(x_i) = \frac{p(x_i)}{q(x_i)}$ được gọi là **Trọng số quan trọng (Importance Weight)**.

★ **Ứng dụng trực tiếp trong Học tăng cường (Off-Policy RL / PPO):** Khi Agent thu thập dữ liệu hành động a_t dưới chính sách cũ $\pi_{\theta_{\text{old}}}(a_t | s_t)$ nhưng cần cập nhật đạo hàm cho chính sách mới $\pi_{\theta}(a_t | s_t)$, hàm mục tiêu PPO dùng đúng tỷ số Importance Sampling:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad \langle \text{PPO cắt ngưỡng clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \text{ để chống nổ phương sai!} \rangle$$

IV. Markov Chain Monte Carlo (MCMC) & Metropolis-Hastings

① **Ý tưởng đột phá của MCMC:** Thay vì lấy các mẫu độc lập bị bác bỏ liên tục trong không gian nhiều chiều, ta thiết kế một **Chuỗi Markov** dịch chuyển cục bộ từ trạng thái hiện tại $x_t \rightarrow x_{t+1}$ sao cho **phân phối dừng (stationary distribution)** của chuỗi trùng khít với $p(x)$.

② **Thuật toán Metropolis-Hastings (MH):** Tại bước t , đứng ở vị trí x , ta đề xuất bước nhảy $x' \sim q(x' | x)$ và chấp nhận bước nhảy $x \rightarrow x'$ với xác suất:

$$A(x \rightarrow x') = \min\left(1, \frac{p(x') q(x | x')}{p(x) q(x' | x)}\right) = \min\left(1, \frac{\tilde{p}(x') q(x | x')}{\tilde{p}(x) q(x' | x)}\right)$$

(Nếu từ chối, chuỗi giữ nguyên trạng thái cho bước kế tiếp: $x_{t+1} = x_t$).

③ **Vì sao hằng số chuẩn hóa bất khả tính $Z = \int \tilde{p}(x) dx$ lại triệt tiêu hoàn toàn?**

- Thay $p(x) = \frac{\tilde{p}(x)}{Z}$ và $p(x') = \frac{\tilde{p}(x')}{Z}$ vào tỷ số chấp nhận:

$$\frac{p(x')}{p(x)} = \frac{\tilde{p}(x')/Z}{\tilde{p}(x)/Z} = \frac{\tilde{p}(x')}{\tilde{p}(x)} \quad \checkmark$$

Nhờ phép chia này, Metropolis-Hastings lấy mẫu được từ **mọi phân phối hậu nghiệm Bayes phức tạp** $P(\theta | \mathcal{D}) = \frac{P(\mathcal{D} | \theta)P(\theta)}{P(\mathcal{D})}$ mà không bao giờ cần tính mẫu số $P(\mathcal{D})$!

- Đặc biệt, nếu phân phối đề xuất đối xứng $q(x' | x) = q(x | x')$ (VD: Random Walk Gaussian $x' \sim \mathcal{N}(x, \sigma^2 I)$), công thức rút gọn thành **thuật toán Metropolis**: $A(x \rightarrow x') = \min\left(1, \frac{\tilde{p}(x')}{\tilde{p}(x)}\right)$ (nếu điểm mới có mật độ cao hơn $\tilde{p}(x') \geq \tilde{p}(x)$ thì luôn nhận 100%; nếu thấp hơn thì vẫn nhận với xác suất $\tilde{p}(x')/\tilde{p}(x)$ để khám phá hết không gian).

③ **Các khái niệm vận hành MCMC thực tế:**

- **Điều kiện Cân bằng Chi tiết (Detailed Balance):** $p(x)T(x \rightarrow x') = p(x')T(x' \rightarrow x)$ bảo chứng toán học rằng $p(x)$ là phân phối dừng của chuỗi.

- **Giai đoạn Khởi động (Burn-in):** Loại bỏ B mẫu đầu tiên khi chuỗi chưa kịp tiến vào vùng mật độ cao ổn định.
- **Độ rộng bước nhảy (Proposal Scale σ):** Nếu σ quá nhỏ $\implies$ bước đi li ti, tự tương quan cao (*slow mixing*); nếu σ quá lớn $\implies$ nhảy vọt ra vùng xác suất bằng 0, bị từ chối liên tục và đứng im tại chỗ!
- **Gibbs Sampling:** Trường hợp đặc biệt của MCMC khi cập nhật lần lượt từng biến $x_j \sim p(x_j \mid x_{-j})$ giữ nguyên các biến còn lại — xác suất chấp nhận luôn bằng 100%.

V. Chiến lược Giải mã & Lấy mẫu Token trong LLM (LLM Decoding)

Ở bước sinh từ thứ t , tầng cuối của Transformer xuất ra vector điểm thô (**logits**) $z = [z_1, z_2, \dots, z_V]^T$. Quá trình biến đổi từ z thành token được chọn đi qua chuỗi bộ lọc toán học sau:

1. Temperature Scaling, Top- k & Top- p (Nucleus Sampling)

① Điều chỉnh Nhiệt độ (Temperature Scaling $T > 0$):

$$P_i(T) = \frac{\exp(z_i/T)}{\sum_{j=1}^V \exp(z_j/T)} \Rightarrow \begin{cases} T \rightarrow 0^+ : \text{ Suy biến thành 1 đỉnh (Greedy } \arg \max_i z_i) \\ T < 1 : \text{ Làm nhọn phân phối (Tập trung, an toàn)} \\ T = 1 : \text{ Giữ nguyên phân phối Softmax chuẩn} \\ T > 1 : \text{ Làm phẳng phân phối (Đa dạng, dễ ảo giác)} \end{cases}$$

② Top- k Sampling vs Top- p (Nucleus) Sampling:

- **Top- k Sampling:** Sắp xếp xác suất giảm dần $P_{(1)} \geq P_{(2)} \geq \dots \geq P_{(V)}$, chỉ giữ lại **đúng k token đầu tiên**, gán toàn bộ các token từ $k+1$ trở đi bằng 0, sau đó **chuẩn hóa lại (renormalize)** để tổng bằng 1:

$$P'_i = \frac{P_i}{\sum_{j \in \mathcal{V}_k} P_j} \quad \text{với } i \in \mathcal{V}_k \text{ (tập } k \text{ token lớn nhất)}$$

- **Top- p (Nucleus) Sampling (Holtzman et al.):** Tìm tập con nhỏ nhất $\mathcal{V}_p$ gồm các token đứng đầu sao cho **tổng xác suất tích lũy vừa vượt ngưỡng p** (VD $p = 0,80$ hoặc $0,90$):

$$\min |\mathcal{V}_p| \quad \text{sao cho} \quad \sum_{i \in \mathcal{V}_p} P_i \geq p \quad \Rightarrow \quad \text{Chuẩn hóa lại } P''_i = \frac{P_i}{\sum_{j \in \mathcal{V}_p} P_j}$$

❓ **Vì sao Top- p (Nucleus) lại ưu việt hơn Top- k trong các LLM hiện đại?**

- Top- k giữ cố định số lượng ứng viên k . Khi ngữ cảnh đã quá rõ ràng (VD: “*Thủ đô của Việt Nam là ___*”, từ “*Hà Nội*” chiếm 96%), việc giữ $k = 50$ vẫn mở cửa cho 49 từ sai lầm lọt vào! Ngược lại, khi ngữ cảnh mờ (“*Hôm nay tôi cảm thấy ___*”), hàng trăm tính từ đều hợp lý thì $k = 10$ lại cắt cụt sự tự nhiên.
- **Top- p tự động co giãn kích thước tập ứng viên** theo độ chắc chắn (Entropy) của mô hình tại từng bước!

2. Ví dụ Số học Từng bước: Tái phân phối Xác suất Token

VD 2: Giả sử tại bước sinh từ tiếp theo, từ điển rút gọn gồm 5 token ứng viên A, B, C, D, E đã qua Softmax (ở $T = 1,0$) có phân phối xác suất giảm dần như sau:

$$P = [P(A) = 0,45, \quad P(B) = 0,25, \quad P(C) = 0,15, \quad P(D) = 0,10, \quad P(E) = 0,05]$$

- **Trường hợp 1 — Greedy Decoding ($T \rightarrow 0$):** Chọn trực tiếp $\arg \max P = \text{Token A}$ với xác suất 100%.

- **Trường hợp 2 — Áp dụng Top- k với $k = 2$:**

- **Bước 1 (Lọc):** Chỉ giữ lại 2 token lớn nhất là $\{A, B\}$, loại bỏ C, D, E (gán xác suất = 0).
- **Bước 2 (Tính tổng phần giữ lại):** $S_k = P(A) + P(B) = 0,45 + 0,25 = 0,70$.
- **Bước 3 (Chuẩn hóa lại — Renormalize):**

$$P'_k = \left[P'(A) = \frac{0,45}{0,70} \approx \mathbf{64,29\%}, \quad P'(B) = \frac{0,25}{0,70} \approx \mathbf{35,71\%}, \quad 0, \quad 0, \quad 0 \right]$$

- **Trường hợp 3 — Áp dụng Top- p (Nucleus) với ngưỡng $p = 0,80$:**

- **Bước 1 (Cộng dồn xác suất tích lũy — Cumulative Sum):**

Xét $\{A\}$: $\text{CumSum}_1 = 0,45 < 0,80$ (chưa đủ)

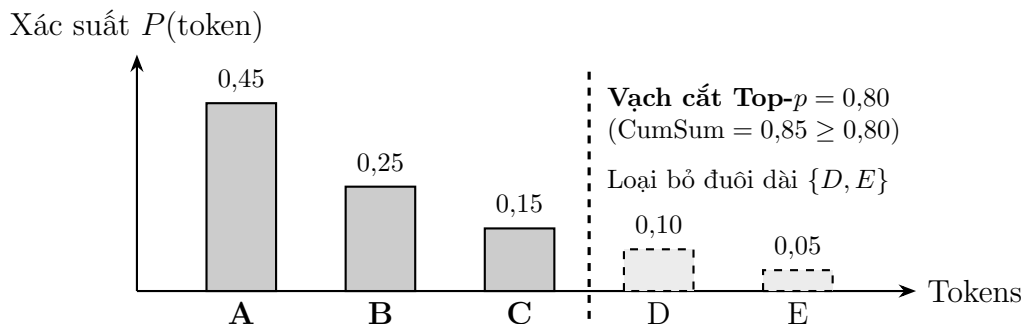
Xét $\{A, B\}$: $\text{CumSum}_2 = 0,45 + 0,25 = 0,70 < 0,80$ (chưa đủ)

Xét $\{A, B, C\}$: $\text{CumSum}_3 = 0,45 + 0,25 + 0,15 = \mathbf{0,85} \geq \mathbf{0,80} \implies \mathbf{DỪNG!}$

- **Bước 2 (Chuẩn hóa lại trên tập hạt nhân $\mathcal{V}_p = \{A, B, C\}$ có tổng $S_p = 0,85$):**

$$P''(A) = \frac{0,45}{0,85} \approx \mathbf{52,94\%}, \quad P''(B) = \frac{0,25}{0,85} \approx \mathbf{29,41\%}, \quad P''(C) = \frac{0,15}{0,85} \approx \mathbf{17,65\%}$$

$$\implies P''_p = [0,5294, \quad 0,2941, \quad 0,1765, \quad 0, \quad 0]$$



VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 16

- **Inverse Transform & Reparameterization Trick:** Rút $U \sim \text{Uniform}(0, 1) \implies X = F^{-1}(U) \sim F(x)$. Trong mạng VAE, tách nhiễu $z = \mu + \sigma \odot \varepsilon$ ($\varepsilon \sim \mathcal{N}(0, I)$) giúp đạo hàm $\frac{\partial z}{\partial \mu} = 1$, $\frac{\partial z}{\partial \sigma} = \varepsilon$ chảy mượt mà qua bước lấy mẫu.
- **Rejection Sampling vs Importance Sampling:** Rejection Sampling chấp nhận mẫu $x \sim q(x)$ với xác suất $\alpha = \frac{\tilde{p}(x)}{Mq(x)}$ nhưng bị sụp đổ theo hàm mũ ở nhiều chiều. Importance Sampling đổi phân phối lấy kỳ vọng: $\mathbb{E}_p[f(x)] = \mathbb{E}_q\left[f(x) \frac{p(x)}{q(x)}\right]$ (nền tảng của thuật toán PPO trong RL).
- **Metropolis-Hastings MCMC:** Tỷ lệ chấp nhận $A(x \rightarrow x') = \min\left(1, \frac{\tilde{p}(x')q(x | x')}{\tilde{p}(x)q(x' | x)}\right)$ giúp triệt tiêu hoàn toàn hằng số chuẩn hóa Z , cho phép lấy mẫu từ mọi phân phối hậu nghiệm nhiều chiều.
- **LLM Token Sampling:** Temperature $P_i \propto \exp(z_i/T)$ điều chỉnh độ nhọn phân phối; **Top- k** giữ cố định k từ cao nhất; **Top- p (Nucleus)** giữ tập từ nhỏ nhất có tổng tích lũy $\geq p$ rồi chuẩn hóa lại xác suất.

Lesson 17: Linear Systems

I. Vấn đề (The Problem)

- Phương trình tuyến tính $\underset{(n \times n)}{A} @ \underset{(n \times 1)}{x} = \underset{(n \times 1)}{b}$ xuất hiện ở tận cùng của hầu hết mọi thuật toán AI và Tính toán Khoa học:
 - 1) **Hồi quy Tuyến tính & Ridge Regression:** Giải hệ phương trình pháp tuyến $(X^T X + \lambda I)w = X^T y$.
 - 2) **Tối ưu hóa bậc hai (Newton's Method):** Tìm bước nhảy Δw qua hệ Hessian $H \cdot \Delta w = -\nabla L(w)$.
 - 3) **Gaussian Processes (GP):** Giải hệ ma trận hạt nhân $(K + \sigma_n^2 I)\alpha = y$ để dự đoán trung bình hậu nghiệm.
 - 4) **Thuật toán PageRank & Nội suy Spline:** Giải hệ trạng thái dừng $(I - d \cdot M^T)r = \frac{1-d}{N}\mathbf{1}$.
- Khi viết code giải $Ax = b$, người mới học thường dịch nguyên xi công thức toán $x = A^{-1}b$ sang Python thành `np.linalg.inv(A) @ b`. Trong kỹ thuật phần mềm AI, đây là **điều cấm kỵ tuyệt đối!**

WRONG — Tại sao `inv(A) @ b` là “tội đồ” trong Production?

- **Cách sai:** Tính ma trận nghịch đảo tường minh A^{-1} rồi nhân với b :
`x = np.linalg.inv(A) @ b` (⊗)
 - **Chậm gấp 3 lần:** Phân rã LU tốn $\frac{2}{3}n^3$ phép tính, trong khi tìm đầy đủ A^{-1} tương đương giải n hệ phương trình với n cột của ma trận đơn vị I , tốn tới $2n^3$ phép tính!
 - **Khuếch đại sai số dấu phẩy động:** Gấp đôi sai số làm tròn (*round-off error*) khi ma trận gần suy biến.
- **Cách đúng chuẩn công nghiệp:** Luôn giải trực tiếp hệ bằng phân rã ma trận (`np.linalg.solve(A, b)` dùng $PA = LU$, hoặc `scipy.linalg.cho_solve` dùng Cholesky $A = LL^T$) (✓)

II. Khử Gauss, Đổi hàng (Partial Pivoting) & Phân rã $PA = LU$

1. Góc nhìn Hàng (Row Picture) vs Góc nhìn Cột (Column Picture)

$$\underset{(2 \times 2)}{\begin{bmatrix} 2 & 1 \\ 1 & -1 \end{bmatrix}} @ \underset{(2 \times 1)}{\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}} = \underset{(2 \times 1)}{\begin{bmatrix} 5 \\ 1 \end{bmatrix}} \iff \begin{cases} \text{Row: Giao điểm của } 2x_1 + x_2 = 5 \text{ và } x_1 - x_2 = 1 \\ \text{Col: Tổ hợp cột: } x_1 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + x_2 \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \end{bmatrix} \end{cases}$$

$\Rightarrow$ Hệ có nghiệm duy nhất $\iff \vec{b} \in \text{Col}(A)$ và các cột của A độc lập tuyến tính ($\det(A) \neq 0$).

2. Khử Gauss & Phân rã $PA = LU$: Giải hệ trong $\mathcal{O}(n^2)$ sau khi phân rã

① **Bản chất Phân rã $A = L \cdot U$** : Khử Gauss (*Gaussian Elimination*) dùng các phép biến đổi sơ cấp dòng để triệt tiêu các phần tử dưới đường chéo chính, biến A thành ma trận tam giác trên U (*Upper triangular*). Nếu ta lưu lại đúng các **hệ số nhân khử** $m_{ik} = \frac{u_{ik}}{u_{kk}}$ vào nửa dưới của ma trận tam giác dưới đơn vị L (*Unit Lower triangular*, có đường chéo bằng 1), ta thu được:

$$A = \begin{matrix} L & @ & U \\ \text{Tam giác dưới} & & \text{Tam giác trên} \end{matrix} \quad (\text{hoặc tổng quát có đổi hàng: } P @ A = L @ U)$$

❓ Vì sao phải tách $PA = LU$ thay vì chỉ khử Gauss trực tiếp trên $[A | b]$?

- Bước phân rã $PA = LU$ chỉ phụ thuộc vào A , tốn $\mathcal{O}(\frac{2}{3}n^3)$ và **chỉ cần làm 1 lần duy nhất**.
- Khi có hàng nghìn vector vế phải $b^{(1)}, b^{(2)}, \dots, b^{(K)}$ mới bay tới, ta giải $Ax = b \iff PAx = Pb \iff L(Ux) = Pb$ qua 2 bước thế tam giác cực nhanh tốn chỉ $\mathcal{O}(n^2)$:

Bước 1 — Thế xuôi (Forward, $\mathcal{O}(n^2)$): Giải $L \cdot y = P \cdot b$ (từ hàng 1 $\rightarrow n$)
Bước 2 — Thế ngược (Back, $\mathcal{O}(n^2)$): Giải $U \cdot x = y$ (từ hàng $n \rightarrow 1$)

VD 1: Phân rã $A = LU$ và giải hệ $Ax = b$ từng bước với:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 2 & 3 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 8 \\ 20 \\ 12 \end{bmatrix}$$

- **Bước 1 (Khử cột 1 dưới phần tử chốt $u_{11} = 2$):**

- Hệ số nhân hàng 2: $l_{21} = \frac{4}{2} = 2 \implies R_2 \leftarrow R_2 - 2R_1 = [4, 3, 3] - [4, 2, 2] = [0, 1, 1]$.
- Hệ số nhân hàng 3: $l_{31} = \frac{2}{2} = 1 \implies R_3 \leftarrow R_3 - 1R_1 = [2, 3, 1] - [2, 1, 1] = [0, 2, 0]$.

- **Bước 2 (Khử cột 2 dưới phần tử chốt $u_{22} = 1$):**

- Hệ số nhân hàng 3: $l_{32} = \frac{2}{1} = 2 \implies R_3 \leftarrow R_3 - 2R_2 = [0, 2, 0] - [0, 2, 2] = [0, 0, -2]$.

- **Bước 3 (Ghi nhận hai ma trận L và U):**

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & -2 \end{bmatrix} \quad (\text{Kiểm tra: } L @ U = A \checkmark)$$

- **Bước 4 (Thế xuôi $Ly = b$ từ trên xuống):**

$$\begin{cases} y_1 = 8 \\ 2y_1 + y_2 = 20 \implies y_2 = 20 - 16 = 4 \\ y_1 + 2y_2 + y_3 = 12 \implies y_3 = 12 - 8 - 8 = -4 \end{cases} \implies y = \begin{bmatrix} 8 \\ 4 \\ -4 \end{bmatrix}$$

- **Bước 5 (Thế ngược $Ux = y$ từ dưới lên):**

$$\begin{cases} -2x_3 = -4 \implies x_3 = 2 \\ x_2 + x_3 = 4 \implies x_2 = 4 - 2 = 2 \\ 2x_1 + x_2 + x_3 = 8 \implies 2x_1 = 8 - 2 - 2 = 4 \implies x_1 = 2 \end{cases} \implies x = \begin{bmatrix} 2 \\ 2 \\ 2 \end{bmatrix} \quad (\checkmark)$$

3. Tại sao Bắt buộc phải chọn Phần tử Chốt (Partial Pivoting)?

Nếu phần tử chốt trên đường chéo $u_{kk} = 0$ (phép chia cho 0 bị sập) hoặc $u_{kk} = 10^{-15}$ (cực nhỏ), hệ số nhân $l_{ik} = \frac{u_{ik}}{u_{kk}}$ sẽ nổ lên thành 10^{15} , làm mất sạch các chữ số có nghĩa khi trừ hai số dấu phẩy động (*catastrophic cancellation*).

- ★ **Quy tắc Partial Pivoting:** Tại cột k , luôn tìm phần tử có **trị tuyệt đối lớn nhất** từ hàng k đến hàng n : $p = \arg \max_{i \geq k} |u_{ik}|$, sau đó **đổi chỗ hàng k và hàng p** (lưu vào ma trận hoán vị P). Khi đó mọi hệ số nhân đều bị chặn chặt: $|l_{ik}| \leq 1$, đảm bảo ổn định số học tuyệt đối!

III. Phân rã Cholesky ($A = LL^T$) cho Ma trận SPD

① **Điều kiện & Định lý:** Khi ma trận vuông $A \in \mathbb{R}^{n \times n}$ là **Đối xứng Xác định Dương** (*Symmetric Positive Definite — SPD*), tức là $A = A^T$ và $x^T A x > 0, \forall x \neq 0$ (mọi trị riêng $\lambda_i > 0$), ta có thể lấy “căn bậc hai” của ma trận dưới dạng:

$$A = \begin{matrix} L & @ & L^T \\ \text{Tam giác dưới} & & \text{Tam giác trên} \end{matrix} \quad \text{với } l_{ii} > 0$$

Công thức đệ quy tính từng phần tử của L (từ cột trái sang phải):

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2} \quad , \quad l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right) \quad (\text{với } i > j)$$

② Vì sao Cholesky là “vũ khí tối thượng” cho Ridge Regression, Covariance Matrix và Gaussian Processes?

- **Nhanh gấp đôi LU:** Tận dụng tính đối xứng $A = A^T$, Cholesky chỉ tốn $\frac{1}{3}n^3$ phép tính (so với $\frac{2}{3}n^3$ của LU).
- **Tiết kiệm 50% bộ nhớ RAM:** Chỉ cần lưu đúng 1 ma trận tam giác dưới L (không cần lưu U riêng và không cần hoán vị P vì ma trận SPD luôn ổn định không cần pivoting!).
- **Tính Log-Determinant tức thì trong GP:** Thay vì tính $\det(A)$ trực tiếp dễ bị tràn số, ta dùng đường chéo của L :

$$\ln \det(A) = \ln (\det(L) \det(L^T)) = 2 \sum_{i=1}^n \ln(l_{ii}) \quad \checkmark$$

VD 2: Thực hiện phân rã Cholesky $A = LL^T$ cho ma trận SPD 3×3 :

$$A = \begin{bmatrix} 4 & 2 & -2 \\ 2 & 10 & 2 \\ -2 & 2 & 5 \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} @ \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

- **Bước 1 (Tính cột 1 của L):**

$$l_{11} = \sqrt{a_{11}} = \sqrt{4} = 2 \quad ; \quad l_{21} = \frac{a_{21}}{l_{11}} = \frac{2}{2} = 1 \quad ; \quad l_{31} = \frac{a_{31}}{l_{11}} = \frac{-2}{2} = -1$$

- **Bước 2 (Tính cột 2 của L):**

$$l_{22} = \sqrt{a_{22} - l_{21}^2} = \sqrt{10 - 1^2} = 3 \quad ; \quad l_{32} = \frac{a_{32} - l_{31}l_{21}}{l_{22}} = \frac{2 - (-1)(1)}{3} = \frac{3}{3} = 1$$

- **Bước 3 (Tính cột 3 của L):**

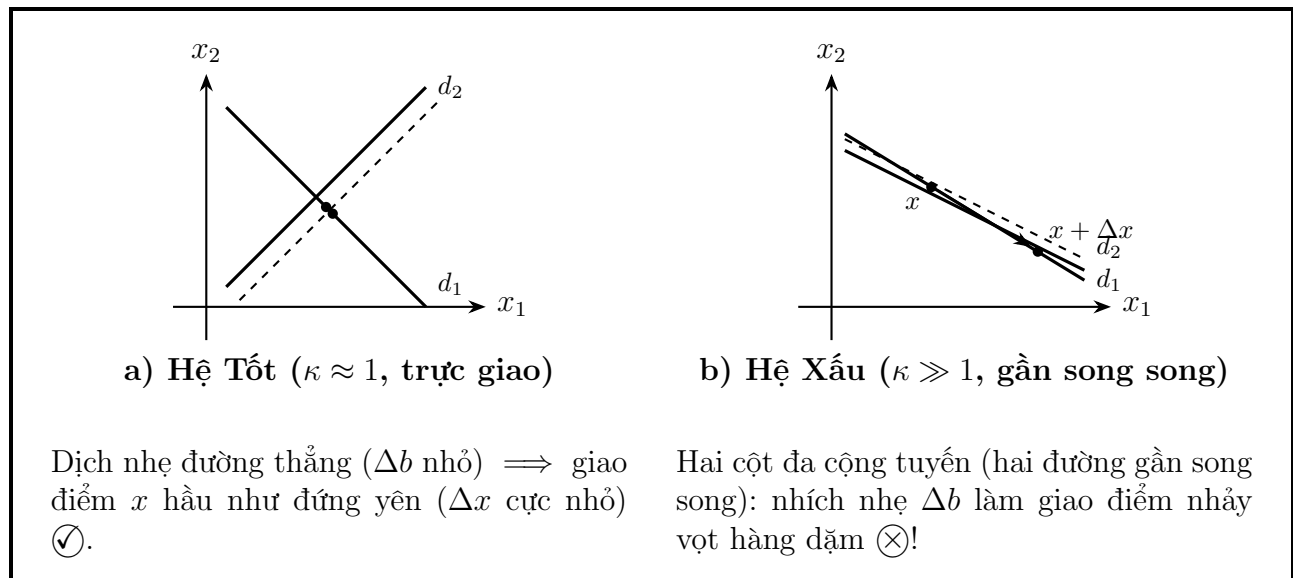
$$l_{33} = \sqrt{a_{33} - l_{31}^2 - l_{32}^2} = \sqrt{5 - (-1)^2 - 1^2} = \sqrt{3} \quad \Rightarrow \quad L = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ -1 & 1 & \sqrt{3} \end{bmatrix} \quad \checkmark$$

IV. Số Điều Kiện $\kappa(A)$ (Condition Number)

① **Định nghĩa & Bất đẳng thức Độ nhạy Sai số:** Giả sử vector nhân b bị nhiễu một lượng nhỏ Δb (do sai số đo đạc hoặc làm tròn máy tính), làm nghiệm dịch chuyển từ $x \rightarrow x + \Delta x$. Độ phóng đại sai số tương đối bị chi phối bởi **Số điều kiện (Condition Number)** $\kappa(A)$:

$$\kappa(A) = \|A\|_2 \cdot \|A^{-1}\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \geq 1 \quad \Rightarrow \quad \frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \cdot \frac{\|\Delta b\|}{\|b\|}$$

(Với ma trận đối xứng xác định dương, $\sigma_i = \lambda_i \Rightarrow \kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$).



★ **Quy tắc Mất chữ số có nghĩa (Rule of Thumb):** Nếu $\kappa(A) \approx 10^k$, bạn sẽ **mất đi khoảng k chữ số thập phân chính xác** khi giải hệ! Với kiểu `float64` (có 15–16 chữ số có nghĩa), nếu $\kappa(A) = 10^8$ thì nghiệm chỉ còn tin cậy được $15 - 8 = 7$ chữ số; nếu $\kappa(A) \geq 10^{16}$ thì kết quả trả về hoàn toàn là rác số học!

V. Hệ Quá Xác Định ($m > n$), Phân rã QR & Ổn định hóa Ridge

1. Phương trình Pháp tuyến (Normal Equations) làm Bình phương Số Điều Kiện!

Trong bài toán Hồi quy tuyến tính với ma trận đặc trưng X ($m \gg n$, nhiều mẫu hơn số đặc trưng), hệ $Xw = y$ vô nghiệm. Để tối thiểu hóa sai số bình phương $\min_w \|Xw - y\|_2^2$, ta lấy đạo hàm theo w bằng 0 và thu được **Hệ phương trình Pháp tuyến (Normal Equations)**:

$$\begin{matrix} (X^T X) & @ & w & = & X^T y \\ (n \times n) & & (n \times 1) & & (n \times 1) \end{matrix}$$

❓ Vì sao giải trực tiếp $X^T X w = X^T y$ lại rất nguy hiểm khi các đặc trưng gần đa cộng tuyến?

- Giả sử phân rã SVD của X có các giá trị kỳ dị $\sigma_{\max}$ và $\sigma_{\min}$, tức là $\kappa(X) = \frac{\sigma_{\max}}{\sigma_{\min}}$.
- Khi nhân $X^T X$, các trị riêng của $X^T X$ chính là $\lambda_i(X^T X) = \sigma_i^2(X)$. Do đó, số điều kiện của ma trận $X^T X$ bị **BÌNH PHƯƠNG LÊN**:

$$\kappa(X^T X) = \frac{\sigma_{\max}^2}{\sigma_{\min}^2} = [\kappa(X)]^2$$

- **Hậu quả:** Nếu dữ liệu gốc có $\kappa(X) = 10^8$ (vẫn giải tốt trong float64), việc lập ma trận $X^T X$ đẩy số điều kiện lên $\kappa(X^T X) = (10^8)^2 = 10^{16} \implies$ mất sạch cả 16 chữ số chính xác!

2. Giải pháp 1 — Phân rã QR ($X = Q \cdot R$): Không bao giờ bình phương $\kappa(X)$

Thay vì nhân $X^T X$, ta phân rã trực chuẩn $X = \begin{matrix} Q \\ (m \times n) \end{matrix} @ \begin{matrix} R \\ (n \times n) \end{matrix}$ (với $Q^T Q = I_n$ và R tam giác trên). Thay vào phương trình pháp tuyến:

$$X^T X w = X^T y \iff R^T \underbrace{(Q^T Q)}_{I_n} R w = R^T Q^T y \iff \boxed{R @ w = Q^T y}$$

Vì Q là phép biến đổi trực giao bảo toàn độ dài nên $\boxed{\kappa(R) = \kappa(X)}$ (không bị bình phương!). Ta chỉ việc giải hệ tam giác trên $Rw = Q^T y$ bằng phép thế ngược (*Back Substitution*) cực kỳ ổn định.

3. Giải pháp 2 — Điều chuẩn Ridge (Tikhonov Regularization)

Khi các cột của X gần như phụ thuộc tuyến tính ($\sigma_{\min}^2 = \lambda_{\min} \approx 0 \implies \kappa(X^T X) \rightarrow \infty$), ta cộng thêm ma trận chéo λI ($\lambda > 0$) vào $X^T X$:

$$\boxed{(X^T X + \lambda I)w = X^T y}$$

① Cơ chế Dịch chuyển Trị riêng (*Eigenvalue Shift*) cứu sống hệ phương trình:

- Gọi $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$ là các trị riêng của ma trận đối xứng bán xác định dương $X^T X$ (ứng với các vector riêng v_i).
- Khi cộng thêm λI , ta có:

$$(X^T X + \lambda I)v_i = X^T X v_i + \lambda v_i = (\lambda_i + \lambda)v_i$$

Nghĩa là **mọi trị riêng đều được nâng lên một lượng hằng số $+\lambda$** : $\lambda_i \longrightarrow \lambda_i + \lambda > 0$.

- Khi đó, số điều kiện mới giảm mạnh từ vô cùng về một con số hữu hạn an toàn:

$$\kappa_{\text{OLS}} = \frac{\lambda_{\max}}{\lambda_{\min}} \quad (\rightarrow \infty \text{ khi } \lambda_{\min} \rightarrow 0) \quad \xrightarrow{+\lambda I} \quad \kappa_{\text{Ridge}} = \frac{\lambda_{\max} + \lambda}{\lambda_{\min} + \lambda} \ll \kappa_{\text{OLS}}$$

VD 3: Giả sử ma trận $X^T X$ có trị riêng lớn nhất $\lambda_{\max} = 100$ và trị riêng nhỏ nhất gần suy biến $\lambda_{\min} = 10^{-8}$.

- **Khi chưa có Ridge** ($\lambda = 0$): $\kappa(X^T X) = \frac{100}{10^{-8}} = \mathbf{10^{10}}$ (mất 10 chữ số chính xác, trọng số w nổ tung).
- **Khi thêm Ridge với $\lambda = 0,1$** : Giảm độ nhạy sai số đi 10 triệu lần:

$$\kappa(X^T X + 0,1I) = \frac{100 + 0,1}{10^{-8} + 0,1} \approx \frac{100,1}{0,1} = \mathbf{1\,001} \approx \mathbf{10^3} \quad \checkmark$$

Hơn nữa, vì $X^T X + \lambda I$ luôn là ma trận **Đối xứng Xác định Dương (SPD)** nghiêm ngặt với mọi $\lambda > 0$, ta luôn giải được nó bằng **Phân rã Cholesky** nhanh gấp đôi LU !

Phương pháp	Điều kiện áp dụng	Độ phức tạp	Ứng dụng chuẩn trong AI / ML
Phân rã $PA = LU$	A vuông, khả nghịch	$\frac{2}{3}n^3$ (factor) $+\mathcal{O}(n^2)$ (solve)	Bộ giải mặc định <code>linalg.solve(A, b)</code> cho hệ vuông tổng quát
Phân rã Cholesky ($A = LL^T$)	A đối xứng xác định dương (SPD)	$\frac{1}{3}n^3$ (Nhanh 2×, tiết kiệm 50% RAM)	Ridge Regression ($X^T X + \lambda I$), Gaussian Processes, Mahalanobis
Phân rã QR ($X = QR$)	X chữ nhật ($m \geq n$), đủ hạng cột	$\mathcal{O}(mn^2)$	Least Squares chuẩn ($Rw = Q^T y$), giữ nguyên $\kappa(R) = \kappa(X)$
Giải nghịch đảo SVD ($X^+ = V\Sigma^+U^T$)	Mọi ma trận X (kể cả khuyết hạng)	$\mathcal{O}(mn^2)$	<code>np.linalg.lstsq</code> / <code>pinv</code> : nghiệm chuẩn nhỏ nhất khi đa cộng tuyến

VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 17

- **Tuyệt đối không dùng $\text{inv}(A)$ @ b:** Nghịch đảo ma trận thường minh chậm gấp 3 lần và kém ổn định số học. Luôn chuyển về giải hệ tam giác bằng phép thế xuôi ($Ly = Pb$) và thế ngược ($Ux = y$) trong $\mathcal{O}(n^2)$.
- **Khử Gauss, Partial Pivoting & $PA = LU$:** Đổi hàng để chọn phần tử chốt lớn nhất ($\max_{i \geq k} |u_{ik}|$) giúp chặn các hệ số nhân $|l_{ik}| \leq 1$, chống tràn số và mất chữ số có nghĩa.
- **Phân rã Cholesky ($A = LL^T$):** Dành riêng cho ma trận Đối xứng Xác định Dương (SPD), tốn chỉ $\frac{1}{3}n^3$ phép tính (nhanh gấp 2 lần LU) và cho phép tính $\ln \det(A) = 2 \sum \ln l_{ii}$ an toàn tuyệt đối.
- **Số điều kiện (Condition Number):** $\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$ đo độ phóng đại sai số tương đối $\frac{\|\Delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\Delta b\|}{\|b\|}$. Với $\kappa(A) \approx 10^k$, ta mất khoảng k chữ số thập phân chính xác.
- **Bình phương tối thiểu & Ridge Stabilization:** Giải trực tiếp $X^T X w = X^T y$ làm bình phương số điều kiện $\kappa(X^T X) = \kappa(X)^2$. Dùng **phân rã QR** ($Rw = Q^T y$) để bảo toàn κ , hoặc cộng **Ridge** λI để dịch mọi trị riêng $\lambda_i \rightarrow \lambda_i + \lambda$, ép số điều kiện giảm về $\kappa_{\text{Ridge}} = \frac{\lambda_{\max} + \lambda}{\lambda_{\min} + \lambda}$.

Lesson 18: Convex Optimization

I. Vấn đề (The Problem)

- Trong Bài 08, ta đã học Gradient Descent, Momentum và Adam — các thuật toán đi bước xuống dốc trên mọi bề mặt hàm mất mát (*loss surface*). Nhưng trên bề mặt không lồi (*non-convex*) của mạng nơ-ron sâu, chúng **không có bất kỳ bảo chứng toán học nào**: thuật toán có thể mắc kẹt ở cực tiểu cục bộ tồi (*bad local minimum*), đứng im trên điểm yên ngựa (*saddle point*), hoặc phụ thuộc nặng nề vào điểm khởi tạo w_0 .
- Ngược lại, hàng loạt bài toán cốt lõi của Machine Learning lại là **bài toán lồi** (*Convex Optimization*): Linear Regression, Ridge, LASSO, Logistic Regression, Support Vector Machines (SVM).
- Định lý vàng của Tối ưu Lồi:**

Với bài toán Tối ưu Lồi: Mọi cực tiểu cục bộ đều là CỰC TIỂU TOÀN CỤC!

- Bề mặt hàm lồi chỉ có **duy nhất một thung lũng** (*single valley*). Bất kỳ thuật toán nào bước xuống dốc đều chắc chắn chạm đáy tối ưu toàn cục (*global minimum*) mà không cần khởi tạo lại ngẫu nhiên (*random restarts*) hay cầu may.

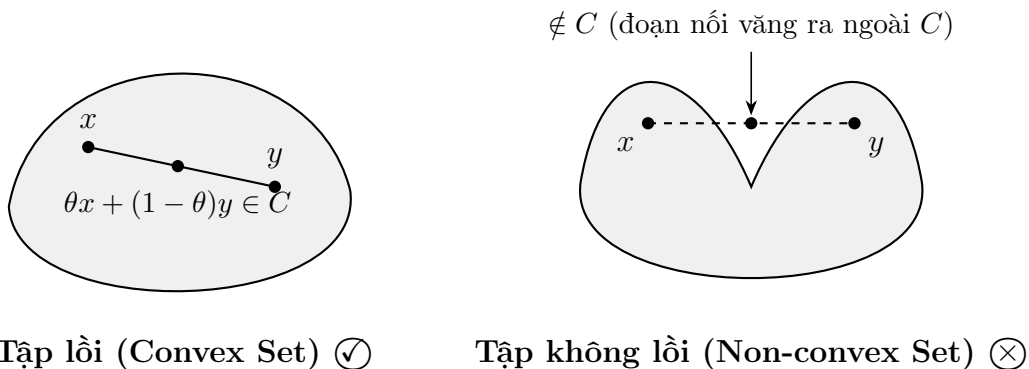
Bài toán ML	Tính lồi?	Bản chất Toán học của Hàm Loss
Linear / Ridge Regression	Lồi ✓	Hàm bậc hai $\ Xw - y\ _2^2 + \lambda\ w\ _2^2$ có Hessian $2(X^T X + \lambda I) \succeq 0$.
LASSO (L_1 Regression)	Lồi ✓	Tổng của hàm lồi bậc hai và chuẩn $\ w\ _1$ (hàm lồi dạng hình thoi).
Logistic Regression	Lồi ✓	Binary Cross-Entropy (Log-loss) có Hessian $X^T \text{diag}(p_i(1 - p_i))X \succeq 0$.
SVM (Hinge Loss / QP)	Lồi ✓	Cực tiểu hóa $\frac{1}{2}\ w\ _2^2$ (bậc hai lồi chặt) dưới các ràng buộc tuyến tính.
Deep Neural Networks	Không lồi ✗	Phép hợp hàm qua các tầng phi tuyến (ReLU, GELU, Softmax) làm vỡ tính lồi.
k -Means / Matrix Factorization	Không lồi ✗	Gán cụm rời rạc hoặc tích hai biến chưa biết $U @ V^T$ tạo nhiều hố cực bộ.

II. Tập Lồi (Convex Sets) & Hàm Lồi (Convex Functions)

1. Tập lồi (Convex Sets)

- ① **Định nghĩa hình học & đại số:** Một tập hợp $C \subseteq \mathbb{R}^d$ được gọi là **tập lồi** nếu lấy bất kỳ hai điểm $x, y \in C$, toàn bộ đoạn thẳng nối giữa x và y đều nằm trọn vẹn bên trong C :

$$\forall x, y \in C, \forall \theta \in [0, 1] \implies \theta x + (1 - \theta)y \in C$$



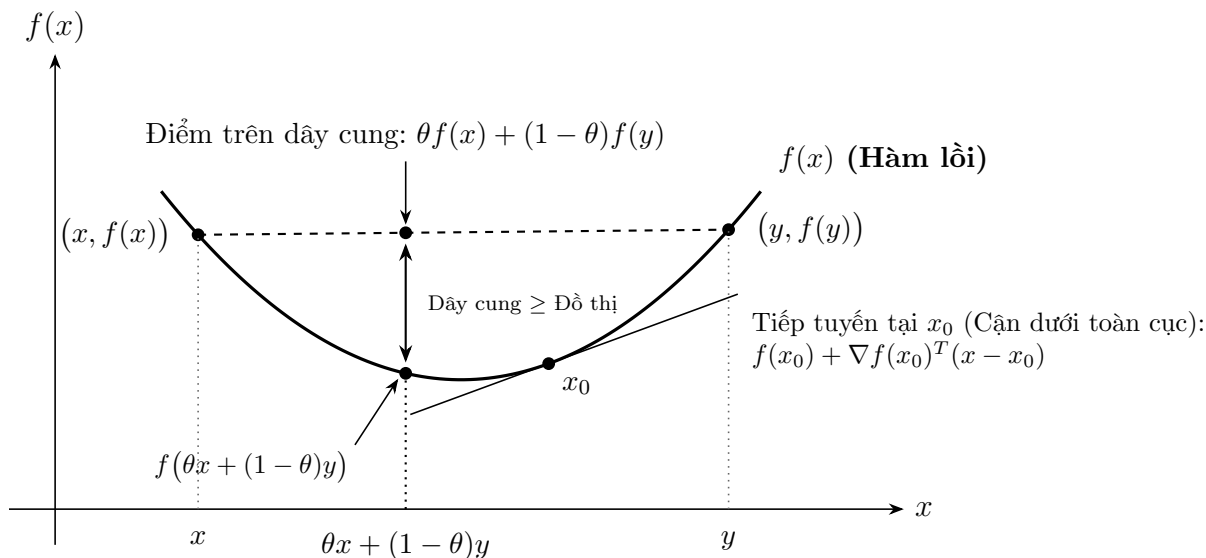
- **Ví dụ tập lồi kinh điển trong AI:** Toàn bộ không gian $\mathbb{R}^d$; siêu phẳng $\{x : a^T x = b\}$; nửa không gian (*halfspace*) $\{x : a^T x \leq b\}$; hình cầu chuẩn L_2 ($\|x\|_2 \leq r$); hình thoi chuẩn L_1 ($\|x\|_1 \leq r$).
- **Tính chất bảo toàn:** Giao (*intersection*) của một số lượng tùy ý các tập lồi **luôn là một tập lồi**. Nhờ đó, miền nghiệm của bài toán Quy hoạch Tuyến tính (LP) hoặc SVM (giao của n nửa không gian ràng buộc) luôn là một đa diện lồi (*convex polyhedron*).

2. Hàm lồi (Convex Functions)

① **Định nghĩa Dây cung (Chord Inequality):** Hàm số $f : C \rightarrow \mathbb{R}$ (với miền xác định C là tập lồi) được gọi là **hàm lồi** nếu với mọi $x, y \in C$ và mọi $\theta \in [0, 1]$:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

- **Ý nghĩa hình học:** Đoạn dây cung (*chord / secant line*) nối hai điểm bất kỳ $(x, f(x))$ và $(y, f(y))$ trên đồ thị **luôn nằm phía trên hoặc chạm vào đường cong đồ thị f** .
- Hàm f gọi là **lồi chặt** (*strictly convex*) nếu dấu “<” xảy ra nghiêm ngặt với mọi $x \neq y$ và $\theta \in (0, 1) \implies$ Cực tiểu toàn cục (nếu có) là **duy nhất** (*unique*).
- Hàm g gọi là **hàm lõm** (*concave function*) nếu $-g$ là hàm lồi (dây cung luôn nằm dưới đồ thị, tiêu biểu là $\log(x)$ với $x > 0$ và hàm Entropy $H(p)$).



III. Điều kiện Khả vi Bậc 1, Bậc 2 & Hội tụ Tuyến tính

1. Điều kiện Bậc 1 (First-Order Condition)

① Tiếp tuyến luôn là cận dưới toàn cục: Giả sử f khả vi trên tập mở lồi. Khi đó f là hàm lồi khi và chỉ khi với mọi x, y :

$$f(y) \geq f(x) + \nabla f(x)^T(y - x)$$

② Vì sao công thức này chứng minh ngay lập tức “ $\nabla f(x^*) = 0 \implies x^*$ là Cực tiểu Toàn cục”?

- Giả sử tại điểm dừng x^* , ta có đạo hàm bằng không: $\nabla f(x^*) = \vec{0}$.
- Thay $x = x^*$ vào bất đẳng thức bậc 1 ở trên với bất kỳ điểm y nào trong toàn bộ không gian:

$$f(y) \geq f(x^*) + \underbrace{\vec{0}^T(y - x^*)}_{=0} \implies \boxed{f(y) \geq f(x^*), \quad \forall y \in C} \quad (\checkmark)$$

- Không thể tồn tại bất kỳ điểm y nào thấp hơn x^* ! Do đó $\nabla f(x^*) = 0$ là **điều kiện cần và đủ** để x^* là nghiệm tối ưu toàn cục của hàm lồi khả vi.

2. Điều kiện Bậc 2 (Second-Order / Hessian Condition) & Newton's Method

① Tiêu chuẩn Đạo hàm bậc hai & Ma trận Hessian: Nếu f khả vi bậc hai trên tập lồi mở C , thì f là hàm lồi khi và chỉ khi độ cong của nó không âm theo mọi hướng:

$$\begin{cases} \text{Không gian 1 chiều } (x \in \mathbb{R}): & \boxed{f''(x) \geq 0, \quad \forall x \in C} \\ \text{Không gian } d \text{ chiều } (x \in \mathbb{R}^d): & \boxed{H(x) = \nabla^2 f(x) \succeq 0 \quad (\text{Positive Semidefinite — PSD})} \end{cases}$$

Trong đó $H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$, và $H(x) \succeq 0$ nghĩa là $v^T H(x) v \geq 0$ với mọi vectơ $v \in \mathbb{R}^d$ (tương đương mọi trị riêng $\lambda_i(H) \geq 0$).

VD 1: Kiểm tra tính lồi của 2 hàm hai biến:

- **Hàm 1**: $f_1(x_1, x_2) = 2x_1^2 + 2x_1x_2 + x_2^2$.
 - **Bước 1 (Tính Gradient)**: $\nabla f_1 = \begin{bmatrix} 4x_1 + 2x_2 \\ 2x_1 + 2x_2 \end{bmatrix}$.
 - **Bước 2 (Tính Hessian)**: $H_1 = \begin{bmatrix} 4 & 2 \\ 2 & 2 \end{bmatrix}$.
 - **Bước 3 (Kiểm tra PSD — định thức con chính Sylvester)**: $\Delta_1 = 4 > 0$ và $\Delta_2 = \det(H_1) = 4 \cdot 2 - 2^2 = 4 > 0 \implies H_1 \succ 0$ (xác định dương) $\implies f_1$ lồi chặt $(\checkmark)$.
- **Hàm 2**: $f_2(x_1, x_2) = x_1^2 + 3x_1x_2 + x_2^2$.

- Hessian $H_2 = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$ có $\det(H_2) = 2 \cdot 2 - 3^2 = -5 < 0$ (hai trị riêng là $\lambda_1 = 5 > 0$, $\lambda_2 = -1 < 0$) $\implies$ Điểm $(0, 0)$ là **điểm yên ngựa** (*saddle point*) $\implies f_2$ **không lồi** $\otimes$.

② **Tính lồi mạnh (Strong Convexity) & Tốc độ hội tụ của Gradient Descent:** Hàm f được gọi là **m -lồi mạnh** (*m -strongly convex*, với $m > 0$) và **L -trơn** (*L -smooth*) nếu mọi trị riêng của Hessian bị chặn trong đoạn $[m, L]$:

$$mI \preceq \nabla^2 f(x) \preceq LI \quad (0 < m \leq L)$$

- Tỷ số $\kappa = \frac{L}{m} = \frac{\lambda_{\max}(H)}{\lambda_{\min}(H)} \geq 1$ được gọi là **Số điều kiện** (*Condition Number*):
 - Nếu $\kappa \approx 1$: Thung lũng tròn đều (đẳng hướng) $\implies$ Gradient Descent lao thẳng về tâm cực nhanh.
 - Nếu $\kappa \gg 1$ (*ill-conditioned*): Thung lũng bị kéo dẹt thành hẻm núi hẹp $\implies$ Gradient Descent đập zig-zag hai bên vách núi, hội tụ tuyến tính với tốc độ phụ thuộc vào $\left(\frac{\kappa-1}{\kappa+1}\right)^t$.
- **Phương pháp Newton (Newton's Method):** Nhân nghịch đảo Hessian để tự động nắn lại toàn bộ độ cong địa phương về hình cầu tròn trịa:

$$\underset{(d \times 1)}{x_{t+1}} = \underset{(d \times 1)}{x_t} - \underset{(d \times d)}{[\nabla^2 f(x_t)]^{-1}} \underset{(d \times 1)}{@ \nabla f(x_t)}$$

Với hàm bậc hai thuần túy $f(x) = \frac{1}{2}x^T Qx + b^T x + c$ ($Q \succ 0$), phương pháp Newton hội tụ về đáy chỉ sau **đúng 1 bước duy nhất** (bất chấp κ lớn cỡ nào!), nhưng chi phí lưu trữ $O(d^2)$ và nghịch đảo ma trận $O(d^3)$ khiến nó không thể áp dụng trực tiếp cho mạng nơ-ron hàng tỷ tham số.

IV. Bất đẳng thức Jensen $\langle$ Jensen's Inequality $\rangle$ — Trụ cột của Xác suất & AI

① **Công thức tổng quát cho Kỳ vọng (Expectation):** Từ bất đẳng thức dây cung 2 điểm, quy nạp cho biến ngẫu nhiên X bất kỳ:

$$\begin{cases} \text{Hàm LỒI } (f \text{ convex}): & \boxed{\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])} & \left(\text{VD: } \mathbb{E}[X^2] \geq (\mathbb{E}[X])^2 \right) \\ \text{Hàm LỖM } (g \text{ concave}): & \boxed{\mathbb{E}[g(X)] \leq g(\mathbb{E}[X])} & \left(\text{VD: } \mathbb{E}[\log X] \leq \log(\mathbb{E}[X]) \right) \end{cases}$$

WRONG

Nhầm lẫn kinh điển khi biến đổi hàm Log-Likelihood ẩn (EM / Variational Autoencoder):

Đưa log chui thẳng vào trong tổng hoặc kỳ vọng:

$$\log \sum_z q(z) \frac{p(x, z)}{q(z)} = \sum_z q(z) \log \frac{p(x, z)}{q(z)} \quad \otimes \langle \text{SAI! log là hàm lõm!} \rangle$$

② Hai ứng dụng sống còn của Bất đẳng thức Jensen trong AI:

1) Chứng minh KL Divergence luôn không âm ($D_{\text{KL}}(P \parallel Q) \geq 0$):

Vì $-\log(u)$ là hàm lồi chặt (đạo hàm bậc hai là $\frac{1}{u^2} > 0$), áp dụng Jensen cho tỷ số $u = \frac{Q(x)}{P(x)}$:

$$D_{\text{KL}}(P \parallel Q) = \mathbb{E}_P \left[-\log \frac{Q(x)}{P(x)} \right] \geq -\log \left(\mathbb{E}_P \left[\frac{Q(x)}{P(x)} \right] \right) = -\log(1) = 0 \quad \checkmark$$

2) Xây dựng cận dưới ELBO (Evidence Lower Bound) trong thuật toán EM & VAE:

Vì $\log(\cdot)$ là hàm lõm, $\log \mathbb{E}_{z \sim q} \left[\frac{p(x, z)}{q(z)} \right] \geq \mathbb{E}_{z \sim q} \left[\log \frac{p(x, z)}{q(z)} \right] = \text{ELBO}(q)$. Nhờ đó, việc cực đại hóa ELBO giúp đẩy $\log p(x)$ tăng lên mà không cần tính tích phân bất khả thi $\int p(x, z) dz$.

V. Tối ưu có Ràng buộc: Nhân tử Lagrange, Đối ngẫu, KKT & SVM

1. Bài toán Gốc (Primal), Hàm Lagrange & Điều kiện KKT

Xét bài toán tối ưu lồi tổng quát có cả ràng buộc bất đẳng thức và đẳng thức:

$$\min_{x \in \mathbb{R}^d} f(x) \quad \text{thỏa mãn (s.t.)} \quad \begin{cases} g_i(x) \leq 0, & i = 1, \dots, m \quad \langle f, g_i \text{ là các hàm lồi} \rangle \\ h_j(x) = 0, & j = 1, \dots, p \quad \langle h_j \text{ là các hàm tuyến tính / affine} \rangle \end{cases}$$

Ta lập **Hàm Lagrange (Lagrangian)** bằng cách cộng các ràng buộc kèm hệ số phạt (nhân tử Lagrange $\lambda_i \geq 0$ và $\nu_j \in \mathbb{R}$):

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \nu_j h_j(x)$$

4 ĐIỀU KIỆN TỐI ƯU KARUSH-KUHN-TUCKER (KKT CONDITIONS)

Với bài toán lồi thỏa điều kiện chuẩn Slater, bộ nghiệm (x^*, λ^*, ν^*) là tối ưu toàn cục khi và chỉ khi thỏa mãn đồng thời 4 điều kiện KKT:

1. **Stationarity (Điểm dừng đạo hàm):** $\nabla_x \mathcal{L}(x^*, \lambda^*, \nu^*) = \nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^p \nu_j^* \nabla h_j(x^*) = \vec{0}$.
2. **Primal Feasibility (Khả thi gốc):** $g_i(x^*) \leq 0 \ (\forall i)$ và $h_j(x^*) = 0 \ (\forall j)$.
3. **Dual Feasibility (Khả thi đối ngẫu):** $\lambda_i^* \geq 0 \ (\forall i = 1, \dots, m)$.
4. **Complementary Slackness (Độ lệch bù):** $\lambda_i^* \cdot g_i(x^*) = 0, \quad \forall i = 1, \dots, m$.

⑦ Ý nghĩa sống còn của điều kiện Độ lệch bù $\lambda_i^* g_i(x^*) = 0$:

- Tích của hai số bằng 0 nghĩa là tại nghiệm tối ưu x^* , với mỗi ràng buộc thứ i , chỉ có thể xảy ra 1 trong 2 kịch bản:
 - **TH 1 — Ràng buộc không chạm biên (*Inactive*):** $g_i(x^*) < 0$ (điểm tối ưu nằm lọt thỏm bên trong vùng khả thi) $\implies$ Bất buộc $\lambda_i^* = 0$ (bức tường ràng buộc thứ i hoàn toàn vô tác dụng, xóa đi cũng không đổi nghiệm!).
 - **TH 2 — Ràng buộc kích hoạt trên biên (*Active / Binding*):** $\lambda_i^* > 0 \implies$ Bất buộc $g_i(x^*) = 0$ (nghiệm x^* nằm ép sát lên mặt biên ràng buộc thứ i).

VD 2: Giải bài toán tối ưu có ràng buộc bất đẳng thức bằng KKT:

$$\min_{x_1, x_2} f(x_1, x_2) = x_1^2 + x_2^2 \quad \text{thỏa mãn} \quad x_1 + x_2 \geq 2 \iff g(x_1, x_2) = 2 - x_1 - x_2 \leq 0$$

- **Bước 1 (Lập hàm Lagrange):** $\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda(2 - x_1 - x_2)$ với $\lambda \geq 0$.
- **Bước 2 (Điều kiện dừng Stationarity):**

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 - \lambda = 0 \implies x_1 = \frac{\lambda}{2} \\ \frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 - \lambda = 0 \implies x_2 = \frac{\lambda}{2} \end{cases}$$

- **Bước 3 (Xét Complementary Slackness $\lambda(2 - x_1 - x_2) = 0$):**
 - Nếu $\lambda = 0 \implies x_1 = x_2 = 0$. Thay vào ràng buộc gốc: $g(0, 0) = 2 - 0 - 0 = 2 > 0$ (vi phạm Primal Feasibility ≤ 0) $\implies$ Loại!
 - Do đó ràng buộc phải kích hoạt ($g = 0$): $2 - x_1 - x_2 = 0 \implies 2 - \frac{\lambda}{2} - \frac{\lambda}{2} = 0 \implies \lambda^* = 2 > 0$ (✓).
- **Bước 4 (Kết luận nghiệm):** $\boxed{(x_1^*, x_2^*) = (1, 1)}$ với giá trị tối ưu $f^* = 1^2 + 1^2 = 2$.

2. Ứng dụng vào SVM & Regularization L_1 vs L_2

① Vì sao chỉ các **Support Vectors** mới quyết định siêu phẳng phân lớp của SVM?
 Bài toán SVM lề cứng (*Hard-Margin SVM*) tìm siêu phẳng $w^T x + b = 0$ cực đại hóa lề $\frac{2}{\|w\|_2}$:

$$\min_{w, b} \frac{1}{2} \|w\|_2^2 \quad \text{s.t.} \quad g_i(w, b) = 1 - y_i(w^T x_i + b) \leq 0, \quad \forall i = 1, \dots, n$$

Lập hàm Lagrange với các nhân tử $\alpha_i \geq 0$:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|_2^2 + \sum_{i=1}^n \alpha_i [1 - y_i(w^T x_i + b)]$$

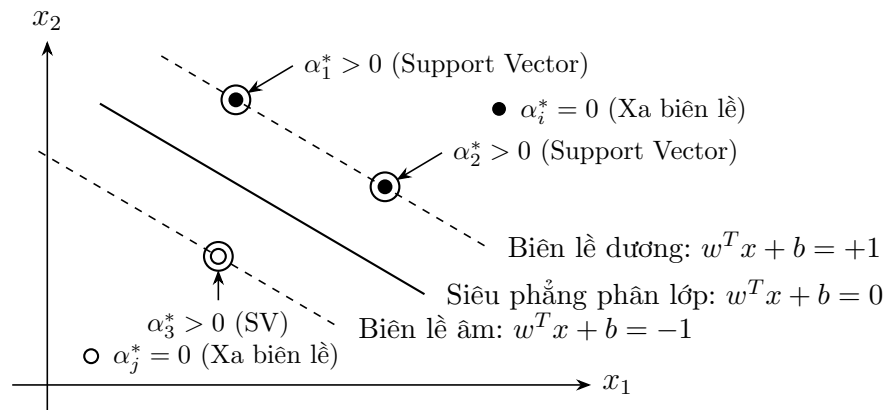
- Từ điều kiện KKT số 1 (*Stationarity*):

$$\nabla_w \mathcal{L} = w - \sum_{i=1}^n \alpha_i y_i x_i = \vec{0} \implies \boxed{w^* = \sum_{i=1}^n \alpha_i^* y_i x_i}$$

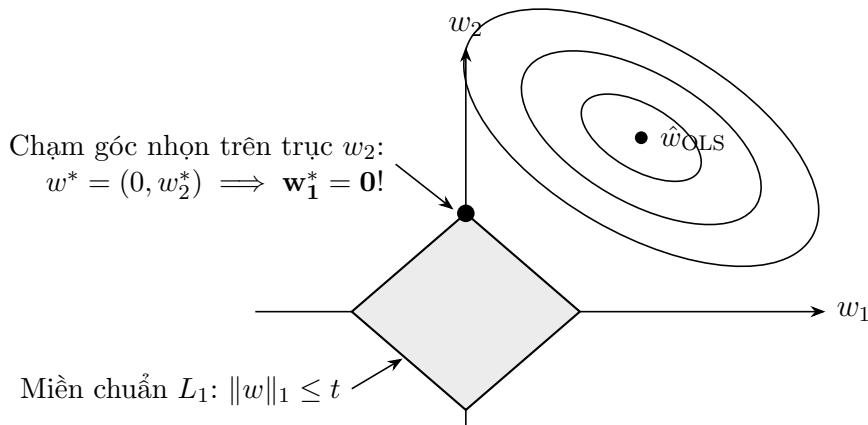
- Từ điều kiện KKT số 4 (*Complementary Slackness*):

$$\alpha_i^* [1 - y_i(w^{*T} x_i + b^*)] = 0, \quad \forall i = 1, \dots, n$$

- Với đại đa số các điểm dữ liệu nằm xa biên quyết định: $y_i(w^{*T} x_i + b^*) > 1 \implies 1 - y_i(w^{*T} x_i + b^*) < 0 \implies$ Bắt buộc $\alpha_i^* = 0$! Các điểm này hoàn toàn biến mất khỏi tổng $w^* = \sum \alpha_i^* y_i x_i$.
- Chỉ những điểm nằm **đúng trên hai đường biên lề** $y_i(w^{*T} x_i + b^*) = 1$ (ràng buộc kích hoạt $g_i = 0$) mới có $\alpha_i^* > 0$. Chúng chính là các **Vectơ Hỗ trợ (Support Vectors)**!



(a) KKT trong SVM: Chỉ các điểm nằm trên biên lề (Support Vectors) mới có $\alpha_i^* > 0$



(b) Ràng buộc L_1 (LASSO): Mặt mức Loss chạm góc nhọn $\implies$ Nghiệm thưa ($w_1^* = 0$)

VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 18

- **Tập lồi & Hàm lồi:** Tập C lồi khi $\theta x + (1 - \theta)y \in C$; hàm f lồi khi dãy cung nằm trên đồ thị $f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) \implies$ Mọi cực tiểu cục bộ đều là cực tiểu toàn cục.
- **Điều kiện Bậc 1 & Bậc 2:** Tiếp tuyến là cận dưới toàn cục $f(y) \geq f(x) + \nabla f(x)^T(y - x)$ ($\nabla f(x^*) = 0 \iff x^*$ là đáy toàn cục); đạo hàm bậc hai $f''(x) \geq 0$ (1D) hoặc ma trận Hessian $\nabla^2 f(x) \succeq 0$ (PSD — mọi trị riêng $\lambda_i \geq 0$).
- **Phương pháp Newton & Số điều kiện:** $x_{t+1} = x_t - [\nabla^2 f(x_t)]^{-1} \nabla f(x_t)$ hội tụ bậc hai (và chỉ mất **1 bước** cho hàm bậc hai), miễn nhiễm với số điều kiện $\kappa = \lambda_{\max}/\lambda_{\min}$, nhưng tốn $O(d^2)$ bộ nhớ và $O(d^3)$ phép tính.
- **Bất đẳng thức Jensen:** Với f lồi thì $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$; với log lõm thì $\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$ — nền tảng chứng minh $D_{\text{KL}}(P \parallel Q) \geq 0$ và cận dưới ELBO trong EM / VAE.
- **Điều kiện KKT & SVM:** Gồm Stationarity, Primal/Dual Feasibility ($\lambda_i \geq 0$) và **Complementary Slackness** ($\lambda_i^* g_i(x^*) = 0$). Trong SVM, chỉ các điểm nằm đúng trên biên lề ($g_i(x^*) = 0$) mới có $\alpha_i^* > 0$ (**Support Vectors**), còn mọi điểm xa biên đều có $\alpha_i^* = 0$.

Lesson 19: Complex Numbers for AI

I. Vấn đề (The Problem)

- Khi mới học toán phổ thông, đơn vị ảo $i = \sqrt{-1}$ ($i^2 = -1$) nghe như một ký hiệu tưởng tượng vô thực. Nhưng khi mở mã nguồn của các kiến trúc AI hiện đại nhất — từ **LLaMA**, **Mistral**, **Qwen**, **DeepSeek** cho tới **Whisper**, **S4**, **Mamba** — bạn sẽ thấy kiểu dữ liệu phức `torch.complex64` và hàm `torch.polar` xuất hiện ở tầng cốt lõi nhất!
- ❓ Vì sao Kỹ sư AI bắt buộc phải nắm vững Số phức?
 - 1) **Rotary Position Embedding (RoPE) trong LLMs**: Mã hóa vị trí tương đối giữa các token trong cơ chế Self-Attention thông qua phép nhân xoay số phức $q \cdot e^{im\theta}$.
 - 2) **Biến đổi Fourier (DFT / FFT) trong Xử lý Âm thanh & Ảnh**: Phân tách sóng âm thanh và ảnh số thành tổ hợp các hàm mũ phức $e^{-i2\pi kn/N}$.
 - 3) **Trị riêng phức & State Space Models (RNN / S4 / Mamba)**: Ma trận chuyển trạng thái bất đối xứng $A \in \mathbb{R}^{d \times d}$ luôn có các cặp trị riêng phức liên hợp $\lambda = re^{\pm i\omega}$ quyết định tần số dao động và độ suy giảm bộ nhớ dài hạn.
- **Chân lý hình học**: Số phức không hề “ảo”. Đơn vị i chính là **toán tử xoay góc 90° ngược chiều kim đồng hồ** trên mặt phẳng 2 chiều! Nhân với i hai lần (i^2) tức là xoay $90^\circ + 90^\circ = 180^\circ$, biến số $+1$ thành -1 .

II. Dạng Đại số (Cartesian Form), Liên hợp & Mặt phẳng Phức

1. Cấu trúc Đại số & Liên hợp (Conjugate, Modulus)

❶ **Định nghĩa & Các phép toán cơ bản**: Một số phức $z \in \mathbb{C}$ gồm phần thực $\text{Re}(z) = a \in \mathbb{R}$ và phần ảo $\text{Im}(z) = b \in \mathbb{R}$:

$$z = a + bi \quad \text{với} \quad i^2 = -1$$

Cho hai số phức $z_1 = a + bi$ và $z_2 = c + di$:

- **Phép cộng / trừ (như cộng vectơ 2D)**: $z_1 \pm z_2 = (a \pm c) + (b \pm d)i$.
- **Phép nhân (nhân phân phối và thay $i^2 = -1$)**:

$$z_1 \cdot z_2 = (a + bi)(c + di) = ac + adi + bci + bdi^2 = (ac - bd) + (ad + bc)i$$

- **Số phức liên hợp (Complex Conjugate — đối xứng qua trục thực)**: $\bar{z} = a - bi$.
- **Mô-đun (Modulus / Độ dài Euclid từ gốc O)**:

$$|z| = \sqrt{a^2 + b^2} \implies z \cdot \bar{z} = (a + bi)(a - bi) = a^2 - b^2i^2 = a^2 + b^2 = |z|^2 \in \mathbb{R}_{\geq 0}$$

- Phép chia (nhân cả tử và mẫu với liên hợp của mẫu để khử i ở mẫu):

$$\frac{z_1}{z_2} = \frac{(a+bi)(c-di)}{(c+di)(c-di)} = \boxed{\frac{(ac+bd) + (bc-ad)i}{c^2+d^2}}$$

WRONG

Sai lầm kinh điển khi tính độ dài (Norm) và Tích vô hướng (Inner Product) trên $\mathbb{C}$:

Nếu áp dụng công thức số thực $z^2 = (a+bi)^2 = a^2 - b^2 + 2abi$, kết quả vẫn là số phức (thậm chí với $z = i \neq 0$ thì $z^2 = -1 < 0$!).

$\Rightarrow$ Trong không gian phức, bình phương độ dài **bắt buộc phải nhân với số phức liên hợp**: $|z|^2 = z \cdot \bar{z} = a^2 + b^2 \geq 0$, và tích vô hướng của hai vectơ phức $\vec{u}, \vec{v} \in \mathbb{C}^d$ là $\langle \vec{u}, \vec{v} \rangle = \vec{u}^H \vec{v} = \sum_{k=1}^d \bar{u}_k v_k$ (chuyển vị liên hợp Hermitian H).

VD 1: Cho $z_1 = 3 + 2i$ và $z_2 = 1 + 4i$. Tính $z_1 \cdot z_2$ và $\frac{z_1}{z_2}$:

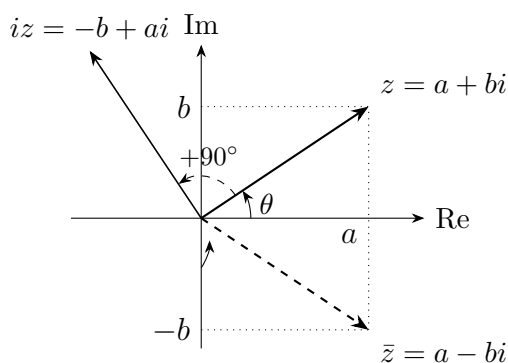
- Bước 1 (Nhân hai số phức):

$$z_1 \cdot z_2 = (3+2i)(1+4i) = 3 \cdot 1 + 3 \cdot 4i + 2i \cdot 1 + 2 \cdot 4 \cdot i^2 = 3 + 14i - 8 = \boxed{-5 + 14i}$$

- Bước 2 (Chia hai số phức — nhân tử và mẫu với $\bar{z}_2 = 1 - 4i$):

$$\frac{z_1}{z_2} = \frac{(3+2i)(1-4i)}{1^2+4^2} = \frac{3-12i+2i-8i^2}{17} = \frac{(3+8)-10i}{17} = \boxed{\frac{11}{17} - \frac{10}{17}i}$$

2. Mặt phẳng Phức Argand (Complex Plane) & Toán tử xoay i



- Mỗi số phức $z = a + bi$ tương ứng 1-1 với điểm (a, b) hoặc vectơ $\begin{bmatrix} a \\ b \end{bmatrix} \in \mathbb{R}^2$.

- **Liên hợp** $\bar{z} = a - bi$: Là ảnh phản chiếu gương qua trục Thực, giữ nguyên độ dài $|\bar{z}| = |z| = r$ nhưng **đảo ngược góc pha**: $\arg(\bar{z}) = -\theta$.

- **Nhân với i** : $i \cdot z = i(a+bi) = -b+ai$. Kiểm tra tích vô hướng 2D: $(a, b) \cdot (-b, a) = 0 \Rightarrow$ Vectơ iz vuông góc 90° với z !

III. Dạng Cực (Polar Form) & Công thức Euler Huyền thoại

1. Chứng minh Công thức Euler bằng Chuỗi Taylor

① Khai triển Taylor tại $x = 0$ (Chuỗi Maclaurin): Từ Giải tích (Bài 04), ta biết khai triển chuỗi lũy thừa hội tụ trên toàn mặt phẳng của e^u , $\cos \theta$ và $\sin \theta$:

$$\cos \theta = 1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \frac{\theta^6}{6!} + \dots \quad \langle \text{chỉ chứa lũy thừa bậc chẵn} \rangle$$

$$\sin \theta = \theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \frac{\theta^7}{7!} + \dots \quad \langle \text{chỉ chứa lũy thừa bậc lẻ} \rangle$$

Thay $u = i\theta$ vào chuỗi $e^u = 1 + u + \frac{u^2}{2!} + \frac{u^3}{3!} + \frac{u^4}{4!} + \frac{u^5}{5!} + \dots$ và dùng chu kỳ lũy thừa của i ($i^1 = i$, $i^2 = -1$, $i^3 = -i$, $i^4 = +1$):

$$\begin{aligned} e^{i\theta} &= 1 + (i\theta) + \frac{(i\theta)^2}{2!} + \frac{(i\theta)^3}{3!} + \frac{(i\theta)^4}{4!} + \frac{(i\theta)^5}{5!} + \dots \\ &= \underbrace{\left(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \dots\right)}_{\cos \theta} + i \underbrace{\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \dots\right)}_{\sin \theta} \Rightarrow \boxed{e^{i\theta} = \cos \theta + i \sin \theta} \end{aligned}$$

- Khi thay $\theta = \pi$: $e^{i\pi} = \cos \pi + i \sin \pi = -1 \Rightarrow \boxed{e^{i\pi} + 1 = 0}$ (Đồng nhất thức Euler kết nối 5 hằng số vĩ đại nhất toán học: $e, i, \pi, 1, 0$).
- Độ dài của $e^{i\theta}$ luôn bằng 1: $|e^{i\theta}| = \sqrt{\cos^2 \theta + \sin^2 \theta} = 1 \Rightarrow$ Mọi điểm $e^{i\theta}$ đều nằm trên đường tròn đơn vị (*unit circle*).

2. Biểu diễn Dạng Cực & Phép nhân Số phức = Phép Xoay 2D

Mọi số phức $z = a + bi$ đều viết được dưới **Dạng Cực (Polar Form)** theo khoảng cách $r = |z| = \sqrt{a^2 + b^2}$ và góc pha $\theta = \text{atan2}(b, a)$:

$$\boxed{z = r(\cos \theta + i \sin \theta) = re^{i\theta}}$$

Khi nhân hai số phức $z_1 = r_1 e^{i\theta_1}$ và $z_2 = r_2 e^{i\theta_2}$:

$$\boxed{z_1 \cdot z_2 = (r_1 e^{i\theta_1}) \cdot (r_2 e^{i\theta_2}) = (r_1 \cdot r_2) e^{i(\theta_1 + \theta_2)}}$$

★ **Ý nghĩa hình học cốt lõi**: Nhân hai số phức thực chất là **nhân hai độ dài** ($r_1 r_2$) và **CỘNG HAI GÓC XOAY** ($\theta_1 + \theta_2$)! Đặc biệt, nếu nhân z với $e^{i\theta}$ (có độ dài $|e^{i\theta}| = 1$), độ dài của z được bảo toàn nguyên vẹn còn hướng của z **xoay đúng một góc θ** .

3. Sự tương đương với Ma trận Xoay 2D & Căn bậc N của Đơn vị

① Nhân với $e^{i\theta}$ $\equiv$ Nhân Ma trận Xoay trực giao $R(\theta)$: Xét điểm $(x, y) \in \mathbb{R}^2$ viết dưới dạng số phức $z = x + yi$. Nhân z với $e^{i\theta} = \cos \theta + i \sin \theta$:

$$z' = z \cdot e^{i\theta} = (x + yi)(\cos \theta + i \sin \theta) = \underbrace{(x \cos \theta - y \sin \theta)}_{x'} + \underbrace{(x \sin \theta + y \cos \theta)}_{y'} i$$

Viết lại dưới dạng phép nhân ma trận – vectơ trong đại số tuyến tính:

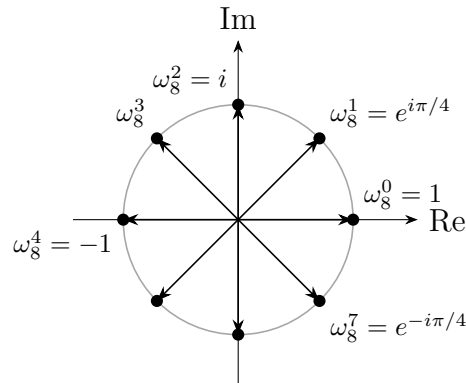
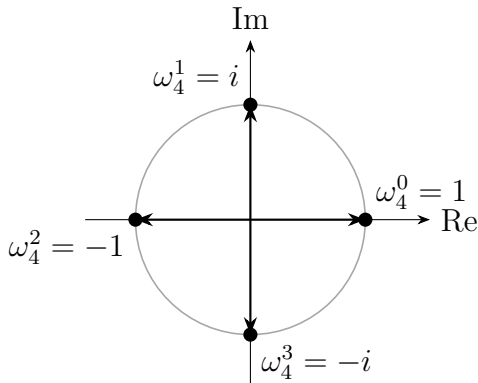
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \underbrace{\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}}_{\substack{R(\theta) \\ (2 \times 2)}} @ \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_{(2 \times 1)} \iff z' = z \cdot e^{i\theta}$$

② **Căn bậc N của đơn vị (Roots of Unity — Trái tim của DFT/FFT):** Phương trình $z^N = 1$ có đúng N nghiệm phức phân biệt chia đều đường tròn đơn vị thành N phần bằng nhau (mỗi bước nhảy góc $\frac{2\pi}{N}$):

$$\omega_N^k = e^{i\frac{2\pi k}{N}} = \cos\left(\frac{2\pi k}{N}\right) + i \sin\left(\frac{2\pi k}{N}\right), \quad k = 0, 1, \dots, N-1$$

- **Tính chất triệt tiêu đối xứng:** Tổng của tất cả N căn bậc N của đơn vị (với $N \geq 2$)

luôn bằng 0: $\sum_{k=0}^{N-1} e^{i\frac{2\pi k}{N}} = 0$. Chính tính chất trực giao này giúp biến đổi Fourier (DFT) có thể nghịch đảo hoàn hảo 100%!



(a) $N = 4$: $\sum_{k=0}^3 \omega_4^k = 1 + i - 1 - i = 0$ (b) $N = 8$: Chia đều góc $\Delta\theta = \frac{2\pi}{8} = 45^\circ$

IV. Ứng dụng Chuyên sâu: Rotary Position Embedding (RoPE) trong LLMs

1. Bài toán Mã hóa Vị trí Tương đối trong Self-Attention

- Cơ chế Self-Attention thuần túy tính điểm chú ý $\text{score}(m, n) = \vec{q}_m^T \vec{k}_n$ hoàn toàn **mù vị trí** (*permutation-invariant*): đảo thứ tự các từ trong câu thì tích vô hướng vẫn giữ nguyên.
- Trong ngôn ngữ tự nhiên, điều quan trọng nhất giữa từ ở vị trí m và từ ở vị trí n không phải là vị trí tuyệt đối của chúng (nằm ở dòng thứ mấy), mà là **khoảng cách tương đối** ($m - n$) (hai từ cách nhau bao nhiêu token).
- Ý tưởng thiên tài của RoPE (Su et al., 2021 — dùng trong LLaMA, Qwen, Mistral, Gemma):

Chia vectơ $\vec{q}, \vec{k} \in \mathbb{R}^d$ thành $d/2$ cặp tọa độ 2 chiều liên tiếp, coi mỗi cặp $(q^{(2j)}, q^{(2j+1)})$ là một số phức $q \in \mathbb{C}$, rồi **xoay token ở vị trí m đi một góc $m\theta$** :

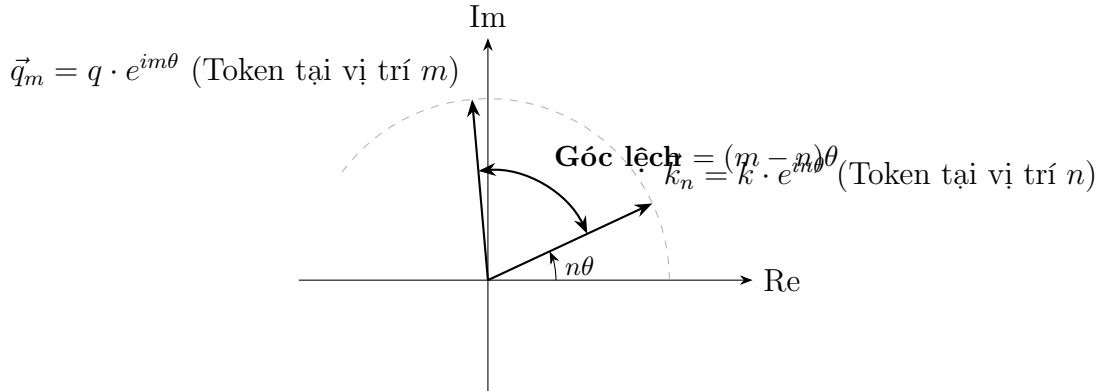
$$q_m = q \cdot e^{im\theta}, \quad k_n = k \cdot e^{in\theta}$$

2. Chứng minh Toán học: Vì sao Tích vô hướng chỉ phụ thuộc vào $(m - n)$?

Tích vô hướng thực giữa hai vectơ 2D $\vec{q}_m$ và $\vec{k}_n$ chính là phần thực của tích phức $\text{Re}(q_m \cdot \bar{k}_n)$:

$$\begin{aligned} \langle \vec{q}_m, \vec{k}_n \rangle &= \text{Re}(q_m \cdot \bar{k}_n) \\ &= \text{Re}\left((q \cdot e^{im\theta}) \cdot \overline{(k \cdot e^{in\theta})}\right) \\ &= \text{Re}(q \cdot e^{im\theta} \cdot \bar{k} \cdot e^{-in\theta}) \quad (\text{vì liên hợp của } e^{in\theta} \text{ là } e^{-in\theta}) \\ &= \boxed{\text{Re}(q \cdot \bar{k} \cdot e^{i(m-n)\theta})} \quad \checkmark \end{aligned}$$

★ **Kết quả kỳ diệu:** Vị trí tuyệt đối m và n đã tự động gom lại thành hiệu số **khoảng cách tương đối** $(m - n)$! Dù cặp từ “con mèo” xuất hiện ở vị trí $(m = 5, n = 4)$ hay ở vị trí $(m = 1005, n = 1004)$, hiệu số vẫn là $m - n = 1$ nên góc lệch pha giữa Query và Key là hoàn toàn giống hệt nhau!



VD 2: Cho vectơ Query 2D $\vec{q} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ($q = 1 + 0i$) và Key $\vec{k} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ ($k = 1 + 0i$), tần số góc cơ sở $\theta = \frac{\pi}{4}$ (45°). Hãy tính Attention Score $\langle \vec{q}_m, \vec{k}_n \rangle$ trong 2 trường hợp cùng khoảng cách $m - n = 1$:

- **TH 1 — Tại vị trí $(m = 1, n = 0)$:**

$$- \vec{q}_1 \text{ xoay góc } 1 \cdot \frac{\pi}{4} = 45^\circ: \vec{q}_1 = \begin{bmatrix} \cos 45^\circ \\ \sin 45^\circ \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix}.$$

$$- \vec{k}_0 \text{ xoay góc } 0^\circ: \vec{k}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \implies \text{Score} = \vec{q}_1^T \vec{k}_0 = \frac{\sqrt{2}}{2} \cdot 1 + \frac{\sqrt{2}}{2} \cdot 0 = \boxed{\frac{\sqrt{2}}{2} \approx 0,707}.$$

- **TH 2 — Dịch đi 100 token tới vị trí $(m = 101, n = 100)$:**

– Theo công thức phức RoPE:

$$\text{Score} = \text{Re} \left(1 \cdot 1 \cdot e^{i(101-100)\frac{\pi}{4}} \right) = \cos \left(\frac{\pi}{4} \right) = \frac{\sqrt{2}}{2} \approx 0,707 \quad \checkmark$$

(Bất biến hoàn hảo với dịch chuyển vị trí tuyệt đối!).

V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

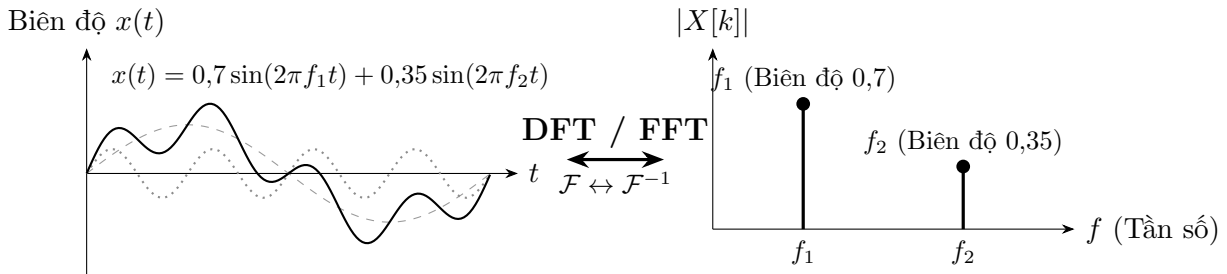
BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 19

- **Số phức, Liên hợp & Mô-đun:** $z = a + bi$ ($i^2 = -1$, nhân với i là xoay 90°). Số phức liên hợp $\bar{z} = a - bi$ giúp tính bình phương độ dài thực: $|z|^2 = z \cdot \bar{z} = a^2 + b^2$ và khử mẫu trong phép chia $\frac{z_1}{z_2} = \frac{z_1 \bar{z}_2}{|z_2|^2}$.
- **Công thức Euler & Dạng cực:** $e^{i\theta} = \cos \theta + i \sin \theta \implies z = r e^{i\theta}$ (với $r = \sqrt{a^2 + b^2}$, $\theta = \text{atan2}(b, a)$). Nhân hai số phức $z_1 z_2 = (r_1 r_2) e^{i(\theta_1 + \theta_2)}$ là **nhân độ dài và cộng góc xoay**.
- **Tương đương Ma trận Xoay 2D:** Nhân số phức $z = x + yi$ với $e^{i\theta}$ hoàn toàn trùng khớp với phép nhân ma trận xoay trực giao $\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$.
- **Căn bậc N của đơn vị (Roots of Unity):** $\omega_N^k = e^{i2\pi k/N}$ ($k = 0, \dots, N-1$) chia đều đường tròn đơn vị thành N điểm đối xứng có tổng bằng 0: $\sum_{k=0}^{N-1} \omega_N^k = 0$ (hệ cơ sở trực giao của DFT/FFT).
- **Rotary Position Embedding (RoPE):** Mã hóa vị trí m, n bằng phép xoay phức $q_m = q e^{im\theta}$, $k_n = k e^{in\theta}$ sao cho Attention Score $\text{Re}(q_m \bar{k}_n) = \text{Re}(q \bar{k} e^{i(m-n)\theta})$ **chỉ phụ thuộc vào khoảng cách tương đối** ($m - n$).

Lesson 20: The Fourier Transform

I. Vấn đề (The Problem) & Trực giác Cốt lõi

- Một đoạn ghi âm giọng nói là chuỗi áp suất không khí theo thời gian $x[n]$; một bức ảnh là lưới cường độ điểm ảnh theo không gian $I(x, y)$. Nếu chỉ nhìn trong **Miền Thời gian / Không gian** (*Time / Spatial Domain*), mọi sóng âm thanh hỗn tạp hay nhiều ảnh đều trộn lẫn vào nhau không thể bóc tách.
- Chân lý của Joseph Fourier:** “Mọi tín hiệu phức tạp bất kỳ đều là tổng chồng chất (*superposition*) của các sóng hình sin thuần túy có tần số, biên độ và góc pha khác nhau.”
- Biến đổi Fourier (Fourier Transform)** là chiếc lăng kính toán học chuyển dữ liệu từ **Miền Thời gian** sang **Miền Tần số** (*Frequency Domain*), đóng vai trò nền tảng cho:
 - Speech & Audio AI (Whisper, AudioLM):** Biến sóng âm thô thành ảnh phổ **Mel-Spectrogram** thông qua biến đổi Fourier thời gian ngắn (STFT).
 - Tích chập nhanh bằng FFT (Convolution Theorem):** Tăng tốc phép tích chập chuỗi dài N từ $O(N^2)$ xuống $O(N \log_2 N)$ trong các mô hình State Space (S4, Hyena) và CNN.
 - Khắc phục Spectral Bias trong Deep Learning:** Giải thích vì sao mạng MLP thông thường bị “mù tần số cao” (làm mờ chi tiết ảnh trong NeRF) và vì sao phải dùng **Fourier Features / Sinusoidal Positional Encoding!**



(a) Miền Thời gian (Time Domain) (b) Miền Tần số (Frequency Spectrum)

II. Biến đổi Fourier Rời rạc (Discrete Fourier Transform — DFT)

1. Công thức DFT, Biến đổi Ngược IDFT & Ma trận Fourier F

① **Định nghĩa Toán học:** Cho tín hiệu rời rạc gồm N mẫu $x = [x_0, x_1, \dots, x_{N-1}]^T \in \mathbb{C}^N$. Biến đổi **DFT** tạo ra N hệ số tần số phức $X = [X_0, X_1, \dots, X_{N-1}]^T \in \mathbb{C}^N$:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i\frac{2\pi}{N}kn} = \sum_{n=0}^{N-1} x_n \left[\cos\left(\frac{2\pi kn}{N}\right) - i \sin\left(\frac{2\pi kn}{N}\right) \right], \quad k = 0, 1, \dots, N-1$$

Và **Biến đổi Fourier Ngược (IDFT — Inverse DFT)** khôi phục chính xác 100% tín hiệu gốc (chỉ cần đổi dấu mũ từ $-i$ sang $+i$ và chia cho N):

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k \cdot e^{+i\frac{2\pi}{N}kn}, \quad n = 0, 1, \dots, N-1$$

② **Góc nhìn Đại số Tuyến tính — Ma trận DFT F** : Đặt $\omega = e^{-i2\pi/N}$. Toàn bộ phép biến đổi DFT chính là phép nhân ma trận Vandermonde $F \in \mathbb{C}^{N \times N}$ (với phần tử $F_{kn} = \omega^{kn}$):

$$\begin{matrix} X \\ (N \times 1) \end{matrix} = \begin{matrix} F \\ (N \times N) \end{matrix} @ \begin{matrix} x \\ (N \times 1) \end{matrix} \quad \text{với} \quad F^H F = N \cdot I_N \implies F^{-1} = \frac{1}{N} F^H = \frac{1}{N} \bar{F}$$

Các cột của $\frac{1}{\sqrt{N}}F$ lập thành một **hệ cơ sở trực chuẩn** (*orthonormal basis*) trong $\mathbb{C}^N$. Nhờ đó, định lý **Parseval** bảo toàn năng lượng tuyệt đối: $\sum_{n=0}^{N-1} |x_n|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X_k|^2$.

③ **Ý nghĩa vật lý của từng hệ số X_k** :

- $X_0 = \sum_{n=0}^{N-1} x_n$ (**Thành phần DC — Tần số 0 Hz**): Tổng toàn bộ các mẫu (tỷ lệ với giá trị trung bình của tín hiệu, tức độ dịch nền không đổi).
- **Mô-đun** $|X_k| = \sqrt{\text{Re}(X_k)^2 + \text{Im}(X_k)^2}$: Biên độ (cường độ) của sóng tần số k chu kỳ trên N mẫu; **Phổ công suất (Power Spectrum)** là $P_k = |X_k|^2$.
- **Tính đối xứng liên hợp** (khi $x_n \in \mathbb{R}$ là tín hiệu thực): $X_{N-k} = \overline{X_k}$. Do đó nửa sau của phổ ($k > N/2$) chỉ là ảnh phản chiếu gương của nửa đầu ($1 \leq k < N/2$).

VD 1: Tính bằng tay biến đổi DFT của tín hiệu 4 điểm $x = [1, 0, -1, 0]^T$ ($N = 4$):

- **Bước 1 (Xác định hệ số xoay cơ sở)**: Với $N = 4$, góc bước nhảy là $\frac{2\pi}{4} = \frac{\pi}{2}$, nên $\omega = e^{-i\pi/2} = \cos\left(-\frac{\pi}{2}\right) + i \sin\left(-\frac{\pi}{2}\right) = -i$.
- **Bước 2 (Tính từng hệ số tần số $X_k = x_0(-i)^0 + x_1(-i)^k + x_2(-i)^{2k} + x_3(-i)^{3k}$)**:
Vì $x_1 = 0$ và $x_3 = 0$, công thức rút gọn còn $X_k = 1 \cdot 1 + (-1) \cdot (-i)^{2k} = 1 - (-1)^k$:

$$\left\{ \begin{array}{l} k = 0 \text{ (DC)} : X_0 = 1 - (-1)^0 = 1 - 1 = 0 \quad \langle \text{trung bình bằng 0} \rangle \\ k = 1 \text{ (Tần số 1)} : X_1 = 1 - (-1)^1 = 1 - (-1) = 2 \quad \langle \text{tập trung ở } k = 1! \rangle \\ k = 2 \text{ (Nyquist)} : X_2 = 1 - (-1)^2 = 1 - 1 = 0 \\ k = 3 \text{ (Đối xứng } \overline{X_1}) : X_3 = 1 - (-1)^3 = 1 - (-1) = 2 \end{array} \right.$$

- **Bước 3 (Kết luận)**: Vector phổ là $X = [0, 2, 0, 2]^T$ — cho thấy tín hiệu $x = [1, 0, -1, 0]^T$ chính là một sóng $\cos\left(\frac{2\pi \cdot 1 \cdot n}{4}\right)$ thuần khiết có tần số đúng bằng $k = 1$ chu kỳ trên 4 mẫu!

III. Thuật toán Fast Fourier Transform (FFT — Cooley-Tukey)

① Vì sao DFT trực tiếp bị tắc nghẽn ($O(N^2)$) và FFT đạt $O(N \log_2 N)$?

- Tính trực tiếp $X = F @ x$ đòi hỏi N vòng lặp ngoài $\times N$ phép nhân cộng bên trong $= O(N^2)$.
- Năm 1965, **Cooley & Tukey** phát hiện ra rằng nếu tách tín hiệu độ dài N (lũy thừa của 2) thành **nửa chỉ số chẵn** x_{2m} và **nửa chỉ số lẻ** x_{2m+1} , các hệ số xoay ω_N^{kn} có tính đối xứng tuần hoàn $\omega_N^{k+N/2} = -\omega_N^k$:

$$X_k = \underbrace{\sum_{m=0}^{N/2-1} x_{2m} e^{-i \frac{2\pi}{N/2} km}}_{E_k \text{ (DFT cỡ } N/2 \text{ của phần chẵn)}} + \underbrace{e^{-i \frac{2\pi}{N} k}}_{\text{Twiddle factor } \omega_N^k} \cdot \underbrace{\sum_{m=0}^{N/2-1} x_{2m+1} e^{-i \frac{2\pi}{N/2} km}}_{O_k \text{ (DFT cỡ } N/2 \text{ của phần lẻ)}}$$

Từ đó ta có **Phép toán Cánh bướm (Butterfly Operation)** tính đồng thời cả nửa đầu và nửa sau chỉ từ một cặp (E_k, O_k) :

$$\begin{cases} X_k = E_k + \omega_N^k \cdot O_k \\ X_{k+N/2} = E_k - \omega_N^k \cdot O_k \end{cases} \quad \left(\forall k = 0, 1, \dots, \frac{N}{2} - 1 \right)$$

- **Độ phức tạp đệ quy:** Chia đôi bài toán liên tiếp qua $\log_2 N$ tầng, mỗi tầng tốn $O(N)$ phép tính $\implies$ Tổng chi phí giảm xuống còn $O(N \log_2 N)$.
- **Con số thực tế:** Với chuỗi âm thanh / chuỗi token dài $N = 10^6 \approx 2^{20}$:

DFT trực tiếp $O(N^2)$: $(10^6)^2 = 10^{12}$ phép tính ($\approx$ vài phút)

FFT Cooley-Tukey $O(N \log_2 N)$: $10^6 \times 20 = 2 \times 10^7$ phép tính (**nhANH gẤP 50 000×**!)

IV. Định lý Lấy mẫu Nyquist-Shannon & Định lý Tích chập (Convolution Theorem)

1. Định lý Nyquist-Shannon & Hiện tượng Chồng phổ (Aliasing)

① **Tần số Nyquist** ($f_{\text{Nyquist}} = f_s/2$): Để thu giữ chính xác một tín hiệu liên tục có tần số cao nhất là f_{max} mà không bị mất thông tin, tần số lấy mẫu f_s (*sampling rate*) bắt buộc phải lớn hơn hoặc bằng **gấp đôi** f_{max} :

$$f_s \geq 2 \cdot f_{\text{max}} \iff f_{\text{max}} \leq f_{\text{Nyquist}} = \frac{f_s}{2}$$

- **Hiện tượng Aliasing (Chồng phổ / Giả dạng tần số):** Nếu tín hiệu có thành phần tần số $f > f_s/2$, sau khi lấy mẫu rồi rạc, tần số cao đó sẽ bị “gập” ngược lại và **nguy trang thành một tần số thấp giả mạo** $f_{\text{alias}} = |f_s - f|$ (giống bánh xe ô tô quay quá nhanh trên phim 24 fps trông như đang quay ngược!).
- **VD 2:** Một sóng âm $f = 90$ Hz bị lấy mẫu ở tần số $f_s = 100$ Hz (thấp hơn ngưỡng Nyquist $2 \times 90 = 180$ Hz). Khi đó các điểm mẫu thu được trùng khít 100% với sóng $f_{\text{alias}} = 100 - 90 = 10$ Hz và không thuật toán nào phân biệt lại được! Trong CNN (ResNet/ StyleGAN), khi giảm kích thước ảnh (*downsampling / stride = 2*), phải đặt bộ lọc làm mờ tần số cao (*anti-aliasing blur filter*) trước khi lấy mẫu để chống hiện tượng răng cưa này.

WRONG

Ngộ nhận phổ biến: “Đệm thêm nhiều số 0 (Zero-padding) vào đuôi tín hiệu giúp tăng độ phân giải tần số thực sự của FFT” $\otimes$

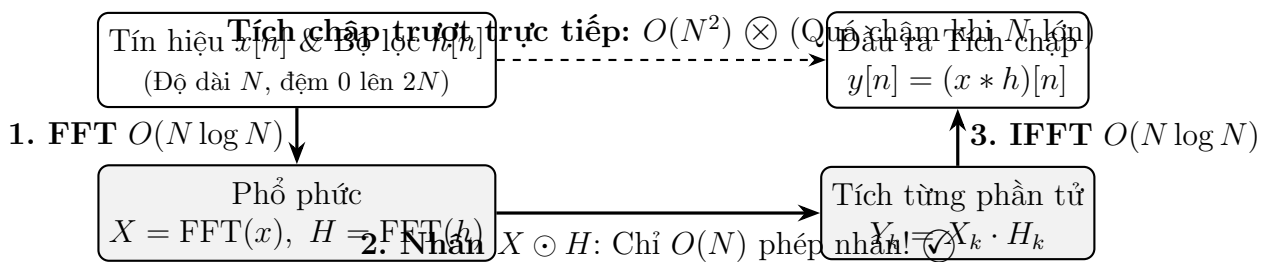
Sự thật: Độ phân giải tần số thực sự $\Delta f = \frac{f_s}{N_{\text{mẫu thật}}} = \frac{1}{T_{\text{quan sát}}}$ chỉ phụ thuộc vào tổng thời gian thu tín hiệu thực tế T . Việc chèn thêm số 0 (zero-padding) chỉ giúp nội suy làm mịn đồ thị phổ giữa các vạch tần số sẵn có, chứ hoàn toàn không thể tách được hai tần số nằm sát nhau hơn $1/T$ Hz!

2. Định lý Tích chập (The Convolution Theorem)

① Tích chập trong miền Thời gian = Nhân từng phần tử trong miền Tần số: Cho tín hiệu đầu vào x và bộ lọc (kernel) h có độ dài N . Phép tích chập $(x * h)[n] = \sum_m x[m]h[n-m]$ nếu tính trượt trực tiếp sẽ mất $O(N^2)$. Định lý Tích chập phát biểu:

$$\mathcal{F}\{x * h\} = \mathcal{F}\{x\} \odot \mathcal{F}\{h\} \iff x * h = \text{IFFT}\left(\text{FFT}(x) \odot \text{FFT}(h)\right)$$

trong đó $\odot$ là phép nhân từng phần tử (element-wise / Hadamard product) chỉ tốn $O(N)$!



- **Lưu ý kỹ thuật sống còn (Circular vs Linear Convolution):** Vì DFT ngầm coi tín hiệu tuần hoàn chu kỳ N , nhân trực tiếp $\text{FFT}(x) \odot \text{FFT}(h)$ sẽ tạo ra **tích chập vòng** (circular convolution — phần đuôi bị tràn vòng lại cộng đè lên phần đầu). Để tính **tích chập tuyến tính chuẩn** của hai chuỗi độ dài N và M , ta **bắt buộc phải đệm số 0** (zero-pad) cho cả hai chuỗi lên độ dài tối thiểu $L \geq N + M - 1$ trước khi chạy FFT!
- **Ứng dụng trong kiến trúc AI hiện đại (S4, Hyena, LongConv):** Thay vì dùng Self-Attention tốn $O(N^2)$ trên ngữ cảnh dài $N = 131\,072$ token, các kiến trúc tích chập toàn cục dùng bộ lọc dài bằng cả ngữ cảnh và tính qua $\text{IFFT}(\text{FFT}(x) \odot \text{FFT}(h))$ chỉ trong $O(N \log N)$.

V. Ứng dụng Chuyên sâu trong AI: STFT, Lọc Ảnh & Spectral Bias

1. STFT & Mel-Spectrogram trong Speech AI (Whisper, AudioLM)

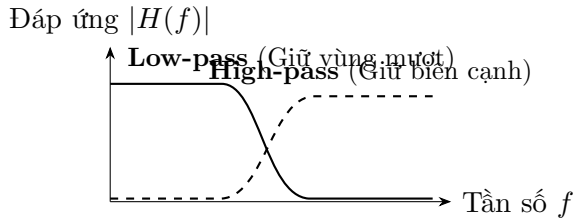
- Một phép FFT duy nhất trên toàn bộ đoạn ghi âm 10 giây cho biết trong đó có những nốt nhạc nào, nhưng **mất hoàn toàn thông tin thời điểm** (không biết từ nào phát

âm trước, từ nào phát âm sau).

- **Short-Time Fourier Transform (STFT):** Cắt tín hiệu âm thanh thành các khung cửa sổ ngắn chồng lấp nhau (VD cửa sổ 25 ms, bước nhảy *hop* 10 ms), nhân với **hàm cửa sổ Hann / Hamming** $w[n] = 0,5(1 - \cos \frac{2\pi n}{N-1})$ (để làm mượt hai mép biên về 0, chống hiện tượng rò rỉ phổ *spectral leakage*), rồi tính FFT trên từng khung.
- Ghép các cột phổ công suất $|X(m, k)|^2$ theo thời gian m và chiều trục tần số lên thang đo cảm thụ thính giác con người (*Mel scale*) ta thu được bức ảnh 2 chiều **Log-Mel Spectrogram** — đầu vào chuẩn mực của **OpenAI Whisper** và mọi mô hình nhận dạng tiếng nói hiện đại!

2. Lọc Tần số trong Ảnh & Spectral Bias của Mạng Nơ-ron

- Trong Xử lý Ảnh 2D:



- **Tần số thấp ($f \approx 0$):** Các vùng màu chuyển đổi êm dịu, bố cục tổng thể. **Bộ lọc thông thấp (Low-pass / Gaussian Blur)** giữ tần số thấp, xóa nhiễu hạt nhưng làm mờ cạnh.
- **Tần số cao (f lớn):** Các đường biên sắc nét (*edges*), vân bề mặt chi tiết (*textures*) và nhiễu hạt. **Bộ lọc thông cao (High-pass / Sobel, Laplacian)** làm nổi bật biên cạnh vật thể.

① Thiên lệch Phổ (Spectral Bias / Frequency Principle) & Fourier Features:

- Một phát hiện lý thuyết cực kỳ quan trọng trong Deep Learning (Rahaman et al., 2019; Tancik et al., 2020): Các mạng nơ-ron MLP tiêu chuẩn khi huấn luyện bằng Gradient Descent luôn **học các thành phần tần số thấp trước tiên và cực kỳ chậm chạp khi học các chi tiết tần số cao (Spectral Bias)**.
- Nếu đưa trực tiếp tọa độ không gian $(x, y, z) \in \mathbb{R}^3$ vào mạng MLP để tái tạo cảnh 3D (**NeRF**), bức ảnh thu được sẽ bị **nhòe mờ hoàn toàn**, mất sạch lông tóc và vân gạch sắc nét!
- **Giải pháp — Fourier Feature Mapping / Sinusoidal Positional Encoding:**
Trước khi đưa tọa độ p vào mạng MLP hoặc Transformer, ta chiếu p lên một dải các sóng hình sin và cosin có tần số tăng dần theo cấp số nhân:

$$\gamma(p) = \left[\sin(2^0 \pi p), \cos(2^0 \pi p), \sin(2^1 \pi p), \cos(2^1 \pi p), \dots, \sin(2^{L-1} \pi p), \cos(2^{L-1} \pi p) \right]^T$$

Các sóng tần số cao $2^{L-1} \pi$ đóng vai trò “kính lúp tần số”, giúp mạng nơ-ron phân biệt tức thì hai vị trí cực gần nhau và hội tụ sắc nét tới từng điểm ảnh!

VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 20

- **DFT & IDFT:** Phân tách tín hiệu N mẫu thành N tần số phức:
$$X_k = \sum_{n=0}^{N-1} x_n e^{-i\frac{2\pi}{N}kn}$$
 và khôi phục hoàn hảo qua $x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{+i\frac{2\pi}{N}kn}$. Ma trận DFT thỏa $F^{-1} = \frac{1}{N}F^H$ và bảo toàn năng lượng (Định lý Parseval).
- **Fast Fourier Transform (FFT — Cooley-Tukey):** Chia để trị theo chỉ số chẵn/lẻ ($X_k = E_k + \omega_N^k O_k$, $X_{k+N/2} = E_k - \omega_N^k O_k$), giảm độ phức tạp từ $O(N^2)$ xuống $O(N \log_2 N)$.
- **Định lý Lấy mẫu Nyquist-Shannon:** Tần số lấy mẫu phải thỏa $f_s \geq 2f_{\max}$ (tức $f_{\max} \leq f_{\text{Nyquist}} = f_s/2$) để không bị chồng phổ (**Aliasing**). Đệm số 0 (*zero-padding*) chỉ nội suy làm mịn phổ chứ không tăng độ phân giải tần số thực $\Delta f = f_s/N$.
- **Định lý Tích chập (Convolution Theorem):** Tích chập trong miền thời gian bằng phép nhân từng phần tử trong miền tần số:
$$x * h = \text{IFFT}(\text{FFT}(x) \odot \text{FFT}(h))$$
 (nhớ đệm số 0 lên độ dài $N + M - 1$ để tránh tràn vòng).
- **Ứng dụng AI (STFT & Spectral Bias):** Dùng cửa sổ trượt STFT (kèm cửa sổ Hann/Hamming giảm rò rỉ phổ) để tạo ảnh **Mel-Spectrogram** cho Speech AI; dùng **Fourier Features** (sin / cos đa tần số) để vượt qua **Spectral Bias** trong NeRF và Transformer.

Lesson 21: Graph Theory for Machine Learning

I. Vấn đề (The Problem)

Trong Học máy truyền thống (Tabular ML, CNN trên ảnh phẳng), dữ liệu được giả định nằm trên một **lưới tọa độ phẳng đều đặn** (Euclidean Grid): mỗi hàng trong bảng dữ liệu độc lập với nhau, hoặc mỗi điểm ảnh (i, j) luôn có đúng 8 điểm lân cận cố định.

❓ Vì sao cấu trúc bảng phẳng thất bại hoàn toàn trên dữ liệu kết nối thực tế?

- Nếu biểu diễn một phân tử hóa học (AlphaFold / Drug Discovery) hoặc mạng xã hội tỷ người dùng thành bảng phẳng, việc đánh số lại thứ tự các đỉnh (Node Permutation) sẽ làm thay đổi hoàn toàn ma trận đầu vào dù cấu trúc vật lý của phân tử không hề đổi!
- Trong thực tế, **mối quan hệ kết nối (Topology)** mang nhiều tín hiệu hơn bản thân thuộc tính rời rạc của từng điểm dữ liệu:

Hệ thống AI thực tế	Đỉnh V (Nodes) & Cạnh E (Edges)	Bài toán học máy trên Đồ thị
Mạng xã hội & Phát hiện gian lận	Đỉnh: Tài khoản, Ví tiền Cạnh: Giao dịch, Kết bạn	Phát hiện vòng rửa tiền (Cycle Detection) và lan truyền tín dụng qua GNN.
Sinh học phân tử (AlphaFold / GNN)	Đỉnh: Nguyên tử (C, N, O) Cạnh: Liên kết hóa học	Dự đoán độc tính, ái lực gắn kết protein (Graph-level Classification).
Hệ thống Gợi ý (Recommender)	Đồ thị 2 phía (Bipartite): User $\leftrightarrow$ Item (Sản phẩm)	Dự đoán cạnh khuyết (Link Prediction) bằng LightGCN / PinSage.
Đồ thị Tri thức (Knowledge Graph)	Đỉnh: Thực thể (Entities) Cạnh có hướng: Quan hệ	Suy luận đa bước (Multi-hop Reasoning) kết hợp GraphRAG cho LLM.
Đồ thị Tính toán (Autograd DAG)	Đỉnh: Phép toán / Tensor Cạnh có hướng: Luồng dữ liệu	Sắp xếp Tô-pô (Topological Sort) để chạy Lan truyền ngược (Backprop).
Transformer Self-Attention	Đỉnh: Các Token trong câu Cạnh: Trọng số α_{ij}	Đồ thị đầy đủ có hướng và có trọng số động (Dynamic Complete Digraph).

$\Rightarrow$ Để máy tính và GPU xử lý được đồ thị $G = (V, E)$ gồm $n = |V|$ đỉnh và $m = |E|$ cạnh, ta phải **đại số hóa đồ thị thành ma trận**.

II. Biểu diễn Đại số của Đồ thị (Algebraic Representation)

1. Ma trận Kề và Ma trận Bậc (A & D)

① Ma trận kề (Adjacency Matrix $A \in \mathbb{R}^{n \times n}$):

$$A_{ij} = \begin{cases} w_{ij} \text{ (hoặc 1)} & \text{nếu có cạnh nối từ đỉnh } i \rightarrow \text{đỉnh } j \quad ((i, j) \in E) \\ 0 & \text{nếu không có cạnh nối trực tiếp} \end{cases}$$

- **Đồ thị vô hướng (Undirected Graph):** Cạnh (i, j) tương đương (j, i) nên ma trận kề luôn **đối xứng**: $A = A^T \implies$ mọi trị riêng của A đều là số thực (theo Định lý Phổ ở Lesson 3).
- **Đồ thị có hướng (Directed Graph / Digraph):** $A_{ij} \neq A_{ji} \implies A \neq A^T$ (bất đối xứng).

② Ma trận Bậc (Degree Matrix $D \in \mathbb{R}^{n \times n}$): Là ma trận đường chéo chứa tổng trọng số các cạnh nối với từng đỉnh $d_i = \sum_{j=1}^n A_{ij}$:

$$D = \text{diag}(d_1, d_2, \dots, d_n) = \begin{bmatrix} d_1 & 0 & \dots & 0 \\ 0 & d_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & d_n \end{bmatrix} \quad \text{với } d_i = \sum_{j=1}^n A_{ij}$$

WRONG — Cạm bẫy tràn RAM khi lưu Ma trận Kề dày đặc (Dense Matrix)

Giả sử mạng xã hội có $n = 10^7$ người dùng (10 triệu đỉnh), trung bình mỗi người có 50 bạn bè ($|E| \approx 5 \times 10^8$ cạnh). Nếu khởi tạo ma trận $A = \text{np.zeros}((10_000_000, 10_000_000))$ kiểu float32 (4 bytes):

$$\text{RAM Dense} = (10^7)^2 \times 4 \text{ bytes} = 4 \times 10^{14} \text{ bytes} \approx \mathbf{400\,000 \text{ GB RAM!}} \quad (\otimes)$$

Trong khi đó, 99,9995% các ô trong A là số 0!

★ **Giải pháp trong PyTorch Geometric (PyG) / DGL:** Chỉ lưu danh sách tọa độ các cạnh khác 0 theo chuẩn **COO (Coordinate Format)** gồm 2 mảng số nguyên edge_index kích thước $\text{idx}_{(2 \times |E|)}$ hoặc chuẩn nén dòng **CSR (Compressed Sparse Row)**:

$$\text{RAM Sparse COO} = 2 \times (5 \times 10^8) \times 4 \text{ bytes} = 4 \times 10^9 \text{ bytes} \approx \mathbf{4 \text{ GB RAM!}} \quad (\checkmark)$$

$\implies$ Độ phức tạp bộ nhớ giảm từ $\mathcal{O}(n^2)$ xuống $\mathcal{O}(|E|)$, giúp nhân ma trận thưa $\text{spmm}(A, X)$ chạy trực tiếp trên 1 GPU!

2. Đường đi và Lũy thừa Ma trận Kề (Walks and Powers of A)

① Định lý Đếm số đường đi (Walks of length k): Cho đồ thị có ma trận kề A (với $A_{ij} \in \{0, 1\}$). Khi lũy thừa ma trận kề lên bậc k ($A^k = \underbrace{A @ A @ \dots @ A}_{k \text{ lần}}$), phần tử ở hàng i , cột j

là $(A^k)_{ij}$ chính là **tổng số đường đi (walks) có độ dài đúng bằng k bước** nối từ đỉnh i đến đỉnh j :

$$(A^k)_{ij} = \sum_{r_1=1}^n \sum_{r_2=1}^n \cdots \sum_{r_{k-1}=1}^n A_{i,r_1} A_{r_1,r_2} \cdots A_{r_{k-1},j} = \text{Số đường đi dài } k \text{ bước từ } i \rightarrow j$$

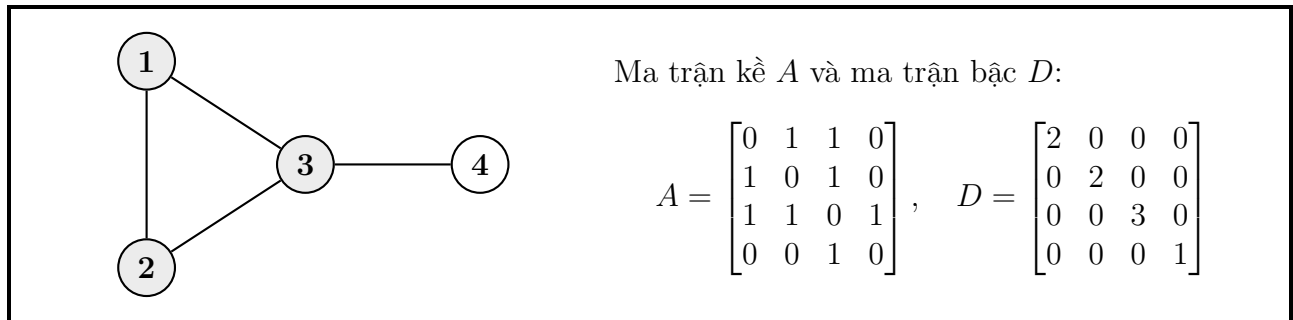
❓ Vì sao phép nhân ma trận lại **tự động đếm được số đường đi trên đồ thị**?

- Xét $k = 2$: Theo định nghĩa nhân hai ma trận, phần tử $(A^2)_{ij}$ được tính bằng tích vô hướng hàng i và cột j :

$$(A^2)_{ij} = \sum_{r=1}^n A_{ir} \cdot A_{rj}$$

- Tích $A_{ir} \cdot A_{rj} = 1$ **khi và chỉ khi** cả $A_{ir} = 1$ (có cạnh nối $i \rightarrow r$) **VÀ** $A_{rj} = 1$ (có cạnh nối $r \rightarrow j$), tức là tồn tại đường đi 2 bước $i \rightarrow r \rightarrow j$ đi qua đỉnh trung gian r . Nếu một trong hai cạnh không tồn tại thì tích bằng 0.
- Do đó, tổng $\sum_{r=1}^n A_{ir} A_{rj}$ quét qua tất cả các đỉnh trung gian $r \in \{1, \dots, n\}$ và cộng dồn toàn bộ các đường đi dài 2 bước từ i đến j ! Quy nạp lên bậc k : $(A^k)_{ij} = \sum_{r=1}^n (A^{k-1})_{ir} A_{rj}$ đếm chính xác số đường đi dài k bước.

VD 1: Xét đồ thị vô hướng gồm 4 đỉnh $\{1, 2, 3, 4\}$ (hình cái điều — gồm tam giác $\{1, 2, 3\}$ và phần đuôi nối từ đỉnh 3 ra đỉnh 4) như hình vẽ:



Bước 1: Tính số đường đi có độ dài $k = 2$ bước bằng phép nhân ma trận $A^2 = A @ A$:

$$A^2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} @ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{2} & 1 & 1 & \mathbf{1} \\ 1 & \mathbf{2} & 1 & 1 \\ 1 & 1 & \mathbf{3} & 0 \\ \mathbf{1} & 1 & 0 & \mathbf{1} \end{bmatrix}$$

Bước 2: Giải mã ý nghĩa vật lý của các con số trong A^2 :

- Trên đường chéo chính** $(A^2)_{ii} = d_i$: $(A^2)_{11} = 2, (A^2)_{22} = 2, (A^2)_{33} = 3, (A^2)_{44} = 1$ chính là **bậc của từng đỉnh**! Vì muốn đi 2 bước từ đỉnh i rồi quay về chính đỉnh i ($i \rightarrow r \rightarrow i$), ta chỉ việc bước sang một hàng xóm r bất kỳ rồi bước ngược lại. Có d_i hàng xóm nên có đúng d_i cách!
- Ngoài đường chéo** $(A^2)_{14} = 1$: Có đúng 1 đường đi dài 2 bước từ đỉnh 1 đến đỉnh 4 (đó là $1 \rightarrow 3 \rightarrow 4$).

- **Ngoài đường chéo** $(A^2)_{34} = 0$: Không có đường đi nào dài đúng 2 bước từ đỉnh 3 đến đỉnh 4 (vì 3 và 4 nối trực tiếp mất 1 bước, còn từ 4 không có cạnh thứ ba nào nối về 3).

Bước 3: Tính tiếp $A^3 = A^2 @ A$ để đếm số đường đi dài $k = 3$ bước và số chu trình tam giác:

$$A^3 = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 3 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix} @ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} \mathbf{2} & 3 & 4 & 1 \\ 3 & \mathbf{2} & 4 & 1 \\ 4 & 4 & \mathbf{2} & 3 \\ 1 & 1 & 3 & \mathbf{0} \end{bmatrix}$$

★ **Hệ quả Đếm số Tam giác (Triangle Counting trong Phân tích mạng xã hội):**

- Đường chéo $(A^3)_{ii}$ đếm số đường đi dài 3 bước xuất phát từ i và quay về chính i (tức là chu trình tam giác đi qua i theo 2 chiều thuận/ngược). Ở đây $(A^3)_{11} = (A^3)_{22} = (A^3)_{33} = 2$ (ứng với $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ và $1 \rightarrow 3 \rightarrow 2 \rightarrow 1$), còn $(A^3)_{44} = 0$ vì đỉnh 4 không thuộc tam giác nào!
- Tổng số tam giác trong đồ thị vô hướng được tính chỉ bằng một công thức Vết ma trận (Trace):

$$\text{Số tam giác trong } G = \frac{1}{6} \text{Trace}(A^3) = \frac{2 + 2 + 2 + 0}{6} = 1 \text{ tam giác}$$

(Chia cho 6 vì mỗi tam giác có 3 đỉnh khởi đầu và 2 chiều đi thuận/ngược: $3 \times 2 = 6$).

3. Thuật toán Duyệt Đồ thị: BFS vs. DFS

Khi cần tìm đường đi ngắn nhất hoặc kiểm tra tính liên thông với chi phí $\mathcal{O}(|V| + |E|)$ mà không cần nhân ma trận $\mathcal{O}(n^3)$:

Thuật toán	Cấu trúc dữ liệu	Cơ chế hoạt động	Ứng dụng cốt lõi
BFS ⟨Breadth-First⟩	Hàng đợi (Queue — FIFO)	Lan tỏa theo từng lớp sóng đồng tâm (hết khoảng cách k mới sang $k + 1$).	Tìm đường đi ngắn nhất (Shortest Hop-Count), lấy mẫu lân cận GNN.
DFS ⟨Depth-First⟩	Ngăn xếp (Stack — LIFO)	Di sâu kịch đường một nhánh rồi mới quay lui (Backtracking).	Phát hiện chu trình (Cycle), sắp xếp Tô-pô (DAG Autograd).

III. Ma trận Graph Laplacian & Lý thuyết Đồ thị Phổ (Spectral Graph Theory)

1. Định nghĩa Ma trận Laplacian ($L = D - A$) và Dạng Toàn phương

① Ma trận Laplacian chưa chuẩn hóa (Unnormalized Graph Laplacian):

$$\boxed{L = D - A} \iff L_{ij} = \begin{cases} d_i & \text{nếu } i = j \text{ (Bậc của đỉnh } i\text{)} \\ -w_{ij} & \text{nếu } i \neq j \text{ và } (i, j) \in E \\ 0 & \text{nếu } i \neq j \text{ và không có cạnh nối} \end{cases}$$

② Chứng minh Dạng toàn phương (Quadratic Form — Trái tim của Lý thuyết Phổ): Giả sử ta gán cho mỗi đỉnh i một giá trị thực $x_i \in \mathbb{R}$, tạo thành vectơ tín hiệu trên đồ thị $\vec{x} = [x_1, x_2, \dots, x_n]^T \in \mathbb{R}^n$. Khi đó dạng toàn phương $\vec{x}^T L \vec{x}$ đo **độ biến thiên** (độ gồ ghề / Dirichlet Energy) của tín hiệu $\vec{x}$ dọc theo các cạnh của đồ thị:

$$\boxed{\vec{x}^T L \vec{x} = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i - x_j)^2 = \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2 \geq 0}$$

★ Chứng minh từng bước (Step-by-step Proof):

- **Bước 1:** Tách $L = D - A$ trong biểu thức $\vec{x}^T L \vec{x}$:

$$\vec{x}^T L \vec{x} = \vec{x}^T (D - A) \vec{x} = \vec{x}^T D \vec{x} - \vec{x}^T A \vec{x} = \sum_{i=1}^n d_i x_i^2 - \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j$$

- **Bước 2:** Thay định nghĩa bậc của đỉnh $d_i = \sum_{j=1}^n w_{ij}$ vào số hạng đầu tiên, rồi nhân đôi và chia 2 (do đồ thị vô hướng có $w_{ij} = w_{ji}$ nên vai trò chỉ số i, j hoàn toàn đối xứng):

$$\begin{aligned} \vec{x}^T L \vec{x} &= \sum_{i=1}^n \left(\sum_{j=1}^n w_{ij} \right) x_i^2 - \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j \\ &= \frac{1}{2} \left(\sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i^2 - 2 \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_i x_j + \sum_{i=1}^n \sum_{j=1}^n w_{ij} x_j^2 \right) \\ &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n w_{ij} (x_i - x_j)^2 = \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2 \quad (\text{đpcm}). \end{aligned}$$

③ Ba tính chất bất biến rút ra từ dạng toàn phương $\vec{x}^T L \vec{x}$:

- 1) **L luôn Xác định Dương Nửa (Positive Semi-Definite — PSD):** Vì trọng số $w_{ij} \geq 0$ và bình phương $(x_i - x_j)^2 \geq 0$ với mọi $\vec{x} \in \mathbb{R}^n$, nên $\vec{x}^T L \vec{x} \geq 0$. Do đó mọi trị riêng của L đều không âm:

$$0 = \lambda_1 \leq \lambda_2 \leq \lambda_3 \leq \dots \leq \lambda_n$$

- 2) **Trị riêng nhỏ nhất luôn bằng 0** ($\lambda_1 = 0$) **với vectơ riêng hằng** $\vec{1} = [1, 1, \dots, 1]^T$: Tổng mỗi hàng của L luôn bằng 0: $(L\vec{1})_i = d_i - \sum_j w_{ij} = 0 \implies L\vec{1} = 0 \cdot \vec{1}$. Khi gán mọi đỉnh cùng một giá trị $x_1 = x_2 = \dots = x_n = c$, độ chênh lệch trên mọi cạnh $(x_i - x_j)^2 = 0 \implies \vec{x}^T L \vec{x} = 0$.
- 3) **Số bội của trị riêng $\lambda = 0$ bằng đúng số Thành phần Liên thông (Connected Components)**: Để $\vec{x}^T L \vec{x} = \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2 = 0$, bắt buộc $x_i = x_j$ trên mọi cặp đỉnh có cạnh nối $(i, j) \in E$. Suy ra $\vec{x}$ phải nhận giá trị hằng số trên từng thành phần liên thông độc lập! Nếu đồ thị có k cụm rời nhau (k connected components), ta có đúng k vectơ riêng độc lập tuyến tính (mỗi vectơ bắt số 1 trên 1 cụm và số 0 trên các cụm còn lại) ứng với $\lambda = 0$.

2. Giá trị Fiedler (λ_2) & Phân cụm Phổ (Spectral Clustering)

① Độ liên thông Đại số (Algebraic Connectivity — Fiedler Value λ_2):

- Với một đồ thị liên thông ($k = 1$), chỉ có duy nhất $\lambda_1 = 0$. Trị riêng thứ hai $\lambda_2 > 0$ (do nhà toán học Miroslav Fiedler phát hiện năm 1973) đo **mức độ gắn kết bền chặt của đồ thị**.
- Nếu $\lambda_2 = 0$: Đồ thị bị đứt rời thành ít nhất 2 mảnh độc lập.
- Nếu $\lambda_2 > 0$ nhưng rất nhỏ ($\lambda_2 \approx 0$): Đồ thị có một **nút cổ chai (bottleneck / cầu nối mỏng manh)** sắp đứt — chỉ cần cắt một vài cạnh là tách đôi được đồ thị!

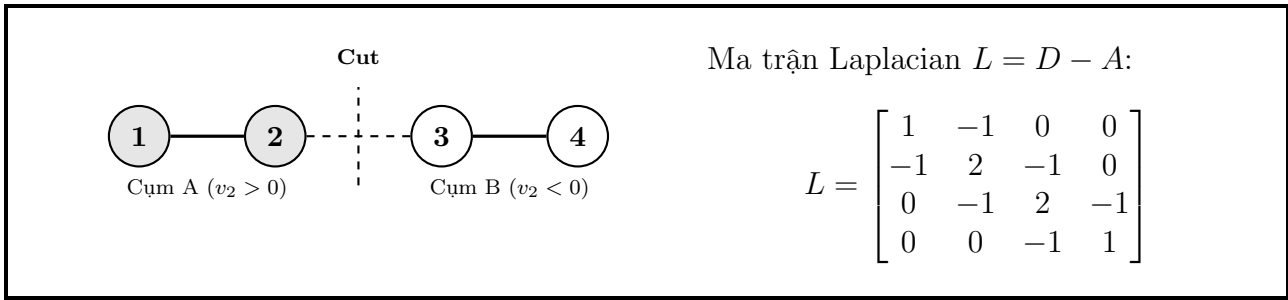
② Tại sao Vectơ Fiedler $\vec{v}_2$ giải được bài toán Cắt Đồ thị (Min-Cut)?

- Theo nguyên lý Courant-Fischer (Rayleigh Quotient), vectơ riêng thứ hai $\vec{v}_2$ là nghiệm của bài toán tối ưu:

$$\vec{v}_2 = \arg \min_{\substack{\vec{x} \perp \vec{1} \\ \|\vec{x}\|=1}} \vec{x}^T L \vec{x} = \arg \min_{\substack{\sum_i x_i = 0 \\ \sum_i x_i^2 = 1}} \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2$$

- Giải thích trực quan tuyệt đẹp:** Điều kiện $\vec{x} \perp \vec{1} \iff \sum_{i=1}^n x_i = 0$ ép các đỉnh phải chia thành **hai phe đối dấu nhau**: một nửa mang dấu dương ($x_i > 0$) và một nửa mang dấu âm ($x_j < 0$) để tổng bằng 0, đồng thời $\sum_i x_i^2 = 1$ cấm việc gán toàn số 0.
- Trong khi đó, hàm mục tiêu bắt tổng $\sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2$ phải **nhỏ nhất**! Nếu hai đỉnh i, j nằm trong cùng một cụm gắn kết chặt (w_{ij} nhiều), chúng buộc phải nhận giá trị **cùng dấu và sát nhau** ($x_i \approx x_j$) để $(x_i - x_j)^2 \approx 0$. Chỉ tại đúng **cầu nối cổ chai (bottleneck)** giữa hai cụm, dấu của x mới dám đổi từ Dương (+) sang Âm (-)!

VD 2: Cho đồ thị hình quả tạ (Barbell Graph) gồm 4 đỉnh chia làm 2 nhóm $\{1, 2\}$ và $\{3, 4\}$, được nối với nhau qua cạnh cầu nối $(2, 3)$:



- **Bước 1:** Giải hệ trị riêng $L\vec{v} = \lambda\vec{v}$ cho đường đi 4 đỉnh, ta thu được 4 trị riêng xếp tăng dần:

$$\lambda_1 = 0, \quad \lambda_2 = 2 - \sqrt{2} \approx 0,5858 \text{ (Fiedler value)}, \quad \lambda_3 = 2, \quad \lambda_4 = 2 + \sqrt{2} \approx 3,4142$$

- **Bước 2:** Tìm vectơ Fiedler $\vec{v}_2$ ứng với $\lambda_2 \approx 0,5858$:

$$\vec{v}_2 \approx \begin{bmatrix} +0,653 \\ +0,271 \\ -0,271 \\ -0,653 \end{bmatrix} \Rightarrow \begin{cases} \text{Đỉnh 1, 2 mang dấu (+) (+0,653 và +0,271)} & \rightarrow \text{Cụm A} \\ \text{Đỉnh 3, 4 mang dấu (-) (-0,271 và -0,653)} & \rightarrow \text{Cụm B} \end{cases}$$

$\Rightarrow$ Không cần chạy bất kỳ vòng lặp ngẫu nhiên nào, chỉ cần nhìn **dấu sign**($\vec{v}_2$) của vectơ Fiedler là cắt đồ thị trúng ngay tâm cầu nối (2,3)!

3. Ma trận Laplacian Chuẩn hóa (Normalized Laplacians)

Khi đồ thị có các **siêu đỉnh (Hub nodes)** có bậc $d_i = 10000$ nằm cạnh các đỉnh lá có bậc $d_j = 1$, ma trận $L = D - A$ chưa chuẩn hóa bị thống trị bởi các đỉnh bậc lớn. Ta dùng 2 dạng chuẩn hóa:

① **Chuẩn hóa Đối xứng ($\mathcal{L}_{\text{sym}}$):** $\mathcal{L}_{\text{sym}} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} A D^{-1/2}$

② **Chuẩn hóa Bước ngẫu nhiên ($\mathcal{L}_{\text{rw}}$):** $\mathcal{L}_{\text{rw}} = D^{-1} L = I - D^{-1} A$

- Trong $\mathcal{L}_{\text{sym}}$, phần tử nối giữa đỉnh i và j được chia cho trung bình nhân bậc của cả hai đầu nút: $\frac{-w_{ij}}{\sqrt{d_i d_j}}$, giữ nguyên tính đối xứng ($\mathcal{L}_{\text{sym}}^T = \mathcal{L}_{\text{sym}}$) và ép toàn bộ trị riêng nằm gọn trong đoạn $\lambda_i \in [0, 2]$.
- Trong $\mathcal{L}_{\text{rw}}$, ma trận $P = D^{-1} A$ có tổng mỗi hàng bằng 1, chính là **ma trận chuyển trạng thái của Bước đi Ngẫu nhiên (Random Walk)** trên đồ thị (sẽ học sâu ở Lesson 22)!

IV. Ứng dụng Chuyên sâu trong AI hiện đại (Deep AI Applications)

1. Graph Neural Networks (GNN / GCN) & Cơ chế Truyền thông điệp (Message Passing)

① Công thức Lan truyền Tầng GCN (Kipf & Welling, ICLR 2017): Giả sử ở tầng thứ l , mỗi đỉnh $i \in \{1, \dots, n\}$ có một vectơ đặc trưng $h_i^{(l)} \in \mathbb{R}^{f_l}$, xếp chồng thành ma trận đặc trưng $H^{(l)}_{(n \times f_l)}$. Công thức cập nhật sang tầng $l + 1$ của mạng Graph Convolutional Network (GCN) là:

$$H^{(l+1)}_{(n \times f_{l+1})} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} @ H^{(l)}_{(n \times f_l)} @ W^{(l)}_{(f_l \times f_{l+1})} \right)$$

trong đó:

- $\tilde{A} = A + I_n$: Ma trận kề được **cộng thêm** ma trận đơn vị I_n (Self-loops / khuyên tự nối).
- $\tilde{D}_{ii} = \sum_{j=1}^n \tilde{A}_{ij} = d_i + 1$: Ma trận bậc mới sau khi đã cộng khuyên tự nối.
- $W^{(l)} \in \mathbb{R}^{f_l \times f_{l+1}}$: Ma trận trọng số học được (Learnable Weights); $\sigma(\cdot)$: hàm kích hoạt phi tuyến (ReLU / GELU).

❓ **Vì sao không viết đơn giản là $\sigma(AH^{(l)}W^{(l)})$ mà bắt buộc phải có $\tilde{A} = A + I$ và $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$?**

- 1) **Tại sao cần cộng I ($\tilde{A} = A + I$)?** Nếu chỉ nhân $AH^{(l)}$, do đường chéo của A toàn số 0 ($A_{ii} = 0$), đỉnh i chỉ gom đặc trưng của các hàng xóm $\sum_{j \in \mathcal{N}(i)} h_j^{(l)}$ mà **quên sạch đặc trưng của chính bản thân nó $h_i^{(l)}$** ! Cộng thêm I_n giúp gắn một cạnh tự nối (self-loop) vào mỗi đỉnh để đỉnh i tổng hợp cả thông tin của chính nó lẫn hàng xóm.
- 2) **Tại sao cần kẹp $\tilde{D}^{-1/2}(\dots)\tilde{D}^{-1/2}$ ở hai bên?** Nếu nhân trực tiếp $\tilde{A}H^{(l)}$, một đỉnh Hub có 1 000 hàng xóm sẽ cộng dồn 1 000 vectơ lại, làm biên độ đặc trưng phóng đại gấp 1 000 lần so với đỉnh bậc 1 $\Rightarrow$ **Nổ đạo hàm (Exploding Gradients)** ngay sau vài tầng! Khi chuẩn hóa đối xứng $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$, phương trình cập nhật tại từng đỉnh i trở thành:

$$\vec{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{(d_i + 1)(d_j + 1)}} \vec{h}_j^{(l)} W^{(l)} \right)$$

★ **Ý nghĩa cực sâu**: Trọng số truyền tin $\frac{1}{\sqrt{(d_i + 1)(d_j + 1)}}$ không chỉ chia trung bình theo bậc của đỉnh nhận d_i , mà còn **giảm trọng số của các hàng xóm quá ồn ào (d_j lớn)**! (Ví dụ: Một tài khoản người nổi tiếng kết bạn với 1 triệu người thì mỗi liên hệ đó ít mang tính cá nhân hóa hơn một người bạn chỉ có 5 liên kết).

② Hiện tượng Quá trơn (Over-smoothing) khi xếp chồng quá nhiều tầng GNN:

- Mỗi tầng GCN thực chất là một bộ lọc thông thấp trên đồ thị (Low-pass Graph Filter) làm giảm năng lượng Dirichlet $\vec{x}^T L \vec{x}$ — tức là kéo đặc trưng của các đỉnh kề nhau lại gần nhau.
- Nếu xếp chồng quá sâu ($k = 16, 32$ tầng GNN mà không có Residual Connection), phép nhân lặp lại $(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2})^k$ sẽ hội tụ về không gian con của trị riêng lớn nhất ($\lambda = 1$), làm **mọi đỉnh trong cùng thành phần liên thông có vectơ biểu diễn giống hệt nhau** ($h_1^{(k)} \approx h_2^{(k)} \approx \dots \approx h_n^{(k)}$)! Đó là lý do các mạng GNN thực tế thường chỉ sâu từ 2 đến 4 tầng.

2. Transformer Self-Attention dưới góc nhìn Lý thuyết Đồ thị

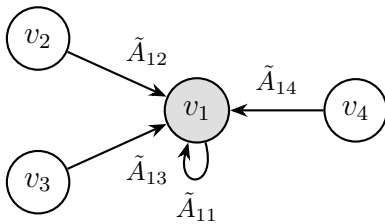
Trong các mô hình ngôn ngữ lớn (LLM / Transformer), một câu văn gồm n token thực chất được mô hình hóa như một **đồ thị đầy đủ có hướng (Complete Directed Graph)** gồm n đỉnh:

$$A_{\text{attn}} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}, \quad H_{\text{out}} = A_{\text{attn}} @ V$$

- Ma trận Attention A_{attn} chính là **ma trận kề có trọng số động (Dynamic Weighted Adjacency Matrix)** được tính lại ở từng tầng dựa trên ngữ cảnh, trong đó $A_{ij} = \alpha_{ij} > 0$ và $\sum_{j=1}^n \alpha_{ij} = 1$ (ma trận ngẫu nhiên theo hàng — Row-stochastic).
- Phép nhân $A_{\text{attn}} @ V$ chính là **1 vòng Message Passing trên đồ thị đầy đủ**: mỗi token i thu thập thông tin V_j từ toàn bộ các token j khác với trọng số cạnh α_{ij} ! (Với Causal Mask trong GPT, ma trận kề A_{attn} là ma trận tam giác dưới — tạo thành Đồ thị có hướng không chu trình DAG chỉ cho phép các từ phía trước truyền tin cho từ phía sau).

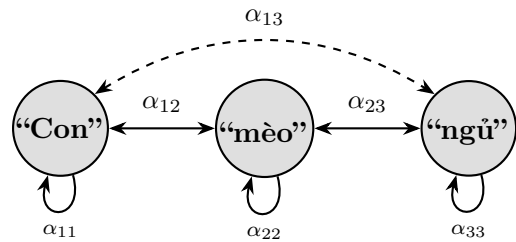
(a) GNN Message Passing

$\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H$ (Chỉ truyền qua láng giềng)



(b) Transformer Self-Attention

$A_{\text{attn}} = \text{softmax}(QK^T / \sqrt{d_k})$ (Đồ thị đầy đủ)



V. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 21

- **Biểu diễn Đại số Đồ thị** $G = (V, E)$: Ma trận kề $A \in \mathbb{R}^{n \times n}$ (với đồ thị thực tế bắt buộc lưu ở dạng thưa **Sparse COO/CSR** tốn $\mathcal{O}(|E|)$ thay vì $\mathcal{O}(n^2)$ RAM) và ma trận bậc đường chéo $D = \text{diag}(d_1, \dots, d_n)$ với $d_i = \sum_j A_{ij}$.
- **Lũy thừa Ma trận Kề & Đếm Đường đi**: Phần tử $(A^k)_{ij}$ bằng đúng **số đường đi (walks)** có độ dài k bước từ đỉnh i đến đỉnh j . Đặc biệt: $(A^2)_{ii} = d_i$ và số tam giác trong đồ thị vô hướng là $\frac{1}{6} \text{Trace}(A^3)$.
- **Ma trận Graph Laplacian** ($L = D - A$): Có dạng toàn phương
$$\vec{x}^T L \vec{x} = \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2 \geq 0$$
, chứng minh L luôn **Xác định Dương Nửa (PSD)** với $\lambda_{\min} = 0$ (vectơ riêng $\vec{1}$).
- **Phân tích Phổ & Spectral Clustering**: Số bội của trị riêng $\lambda = 0$ của L bằng đúng **số thành phần liên thông**. Trị riêng nhỏ thứ hai λ_2 (**Fiedler value**) đo độ liên thông đại số; dấu của vectơ riêng $\vec{v}_2$ (**Fiedler vector**) chia đôi đồ thị tối ưu tại nút cổ chai (Min-Cut).
- **Mạng GNN (GCN) & Self-Attention**: Tầng GCN
$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)})$$
 thêm khuyến tự nối $\tilde{A} = A + I$ để giữ đặc trưng bản thân và chuẩn hóa đối xứng qua $\tilde{D}^{-1/2}$ để chống nổ đạo hàm tại các đỉnh Hub. Trong khi đó, Transformer Attention $A_{\text{attn}} = \text{softmax}(QK^T/\sqrt{d_k})$ chính là GNN trên đồ thị đầy đủ có trọng số động.

Lesson 22: Stochastic Processes

I. Vấn đề (The Problem)

Ở các bài Xác suất Thống kê cơ bản (Lesson 6–9), ta thường xét một biến ngẫu nhiên tĩnh X (tung xúc xắc 1 lần, rút 1 mẫu ảnh độc lập $X_i \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}$). Tuy nhiên, trong thế giới thực và các hệ thống AI hiện đại, **sự ngẫu nhiên tiến hóa liên tục theo thời gian t** , nơi trạng thái ở bước tiếp theo phụ thuộc chặt chẽ vào trạng thái hiện tại!

① **Định nghĩa Quá trình Ngẫu nhiên (Stochastic Process $\{X_t\}_{t \in T}$):**

Một quá trình ngẫu nhiên là một **họ các biến ngẫu nhiên $\{X_t\}_{t \in T}$** được đánh chỉ số theo tập thời gian T :

- **Thời gian rời rạc (Discrete-Time):** $T = \{0, 1, 2, \dots, n\}$ (VD: Chuỗi sinh token của LLM, các bước hành động trong Reinforcement Learning, $T = 1000$ bước khởi nhiễu của DDPM).
- **Thời gian liên tục (Continuous-Time):** $T = [0, +\infty)$ (VD: Chuyển động Brown W_t , Phương trình vi phân ngẫu nhiên SDE trong Score-Based Diffusion / Flow Matching).

Trụ cột AI hiện đại	Quá trình Ngẫu nhiên cốt lõi	Cơ chế toán học vận hành
Mô hình Khuếch tán ⟨DDPM / Diffusion / Sora⟩	Chuỗi Markov Thuận & Ngược + Chuyển động Brown / Langevin	Quá trình thuận cộng nhiễu Gaussian dần dần tới $\mathcal{N}(0, I)$; mạng Neural học đảo ngược chuỗi Markov để sinh ảnh/video.
Học tăng cường (RL) ⟨AlphaGo / RLHF LLM⟩	Quá trình Quyết định Markov ⟨MDP: (S, A, P, R, γ) ⟩	Agent chọn hành động $a_t \sim \pi(a s_t)$, môi trường chuyển trạng thái $s_{t+1} \sim P(s' s_t, a_t)$ và tối ưu Phương trình Bellman.
Suy diễn Bayes ⟨MCMC / Metropolis⟩	Chuỗi Markov Thời gian Rời rạc có Cân bằng Chi tiết	Thiết kế ma trận chuyển P sao cho phân phối dừng $\vec{\pi}$ trùng khớp với phân phối hậu nghiệm phức tạp $p(\theta \mathcal{D})$.
Mô hình Ngôn ngữ ⟨Autoregressive LLM⟩	Quá trình Ngẫu nhiên Tự hồi quy điều khiển bởi Temperature T	Mỗi bước lấy mẫu token $x_{t+1} \sim \text{softmax}(z_t/T)$ rồi nối vào ngữ cảnh để dịch chuyển trạng thái.
Xếp hạng Đồ thị ⟨PageRank / Node2Vec⟩	Bước đi Ngẫu nhiên (Random Walk) trên Đồ thị	Phân phối dừng $\vec{\pi} = \vec{\pi}P$ đo xác suất lướt web dài hạn tại mỗi trang.

II. Chuỗi Markov Thời gian Rời rạc (DTMC)

1. Tính chất Markov (Không nhớ) & Ma trận Chuyển trạng thái P

① Tính chất Markov (Markov Property — Memorylessness):

Quá trình ngẫu nhiên rời rạc $\{X_0, X_1, X_2, \dots\}$ nhận giá trị trong không gian trạng thái hữu hạn $S = \{1, 2, \dots, n\}$ được gọi là một **Chuỗi Markov bậc 1** nếu xác suất chuyển sang trạng thái tương lai X_{t+1} **chỉ phụ thuộc vào trạng thái hiện tại X_t** , mà hoàn toàn độc lập với toàn bộ lịch sử quá khứ $\{X_0, \dots, X_{t-1}\}$:

$$P(X_{t+1} = j \mid X_t = i, X_{t-1} = i_{t-1}, \dots, X_0 = i_0) = P(X_{t+1} = j \mid X_t = i) = P_{ij}$$

★ **Ý nghĩa tính toán:** Nếu không có tính chất Markov, để dự đoán bước thứ $t+1$ với n trạng thái, ta phải lưu bảng xác suất có kích thước bùng nổ hàm mũ n^{t+1} . Nhờ tính chất Markov (“Tương lai độc lập với quá khứ khi đã biết hiện tại”), toàn bộ động lực học của hệ thống được nén gọn vào duy nhất một **ma trận vuông** $P \in \mathbb{R}^{n \times n}$!

② Ma trận Chuyển trạng thái P & Phương trình Chapman-Kolmogorov:

$$P = \begin{bmatrix} P_{11} & P_{12} & \dots & P_{1n} \\ P_{21} & P_{22} & \dots & P_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ P_{n1} & P_{n2} & \dots & P_{nn} \end{bmatrix} \quad (\text{Row-stochastic}): \begin{cases} P_{ij} \geq 0 & \forall i, j \\ \sum_{j=1}^n P_{ij} = 1 & \forall i \end{cases}$$

- **Tại sao tổng mỗi hàng bắt buộc bằng 1** ($\sum_{j=1}^n P_{ij} = 1$)? Vì khi đang đứng ở trạng thái i tại thời điểm t , sang thời điểm $t+1$ hệ thống bắt buộc phải chuyển sang một trạng thái $j \in \{1, \dots, n\}$ nào đó (kể cả việc đứng yên tại i với xác suất P_{ii}).
- **Biểu diễn phân phối qua phép nhân ma trận:** Gọi $\vec{\mu}^{(t)} = [\mu_1^{(t)}, \mu_2^{(t)}, \dots, \mu_n^{(t)}] \in \mathbb{R}^{1 \times n}$ là **vector hàng** biểu diễn phân phối xác suất tại bước t (với $\mu_i^{(t)} = P(X_t = i)$). Khi đó phân phối tại bước $t+1$ và sau k bước là:

$$\begin{matrix} \vec{\mu}^{(t+1)} \\ (1 \times n) \end{matrix} = \begin{matrix} \vec{\mu}^{(t)} \\ (1 \times n) \end{matrix} @ \begin{matrix} P \\ (n \times n) \end{matrix} \implies \begin{matrix} \vec{\mu}^{(t+k)} \\ (1 \times n) \end{matrix} = \begin{matrix} \vec{\mu}^{(t)} \\ (1 \times n) \end{matrix} @ P^k \quad \left(\text{với } P^{(k)} = \underbrace{P @ P @ \dots @ P}_{k \text{ lần}} \right)$$

Phương trình $P^{(m+k)} = P^{(m)} @ P^{(k)}$ hay $P_{ij}^{(m+k)} = \sum_{r=1}^n P_{ir}^{(m)} P_{rj}^{(k)}$ được gọi là **Phương trình Chapman-Kolmogorov** (xác suất đi từ $i \rightarrow j$ sau $m+k$ bước bằng tổng xác suất đi qua mọi trạng thái trung gian r ở bước thứ m).

WRONG — Nhầm lẫn Quy ước Nhân Trái ($\vec{\pi}P$) và Nhân Phải ($P\vec{\pi}$)

- Trong sách Xác suất và Thư viện NumPy/PyTorch (chuẩn Row-stochastic: $P_{ij} = P(i \rightarrow j)$, **tổng mỗi hàng** = 1): vectơ phân phối $\vec{\pi}$ được viết dạng **vectơ hàng** ($1 \times n$) và nhân bên trái (hoặc chuyển vị $\vec{\mu}_{\text{col}}^{(t+1)} = P^T @ \vec{\mu}_{\text{col}}^{(t)}$ nếu dùng vectơ cột):

$$\mu_j^{(t+1)} = \sum_{i=1}^n \mu_i^{(t)} P_{ij} \iff \vec{\mu}^{(t+1)} = \vec{\mu}^{(t)} @ P \quad \checkmark$$

- Nếu lấy ma trận tổng hàng bằng 1 mà nhân trực tiếp với vectơ cột $P @ \vec{\mu}_{\text{col}}$ thì mỗi phần tử kết quả là $\sum_{j=1}^n P_{ij} \mu_j$ (lấy xác suất xuất phát từ i nhân với phân phối tại j) — **sai hoàn toàn bản chất xác suất** và tổng các phần tử không còn bằng 1! $\otimes$

2. Phân phối Dừng ($\vec{\pi}$), Tính Ergodic & Cân bằng Chi tiết

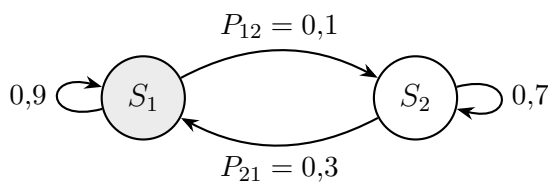
① Định nghĩa Phân phối Dừng (Stationary / Equilibrium Distribution $\vec{\pi}$):

Khi chuỗi Markov chạy đủ lâu ($t \rightarrow \infty$), phân phối xác suất tiến tới trạng thái cân bằng động $\vec{\pi} = [\pi_1, \pi_2, \dots, \pi_n]$ sao cho sau khi bước thêm 1 bước nữa, phân phối vẫn giữ nguyên không đổi:

$$\vec{\pi} @ P = \vec{\pi} \iff P^T \vec{\pi}^T = 1 \cdot \vec{\pi}^T \quad \text{với điều kiện chuẩn hóa } \sum_{i=1}^n \pi_i = 1, \pi_i \geq 0$$

$\Rightarrow$ Về mặt Đại số tuyến tính, phân phối dừng $\vec{\pi}$ chính là **vectơ riêng bên trái (Left Eigenvector)** của ma trận chuyển P (hoặc vectơ riêng bên phải của P^T) ứng với **trị riêng lớn nhất** $\lambda_1 = 1$!

VD 1: Xét mô hình Thời tiết (hoặc trạng thái Server AI) gồm 2 trạng thái: Trạng thái 1 = **Nắng / Tải thấp** (S_1) và Trạng thái 2 = **Mưa / Quá tải** (S_2) với sơ đồ chuyển trạng thái:



Ma trận chuyển trạng thái P :

$$P = \begin{bmatrix} 0,9 & 0,1 \\ 0,3 & 0,7 \end{bmatrix} \begin{array}{l} \rightarrow \text{Tổng hàng 1} = 1,0 \\ \rightarrow \text{Tổng hàng 2} = 1,0 \end{array}$$

- Bước 1 (Tính phân phối sau $k = 1, 2$ ngày nếu ngày đầu chắc chắn Nắng $\vec{\mu}^{(0)} = [1, 0]$):**

$$\vec{\mu}^{(1)} = \vec{\mu}^{(0)} @ P = [1, 0] \begin{bmatrix} 0,9 & 0,1 \\ 0,3 & 0,7 \end{bmatrix} = [0,9, \quad 0,1]$$

$$\vec{\mu}^{(2)} = \vec{\mu}^{(1)} @ P = [0,9, 0,1] \begin{bmatrix} 0,9 & 0,1 \\ 0,3 & 0,7 \end{bmatrix} = [0,81 + 0,03, \quad 0,09 + 0,07] = [0,84, \quad 0,16]$$

- Bước 2 (Tìm chính xác Phân phối Dừng $\vec{\pi} = [\pi_1, \pi_2]$ khi $t \rightarrow \infty$):** Giải hệ phương

trình $\vec{\pi}P = \vec{\pi}$ kết hợp điều kiện tổng xác suất $\pi_1 + \pi_2 = 1$:

$$\begin{cases} 0,9\pi_1 + 0,3\pi_2 = \pi_1 \\ 0,1\pi_1 + 0,7\pi_2 = \pi_2 \\ \pi_1 + \pi_2 = 1 \end{cases} \iff \begin{cases} \pi_1 = 3\pi_2 \\ \pi_1 + \pi_2 = 1 \end{cases} \iff \boxed{\vec{\pi} = [\pi_1, \pi_2] = [0,75, 0,25]}$$

- **Bước 3 (Kiểm tra Tốc độ Hội tụ — Mixing Time qua Khoảng cách Phổ / Spectral Gap):** Ma trận P có $\text{Trace}(P) = 0,9 + 0,7 = 1,6$ và $\det(P) = 0,63 - 0,03 = 0,60 \implies$ 2 trị riêng là $\lambda_1 = 1$ và $\lambda_2 = 0,6$.

Khoảng cách phổ (Spectral Gap) là $\gamma = 1 - |\lambda_2| = 1 - 0,6 = 0,4$. Sai số so với trạng thái dừng tắt dần theo hàm mũ $|\lambda_2|^t = 0,6^t \implies$ Thời gian trộn (Mixing time) chỉ khoảng $\tau_{\text{mix}} \approx \frac{1}{\gamma} = \frac{1}{0,4} = 2,5$ bước!

② Điều kiện Hội tụ Duy nhất (Ergodic Theorem) & Cân bằng Chi tiết (Detailed Balance):

- 1) **Tính tối giản (Irreducibility — Liên thông mạnh):** Mọi trạng thái i đều có thể đi tới mọi trạng thái j khác sau hữu hạn bước ($\forall i, j, \exists k \geq 1 : (P^k)_{ij} > 0$). Nếu đồ thị bị chia cắt hoặc có trạng thái hấp thụ (Absorbing state $P_{ii} = 1$), phân phối cuối cùng sẽ phụ thuộc vào điểm xuất phát.
- 2) **Tính phi tuần hoàn (Aperiodicity):** Chuỗi không bị kẹt trong vòng lặp chẵn/lẻ cố định (VD: đồ thị 2 phía nhảy qua lại $1 \leftrightarrow 2$ với $P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ có $\lambda_2 = -1$ sẽ dao động mãi mãi mà không hội tụ!).
 $\implies$ **Định lý Ergodic:** Một chuỗi Markov hữu hạn vừa **Tối giản** vừa **Phi tuần hoàn** được gọi là **Ergodic** — nó luôn hội tụ về **duy nhất một phân phối dừng** $\vec{\pi}$ bất chấp phân phối khởi tạo $\vec{\mu}^{(0)}$ là gì: $\lim_{k \rightarrow \infty} (P^k)_{ij} = \pi_j$.
- 3) **Phương trình Cân bằng Chi tiết (Detailed Balance / Time-Reversibility — Tráit tìm của MCMC):**

$$\boxed{\pi_i P_{ij} = \pi_j P_{ji} \quad \forall i, j \in S}$$

Ý nghĩa vật lý: Lượng xác suất chảy từ $i \rightarrow j$ ($\pi_i P_{ij}$) bằng đúng lượng xác suất chảy ngược từ $j \rightarrow i$ ($\pi_j P_{ji}$). Nếu thỏa Cân bằng chi tiết thì lấy tổng theo i hai vế: $\sum_i \pi_i P_{ij} = \sum_i \pi_j P_{ji} = \pi_j \sum_i P_{ji} = \pi_j \implies \vec{\pi}P = \vec{\pi}$ (đảm bảo $\vec{\pi}$ chắc chắn là phân phối dừng!). Đây chính là nguyên lý thiết kế thuật toán lấy mẫu **Metropolis-Hastings MCMC**: với phân phối mục tiêu $p(x)$ và phân phối đề xuất $Q(x' | x)$, ta chấp nhận bước nhảy $x \rightarrow x'$ với xác suất $\alpha(x, x') = \min\left(1, \frac{p(x')Q(x | x')}{p(x)Q(x' | x)}\right)$ để ép hệ thống thỏa mãn Cân bằng chi tiết!

III. Bước đi Ngẫu nhiên, Quá trình Poisson & Chuyển động Brown

1. Bước đi Ngẫu nhiên 1D (Random Walk) & Quy luật Căn bậc hai $\sqrt{t}$

Xét một hạt xuất phát tại gốc tọa độ $S_0 = 0$. Ở mỗi bước thời gian $i = 1, 2, \dots, n$, hạt tung đồng xu cân bằng để bước sang phải ($X_i = +1$ với xác suất $1/2$) hoặc bước sang trái ($X_i = -1$ với xác suất $1/2$). Vị trí sau n bước là:

$$S_n = \sum_{i=1}^n X_i, \quad \text{với } \mathbb{E}[X_i] = 0, \quad \text{Var}(X_i) = \mathbb{E}[X_i^2] - (\mathbb{E}[X_i])^2 = 1 - 0 = 1$$

Do các bước nhảy $X_1, \dots, X_n$ độc lập đôi một, kỳ vọng và phương sai của vị trí S_n sau n bước là:

$$\mathbb{E}[S_n] = 0, \quad \text{Var}(S_n) = \sum_{i=1}^n \text{Var}(X_i) = n \implies \sigma(S_n) = \sqrt{\text{Var}(S_n)} = \sqrt{n}$$

❓ Vì sao khoảng cách dịch chuyển điển hình lại tỷ lệ với $\sqrt{n}$ chứ không phải n hay 0?

- Nhiều người lầm tưởng vì xác suất trái/phải bằng nhau (50%/50%) nên đi 10 000 bước sẽ triệt tiêu nhau và đứng ngay sát 0. Thực tế, $\mathbb{E}[S_n] = 0$ chỉ có nghĩa là trung bình có đối xứng dấu (+ và -), còn **khoảng cách thực tế tới gốc tọa độ** $\sqrt{\mathbb{E}[S_n^2]} = \sqrt{n} = \sqrt{10\,000} = 100$ bước!
- Quy luật $\sqrt{n}$ này giải thích vì sao trong Transformer Attention (Lesson 21), tích vô hướng $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ của hai vectơ ngẫu nhiên độc lập có phương sai tăng tuyến tính theo d_k và độ lệch chuẩn tăng theo $\sqrt{d_k} \implies$ **bắt buộc phải chia cho $\sqrt{d_k}$** trong $\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$ để giữ phương sai bằng 1!

② Bài toán Sạt nghiệp của Con bạc (Gambler's Ruin & Martingale):

Nếu người chơi bắt đầu với số vốn k đô-la ($0 < k < N$), mỗi ván thắng $+1\$$ hoặc thua $-1\$$ với xác suất $1/2$, và dừng cuộc chơi khi chạm vạch phá sản 0 hoặc đạt mục tiêu N (hai trạng thái hấp thụ — Absorbing Barriers). Do trò chơi công bằng là một **Martingale** ($\mathbb{E}[S_T] = S_0 = k$), gọi P_k là xác suất chạm đích N trước khi chạm 0:

$$\mathbb{E}[S_T] = N \cdot P_k + 0 \cdot (1 - P_k) = k \implies P(\text{Đạt } N \text{ trước khi phá sản } 0 \mid S_0 = k) = \frac{k}{N}$$

(Nếu $N \rightarrow \infty$ như sòng bạc vô hạn vốn thì $P_k \rightarrow 0$: người chơi chắc chắn 100% sẽ phá sản!).

2. Quá trình Poisson & Chuyển động Brown (Wiener Process W_t)

- Quá trình đếm Poisson $N(t)$ (Poisson Process):** Đếm số sự kiện rời rạc (request gửi vào server LLM, gói tin mạng) xảy ra trong khoảng thời gian $[0, t]$ với tốc độ trung bình $\lambda > 0$. Số sự kiện trong khoảng Δt tuân theo phân phối Poisson: $N(t + \Delta t) - N(t) \sim \text{Poisson}(\lambda \Delta t)$, và thời gian chờ giữa 2 sự kiện liên tiếp tuân theo **phân phối Mũ** $\text{Exp}(\lambda)$.

- **Chuyển động Brown / Quá trình Wiener W_t (Giới hạn liên tục của Random Walk):** Chia nhỏ khoảng thời gian $[0, t]$ thành n bước vi phân $\Delta t = t/n$, mỗi bước hạt nhảy $\pm\sqrt{\Delta t}$. Khi $n \rightarrow \infty$ ($\Delta t \rightarrow dt$), theo Định lý Giới hạn Trung tâm (CLT), quá trình hội tụ về **Chuyển động Brown chuẩn W_t** thỏa mãn 3 tiên đề:

$$\boxed{1) W_0 = 0 \quad 2) \text{ Số gia độc lập } dW_t \quad 3) W_t - W_s \sim \mathcal{N}(0, t - s) \quad (\forall t > s \geq 0)}$$

★ **Quy tắc Vi phân Ngẫu nhiên Itô (Euler-Maruyama Discretization):** Trong mô phỏng máy tính với bước thời gian nhỏ Δt , vi phân chuyển động Brown luôn được lấy mẫu bằng:

$$\boxed{\Delta W_t = W_{t+\Delta t} - W_t = \sqrt{\Delta t} \cdot z, \quad \text{với } z \sim \mathcal{N}(0, 1)}$$

(Lưu ý: Nhân với $\sqrt{\Delta t}$ chứ không phải Δt , vì $\text{Var}(\sqrt{\Delta t} z) = (\sqrt{\Delta t})^2 \text{Var}(z) = \Delta t$!).

IV. Ứng dụng Chuyên sâu 1: Học Tăng cường & Phương trình Bellman (MDP)

Trong Học tăng cường (Reinforcement Learning — RL, từ AlphaGo đến bước tinh chỉnh RLHF / GRPO cho các mô hình ngôn ngữ lớn), một chuỗi Markov thuần túy chưa đủ vì chưa có **người ra quyết định (Agent)**. Khi bổ sung thêm hành động a_t và phần thưởng r_t , chuỗi Markov mở rộng thành **Quá trình Quyết định Markov (Markov Decision Process — MDP)** gồm bộ 5 thành phần $\langle S, A, P, R, \gamma \rangle$:

- S : Tập trạng thái (States — VD: chuỗi văn bản prompt + các token đã sinh).
- A : Tập hành động (Actions — VD: từ vựng gồm 128 000 token tiếp theo có thể chọn).
- $P(s' | s, a) = P(S_{t+1} = s' | S_t = s, A_t = a)$: Xác suất chuyển trạng thái có điều kiện theo hành động.
- $R(s, a, s')$: Phần thưởng tức thời (Immediate Reward); $\gamma \in [0, 1]$: Hệ số chiết khấu tương lai (Discount factor).

① Phương trình Kỳ vọng Bellman (Bellman Expectation Equation):

Với chiến lược hành động ngẫu nhiên (Policy) $\pi(a | s) = P(A_t = a | S_t = s)$, hàm giá trị trạng thái $V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$ thỏa mãn hệ phương trình tuyến tính đệ quy:

$$\boxed{V^\pi(s) = \sum_{a \in A} \pi(a | s) \sum_{s' \in S} P(s' | s, a) \left[R(s, a, s') + \gamma V^\pi(s') \right]}$$

$\Rightarrow$ Dưới dạng ma trận (khi cố định Policy π), đặt $R_s^\pi = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) R(s, a, s')$ và $P_{ss'}^\pi = \sum_a \pi(a | s) P(s' | s, a)$, phương trình Bellman trở thành:

$$\boxed{\vec{V}^\pi = \vec{R}^\pi + \gamma P^\pi \vec{V}^\pi \iff (I - \gamma P^\pi) \vec{V}^\pi = \vec{R}^\pi \iff \vec{V}^\pi = (I - \gamma P^\pi)^{-1} \vec{R}^\pi}$$

Vì P^π là ma trận Markov có trị riêng lớn nhất bằng 1 và hệ số chiết khấu $\gamma < 1$, mọi trị riêng của $(I - \gamma P^\pi)$ đều dương ($\geq 1 - \gamma > 0$), đảm bảo ma trận $(I - \gamma P^\pi)$ luôn khả nghịch!

V. Ứng dụng Chuyên sâu 2: Mô hình Khuếch tán (DDPM & Langevin)

1. Quá trình Khuếch tán Thuận (Forward Diffusion Process)

Cho một bức ảnh gốc sắc nét $x_0 \sim q(x_0)$. Mô hình Denoising Diffusion Probabilistic Models (DDPM — Ho et al., NeurIPS 2020) định nghĩa một **chuỗi Markov thuận** $q(x_{1:T} | x_0)$ cố định không cần học, làm nhiệm vụ cộng thêm một chút nhiễu Gaussian tại mỗi bước $t = 1, 2, \dots, T$ (với lịch trình phương sai nhiễu nhỏ dần $0 < \beta_1 < \beta_2 < \dots < \beta_T < 1$):

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I) \iff x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I)$$

❓ Vì sao hệ số nhân vào x_{t-1} phải là $\sqrt{1 - \beta_t}$ mà không phải $(1 - \beta_t)$?

- Giả sử dữ liệu đầu vào đã chuẩn hóa có phương sai đơn vị $\text{Var}(x_{t-1}) = I$. Khi lấy tổ hợp tuyến tính của hai biến độc lập x_{t-1} và ε_t :

$$\text{Var}(x_t) = \left(\sqrt{1 - \beta_t}\right)^2 \text{Var}(x_{t-1}) + \left(\sqrt{\beta_t}\right)^2 \text{Var}(\varepsilon_t) = (1 - \beta_t)I + \beta_t I = I$$

- Nhờ căn bậc hai $\sqrt{1 - \beta_t}$ (Variance-Preserving), phương sai của tín hiệu được **bảo toàn bằng 1 xuyên suốt toàn bộ** $T = 1000$ **bước**, không bị nổ lên ∞ hay tắt lịm về 0, đảm bảo khi $t \rightarrow T$ thì x_T hội tụ chuẩn xác về nhiễu chuẩn tắc $\mathcal{N}(0, I)$!

② Chứng minh Công thức Nhảy cóc Đóng (Closed-form Jump từ $x_0 \rightarrow x_t$):

Khi huấn luyện mạng Diffusion với $T = 1000$ bước, nếu mỗi bức ảnh trong batch đều phải chạy vòng lặp **for** 1000 lần từ $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_t$ thì tốc độ huấn luyện sẽ chậm gấp 1000 lần!

Đặt $\alpha_t = 1 - \beta_t$ và tích lũy $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s = \alpha_1 \alpha_2 \dots \alpha_t$. Ta chứng minh có thể nhảy thẳng từ

ảnh gốc x_0 tới trạng thái nhiễu ở bước t bất kỳ chỉ bằng **1 phép tính duy nhất**:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I) \iff x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

★ Chứng minh từng bước bằng tính chất cộng biến Gaussian độc lập:

- **Bước 1:** Viết công thức truy hồi cho 2 bước liên tiếp $t-1$ và t (với $\varepsilon_{t-1}, \varepsilon_t \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, I)$):

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t \quad \text{và} \quad x_{t-1} = \sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-1}$$

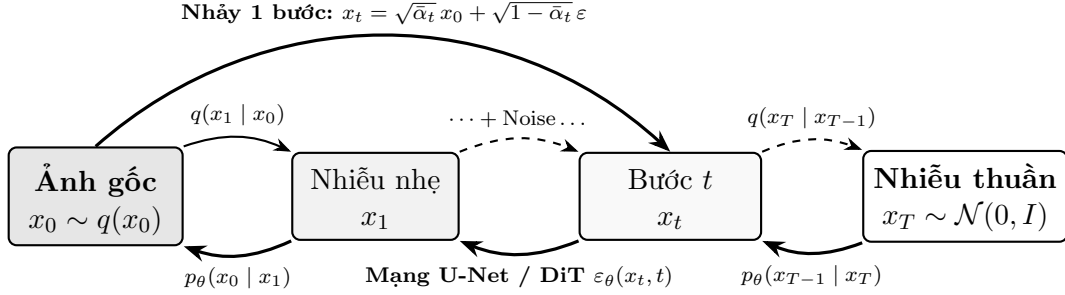
- **Bước 2:** Thế x_{t-1} vào biểu thức của x_t :

$$\begin{aligned} x_t &= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_{t-1}} \varepsilon_{t-1} \right) + \sqrt{1 - \alpha_t} \varepsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \underbrace{\sqrt{\alpha_t (1 - \alpha_{t-1})} \varepsilon_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t}_{\text{Tổng của 2 biến Gaussian độc lập}} \end{aligned}$$

- **Bước 3:** Áp dụng quy tắc cộng hai biến Gaussian độc lập $\mathcal{N}(0, \sigma_1^2 I) + \mathcal{N}(0, \sigma_2^2 I) = \mathcal{N}(0, (\sigma_1^2 + \sigma_2^2)I)$, tổng phương sai của hai thành phần nhiễu là:

$$\sigma_{\text{gộp}}^2 = \alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = \alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t = 1 - \alpha_t\alpha_{t-1}$$

Do đó hai nhiễu gộp lại thành một nhiễu chuẩn duy nhất $\sqrt{1 - \alpha_t\alpha_{t-1}} \bar{\varepsilon}_{t-2}$. Lặp lại quy nạp lùi về tận x_0 , ta được $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$ (đpcm)!



2. Quá trình Khử nhiễu Ngược & Động lực học Langevin

① Học Chuỗi Markov Ngược bằng Mạng Neural Dự đoán Nhiễu $\varepsilon_\theta(x_t, t)$:

Để sinh ảnh mới, ta lấy mẫu nhiễu trắng $x_T \sim \mathcal{N}(0, I)$ rồi chạy **chuỗi Markov ngược** $p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$ lùi từ $T \rightarrow 0$. Thay vì bắt mạng Neural dự đoán trực tiếp bức ảnh, DDPM huấn luyện mạng U-Net / Diffusion Transformer (DiT) $\varepsilon_\theta(x_t, t)$ **dự đoán lớp nhiễu ε đã cộng vào x_0** qua hàm mất mát MSE cực kỳ đơn giản:

$$\mathcal{L}_{\text{DDPM}}(\theta) = \mathbb{E}_{t, x_0, \varepsilon \sim \mathcal{N}(0, I)} \left[\left\| \varepsilon - \varepsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, t) \right\|^2 \right]$$

Khi đã đoán được nhiễu $\varepsilon_\theta(x_t, t)$, công thức bước nhảy Markov ngược để khử nhiễu từ $x_t \rightarrow x_{t-1}$ là:

$$x_{t-1} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I)$$

② Động lực học Langevin (Langevin Dynamics & Score-Based Models):

Nếu muốn lấy mẫu từ một phân phối xác suất bất kỳ $p(x) = \frac{1}{Z} e^{-U(x)/T}$ (với hàm năng lượng $U(x)$ và hằng số chuẩn hóa $Z = \int e^{-U(x)/T} dx$ không thể tính tích phân được), **Động lực học Langevin** kết hợp Gradient Descent với Chuyển động Brown:

$$x_{t+1} = x_t - \eta \nabla_x U(x_t) + \sqrt{2T\eta} z_t \iff x_{t+1} = x_t + \eta \underbrace{\nabla_x \log p(x_t)}_{\text{Score } s_\theta(x_t)} + \sqrt{2\eta} z_t$$

- Tại sao $\nabla_x \log p(x)$ (Score Function) lại khử mất hằng số Z ? Vì $\log p(x) = -\frac{U(x)}{T} - \log Z$, khi lấy đạo hàm theo x thì hằng số $\nabla_x \log Z = 0$ biến mất hoàn toàn!
- Hai lực đối kháng tạo nên sự kỳ diệu:

- 1) **Lực dẫn đường (Drift / Gradient Force $+\eta \nabla_x \log p(x_t)$):** Đẩy hạt về vùng có mật độ xác suất cao (năng lượng thấp $U(x)$ nhỏ). Khi nhiệt độ $T \rightarrow 0$, thành phần nhiễu tắt hẳn và Langevin Dynamics suy biến về **Gradient Descent thuần túy** (chỉ rơi vào 1 cực tiểu cục bộ).

2) **Lực khuếch tán ngẫu nhiên (Brownian Noise + $\sqrt{2T\eta}z_t$):** Ngăn hạt bị kẹt chết ở một điểm cực trị, giúp hạt khám phá toàn bộ không gian và hội tụ về đúng phân phối mục tiêu $p(x)$!

- **Mối liên hệ thống nhất giữa Score Function và Nhiễu DDPM:** Với $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$, lấy đạo hàm log-mật độ Gaussian theo x_t cho ta:

$$\nabla_{x_t} \log q(x_t | x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1 - \bar{\alpha}_t} = -\frac{\varepsilon}{\sqrt{1 - \bar{\alpha}_t}} \implies \boxed{s_\theta(x_t, t) = -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}}$$

$\implies$ Dự đoán nhiễu ε_θ trong DDPM và ước lượng đạo hàm mật độ $\nabla_x \log p(x)$ trong Score-based Langevin Dynamics thực chất là **cùng một bài toán toán học** chỉ khác nhau hệ số tỷ lệ $-\frac{1}{\sqrt{1 - \bar{\alpha}_t}}$!

VI. Tổng hợp Kiến thức Trọng tâm (Key Takeaways)

BẢNG TÓM TẮT CÔNG THỨC & BẢN CHẤT — LESSON 22

- **Tính chất Markov & Ma trận chuyển P :** Tương lai chỉ phụ thuộc vào hiện tại: $P(X_{t+1} = j | X_t = i, \dots, X_0) = P_{ij}$. Với P chuẩn hóa theo hàng ($\sum_j P_{ij} = 1$), phân phối sau k bước là $\boxed{\vec{\mu}^{(t+k)} = \vec{\mu}^{(t)} @ P^k}$ (Phương trình Chapman-Kolmogorov).
- **Phân phối Dừng ($\vec{\pi} = \vec{\pi}P$) & Cân bằng Chi tiết:** $\vec{\pi}$ là vectơ riêng bên trái của P ứng với $\lambda_1 = 1$. Chuỗi **Ergodic** (Tối giản + Phi tuần hoàn) luôn hội tụ về duy nhất một $\vec{\pi}$ với tốc độ quyết định bởi khoảng cách phổ $\gamma = 1 - |\lambda_2|$. Điều kiện Cân bằng chi tiết $\boxed{\pi_i P_{ij} = \pi_j P_{ji}}$ đảm bảo hội tụ trong lấy mẫu **Metropolis-Hastings MCMC**.
- **Random Walk & Chuyển động Brown W_t :** Tổng n bước ngẫu nhiên độc lập có phương sai $\text{Var}(S_n) = n \implies$ độ lệch chuẩn tăng theo $\sqrt{n}$. Giới hạn liên tục sinh ra Chuyển động Brown $W_t - W_s \sim \mathcal{N}(0, t - s)$ với vi phân ngẫu nhiên $\Delta W_t = \sqrt{\Delta t} z$ ($z \sim \mathcal{N}(0, 1)$).
- **Quá trình Quyết định Markov (MDP) trong RL:** Mở rộng chuỗi Markov với hành động a và phần thưởng R , giải qua Phương trình Bellman:

$$\boxed{V^\pi(s) = \sum_{a \in A} \pi(a | s) \sum_{s' \in S} P(s' | s, a) \left[R(s, a, s') + \gamma V^\pi(s') \right]}$$

- **Mô hình Khuếch tán (DDPM) & Langevin Dynamics:** Quá trình thuận cộng nhiễu nhảy thẳng 1 bước $\boxed{x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\varepsilon}$; quá trình ngược dùng mạng Neural dự đoán nhiễu $\varepsilon_\theta(x_t, t) = -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p(x_t)$ để khử nhiễu từ $\mathcal{N}(0, I)$ về dữ liệu thật.

NGUỒN TÀI LIỆU & MÃ NGUỒN (REPOSITORY SOURCES)

- **Người soạn:** Nguyễn Việt Thành - K70 HUST
- **Kho lưu trữ tài liệu Tiếng Việt (GitHub):**
https://github.com/vietthanh-ng/math_foundations_for_ai_vietnamese.git
- **Nguồn lộ trình tham khảo gốc (Reference Curriculum):**
<https://github.com/rohitg00/ai-engineering-from-scratch>